

Babel

Code

Version 26.11.134271
2026/09/03

Javier Bezos
Current maintainer

Johannes L. Braams
Original author

Localization and
internationalization

Unicode

T_EX

LuaT_EX

pdfT_EX

XeT_EX

Contents

1	Identification and loading of required files	3
2	locale directory	3
3	Tools	3
3.1	A few core definitions	8
3.2	$\LaTeX$: babel.sty (start)	8
3.3	base	9
3.4	key=value options and other general option	10
3.5	Post-process some options	11
3.6	Plain: babel.def (start)	13
4	babel.sty and babel.def (common)	13
4.1	Selecting the language	15
4.2	Errors	23
4.3	More on selection	24
4.4	Short tags	26
4.5	Compatibility with language.def	26
4.6	Hooks	27
4.7	Setting up language files	27
4.8	Shorthands	30
4.9	Language attributes	38
4.10	Support for saving and redefining macros	40
4.11	French spacing	41
4.12	Hyphens	42
4.13	Multiencoding strings	44
4.14	Tailor captions	48
4.15	Making glyphs available	49
4.15.1	Quotation marks	50
4.15.2	Letters	51
4.15.3	Shorthands for quotation marks	52
4.15.4	Umlauts and tremas	53
4.16	Layout	54
4.17	Load engine specific macros	54
4.18	Creating and modifying languages	54
4.19	Main loop in ‘provide’	63
4.20	Processing keys in ini	67
4.21	French spacing (again)	73
4.22	Handle language system	74
4.23	Numerals	75
4.24	Casing	77
4.25	Getting info	77
4.26	BCP 47 related commands	78
5	Adjusting the Babel behavior	79
5.1	Cross referencing macros	82
5.2	Layout	84
5.3	Marks	86
5.4	Other packages	87
5.4.1	ifthen	87
5.4.2	varioref	87
5.4.3	hhline	88
5.5	Encoding and fonts	89
5.6	Basic bidi support	90
5.7	Local Language Configuration	93
5.8	Language options	94

6	The kernel of Babel	98
7	Error messages	98
8	Loading hyphenation patterns	102
9	luatex + xetex: common stuff	106
10	Hooks for XeTeX and LuaTeX	109
10.1	XeTeX	109
10.2	Support for interchar	111
10.3	Layout	113
10.4	8-bit TeX	115
10.5	LuaTeX	115
10.6	Southeast Asian scripts	122
10.7	CJK line breaking	123
10.8	Arabic justification	125
10.9	Common stuff	130
10.10	Automatic fonts and ids switching	130
10.11	Bidi	137
10.12	Layout	139
10.13	Lua: transforms	148
10.14	Lua: Auto bidi with basic and basic-r	158
11	Data for CJK	170
12	The ‘nil’ language	170
13	Calendars	171
13.1	Islamic	171
13.2	Hebrew	173
13.3	Persian	177
13.4	Coptic and Ethiopic	177
13.5	Julian	178
13.6	Buddhist	178
14	Support for Plain TeX (plain.def)	180
14.1	Not renaming hyphen.tex	180
14.2	Emulating some L ^A T _E X features	181
14.3	General tools	181
14.4	Encoding related macros	185
15	Acknowledgements	187

The babel package is being developed incrementally, which means parts of the code are under development and therefore incomplete. Only documented features are considered complete. In other words, use babel in real documents only as documented (except, of course, if you want to explore and test them).

1 Identification and loading of required files

The babel package after unpacking consists of the following files:

babel.sty is the $\text{\LaTeX}$ package, which set options and load language styles.

babel.def is loaded by Plain.

switch.def defines macros to set and switch languages (it loads part babel.def).

plain.def is not used, and just loads babel.def, for compatibility.

hyphen.cfg is the file to be used when generating the formats to load hyphenation patterns.

There some additional tex, def and lua files.

The babel installer extends docstrip with a few “pseudo-guards” to set “variables” used at installation time. They are used with `<@name@>` at the appropriate places in the source code and defined with either `{(name=value)}`, or with a series of lines between `{(*name)}` and `{(/name)}`. The latter is cumulative (e.g., with *More package options*). That brings a little bit of literate programming. The guards `<-name>` and `<+name>` have been redefined, too. See babel.ins for further details.

2 locale directory

A required component of babel is a set of ini files with basic definitions for about 300 languages. They are distributed as a separate zip file, not packed as dtx. Many of them are essentially finished (except bugs and mistakes, of course). Some of them are still incomplete (but they will be usable), and there are some omissions (e.g., there are no geographic areas in Spanish). Not all include LICR variants.

babel-*.ini files contain the actual data; babel-*.tex files are basically proxies to the corresponding ini files.

See [Keys in ini files](#) in the the babel site.

3 Tools

```
1 {(version=26.11.134271)}
2 {(date=2026/09/03)}
```

Do not use the following macros in ldf files. They may change in the future. This applies mainly to those recently added for replacing, trimming and looping. The older ones, like `\bbl@afterfi`, will not change. We define some basic macros which just make the code cleaner. `\bbl@add` is now used internally instead of `\addto` because of the unpredictable behavior of the latter. Used in babel.def and in babel.sty, which means in $\text{\LaTeX}$ is executed twice, but we need them when defining options and babel.def cannot be load until options have been defined. This does not hurt, but should be fixed somehow.

```
3 {( *Basic macros)} ≡
4 \bbl@trace{Basic macros}
5 \def\bbl@stripslash{\expandafter\@gobble\string}
6 \def\bbl@add#1#2{%
7   \bbl@ifunset{\bbl@stripslash#1}%
8     {\def#1{#2}}%
9     {\expandafter\def\expandafter#1\expandafter{#1#2}}
10 \def\bbl@xin@{\@expandtwoargs\in@}
11 \def\bbl@carg#1#2{\expandafter#1\csname#2\endcsname}%
12 \def\bbl@ncarg#1#2#3{\expandafter#1\expandafter#2\csname#3\endcsname}%
13 \def\bbl@ccarg#1#2#3{%
14   \expandafter#1\csname#2\expandafter\endcsname\csname#3\endcsname}%
15 \def\bbl@carg#1#2{\expandafter#1\csname bbl@#2\endcsname}%
16 \def\bbl@cs#1{\csname bbl@#1\endcsname}
17 \def\bbl@c1#1{\csname bbl@#1\language\endcsname}
18 \def\bbl@loop#1#2#3{\bbl@loop#1{#3}#2,\@nnil,}
```

```

19 \def\bbl@loopx#1#2{\expandafter\bbl@loop\expandafter#1\expandafter{#2}}
20 \def\bbl@loop#1#2#3,{%
21   \ifx\@nnil#3\relax\else
22     \def#1{#3}#2\bbl@afterfi\bbl@loop#1{#2}%
23   \fi}
24 \def\bbl@for#1#2#3{\bbl@loopx#1{#2}{\ifx#1\@empty\else#3\fi}}

```

\bbl@add@list This internal macro adds its second argument to a comma separated list in its first argument. When the list is not defined yet (or empty), it will be initiated. It presumes expandable character strings.

```

25 \def\bbl@add@list#1#2{%
26   \edef#1{%
27     \bbl@ifunset{\bbl@stripslash#1}%
28     }%
29     {\ifx#1\@empty\else#1,\fi}%
30   #2}}

```

\bbl@afterelse

\bbl@afterfi Because the code that is used in the handling of active characters may need to look ahead, we take extra care to ‘throw’ it over the `\else` and `\fi` parts of an `\if`-statement¹. These macros will break if another `\if... \fi` statement appears in one of the arguments and it is not enclosed in braces.

```

31 \long\def\bbl@afterelse#1\else#2\fi{\fi#1}
32 \long\def\bbl@afterfi#1\fi{\fi#1}

```

\bbl@exp Now, just syntactical sugar, but it makes partial expansion of some code a lot more simple and readable. Here `\` stands for `\noexpand`, `\<.` for `\noexpand` applied to a built macro name (which does not define the macro if undefined to `\relax`, because it is created locally), and `\[. . .]` for one-level expansion (where `. . .` is the macro name without the backslash). The result may be followed by extra arguments, if necessary.

```

33 \def\bbl@exp#1{%
34   \begingroup
35   \let\<\noexpand
36   \let\<\bbl@exp@en
37   \let\[\bbl@exp@ue
38   \edef\bbl@exp@aux{\endgroup#1}%
39   \bbl@exp@aux}
40 \def\bbl@exp@en#1>{\expandafter\noexpand\csname#1\endcsname}%
41 \def\bbl@exp@ue#1]{%
42   \unexpanded\expandafter\expandafter\expandafter{\csname#1\endcsname}}%

```

\bbl@trim The following piece of code is stolen (with some changes) from `keyval`, by David Carlisle. It defines two macros: `\bbl@trim` and `\bbl@trim@def`. The first one strips the leading and trailing spaces from the second argument and then applies the first argument (a macro, `\toks@` and the like). The second one, as its name suggests, defines the first argument as the stripped second argument.

```

43 \def\bbl@tempa#1{%
44   \long\def\bbl@trim##1##2{%
45     \futurelet\bbl@trim@a\bbl@trim@c##2\@nil\@nil#1\@nil\relax{##1}}%
46   \def\bbl@trim@c{%
47     \ifx\bbl@trim@a\@sptoken
48       \expandafter\bbl@trim@b
49     \else
50       \expandafter\bbl@trim@b\expandafter#1%
51     \fi}%
52   \long\def\bbl@trim@b#1##1 \@nil{\bbl@trim@i##1}}
53 \bbl@tempa{ }
54 \long\def\bbl@trim@i#1\@nil#2\relax#3{#3{#1}}
55 \long\def\bbl@trim@def#1{\bbl@trim{\def#1}}

```

¹This code is based on code presented in TUGboat vol. 12, no2, June 1991 in “An expansion Power Lemma” by Sonja Maus.

\bbl@ifunset To check if a macro is defined, we create a new macro, which does the same as `\ifundefined`. However, in an ϵ -tex engine, it is based on `\ifcsname`, which is more efficient, and does not waste memory. Defined inside a group, to avoid `\ifcsname` being implicitly set to `\relax` by the `\csname` test.

```

56 \begingroup
57 \gdef\bbl@ifunset#1{%
58   \expandafter\ifx\csname#1\endcsname\relax
59     \expandafter\@firstoftwo
60   \else
61     \expandafter\@secondoftwo
62   \fi}
63 \bbl@ifunset{ifcsname}%
64 {}%
65 {\gdef\bbl@ifunset#1{%
66   \ifcsname#1\endcsname
67     \expandafter\ifx\csname#1\endcsname\relax
68       \bbl@afterelse\expandafter\@firstoftwo
69     \else
70       \bbl@afterfi\expandafter\@secondoftwo
71     \fi
72   \else
73     \expandafter\@firstoftwo
74   \fi}}
75 \endgroup

```

\bbl@ifblank A tool from url, by Donald Arseneau, which tests if a string is empty or space. The companion macros tests if a macro is defined with some ‘real’ value, i.e., not `\relax` and not empty,

```

76 \def\bbl@ifblank#1{%
77   \bbl@ifblank@i#1\@nil\@nil\@secondoftwo\@firstoftwo\@nil}
78 \long\def\bbl@ifblank@i#1#2\@nil#3#4#5\@nil{#4}
79 \def\bbl@ifset#1#2#3{%
80   \bbl@ifunset{#1}{#3}{\bbl@exp{\bbl@ifblank{\@nameuse{#1}}}{#3}{#2}}

```

For each element in the comma separated `<key>=<value>` list, execute `<code>` with #1 and #2 as the key and the value of current item (trimmed). In addition, the item is passed verbatim as #3. With the `<key>` alone, it passes `\@empty` as value (i.e., the macro thus named, not an empty argument, which is what you get with `<key>=` and no value).

```

81 \def\bbl@forkv#1#2{%
82   \def\bbl@kvcmd##1##2##3{#2}%
83   \bbl@kvnext#1,\@nil,}
84 \def\bbl@kvnext#1,{%
85   \ifx\@nil#1\relax\else
86     \bbl@ifblank{#1}{\bbl@forkv@eq#1=\@empty=\@nil{#1}}%
87     \expandafter\bbl@kvnext
88   \fi}
89 \def\bbl@forkv@eq#1=#2=#3\@nil#4{%
90   \bbl@trim\def\bbl@forkv@a{#1}%
91   \bbl@trim{\expandafter\bbl@kvcmd\expandafter{\bbl@forkv@a}}{#2}{#4}}

```

A *for* loop. Each item (trimmed) is #1. It cannot be nested (it’s doable, but we don’t need it).

```

92 \def\bbl@vforeach#1#2{%
93   \def\bbl@forcmd##1{#2}%
94   \bbl@fornext#1,\@nil,}
95 \def\bbl@fornext#1,{%
96   \ifx\@nil#1\relax\else
97     \bbl@ifblank{#1}{\bbl@trim\bbl@forcmd{#1}}%
98     \expandafter\bbl@fornext
99   \fi}
100 \def\bbl@foreach#1{\expandafter\bbl@vforeach\expandafter{#1}}

```

Some code should be executed once. The first argument is a flag.

```

101 \global\let\bbl@done\@empty
102 \def\bbl@once#1#2{%

```

```

103 \bbl@xin@{, #1, }{, \bbl@done,}%
104 \ifin@ \else
105     #2%
106 \xdef\bbl@done{\bbl@done, #1,}%
107 \fi}
108 % \end{macrodef}
109 %
110 % \macro{\bbl@replace}
111 %
112 % Returns implicitly |\toks@| with the modified string.
113 %
114 % \begin{macrocode}
115 \def\bbl@replace#1#2#3{% in #1 -> repl #2 by #3
116 \toks@{}}%
117 \def\bbl@replace@aux##1#2##2#2{%
118 \ifx\bbl@nil##2%
119 \toks@{\expandafter{\the\toks@##1}}%
120 \else
121 \toks@{\expandafter{\the\toks@##1#3}}%
122 \bbl@afterfi
123 \bbl@replace@aux##2#2%
124 \fi}%
125 \expandafter\bbl@replace@aux#1#2\bbl@nil#2%
126 \edef#1{\the\toks@}}

```

An extension to the previous macro. It takes into account the parameters, and it is string based (i.e., if you replace elax by ho, then \relax becomes \rho). No checking is done at all, because it is not a general purpose macro, and it is used by babel only when it works (an example where it does *not* work is in \bbl@TG@@date, and also fails if there are macros with spaces, because they are retokenized). It may change! (or even merged with \bbl@replace; I'm not sure checking the replacement is really necessary or just paranoia).

```

127 \ifx\detokenize@undefined\else % Unused macros if old Plain TeX
128 \bbl@exp{\def\\bbl@parsedef##1\detokenize{macro:}}#2->#3\relax{%
129 \def\bbl@tempa{#1}%
130 \def\bbl@tempb{#2}%
131 \def\bbl@tempe{#3}}
132 \def\bbl@sreplace#1#2#3{%
133 \begingroup
134 \expandafter\bbl@parsedef\meaning#1\relax
135 \def\bbl@tempc{#2}%
136 \edef\bbl@tempc{\expandafter\strip@prefix\meaning\bbl@tempc}%
137 \def\bbl@tempd{#3}%
138 \edef\bbl@tempd{\expandafter\strip@prefix\meaning\bbl@tempd}%
139 \bbl@xin@{\bbl@tempc}{\bbl@tempe}% If not in macro, do nothing
140 \ifin@
141 \bbl@exp{\\bbl@replace\\bbl@tempe{\bbl@tempc}{\bbl@tempd}}%
142 \def\bbl@tempc{% Expanded an executed below as 'uplevel'
143 \\makeatletter % "internal" macros with @ are assumed
144 \\scantokens{%
145 \bbl@tempa\\@namedef{\bbl@stripslash#1}\bbl@tempb{\bbl@tempe}%
146 \noexpand\noexpand}%
147 \catcode64=\the\catcode64\relax}% Restore @
148 \else
149 \let\bbl@tempc\@empty % Not \relax
150 \fi
151 \bbl@exp{% For the 'uplevel' assignments
152 \endgroup
153 \bbl@tempc}} % empty or expand to set #1 with changes
154 \fi

```

Two further tools. \bbl@ifsamestring first expand its arguments and then compare their expansion (sanitized, so that the catcodes do not matter). \bbl@engine takes the following values: 0 is pdfTeX, 1 is luatex, and 2 is xetex. You may use the latter it in your language style if you want.

```

155 \def\bbl@ifsamestring#1#2{%

```

```

156 \begingroup
157   \protected@edef\bbl@tempb{#1}%
158   \edef\bbl@tempb{\expandafter\strip@prefix\meaning\bbl@tempb}%
159   \protected@edef\bbl@tempc{#2}%
160   \edef\bbl@tempc{\expandafter\strip@prefix\meaning\bbl@tempc}%
161   \ifx\bbl@tempb\bbl@tempc
162     \aftergroup\@firstoftwo
163   \else
164     \aftergroup\@secondoftwo
165   \fi
166 \endgroup}
167 \chardef\bbl@engine=%
168 \ifx\directlua\@undefined
169   \ifx\XeTeXinputencoding\@undefined
170     \z@
171   \else
172     \tw@
173   \fi
174 \else
175   \@ne
176 \fi

```

A somewhat hackish tool (hence its name) to avoid spurious spaces in some contexts.

```

177 \def\bbl@bsphack{%
178   \ifhmode
179     \hskip\z@skip
180     \def\bbl@esphack{\loop\ifdim\lastskip>\z@\unskip\repeat\unskip}%
181   \else
182     \let\bbl@esphack\@empty
183   \fi}

```

Another hackish tool, to apply case changes inside a protected macros. It's based on the internal \let's made by \MakeUppercase and \MakeLowercase between things like \oe and \OE.

```

184 \def\bbl@cased{%
185   \ifx\oe\OE
186     \expandafter\in@\expandafter
187     {\expandafter\OE\expandafter}\expandafter{\oe}%
188     \ifin@
189       \bbl@afterelse\expandafter\MakeUppercase
190     \else
191       \bbl@afterfi\expandafter\MakeLowercase
192     \fi
193   \else
194     \expandafter\@firstofone
195   \fi}

```

The following adds some code to \extras... both before and after, while avoiding doing it twice. It's somewhat convoluted, to deal with #'s. Used to deal with alph, Alph and frenchspacing when there are already changes (with \babel@save).

```

196 \def\bbl@extras@wrap#1#2#3{% 1:in-test, 2:before, 3:after
197   \toks@\expandafter\expandafter\expandafter{%
198     \csname extras\language\endcsname}%
199   \bbl@exp{\in{#1}{\the\toks@}}%
200   \ifin@else
201     \@temptokena{#2}%
202     \edef\bbl@tempc{\the\@temptokena\the\toks@}%
203     \toks@\expandafter{\bbl@tempc#3}%
204     \expandafter\edef\csname extras\language\endcsname{\the\toks@}%
205   \fi}
206 <(/Basic macros)>

```

Some files identify themselves with a $\LaTeX$ macro. The following code is placed before them to define (and then undefine) if not in $\LaTeX$.

```

207 <(*Make sure ProvidesFile is defined)>  ≡

```



```

208 \ifx\ProvidesFile\@undefined
209 \def\ProvidesFile#1[#2 #3 #4]{%
210 \wlog{File: #1 #4 #3 <#2>}%
211 \let\ProvidesFile\@undefined}
212 \fi
213 <(/Make sure ProvidesFile is defined)>

```

3.1 A few core definitions

\language Just for compatibility, for not to touch `hyphen.cfg`.

```

214 <(*Define core switching macros)> ≡
215 \ifx\language\@undefined
216 \csname newcount\endcsname\language
217 \fi
218 <(/Define core switching macros)>

```

\last@language Another counter is used to keep track of the allocated languages. $\text{T}_{\text{E}}\text{X}$ and $\text{L}_{\text{A}}\text{T}_{\text{E}}\text{X}$ reserves for this purpose the count 19.

\addlanguage This macro was introduced for $\text{T}_{\text{E}}\text{X} < 2$. Preserved for compatibility.

```

219 <(*Define core switching macros)> ≡
220 \countdef\last@language=19
221 \def\addlanguage{\csname newlanguage\endcsname}
222 <(/Define core switching macros)>

```

Now we make sure all required files are loaded. When the command `\AtBeginDocument` doesn't exist we assume that we are dealing with a plain-based format. In that case the file `plain.def` is needed (which also defines `\AtBeginDocument`, and therefore it is not loaded twice). We need the first part when the format is created, and `\orig@dump` is used as a flag. Otherwise, we need to use the second part, so `\orig@dump` is not defined (`plain.def` undefines it).

Check if the current version of `switch.def` has been previously loaded (mainly, `hyphen.cfg`). If not, load it now. We cannot load `babel.def` here because we first need to declare and process the package options.

3.2 $\text{L}_{\text{A}}\text{T}_{\text{E}}\text{X}$: `babel.sty` (start)

Here starts the style file for $\text{L}_{\text{A}}\text{T}_{\text{E}}\text{X}$. It also takes care of a number of compatibility issues with other packages.

```

223 <(*package)>
224 \NeedsTeXFormat{LaTeX2e}
225 \ProvidesPackage{babel}%
226 [<@date@> v<@version@>]
227 The multilingual framework for LuaLaTeX, pdfLaTeX and XeLaTeX]

```

Start with some “private” debugging tools, and then define macros for errors. The global lua ‘space’ Babel is declared here, too (inside the test for debug).

```

228 \@ifpackagewith{babel}{debug}
229 {\providecommand\bbl@trace[1]{\message{^^J[ #1 ]}}%
230 \let\bbl@debug\@firstofone
231 \ifx\directlua\@undefined\else
232 \directlua{
233 Babel = Babel or {}
234 Babel.debug = true }%
235 \input{babel-debug.tex}%
236 \fi}
237 {\providecommand\bbl@trace[1]{}%
238 \let\bbl@debug\@gobble
239 \ifx\directlua\@undefined\else
240 \directlua{
241 Babel = Babel or {}
242 Babel.debug = false }%
243 \fi}

```

Macros to deal with errors, warnings, etc. Errors are stored in a separate file.

```

244 \def\bbl@error#1{% Implicit #2#3#4
245   \begingroup
246     \catcode`\=0 \catcode`\==12 \catcode`\`=12
247     \input errbabel.def
248   \endgroup
249   \bbl@error{#1}}
250 \def\bbl@warning#1{%
251   \begingroup
252     \def\{\MessageBreak}%
253     \PackageWarning{babel}{#1}%
254   \endgroup}
255 \def\bbl@infowarn#1{%
256   \begingroup
257     \def\{\MessageBreak}%
258     \PackageNote{babel}{#1}%
259   \endgroup}
260 \def\bbl@info#1{%
261   \begingroup
262     \def\{\MessageBreak}%
263     \PackageInfo{babel}{#1}%
264   \endgroup}

```

Many of the following options don't do anything themselves, they are just defined in order to make it possible for babel and language definition files to check if one of them was specified by the user.

But first, include here the *Basic macros* defined above.

```

265 <@Basic macros@>
266 \ifpackagewith{babel}{silent}
267   {\let\bbl@info\@gobble
268    \let\bbl@infowarn\@gobble
269    \let\bbl@warning\@gobble}
270   {}
271 %
272 \def\AfterBabelLanguage#1{%
273   \global\expandafter\bbl@add\csname#1. ldf-h@k\endcsname}%

```

If the format created a list of loaded languages (in \bbl@languages), get the name of the 0-th to show the actual language used. Also available with base, because it just shows info.

```

274 \ifx\bbl@languages\undefined\else
275   \begingroup
276     \catcode`\^^I=12
277     \@ifpackagewith{babel}{showlanguages}{%
278       \begingroup
279         \def\bbl@elt#1#2#3#4{\wlog{#2^^I#1^^I#3^^I#4}}%
280         \wlog{<*languages>}%
281         \bbl@languages
282         \wlog{</languages>}%
283       \endgroup{}}
284   \endgroup
285   \def\bbl@elt#1#2#3#4{%
286     \ifnum#2=\z@
287       \gdef\bbl@nulllanguage{#1}%
288       \def\bbl@elt##1##2##3##4{%
289         \fi}%
290   \bbl@languages
291 \fi

```

3.3 base

The first 'real' option to be processed is base, which set the hyphenation patterns then resets ver@babel.sty so that L^AT_EX forgets about the first loading. After a subset of babel.def has been loaded (the old switch.def) and \AfterBabelLanguage defined, it exits.

Now the base option. With it we can define (and load, with luatex) hyphenation patterns, even if we are not interested in the rest of babel.

```

292 \bbl@trace{Defining option 'base'}
293 \@ifpackagewith{babel}{base}{%
294   \let\bbl@onlyswitch\@empty
295   \let\bbl@provide@locale\relax
296   \input babel.def
297   \let\bbl@onlyswitch\@undefined
298   \ifx\directlua\@undefined
299     \DeclareOption*{\bbl@patterns{\CurrentOption}}%
300   \else
301     \input luababel.def
302     \DeclareOption*{\bbl@patterns@lua{\CurrentOption}}%
303   \fi
304   \DeclareOption{base}{}%
305   \DeclareOption{showlanguages}{}%
306   \ProcessOptions
307   \global\expandafter\let\csname opt@babel.sty\endcsname\relax
308   \global\expandafter\let\csname ver@babel.sty\endcsname\relax
309   \global\let\@ifl@ter@\@ifl@ter
310   \def\@ifl@ter#1#2#3#4#5{\global\let\@ifl@ter\@ifl@ter@}%
311   \endinput}{}%

```

3.4 key=value options and other general option

The following macros extract language modifiers, and only real package options are kept in the option list. Modifiers are saved and assigned to \BabelModifiers at \bbl@load@language; when no modifiers have been given, the former is \relax.

```

312 \bbl@trace{key=value and another general options}
313 \bbl@csarg\let{tempa\expandafter}\csname opt@babel.sty\endcsname
314 \def\bbl@tempb#1.#2{% Removes trailing dot
315   #1\ifx\@empty#2\else,\bbl@afterfi\bbl@tempb#2\fi}%
316 \def\bbl@tempe#1=#2\@@{%
317   \bbl@csarg\edef{mod@#1}{\bbl@tempb#2}}
318 \def\bbl@tempd#1.#2\@nnil{%
319   \ifx\@empty#2%
320     \edef\bbl@tempc{\ifx\bbl@tempc\@empty\else\bbl@tempc,\fi#1}%
321   \else
322     \in@{,provide=}{, #1}%
323     \ifin@
324       \edef\bbl@tempc{%
325         \ifx\bbl@tempc\@empty\else\bbl@tempc,\fi#1.\bbl@tempb#2}%
326     \else
327       \in@{ $modifiers$ }{ $ #1 $}%
328       \ifin@
329         \bbl@tempe#2\@@
330       \else
331         \in@{ = }{ #1}%
332         \ifin@
333           \edef\bbl@tempc{\ifx\bbl@tempc\@empty\else\bbl@tempc,\fi#1.#2}%
334         \else
335           \edef\bbl@tempc{\ifx\bbl@tempc\@empty\else\bbl@tempc,\fi#1}%
336           \bbl@csarg\edef{mod@#1}{\bbl@tempb#2}%
337         \fi
338       \fi
339     \fi
340   \fi}
341 \let\bbl@tempc\@empty
342 \bbl@foreach\bbl@tempa{\bbl@tempd#1.\@empty\@nnil}
343 \expandafter\let\csname opt@babel.sty\endcsname\bbl@tempc

```

The next option tells babel to leave shorthand characters active at the end of processing the package. This is *not* the default as it can cause problems with other packages, but for those who want

to use the shorthand characters in the preamble of their documents this can help.

```

344 \DeclareOption{KeepShorthandsActive}{}
345 \DeclareOption{activeacute}{}
346 \DeclareOption{activegrave}{}
347 \DeclareOption{debug}{}
348 \DeclareOption{debug-german}{} % Temporary
349 \DeclareOption{noconfigs}{}
350 \DeclareOption{showlanguages}{}
351 \DeclareOption{silent}{}
352 \DeclareOption{shorthands=off}{\bbl@tempa shorthands=\bbl@tempa}
353 \chardef\bbl@iniflag\z@
354 \DeclareOption{provide=*}{\chardef\bbl@iniflag@ne} % main = 1
355 \DeclareOption{provide+=*}{\chardef\bbl@iniflag\tw@} % second = 2
356 \DeclareOption{provide*=*}{\chardef\bbl@iniflag\thr@@} % second + main
357 \chardef\bbl@ldfflag\z@
358 \DeclareOption{provide=!}{\chardef\bbl@ldfflag@ne} % main = 1
359 \DeclareOption{provide+=!}{\chardef\bbl@ldfflag\tw@} % second = 2
360 \DeclareOption{provide*=!}{\chardef\bbl@ldfflag\thr@@} % second + main
361 \DeclareOption{licr=extended}{}
362 \DeclareOption{licr=unextended}{}
363 % Don't use. Experimental.
364 \newif\ifbbl@single
365 \DeclareOption{selectors=off}{\bbl@singletrue}
366 <@More package options@>

```

Handling of package options is done in three passes. (I [JBL] am not very happy with the idea, anyway.) The first one processes options which has been declared above or follow the syntax $\langle key \rangle = \langle value \rangle$, the second one loads the requested languages, except the main one if set with the key `main`, and the third one loads the latter. First, we “flag” valid keys with a nil value.

```

367 \let\bbl@opt@shorthands\@nnil
368 \let\bbl@opt@config\@nnil
369 \let\bbl@opt@main\@nnil
370 \let\bbl@opt@headfoot\@nnil
371 \let\bbl@opt@layout\@nnil
372 \let\bbl@opt@provide\@nnil

```

The following tool is defined temporarily to store the values of options.

```

373 \def\bbl@tempa#1=#2\bbl@tempa{%
374   \bbl@csarg\ifx{opt@#1}\@nnil
375   \bbl@csarg\edef{opt@#1}{#2}%
376   \else
377   \bbl@error{bad-package-option}{#1}{#2}{}%
378   \fi}

```

Now the option list is processed, taking into account only currently declared options (including those declared with a `=`), and $\langle key \rangle = \langle value \rangle$ options (the former take precedence). Unrecognized options are saved in `\bbl@language@opts`, because they are language options.

```

379 \let\bbl@language@opts\@empty
380 \DeclareOption*{%
381   \bbl@xin@{\string=}{\CurrentOption}%
382   \ifin@
383   \expandafter\bbl@tempa\CurrentOption\bbl@tempa
384   \else
385   \bbl@add@list\bbl@language@opts{\CurrentOption}%
386   \fi}

```

Now we finish the first pass (and start over).

```

387 \ProcessOptions*

```

3.5 Post-process some options

```

388 \ifx\bbl@opt@provide\@nnil
389   \let\bbl@opt@provide\@empty % %%% MOVE above
390 \else

```

```

391 \chardef\bbl@iniflag\@ne
392 \bbl@exp{\bbl@forkv{\@nameuse{@raw@opt@babel.sty}}}{%
393   \in@{,provide,},{, #1,}%
394   \ifin@
395     \def\bbl@opt@provide{#2}%
396   \fi}
397 \fi

```

If there is no `shorthands=<chars>`, the original babel macros are left untouched, but if there is, these macros are wrapped (in `babel.def`) to define only those given.

A bit of optimization: if there is no `shorthands=`, then `\bbl@ifshorthand` is always true, and it is always false if `shorthands` is empty. Also, some code makes sense only with `shorthands=...`

```

398 \bbl@trace{Conditional loading of shorthands}
399 \def\bbl@sh@string#1{%
400   \ifx#1@empty\else
401     \ifx#1t\string~%
402     \else\ifx#1c\string,%
403     \else\string#1%
404     \fi\fi
405     \expandafter\bbl@sh@string
406   \fi}
407 \ifx\bbl@opt@shorthands\@nnil
408   \def\bbl@ifshorthand#1#2#3{#2}%
409 \else\ifx\bbl@opt@shorthands@empty
410   \def\bbl@ifshorthand#1#2#3{#3}%
411 \else

```

The following macro tests if a shorthand is one of the allowed ones.

```

412 \def\bbl@ifshorthand#1{%
413   \bbl@xin@{\string#1}{\bbl@opt@shorthands}%
414   \ifin@
415     \expandafter\@firstoftwo
416   \else
417     \expandafter\@secondoftwo
418   \fi}

```

We make sure all chars in the string are ‘other’, with the help of an auxiliary macro defined above (which also zaps spaces).

```

419 \edef\bbl@opt@shorthands{%
420   \expandafter\bbl@sh@string\bbl@opt@shorthands@empty}%

```

The following is ignored with `shorthands=off`, since it is intended to take some additional actions for certain chars.

```

421 \bbl@ifshorthand{'}%
422   {\PassOptionsToPackage{activeacute}{babel}}{}
423 \bbl@ifshorthand{`}%
424   {\PassOptionsToPackage{activegrave}{babel}}{}
425 \fi\fi

```

With `headfoot=lang` we can set the language used in heads/feet. For example, in `babel/3796` just add `headfoot=english`. It misuses `\@resetactivechars`, but seems to work.

```

426 \ifx\bbl@opt@headfoot\@nnil\else
427   \g@addto@macro\@resetactivechars{%
428     \set@typeset@protect
429     \expandafter\select@language@x\expandafter{\bbl@opt@headfoot}%
430     \let\protect\noexpand}
431 \fi

```

For the option `safe` we use a different approach – `\bbl@opt@safe` says which macros are redefined (B for bibs and R for refs). By default, both are currently set, but in a future release it will be set to none.

```

432 \ifx\bbl@opt@safe\@undefined
433   \def\bbl@opt@safe{BR}
434   % \let\bbl@opt@safe@empty % Pending of \cite
435 \fi

```

For layout an auxiliary macro is provided, available for packages and language styles.
Optimization: if there is no layout, just do nothing.

```

436 \bbl@trace{Defining IfBabelLayout}
437 \ifx\bbl@opt@layout\@nnil
438   \newcommand\IfBabelLayout[3]{#3}%
439 \else
440   \bbl@exp{\bbl@forkv{\@nameuse{@raw@opt@babel.sty}}}{%
441     \in@{, layout,}{, #1,}%
442     \ifin@
443       \def\bbl@opt@layout{#2}%
444       \bbl@replace\bbl@opt@layout{ }{.}%
445       \fi}
446 \newcommand\IfBabelLayout[1]{%
447   \@expandtwoargs\in@{.#1.}{.\bbl@opt@layout.}%
448   \ifin@
449     \expandafter\@firstoftwo
450   \else
451     \expandafter\@secondoftwo
452   \fi}
453 \fi
454 (/package)

```

3.6 Plain: babel.def (start)

Because of the way docstrip works, we need to insert some code for Plain here. However, the tools provided by the babel installer for literate programming makes this section a short interlude, because the actual code is below, tagged as *Emulate LaTeX*.

First, exit immediately if previously loaded.

```

455 (*core)
456 \ifx\ldf@quit\@undefined\else
457 \endinput\fi % Same line!
458 <@Make sure ProvidesFile is defined@>
459 \ProvidesFile{babel.def}[<@date@> v<@version@> Babel common definitions]
460 \ifx\AtBeginDocument\@undefined
461   <@Emulate LaTeX@>
462 \fi
463 <@Basic macros@>
464 (/core)

```

That is all for the moment. Now follows some common stuff, for both Plain and $\LaTeX$. After it, we will resume the $\LaTeX$ -only stuff.

4 babel.sty and babel.def (common)

```

465 (*package|core)
466 \def\bbl@version{<@version@>}
467 \def\bbl@date{<@date@>}
468 <@Define core switching macros@>

```

\adddialect The macro `\adddialect` can be used to add the name of a dialect or variant language, for which an already defined hyphenation table can be used.

```

469 \def\adddialect#1#2{%
470   \global\chardef#1#2\relax
471   \bbl@usehooks{adddialect}{#1}{#2}%
472   \begingroup
473     \count@#1\relax
474     \def\bbl@elt##1##2##3##4{%
475       \ifnum\count@=#2\relax
476         \edef\bbl@tempa{\expandafter\@gobbletwo\string#1}%
477         \bbl@info{Hyphen rules for '\expandafter\@gobble\bbl@tempa'
478           set to \expandafter\string\csname l@##1\endcsname\\%
479           (\string\language\the\count@). Reported}%

```

```

480     \def\bbl@elt####1####2####3####4{%
481     \fi}%
482     \bbl@cs{languages}%
483 \endgroup}

```

\bbl@iflanguage executes code only if the language l@ exists. Otherwise raises an error.

The argument of \bbl@fixname has to be a macro name, as it may get “fixed” if casing (lc/uc) is wrong. It’s an attempt to fix a long-standing bug when \foreignlanguage and the like appear in a \MakeXXXcase. However, a lowercase form is not imposed to improve backward compatibility (perhaps you defined a language named MYLANG, but unfortunately mixed case names cannot be trapped). Note l@ is encapsulated, so that its case does not change.

```

484 \def\bbl@fixname#1{%
485   \begingroup
486   \def\bbl@tempe{l}%
487   \edef\bbl@tempd{\noexpand\@ifundefined{\noexpand\bbl@tempe#1}}%
488   \bbl@tempd
489     {\lowercase\expandafter{\bbl@tempd}%
490     {\uppercase\expandafter{\bbl@tempd}%
491     \@empty
492     {\edef\bbl@tempd{\def\noexpand#1{#1}}%
493     \uppercase\expandafter{\bbl@tempd}}}%
494     {\edef\bbl@tempd{\def\noexpand#1{#1}}%
495     \lowercase\expandafter{\bbl@tempd}}}%
496   \@empty
497   \edef\bbl@tempd{\endgroup\def\noexpand#1{#1}}%
498   \bbl@tempd
499   \bbl@exp{\bbl@usehooks{language}{\language}{#1}}}
500 \def\bbl@iflanguage#1{%
501   \@ifundefined{l#1}{\@nolanerr{#1}\@gobble}\@firstofone}

```

After a name has been ‘fixed’, the selectors will try to load the language. If even the fixed name is not defined, will load it on the fly, either based on its name, or if activated, its BCP 47 code.

We first need a couple of macros for a simple BCP 47 look up. It also makes sure, casing is the usual one, so that sr-latn-ba becomes fr-Latn-BA.

\bbl@bcplookup returns \bbl@bcp with either the found ini tag or \relax. Temporary version, to be refactored with the new \text_bcp_parse:n, which is not currently fully expandable with invalid BCP 47 tags. The argument may be is some cases either a language name or a tag (e.g., \foreignlanguage. This was a huge mistake, because of the potential ambiguities (for example, the name vai-latin is also a syntactically valid tag).

```

502 \def\bbl@cntchars#1{\bbl@cntchars@i#1876543210\@@}
503 \def\bbl@cntchars@i#1#2#3#4#5#6#7#8#9\@@{\@car#9\@nil}%
504 \def\bbl@bcplookup#1{%
505   \let\bbl@templ\@empty % language
506   \let\bbl@temps\@empty % script
507   \let\bbl@tempr\@empty % regions
508   \let\bbl@tempv\@empty % variant
509   \edef\bbl@tempa{#1}%
510   \bbl@replace\bbl@tempa{-}{,}%
511   \bbl@replace\bbl@tempa_{,}%
512   \bbl@foreach\bbl@tempa{%
513     \ifx\bbl@templ\@empty
514       \lowercase{\def\bbl@templ{##1}}%
515     \else\ifnum\bbl@cntchars{##1}=4
516       \bbl@xin{\@car##1\@nil}{0123456789}%
517       \ifin@
518         \def\bbl@tempv{##1}% eg, 1901
519       \else
520         \uppercase{\edef\bbl@tempb{\@car##1\@nil}}%
521         \lowercase{\edef\bbl@tempb{\bbl@tempb\@gobble##1}}%
522       \fi
523     \else\ifnum\bbl@cntchars{##1}<4
524       \uppercase{\def\bbl@tempr{##1}}%
525     \else\ifnum\bbl@cntchars{##1}>4
526       \lowercase{\def\bbl@tempv{##1}}%

```

```

527 \fi\fi\fi\fi}%
528 \bbl@exp{\bbl@bcplookup@i{\bbl@templ}{\bbl@temps}{\bbl@tempr}{\bbl@tempv}}
529 \def\bbl@bcplookup@try#1{%
530 \ifx\bbl@bcp\relax
531 \IfFileExists{babel-#1\bbl@tempe.ini}{\edef\bbl@bcp{#1\bbl@tempe}}{}%
532 \fi}
533 \def\bbl@bcplookup@i#1#2#3#4{%
534 \let\bbl@bcp\relax
535 \def\bbl@tempe{#4}%
536 \ifx\bbl@tempe\@empty\else
537 \edef\bbl@tempe{-#4}%
538 \fi
539 \edef\bbl@tempa{#2#3\bbl@tempe}% en
540 \ifx\bbl@tempa\@empty
541 \bbl@bcplookup@try{#1}%
542 \else
543 \edef\bbl@tempa{#3\bbl@tempe}% en-Latn
544 \ifx\bbl@tempa\@empty
545 \bbl@bcplookup@try{#1-#2}%
546 \bbl@bcplookup@try{#1}%
547 \else
548 \edef\bbl@tempa{#2\bbl@tempe}% en-US, en-001
549 \ifx\bbl@tempa\@empty
550 \bbl@bcplookup@try{#1-#3}%
551 \bbl@bcplookup@try{#1}%
552 \else % en-Latn-US
553 \bbl@bcplookup@try{#1-#2-#3}%
554 \bbl@bcplookup@try{#1-#2}%
555 \bbl@bcplookup@try{#1-#3}%
556 \bbl@bcplookup@try{#1}%
557 \fi
558 \fi
559 \fi
560 \ifx\bbl@bcp\relax
561 \ifx\bbl@tempe\@empty\else\bbl@bcplookup@i{#1}{#2}{#3}{\fi}%
562 \fi}
563 %
564 \let\bbl@initoload\relax

```

\iflanguage Users might want to test (in a private package for instance) which language is currently active. For this we provide a test macro, `\iflanguage`, that has three arguments. It checks whether the first argument is a known language. If so, it compares the first argument with the value of `\language`. Then, depending on the result of the comparison, it executes either the second or the third argument.

```

565 \def\iflanguage#1{%
566 \bbl@iflanguage{#1}{%
567 \ifnum\csname l@#1\endcsname=\language
568 \expandafter\@firstoftwo
569 \else
570 \expandafter\@secondoftwo
571 \fi}}

```

4.1 Selecting the language

\selectlanguage It checks whether the language is already defined before it performs its actual task, which is to update `\language` and activate language-specific definitions.

```

572 \let\bbl@select@type\z@
573 \edef\selectlanguage{%
574 \noexpand\protect
575 \expandafter\noexpand\csname selectlanguage \endcsname}

```

Because the command `\selectlanguage` could be used in a moving argument it expands to `\protect\selectlanguageL`. Therefore, we have to make sure that a macro `\protect` exists. If it

doesn't it is \let to \relax.

```
576 \ifx\@undefined\protect\let\protect\relax\fi
```

The following definition is preserved for backwards compatibility (e.g., arabi, koma). It is related to a trick for 2.09, now discarded.

```
577 \let\xstring\string
```

Since version 3.5 babel writes entries to the auxiliary files in order to typeset table of contents etc. in the correct language environment.

\bbl@pop@language But when the language change happens *inside* a group the end of the group doesn't write anything to the auxiliary files. Therefore we need TeX's *aftergroup* mechanism to help us. The command \aftergroup stores the token immediately following it to be executed when the current group is closed. So we define a temporary control sequence \bbl@pop@language to be executed at the end of the group. It calls \bbl@set@language with the name of the current language as its argument.

\bbl@language@stack The previous solution works for one level of nesting groups, but as soon as more levels are used it is no longer adequate. For that case we need to keep track of the nested languages using a stack mechanism. This stack is called \bbl@language@stack and initially empty.

```
578 \def\bbl@language@stack{}
```

When using a stack we need a mechanism to push an element on the stack and to retrieve the information afterwards.

\bbl@push@language

\bbl@pop@language The stack is simply a list of languagenames, separated with a '+' sign; the push function can be simple:

```
579 \def\bbl@push@language{%
580   \ifx\language\@undefined\else
581     \ifx\currentgrouplevel\@undefined
582       \xdef\bbl@language@stack{\language+\bbl@language@stack}%
583     \else
584       \ifnum\currentgrouplevel=\z@
585         \xdef\bbl@language@stack{\language+}%
586       \else
587         \xdef\bbl@language@stack{\language+\bbl@language@stack}%
588       \fi
589     \fi
590 }
```

Retrieving information from the stack is a little bit less simple, as we need to remove the element from the stack while storing it in the macro \language. For this we first define a helper function.

\bbl@pop@lang This macro stores its first element (which is delimited by the '+'-sign) in \language and stores the rest of the string in \bbl@language@stack.

```
591 \def\bbl@pop@lang#1+#2\@@{%
592   \edef\language{#1}%
593   \xdef\bbl@language@stack{#2}}
```

The reason for the somewhat weird arrangement of arguments to the helper function is the fact it is called in the following way. This means that before \bbl@pop@lang is executed TeX first *expands* the stack, stored in \bbl@language@stack. The result of that is that the argument string of \bbl@pop@lang contains one or more language names, each followed by a '+'-sign (zero language names won't occur as this macro will only be called after something has been pushed on the stack).

```
594 \let\bbl@ifrestoring\@secondoftwo
595 \def\bbl@pop@language{%
596   \expandafter\bbl@pop@lang\bbl@language@stack\@@
597   \let\bbl@ifrestoring\@firstoftwo
598   \expandafter\bbl@set@language\expandafter{\language}%
599   \let\bbl@ifrestoring\@secondoftwo}
```

Once the name of the previous language is retrieved from the stack, it is fed to `\bbl@set@language` to do the actual work of switching everything that needs switching.

An alternative way to identify languages (in the babel sense) with a numerical value is introduced in 3.30. This is one of the first steps for a new interface based on the concept of locale, which explains the name of `\localeid`. This means `\l@. . .` will be reserved for hyphenation patterns (so that two locales can share the same rules).

```

600 \chardef\localeid\z@
601 \gdef\bbl@id@last{0}    % No real need for a new counter
602 \def\bbl@id@assign{%
603   \bbl@ifunset\bbl@id@@\language\name}%
604   {\count@\bbl@id@last\relax
605    \advance\count@\@ne
606    \global\bbl@csarg\chardef{id@@\language\name}\count@
607    \xdef\bbl@id@last{\the\count@}%
608    \ifcase\bbl@engine\or
609      \directlua{
610        Babel.locale_props[\bbl@id@last] = {}
611        Babel.locale_props[\bbl@id@last].name = '\language'
612        Babel.locale_props[\bbl@id@last].vars = {}
613      }%
614    \fi}%
615  }%
616  \chardef\localeid\bbl@cl{id@}

```

The unprotected part of `\selectlanguage`. In case it is used as environment, declare `\endselectlanguage`, just for safety.

```

617 \let\bbl@select@opts@empty
618 \expandafter\def\csname selectlanguage \endcsname{%
619   \ifnextchar\bbl@select@s{\bbl@select@s[]}%
620 \def\bbl@select@s[#1]#2{%
621   \def\bbl@select@opts{#1}%
622   \ifnum\bbl@hymapset=\@ccclv\let\bbl@hymapset\tw@fi
623   \bbl@push@language
624   \aftergroup\bbl@pop@language
625   \bbl@set@language{#2}%
626 \let\endselectlanguage\relax

```

`\bbl@set@language` The macro `\bbl@set@language` takes care of switching the language environment *and* of writing entries on the auxiliary files. For historical reasons, language names can be either language of `\language`. To catch either form a trick is used, but unfortunately as a side effect the catcodes of letters in `\language` are messed up. This is a bug, but preserved for backwards compatibility. The list of auxiliary files can be extended by redefining `\BabelContentsFiles`, but make sure they are loaded inside a group (as `aux`, `toc`, `lof`, and `lot` do) or the last language of the document will remain active afterwards.

We also write a command to change the current language in the auxiliary files.

`\bbl@savelastskip` is used to deal with skips before the write whatsit (as suggested by U Fischer). Adapted from `hyperref`, but it might fail, so I'll consider it a temporary hack, while I study other options (the ideal, but very likely unfeasible except perhaps in `luatex`, is to avoid the `\write` altogether when not needed).

```

627 \def\BabelContentsFiles{toc,lof,lot}
628 \def\bbl@set@language#1{% from selectlanguage, pop@
629   % The old buggy way. Preserved for compatibility, but simplified
630   \edef\language{\expandafter\string#1\@empty}%
631   \select@language{\language}%
632   \bbl@xin@{,main,},{,\bbl@select@opts,}%
633   \ifin@
634     \let\bbl@main@language\localename
635     \let\mainlocalename\localename
636   \fi
637   \let\bbl@select@opts@empty
638   % write to aux files
639   \expandafter\ifx\csname date\language\endcsname\relax\else

```

```

640 \if@filesw
641 \bbl@xin@{,nofiles,}{,\bbl@select@opts,}%
642 \ifin@else
643 \ifx\babel@aux@gobbletwo\else % Set if single in the first, redundant
644 \bbl@savelastskip
645 \protected@write\@auxout{ }\string\babel@aux{\bbl@auxname}{}}%
646 \bbl@restorelastskip
647 \fi
648 \bbl@usehooks{write}{}%
649 \fi
650 \fi
651 \fi}
652 %
653 \let\bbl@restorelastskip\relax
654 \let\bbl@savelastskip\relax
655 %
656 \def\select@language#1{% from set@, babel@aux, babel@toc
657 \ifx\bbl@selectorname\@empty
658 \def\bbl@selectorname{select}%
659 \fi
660 % set hymap
661 \ifnum\bbl@hymapsel=\@cclv\chardef\bbl@hymapsel4\relax\fi
662 % set name (when coming from babel@aux)
663 \edef\language{#1}%
664 \bbl@fixname\language
665 % define \localename when coming from set@, with a trick
666 \ifx\scantokens\@undefined
667 \def\localename{??}%
668 \else
669 \bbl@exp{\scantokens{\def\localename{\language}\noexpand}\relax}%
670 \fi
671 \bbl@provide@locale
672 \bbl@iflanguage\language{%
673 \let\bbl@select@type\z@
674 \expandafter\bbl@switch\expandafter{\language}}
675 \def\babel@aux#1#2{%
676 \select@language{#1}%
677 \bbl@foreach\BabelContentsFiles{% \relax -> don't assume vertical mode
678 \@writefile{##1}{\babel@toc{#1}{#2}\relax}}}%
679 \def\babel@toc#1#2{%
680 \select@language{#1}}

```

First, check if the user asks for a known language. If so, update the value of `\language` and call `\originalTeX` to bring $\TeX$ in a certain pre-defined state.

The name of the language is stored in the control sequence `\language`.

Then we have to *redefine* `\originalTeX` to compensate for the things that have been activated. To save memory space for the macro definition of `\originalTeX`, we construct the control sequence name for the `\noextras<language>` command at definition time by expanding the `\csname` primitive.

Now activate the language-specific definitions. This is done by constructing the names of three macros by concatenating three words with the argument of `\selectlanguage`, and calling these macros.

The switching of the values of `\lefthyphenmin` and `\righthyphenmin` is somewhat different. First we save their current values, then we check if `\<language>hyphenmins` is defined. If it is not, we set default values (2 and 3), otherwise the values in `\<language>hyphenmins` will be used.

No text is supposed to be added with switching captions and date, so we remove any spurious spaces with `\bbl@bsphack` and `\bbl@esphack`.

```

681 \newif\ifbbl@usedategroup
682 \let\bbl@savextras\@empty
683 \def\bbl@switch#1{% from select@, foreign@
684 % restore
685 \originalTeX
686 \expandafter\def\expandafter\originalTeX\expandafter{%
687 \csname noextras#1\endcsname

```

```

688 \let\originalTeX\@empty
689 \babel@beginsave}%
690 \bbl@usehooks{afterreset}{}%
691 \languageshorthands{none}%
692 % set the locale id
693 \bbl@id@assign
694 % switch captions, date
695 \bbl@bsphack
696 \ifcase\bbl@select@type
697 \csname captions#1\endcsname\relax
698 \csname date#1\endcsname\relax
699 \else
700 \bbl@xin@{,captions,}{,\bbl@select@opts,}%
701 \ifin@
702 \csname captions#1\endcsname\relax
703 \fi
704 \bbl@xin@{,date,}{,\bbl@select@opts,}%
705 \ifin@ % if \foreign... within \<language>date
706 \csname date#1\endcsname\relax
707 \fi
708 \fi
709 \bbl@esphack
710 % switch extras
711 \csname bbl@preextras@#1\endcsname
712 \bbl@usehooks{beforeextras}{}%
713 \csname extras#1\endcsname\relax
714 \bbl@usehooks{afterextras}{}%
715 % > babel-ensure
716 % > babel-sh-<short>
717 % > babel-bidi
718 % > babel-fontspec
719 \let\bbl@savextras\@empty
720 % hyphenation - case mapping
721 \ifcase\bbl@opt@hyphenmap\or
722 \def\BabelLower##1##2{\lccode##1=##2\relax}%
723 \ifnum\bbl@hymap>4\else
724 \csname\language @bbl@hyphenmap\endcsname
725 \fi
726 \chardef\bbl@opt@hyphenmap\z@
727 \else
728 \ifnum\bbl@hymap>\bbl@opt@hyphenmap\else
729 \csname\language @bbl@hyphenmap\endcsname
730 \fi
731 \fi
732 \let\bbl@hymap\@cclv
733 % hyphenation - select rules
734 \ifnum\csname l@\language\endcsname=\l@unhyphenated
735 \edef\bbl@tempa{u}%
736 \else
737 \edef\bbl@tempa{\bbl@cl{\lnbrk}}%
738 \fi
739 % linebreaking - handle u, e, k (v in the future)
740 \bbl@xin@{/u}{/\bbl@tempa}%
741 \ifin@\else\bbl@xin@{/e}{/\bbl@tempa}\fi % elongated forms
742 \ifin@\else\bbl@xin@{/k}{/\bbl@tempa}\fi % only kashida
743 \ifin@\else\bbl@xin@{/p}{/\bbl@tempa}\fi % padding (e.g., Tibetan)
744 \ifin@\else\bbl@xin@{/v}{/\bbl@tempa}\fi % variable font
745 % hyphenation - save mins
746 \babel@savevariable\lefthyphenmin
747 \babel@savevariable\righthyphenmin
748 \ifnum\bbl@engine=@ne
749 \babel@savevariable\hyphenationmin
750 \fi

```

```

751 \ifin@
752 % unhyphenated/kashida/elongated/padding = allow stretching
753 \language\l@unhyphenated
754 \babel@savevariable\emergencystretch
755 \emergencystretch\maxdimen
756 \babel@savevariable\hbadness
757 \hbadness\@M
758 \else
759 % other = select patterns
760 \bbl@patterns{#1}%
761 \fi
762 % hyphenation - set mins
763 \expandafter\ifx\csname #1hyphenmins\endcsname\relax
764 \set@hyphenmins\tw@\thr@@\relax
765 \@nameuse{bbl@hyphenmins@}%
766 \else
767 \expandafter\expandafter\expandafter\set@hyphenmins
768 \csname #1hyphenmins\endcsname\relax
769 \fi
770 \@nameuse{bbl@hyphenmins@}%
771 \@nameuse{bbl@hyphenmins@\language\name}%
772 \@nameuse{bbl@hyphenatmin@}%
773 \@nameuse{bbl@hyphenatmin@\language\name}%
774 \let\bbl@selectortname\empty}

```

otherlanguage It can be used as an alternative to using the `\selectlanguage` declarative command. The `\ignorespaces` command is necessary to hide the environment when it is entered in horizontal mode.

```

775 \edef\otherlanguage{%
776 \noexpand\protect
777 \expandafter\noexpand\csname otherlanguage \endcsname}
778 \expandafter\def\csname otherlanguage \endcsname{%
779 \@ifstar{\@nameuse{otherlanguage*}}\bbl@otherlanguage}
780 \def\bbl@otherlanguage#1{%
781 \def\bbl@selectortname{other}%
782 \ifnum\bbl@hymapsel=\@ccclv\let\bbl@hymapsel\thr@@\fi
783 \csname selectlanguage \endcsname{#1}%
784 \ignorespaces}

```

The `\endotherlanguage` part of the environment tries to hide itself when it is called in horizontal mode.

```

785 \long\def\endotherlanguage{\@ignoretrue\ignorespaces}

```

otherlanguage* It is meant to be used when a large part of text from a different language needs to be typeset, but without changing the translation of words such as ‘figure’. It makes use of `\foreign@language`.

```

786 \expandafter\def\csname otherlanguage*\endcsname{%
787 \@ifnextchar[\bbl@otherlanguage@s{\bbl@otherlanguage@s[]}}
788 \def\bbl@otherlanguage@s[#1]#2{%
789 \def\bbl@selectortname{other*}%
790 \ifnum\bbl@hymapsel=\@ccclv\chardef\bbl@hymapsel4\relax\fi
791 \def\bbl@select@opts{#1}%
792 \foreign@language{#2}}

```

At the end of the environment we need to switch off the extra definitions. The grouping mechanism of the environment will take care of resetting the correct hyphenation rules and “extras”.

```

793 \expandafter\let\csname endotherlanguage*\endcsname\relax

```

\foreignlanguage This command takes two arguments, the first argument is the name of the language to use for typesetting the text specified in the second argument.

Unlike `\selectlanguage` this command doesn’t switch *everything*, it only switches the hyphenation rules and the extra definitions for the language specified. It does this within a group and assumes the

`\extras<language>` command doesn't make any `\global` changes. The coding is very similar to part of `\selectlanguage`.

`\bbl@beforeforeign` is a trick to fix a bug in bidi texts. `\foreignlanguage` is supposed to be a 'text' command, and therefore it must emit a `\leavevmode`, but it does not, and therefore the indent is placed on the opposite margin. For backward compatibility, however, it is done only if a right-to-left script is requested; otherwise, it is no-op.

(3.11) `\foreignlanguage*` is a temporary, experimental macro for a few lines with a different script direction, while preserving the paragraph format (thank the braces around `\par`, things like `\hangindent` are not reset). Do not use it in production, because its semantics and its syntax may change (and very likely will, or even it could be removed altogether). Currently it enters in vmode and then selects the language (which in turn sets the paragraph direction).

(3.11) Also experimental are the hook `foreign` and `foreign*`. With them you can redefine `\BabelText` which by default does nothing. Its behavior is not well defined yet. So, use it in horizontal mode only if you do not want surprises.

In other words, at the beginning of a paragraph `\foreignlanguage` enters into hmode with the surrounding lang, and with `\foreignlanguage*` with the new lang.

```

794 \providecommand\bbl@beforeforeign{}
795 \edef\foreignlanguage{%
796   \noexpand\protect
797   \expandafter\noexpand\csname foreignlanguage \endcsname}
798 \expandafter\def\csname foreignlanguage \endcsname{%
799   \@ifstar\bbl@foreign@s\bbl@foreign@x}
800 \providecommand\bbl@foreign@x[3][]{%
801   \begingroup
802     \def\bbl@select@name{foreign}%
803     \def\bbl@select@opts{#1}%
804     \let\BabelText\@firstofone
805     \bbl@beforeforeign
806     \foreign@language{#2}%
807     \bbl@usehooks{foreign}{}%
808     \BabelText{#3}% Now in horizontal mode!
809   \endgroup}
810 \def\bbl@foreign@s#1#2{%
811   \begingroup
812     {\par}%
813     \def\bbl@select@name{foreign*}%
814     \let\bbl@select@opts\@empty
815     \let\BabelText\@firstofone
816     \foreign@language{#1}%
817     \bbl@usehooks{foreign*}{}%
818     \bbl@dirparastext
819     \BabelText{#2}% Still in vertical mode!
820   {\par}%
821   \endgroup}
822 \providecommand\BabelWrapText[1]{%
823   \def\bbl@tempa{\def\BabelText###1}%
824   \expandafter\bbl@tempa\expandafter{\BabelText{#1}}}
```

\foreign@language This macro does the work for `\foreignlanguage` and the `otherlanguage*` environment. First we need to store the name of the language and check that it is a known language. Then it just calls `bbl@switch`.

```

825 \def\foreign@language#1{%
826   % set name
827   \edef\language#1%
828   \ifbbl@usedatgroup
829     \bbl@add\bbl@select@opts{,date,}%
830     \bbl@usedatgroupfalse
831   \fi
832   \bbl@fixname\language
833   \let\localname\language
834   \bbl@provide@locale
835   \bbl@iflanguage\language%
```

```

836 \let\bbl@select@type\@ne
837 \expandafter\bbl@switch\expandafter{\language}\@empty}}

```

The following macro executes conditionally some code based on the selector being used.

```

838 \def\IfBabelSelectorTF#1{%
839 \bbl@xin@{\bbl@select@name,}{,\zap@space#1 \@empty,}%
840 \ifin@
841 \expandafter\@firstoftwo
842 \else
843 \expandafter\@secondoftwo
844 \fi}

```

\bbl@patterns This macro selects the hyphenation patterns by changing the \language register. If special hyphenation patterns are available specifically for the current font encoding, use them instead of the default.

It also sets hyphenation exceptions, but only once, because they are global (here language \lccode's has been set, too). \bbl@hyphenation@ is set to relax until the very first \babelhyphenation, so do nothing with this value. If the exceptions for a language (by its number, not its name, so that :ENC is taken into account) has been set, then use \hyphenation with both global and language exceptions and empty the latter to mark they must not be set again.

```

845 \let\bbl@hyphlist\@empty
846 \let\bbl@hyphenation@\relax
847 \let\bbl@pttnlist\@empty
848 \let\bbl@patterns@\relax
849 \let\bbl@hymapsel=\ccclv
850 \def\bbl@patterns#1{%
851 \language=\expandafter\ifx\csname l@#1:\f@encoding\endcsname\relax
852 \csname l@#1\endcsname
853 \edef\bbl@tempa{#1}%
854 \else
855 \csname l@#1:\f@encoding\endcsname
856 \edef\bbl@tempa{#1:\f@encoding}%
857 \fi
858 \@expandtwoargs\bbl@usehooks{patterns}{#1}{\bbl@tempa}}%
859 % > luatex
860 \@ifundefined{bbl@hyphenation@}{% Can be \relax!
861 \begingroup
862 \bbl@xin@{\number\language,}{,\bbl@hyphlist}%
863 \ifin@\else
864 \@expandtwoargs\bbl@usehooks{hyphenation}{#1}{\bbl@tempa}}%
865 \hyphenation{%
866 \bbl@hyphenation@
867 \@ifundefined{bbl@hyphenation@#1}%
868 \@empty
869 {\space\csname bbl@hyphenation@#1\endcsname}}%
870 \xdef\bbl@hyphlist{\bbl@hyphlist\number\language,}%
871 \fi
872 \endgroup}}

```

hyphenrules It can be used to select *just* the hyphenation rules. It does *not* change \language and when the hyphenation rules specified were not loaded it has no effect. Note however, \lccode's and font encodings are not set at all, so in most cases you should use otherlanguage*.

```

873 \def\hyphenrules#1{%
874 \edef\bbl@tempf{#1}%
875 \bbl@fixname\bbl@tempf
876 \bbl@iflanguage\bbl@tempf{%
877 \expandafter\bbl@patterns\expandafter{\bbl@tempf}%
878 \ifx\languageshorthands\@undefined\else
879 \languageshorthands{none}%
880 \fi
881 \expandafter\ifx\csname\bbl@tempf hyphenmins\endcsname\relax
882 \set@hyphenmins\tw@thr@{\relax

```

```

883 \else
884 \expandafter\expandafter\expandafter\set@hyphenmins
885 \csname\bbl@tempf hyphenmins\endcsname\relax
886 \fi}}
887 \let\endhyphenrules\@empty

```

\providehyphenmins The macro `\providehyphenmins` should be used in the language definition files to provide a *default* setting for the hyphenation parameters `\lefthyphenmin` and `\righthyphenmin`. If the macro `\<language>hyphenmins` is already defined this command has no effect.

```

888 \def\providehyphenmins#1#2{%
889 \expandafter\ifx\csname #1hyphenmins\endcsname\relax
890 \@namedef{#1hyphenmins}{#2}%
891 \fi}

```

\set@hyphenmins This macro sets the values of `\lefthyphenmin` and `\righthyphenmin`. It expects two values as its argument.

```

892 \def\set@hyphenmins#1#2{%
893 \lefthyphenmin#1\relax
894 \righthyphenmin#2\relax}

```

\ProvidesLanguage The identification code for each file is something that was introduced in $\text{\TeX 2.}\epsilon$. When the command `\ProvidesFile` does not exist, a dummy definition is provided temporarily. For use in the language definition file the command `\ProvidesLanguage` is defined by babel.

Depending on the format, i.e., or if the former is defined, we use a similar definition or not.

```

895 \ifx\ProvidesFile\@undefined
896 \def\ProvidesLanguage#1[#2 #3 #4]{%
897 \wlog{Language: #1 #4 #3 <#2>}%
898 }
899 \else
900 \def\ProvidesLanguage#1{%
901 \begingroup
902 \catcode`\ 10 %
903 \@makeother\/%
904 \@ifnextchar[%]
905 {\@provideslanguage{#1}}{\@provideslanguage{#1}[]}}
906 \def\@provideslanguage#1[#2]{%
907 \wlog{Language: #1 #2}%
908 \expandafter\xdef\csname ver@#1.ldf\endcsname{#2}%
909 \endgroup}
910 \fi

```

\originalTeX The macro `\originalTeX` should be known to $\text{\TeX}$ at this moment. As it has to be expandable we `\let` it to `\@empty` instead of `\relax`.

```

911 \ifx\originalTeX\@undefined\let\originalTeX\@empty\fi

```

Because this part of the code can be included in a format, we make sure that the macro which initializes the save mechanism, `\babel@beginsave`, is not considered to be undefined.

```

912 \ifx\babel@beginsave\@undefined\let\babel@beginsave\relax\fi

```

A few macro names are reserved for future releases of babel, which will use the concept of ‘locale’:

```

913 \providecommand\setlocale{\bbl@error{not-yet-available}}{}{}{}
914 \let\uselocale\setlocale
915 \let\locale\setlocale
916 \let\selectlocale\setlocale
917 \let\textlocale\setlocale
918 \let\textlanguage\setlocale
919 \let\languagegettext\setlocale

```

4.2 Errors

\@nolanerr

\@nopatterns The babel package will signal an error when a documents tries to select a language that hasn't been defined earlier. When a user selects a language for which no hyphenation patterns were loaded into the format he will be given a warning about that fact. We revert to the patterns for `\language=0` in that case. In most formats that will be (US)english, but it might also be empty.

\@noopterr When the package was loaded without options not everything will work as expected. An error message is issued in that case.

When the format knows about `\PackageError` it must be $\text{\LaTeX 2}_{\epsilon}$, so we can safely use its error handling interface. Otherwise we'll have to 'keep it simple'.

Infos are not written to the console, but on the other hand many people think warnings are errors, so a further message type is defined: an important info which is sent to the console.

```

920 \edef\bbl@nulllanguage{\string\language=0}
921 \def\bbl@nocaption#1#2{\NoCaseChange{\protect\bbl@nocaption@i{#1}{#2}}}
922 \def\bbl@nocaption@i#1#2{% 1: text to be printed 2: caption \langXname
923   \global\@namedef{#2}{\textbf{?#1?}}}%
924   \@nameuse{#2}%
925   \edef\bbl@tempa{#1}%
926   \bbl@replace\bbl@tempa{name}{}}%
927   \bbl@warning{%
928     \@backslashchar#1 not set for '\language'. Please,\\%
929     define it after the language has been loaded\\%
930     (typically in the preamble) with:\\%
931     \string\setlocalecaption{\language}{\bbl@tempa}{..}\\%
932     Feel free to contribute on github.com/latex3/babel.\\%
933     Reported}}
934 \def\bbl@tentative{\protect\bbl@tentative@i}
935 \def\bbl@tentative@i#1{%
936   \bbl@warning{%
937     Some functions for '#1' are tentative.\\%
938     They might not work as expected and their behavior\\%
939     could change in the future.\\%
940     Reported}}
941 \def\@nolanerr#1{\bbl@error{undefined-language}{#1}{}}
942 \def\@nopatterns#1{%
943   \bbl@warning
944     {No hyphenation patterns were preloaded for\\%
945     the language '#1' into the format.\\%
946     Please, configure your TeX system to add them and\\%
947     rebuild the format. Now I will use the patterns\\%
948     preloaded for \bbl@nulllanguage\space instead}}
949 \let\bbl@usehooks\@gobbletwo

Here ended the now discarded switch.def.
Here also (currently) ends the base option.

950 \ifx\bbl@onlyswitch\@empty\endinput\fi

```

4.3 More on selection

\babelensure The user command just parses the optional argument and creates a new macro named `\bbl@e@<language>`. We register a hook at the `afterextras` event which just executes this macro in a “complete” selection (which, if undefined, is `\relax` and does nothing). This part is somewhat involved because we have to make sure things are expanded the correct number of times.

The macro `\bbl@e@<language>` contains `\bbl@ensure{<include>}{<exclude>}{<fontenc>}`, which in turn loops over the macros names in `\bbl@captionslist`, excluding (with the help of `\in@`) those in the exclude list. If the fontenc is given (and not `\relax`), the `\fontencoding` is also added. Then we loop over the include list, but if the macro already contains `\foreignlanguage`, nothing is done. Note this macro (1) is not restricted to the preamble, and (2) changes are local.

```

951 \bbl@trace{Defining babelensure}
952 \newcommand\babelensure[2][{}]{%
953   \AddBabelHook{babel-ensure}{afterextras}{%
954     \ifcase\bbl@select@type
955       \bbl@cl{e}%

```

```

956 \fi}%
957 \begingroup
958 \let\bbbl@ens@include\@empty
959 \let\bbbl@ens@exclude\@empty
960 \def\bbbl@ens@fontenc{\relax}%
961 \def\bbbl@tempb##1{%
962   \ifx\@empty##1\else\noexpand##1\expandafter\bbbl@tempb\fi}%
963 \edef\bbbl@tempa{\bbbl@tempb#1\@empty}%
964 \def\bbbl@tempb##1=##2\@@{\@namedef{\bbbl@ens@##1}{##2}}%
965 \bbbl@foreach\bbbl@tempa{\bbbl@tempb##1\@@}%
966 \def\bbbl@tempc{\bbbl@ensure}%
967 \expandafter\bbbl@add\expandafter\bbbl@tempc\expandafter{%
968   \expandafter\bbbl@ens@include}%
969 \expandafter\bbbl@add\expandafter\bbbl@tempc\expandafter{%
970   \expandafter\bbbl@ens@exclude}%
971 \toks@{\expandafter\bbbl@tempc}%
972 \bbbl@exp{%
973 \endgroup
974 \def\<bbbl@e@#2>{\the\toks@{\bbbl@ens@fontenc}}}%
975 \def\bbbl@ensure#1#2#3{% 1: include 2: exclude 3: fontenc
976 \def\bbbl@tempb##1{% elt for (excluding) \bbbl@captionslist list
977   \ifx##1\@undefined % 3.32 - Don't assume the macro exists
978     \edef##1{\noexpand\bbbl@nocaption
979       {\bbbl@stripslash##1}{\language\bbbl@stripslash##1}}%
980   \fi
981   \ifx##1\@empty\else
982     \in@{##1}{#2}%
983     \ifin@else
984       \let\bbbl@tempc\language\language
985       \bbbl@replace\bbbl@tempc{-}{@}%
986       \bbbl@ifunset{\bbbl@ensure@\bbbl@tempc}%
987       {\bbbl@exp{%
988         \\DeclareRobustCommand\<bbbl@ensure@\bbbl@tempc>[1]{%
989           \\foreignlanguage{\language\language}%
990           {\ifx\relax#3\else
991             \\fontencoding{#3}\\selectfont
992             \fi
993             #####1}}}%
994       }%
995       \toks@{\expandafter{##1}%
996       \edef##1{%
997         \bbbl@csarg\noexpand{ensure@\bbbl@tempc}%
998         {\the\toks@}}}%
999     \fi
1000   \expandafter\bbbl@tempb
1001   \fi}%
1002 \expandafter\bbbl@tempb\bbbl@captionslist\today\@empty
1003 \def\bbbl@tempa##1{% elt for include list
1004   \ifx##1\@empty\else
1005     \let\bbbl@tempc\language\language
1006     \bbbl@replace\bbbl@tempc{-}{@}%
1007     \bbbl@csarg\in@{ensure@\bbbl@tempc\expandafter}\expandafter{##1}%
1008     \ifin@else
1009       \bbbl@tempb##1\@empty
1010     \fi
1011     \expandafter\bbbl@tempa
1012     \fi}%
1013 \bbbl@tempa#1\@empty}
1014 \def\bbbl@captionslist{%
1015 \prefacename\refname\abstractname\bibname\chaptername\appendixname
1016 \contentsname\listfigurename\listtablename\indexname\figurename
1017 \tablename\partname\enclname\ccname\headtoname\pagename\seename
1018 \alsoname\proofname\glossaryname}

```

4.4 Short tags

\babeltags This macro is straightforward. After zapping spaces, we loop over the list and define the macros `\text<tag>` and `\<tag>`. Definitions are first expanded so that they don't contain `\csname` but the actual macro.

```
1019 \bbl@trace{Short tags}
1020 \newcommand\babeltags[1]{%
1021   \edef\bbl@tempa{\zap@space#1 \@empty}%
1022   \def\bbl@tempb##1=##2\@{;%
1023     \edef\bbl@tempc{%
1024       \noexpand\newcommand
1025       \expandafter\noexpand\csname ##1\endcsname{%
1026         \noexpand\protect
1027         \expandafter\noexpand\csname otherlanguage*\endcsname{##2}}
1028       \noexpand\newcommand
1029       \expandafter\noexpand\csname text##1\endcsname{%
1030         \noexpand\foreignlanguage{##2}}
1031     \bbl@tempc}%
1032   \bbl@for\bbl@tempa\bbl@tempa{%
1033     \expandafter\bbl@tempb\bbl@tempa\@{}}
```

4.5 Compatibility with language.def

Plain e-TeX doesn't rely on language.dat, but babel can be made compatible with this format easily.

```
1034 \bbl@trace{Compatibility with language.def}
1035 \ifx\directlua\@undefined\else
1036   \ifx\bbl@luapatterns\@undefined
1037     \input luababel.def
1038   \fi
1039 \fi
1040 \ifx\bbl@languages\@undefined
1041   \ifx\directlua\@undefined
1042     \openin0 = language.def
1043     \ifeof0
1044       \closein0
1045       \message{I couldn't find the file language.def}
1046     \else
1047       \closein0
1048       \begingroup
1049         \def\addlanguage#1#2#3#4#5{%
1050           \expandafter\ifx\csname lang@#1\endcsname\relax\else
1051             \global\expandafter\let\csname l@#1\expandafter\endcsname
1052             \csname lang@#1\endcsname
1053           \fi}%
1054         \def\uselanguage#1{%
1055           \input language.def
1056         \endgroup
1057       \fi
1058     \fi
1059   \chardef\l@english\z@
1060 \fi
```

\addto It takes two arguments, a *<control sequence>* and TeX-code to be added to the *<control sequence>*.

If the *<control sequence>* has not been defined before it is defined now. The control sequence could also expand to `\relax`, in which case a circular definition results. The net result is a stack overflow. Note there is an inconsistency, because the assignment in the last branch is global.

```
1061 \def\addto#1#2{%
1062   \ifx#1\@undefined
1063     \def#1{#2}%
1064   \else
1065     \ifx#1\relax
```

```

1066     \def#1{#2}%
1067     \else
1068     {\toks@ \expandafter{#1#2}%
1069     \xdef#1{\the\toks@}}%
1070     \fi
1071 \fi}

```

4.6 Hooks

Admittedly, the current implementation is a somewhat simplistic and does very little to catch errors, but it is meant for developers, after all. `\bbl@usehooks` is the commands used by babel to execute hooks defined for an event.

```

1072 \bbl@trace{Hooks}
1073 \newcommand\AddBabelHook[3][]{%
1074   \bbl@i funset{\bbl@hk@#2}{\EnableBabelHook{#2}}}%
1075   \def\bbl@tempa##1,##3=##2,##3\@empty{\def\bbl@tempb{##2}}%
1076   \expandafter\bbl@tempa\bbl@evargs,##3=,\@empty
1077   \bbl@i funset{\bbl@ev@#2@#3@#1}%
1078     {\bbl@csarg\bbl@add{ev@#3@#1}{\bbl@elth{#2}}}%
1079     {\bbl@csarg\let{ev@#2@#3@#1}\relax}%
1080   \bbl@csarg\newcommand{ev@#2@#3@#1}[\bbl@tempb]
1081 \newcommand\EnableBabelHook[1]{\bbl@csarg\let{hk@#1}\@firstofone}
1082 \newcommand\DisableBabelHook[1]{\bbl@csarg\let{hk@#1}\@gobble}
1083 \def\bbl@usehooks{\bbl@usehooks@lang\language}
1084 \def\bbl@usehooks@lang#1#2#3{% Test for Plain
1085   \ifx\UseHook\@undefined\else\UseHook{babel/*/#2}\fi
1086   \def\bbl@elth##1{%
1087     \bbl@cs{hk@##1}{\bbl@cs{ev@##1@#2@#3}}%
1088     \bbl@cs{ev@#2@#3}%
1089   \ifx\language\@undefined\else % Test required for Plain (?)
1090     \ifx\UseHook\@undefined\else\UseHook{babel/#1/#2}\fi
1091     \def\bbl@elth##1{%
1092       \bbl@cs{hk@##1}{\bbl@cs{ev@##1@#2@#1}#3}}%
1093     \bbl@cs{ev@#2@#1}%
1094   \fi}

```

To ensure forward compatibility, arguments in hooks are set implicitly. So, if a further argument is added in the future, there is no need to change the existing code. Note events intended for `hyphen.cfg` are also loaded (just in case you need them for some reason).

```

1095 \def\bbl@evargs{,% <- don't delete this comma
1096   everylanguage=1,loadkernel=1,loadpatterns=1,loadexceptions=1,%
1097   adddialect=2,patterns=2,defaultcommands=0,encodedcommands=2,write=0,%
1098   beforeextras=0,afterextras=0,stopcommands=0,stringprocess=0,%
1099   hyphenation=2,initiateactive=3,afterreset=0,foreign=0,foreign*=0,%
1100   beforestart=0,language=2,begindocument=1}
1101 \ifx\NewHook\@undefined\else % Test for Plain (?)
1102   \def\bbl@tempa#1=#2\@@{\NewHook{babel/#1}\NewHook{babel/*/#1}}
1103   \bbl@foreach\bbl@evargs{\bbl@tempa#1\@@}
1104 \fi

```

Since the following command is meant for a hook (although a $\TeX$ one), it's placed here.

```

1105 \providecommand\PassOptionsToLocale[2]{%
1106   \bbl@csarg\bbl@add@list{passto#2}{#1}}

```

4.7 Setting up language files

\LdfInit `\LdfInit` macro takes two arguments. The first argument is the name of the language that will be defined in the language definition file; the second argument is either a control sequence or a string from which a control sequence should be constructed. The existence of the control sequence indicates that the file has been processed before.

At the start of processing a language definition file we always check the category code of the at-sign. We make sure that it is a 'letter' during the processing of the file. We also save its name as the last called option, even if not loaded.

Another character that needs to have the correct category code during processing of language definition files is the equals sign, ‘=’, because it is sometimes used in constructions with the `\let` primitive. Therefore we store its current catcode and restore it later on.

Now we check whether we should perhaps stop the processing of this file. To do this we first need to check whether the second argument that is passed to `\LdfInit` is a control sequence. We do that by looking at the first token after passing #2 through `string`. When it is equal to `\@backslashchar` we are dealing with a control sequence which we can compare with `\@undefined`.

If so, we call `\ldf@quit` to set the main language, restore the category code of the `@`-sign and call `\endinput`

When #2 was *not* a control sequence we construct one and compare it with `\relax`.

Finally we check `\originalTeX`.

```

1107 \bbl@trace{Macros for setting language files up}
1108 \def\bbl@ldfinit{%
1109   \let\bbl@screset\@empty
1110   \let\BabelStrings\bbl@opt@string
1111   \let\BabelOptions\@empty
1112   \let\BabelLanguages\relax
1113   \ifx\originalTeX\@undefined
1114     \let\originalTeX\@empty
1115   \else
1116     \originalTeX
1117   \fi}
1118 \def\LdfInit#1#2{%
1119   \chardef\atcatcode=\catcode`\@
1120   \catcode`\@=11\relax
1121   \chardef\eqcatcode=\catcode`\=
1122   \catcode`\==12\relax
1123   \@ifpackagewith{babel}{ensureinfo=off}{}%
1124     {\ifx\InputIfFileExists\@undefined\else
1125       \bbl@ifunset{\bbl@lname@#1}%
1126       {{\let\bbl@ensuring\@empty % Flag used in babel-serbianc.tex
1127         \def\languagename{#1}%
1128         \bbl@id@assign
1129         \bbl@load@info{#1}}}%
1130       }%
1131     \fi}%
1132   \expandafter\if\expandafter\@backslashchar
1133     \expandafter\@car\string#2\@nil
1134     \ifx#2\@undefined\else
1135       \ldf@quit{#1}%
1136     \fi
1137   \else
1138     \expandafter\ifx\csname#2\endcsname\relax\else
1139       \ldf@quit{#1}%
1140     \fi
1141   \fi
1142   \bbl@ldfinit}

```

\ldf@quit This macro interrupts the processing of a language definition file. Remember `\endinput` is not executed immediately, but delayed to the end of the current line in the input file.

```

1143 \def\ldf@quit#1{%
1144   \expandafter\main@language\expandafter{#1}%
1145   \catcode`\@=\atcatcode \let\atcatcode\relax
1146   \catcode`\==\eqcatcode \let\eqcatcode\relax
1147   \endinput}

```

\ldf@finish This macro takes one argument. It is the name of the language that was defined in the language definition file.

We load the local configuration file if one is present, we set the main language (taking into account that the argument might be a control sequence that needs to be expanded) and reset the category code of the `@`-sign.

```

1148 \def\bbl@afterldf{%

```

```

1149 \bbl@afterlang
1150 \let\bbl@afterlang\relax
1151 \let\BabelModifiers\relax
1152 \let\bbl@screset\relax}%
1153 \def\ldf@finish#1{%
1154 \loadlocalcfg{#1}%
1155 \bbl@afterldf
1156 \expandafter\main@language\expandafter{#1}%
1157 \catcode`\@=\atcatcode \let\atcatcode\relax
1158 \catcode`\==\eqcatcode \let\eqcatcode\relax}

```

After the preamble of the document the commands `\LdfInit`, `\ldf@quit` and `\ldf@finish` are no longer needed. Therefore they are turned into warning messages in `ltxex`.

```

1159 \@onlypreamble\LdfInit
1160 \@onlypreamble\ldf@quit
1161 \@onlypreamble\ldf@finish

```

\main@language

\bbl@main@language This command should be used in the various language definition files. It stores its argument in `\bbl@main@language`; to be used to switch to the correct language at the beginning of the document.

```

1162 \def\main@language#1{%
1163 \def\bbl@main@language{#1}%
1164 \let\language\name\bbl@main@language
1165 \let\localename\bbl@main@language
1166 \let\mainlocalename\bbl@main@language
1167 \bbl@id@assign
1168 \ifcase\bbl@engine\or
1169 \ifx\setattribute\undefined\else
1170 \setattribute\bbl@attr@locale\localeid
1171 \fi
1172 \fi
1173 \bbl@patterns{\language\name}}

```

We also have to make sure that some code gets executed at the beginning of the document, either when the aux file is read or, if it does not exist, when the `\AtBeginDocument` is executed. Languages do not set `\pagedir`, so we set here for the whole document to the main `\bodydir`.

The code written to the aux file attempts to avoid errors if babel is removed from the document.

```

1174 \def\bbl@beforestart{%
1175 \def\@nolanerr##1{%
1176 \bbl@carg\chardef{l@##1}\z@
1177 \bbl@warning{Undefined language '##1' in aux.\Reported}}%
1178 \bbl@usehooks{beforestart}{}%
1179 \global\let\bbl@beforestart\relax}
1180 \AtBeginDocument{%
1181 {\@nameuse\bbl@beforestart}}% Group!
1182 \if@filesw
1183 \providecommand\babel@aux[2]{}%
1184 \immediate\write\@mainaux{\unexpanded{%
1185 \providecommand\babel@aux[2]{\global\let\babel@toc\@gobbletwo}}}%
1186 \immediate\write\@mainaux{\string\@nameuse\bbl@beforestart}}%
1187 \fi
1188 \expandafter\selectlanguage\expandafter{\bbl@main@language}%
1189 \ifbbl@single % must go after the line above.
1190 \renewcommand\selectlanguage[1]{}%
1191 \renewcommand\foreignlanguage[2]{#2}%
1192 \global\let\babel@aux\@gobbletwo % Also as flag
1193 \fi}

```

A bit of optimization. Select in heads/feet the language only if necessary.

```

1194 \def\select@language@x#1{%
1195 \ifcase\bbl@select@type
1196 \bbl@ifsamestring\language\name{#1}{\select@language{#1}}%

```

```

1197 \else
1198   \select@language{#1}%
1199 \fi}

```

4.8 Shorthands

The macro `\initiate@active@char` below takes all the necessary actions to make its argument a shorthand character. The real work is performed once for each character. But first we define a little tool.

```

1200 \bbl@trace{Shorhands}
1201 \def\bbl@withactive#1#2{%
1202   \begingroup
1203     \lccode`~=`#2\relax
1204     \lowercase{\endgroup#1~}}

```

\bbl@add@special The macro `\bbl@add@special` is used to add a new character (or single character control sequence) to the macro `\dospecials` (and `\@sanitize` if $\TeX$ is used). It is used only at one place, namely when `\initiate@active@char` is called (which is ignored if the char has been made active before). Because `\@sanitize` can be undefined, we put the definition inside a conditional.

Items are added to the lists without checking its existence or the original catcode. It does not hurt, but should be fixed. It's already done with `\nfss@catcodes`, added in 3.10.

```

1205 \def\bbl@add@special#1{% 1:a macro like "\", \?, etc.
1206   \bbl@add\dospecials{\do#1}% test \@sanitize = \relax, for back. compat.
1207   \bbl@ifunset{\@sanitize}{\bbl@add\@sanitize{\@makeother#1}}%
1208   \ifx\nfss@catcodes\undefined\else
1209     \begingroup
1210       \catcode`#1\active
1211       \nfss@catcodes
1212       \ifnum\catcode`#1=\active
1213         \endgroup
1214         \bbl@add\nfss@catcodes{\@makeother#1}%
1215       \else
1216         \endgroup
1217     \fi
1218   \fi}

```

\initiate@active@char A language definition file can call this macro to make a character active. This macro takes one argument, the character that is to be made active. When the character was already active this macro does nothing. Otherwise, this macro defines the control sequence `\normal@char<char>` to expand to the character in its ‘normal state’ and it defines the active character to expand to `\normal@char<char>` by default (`<char>` being the character to be made active). Later its definition can be changed to expand to `\active@char<char>` by calling `\bbl@activate{<char>}`.

For example, to make the double quote character active one could have `\initiate@active@char{"}` in a language definition file. This defines " as `\active@prefix "\active@char` (where the first " is the character with its original catcode, when the shorthand is created, and `\active@char` is a single token). In protected contexts, it expands to `\protect "` or `\noexpand "` (i.e., with the original "); otherwise `\active@char` is executed. This macro in turn expands to `\normal@char` in “safe” contexts (e.g., `\label`), but `\user@active` in normal “unsafe” ones. The latter search a definition in the user, language and system levels, in this order, but if none is found, `\normal@char` is used. However, a deactivated shorthand (with `\bbl@deactivate` defined as `\active@prefix "\normal@char`).

The following macro is used to define shorthands in the three levels. It takes 4 arguments: the (string'ed) character, `\<level>@group`, `\<level>@active` and `\<next-level>@active` (except in system).

```

1219 \def\bbl@active@def#1#2#3#4{%
1220   \namedef{#3#1}{%
1221     \expandafter\ifx\csname#2@sh@#1\endcsname\relax
1222     \bbl@afterelse\bbl@sh@select#2#1{#3@arg#1}{#4#1}%
1223   \else
1224     \bbl@afterfi\csname#2@sh@#1\endcsname
1225   \fi}%

```

When there is also no current-level shorthand with an argument we will check whether there is a next-level defined shorthand for this active character.

```

1226 \long\@namedef{#3@arg#1}##1{%
1227   \expandafter\ifx\csname#2@sh@#1\string##1@endcsname\relax
1228     \bbl@afterelse\csname#4#1\endcsname##1%
1229   \else
1230     \bbl@afterfi\csname#2@sh@#1\string##1\endcsname
1231   \fi}}%

```

\initiate@active@char calls \@initiate@active@char with 3 arguments. All of them are the same character with different catcodes: active, other (\string'ed) and the original one. This trick simplifies the code a lot.

```

1232 \def\initiate@active@char#1{%
1233   \bbl@ifunset{active@char\string#1}%
1234   {\bbl@withactive
1235     {\expandafter\@initiate@active@char\expandafter}#1\string#1}%
1236   {}}

```

The very first thing to do is saving the original catcode and the original definition, even if not active, which is possible (undefined characters require a special treatment to avoid making them \relax and preserving some degree of protection).

```

1237 \def\@initiate@active@char#1#2#3{%
1238   \bbl@csarg\edef{oricat@#2}{\catcode`#2=\the\catcode`#2\relax}%
1239   \ifx#1\@undefined
1240     \bbl@csarg\def{oridef@#2}{\def#1{\active@prefix#1\@undefined}}%
1241   \else
1242     \bbl@csarg\let{oridef@#2}#1%
1243     \bbl@csarg\edef{oridef@#2}{%
1244       \let\noexpand#1%
1245       \expandafter\noexpand\csname bbl@oridef@#2\endcsname}%
1246   \fi

```

If the character is already active we provide the default expansion under this shorthand mechanism. Otherwise we write a message in the transcript file, and define \normal@char<char> to expand to the character in its default state. If the character is mathematically active when babel is loaded (for example ') the normal expansion is somewhat different to avoid an infinite loop (but it does not prevent the loop if the mathcode is set to "8000 *a posteriori*").

```

1247 \ifx#1#3\relax
1248   \expandafter\let\csname normal@char#2\endcsname#3%
1249 \else
1250   \bbl@info{Making #2 an active character}%
1251   \ifnum\mathcode`#2=\ifodd\bbl@engine"1000000 \else"8000 \fi
1252     \@namedef{normal@char#2}{%
1253       \textormath{#3}{\csname bbl@oridef@#2\endcsname}}%
1254   \else
1255     \@namedef{normal@char#2}{#3}%
1256   \fi

```

To prevent problems with the loading of other packages after babel we reset the catcode of the character to the original one at the end of the package and of each language file (except with KeepShorthandsActive). It is re-activate again at \begin{document}. We also need to make sure that the shorthands are active during the processing of the aux file. Otherwise some citations may give unexpected results in the printout when a shorthand was used in the optional argument of \bibitem for example. Then we make it active (not strictly necessary, but done for backward compatibility).

```

1257 \bbl@restoreactive{#2}%
1258 \AtBeginDocument{%
1259   \catcode`#2\active
1260   \if@filesw
1261     \immediate\write\@mainaux{\catcode`\string#2\active}%
1262   \fi}%
1263 \expandafter\bbl@add@special\csname#2\endcsname
1264 \catcode`#2\active
1265 \fi

```


Now we have set `\normal@char<char>`, we must define `\active@char<char>`, to be executed when the character is activated. We define the first level expansion of `\active@char<char>` to check the status of the `@safe@actives` flag. If it is set to true we expand to the ‘normal’ version of this character, otherwise we call `\user@active<char>` to start the search of a definition in the user, language and system levels (or eventually `normal@char<char>`).

```

1266 \let\bbl@tempa\@firstoftwo
1267 \if\string^#2%
1268 \def\bbl@tempa{\noexpand\textormath}%
1269 \else
1270 \ifx\bbl@mathnormal\@undefined\else
1271 \let\bbl@tempa\bbl@mathnormal
1272 \fi
1273 \fi
1274 \expandafter\edef\csname active@char#2\endcsname{%
1275 \bbl@tempa
1276 {\noexpand\if@safe@actives
1277 \noexpand\expandafter
1278 \expandafter\noexpand\csname normal@char#2\endcsname
1279 \noexpand\else
1280 \noexpand\expandafter
1281 \expandafter\noexpand\csname bbl@doactive#2\endcsname
1282 \noexpand\fi}%
1283 {\expandafter\noexpand\csname normal@char#2\endcsname}}%
1284 \bbl@csarg\edef{doactive#2}{%
1285 \expandafter\noexpand\csname user@active#2\endcsname}%

```

We now define the default values which the shorthand is set to when activated or deactivated. It is set to the deactivated form (globally), so that the character expands to

`\active@prefix<char>\normal@char<char>`

(where `\active@char<char>` is *one* control sequence!).

```

1286 \bbl@csarg\edef{active@#2}{%
1287 \noexpand\active@prefix\noexpand#1%
1288 \expandafter\noexpand\csname active@char#2\endcsname}%
1289 \bbl@csarg\edef{normal@#2}{%
1290 \noexpand\active@prefix\noexpand#1%
1291 \expandafter\noexpand\csname normal@char#2\endcsname}%
1292 \bbl@ncarg\let#1\bbl@normal@#2}%

```

The next level of the code checks whether a user has defined a shorthand for himself with this character. First we check for a single character shorthand. If that doesn’t exist we check for a shorthand with an argument.

```

1293 \bbl@active@def#2\user@group{user@active}{language@active}%
1294 \bbl@active@def#2\language@group{language@active}{system@active}%
1295 \bbl@active@def#2\system@group{system@active}{normal@char}%

```

In order to do the right thing when a shorthand with an argument is used by itself at the end of the line we provide a definition for the case of an empty argument. For that case we let the shorthand character expand to its non-active self. Also, When a shorthand combination such as ‘ ’ ends up in a heading $\TeX$ would see `\protect'\protect'`. To prevent this from happening a couple of shorthand needs to be defined at user level.

```

1296 \expandafter\edef\csname\user@group @sh@#2@@\endcsname
1297 {\expandafter\noexpand\csname normal@char#2\endcsname}%
1298 \expandafter\edef\csname\user@group @sh@#2@\string\protect@\endcsname
1299 {\expandafter\noexpand\csname user@active#2\endcsname}%

```

Finally, a couple of special cases are taken care of. (1) If we are making the right quote (‘) active we need to change `\pr@m@s` as well. Also, make sure that a single ‘ in math mode ‘does the right thing’. (2) If we are using the caret (^) as a shorthand character special care should be taken to make sure math still works. Therefore an extra level of expansion is introduced with a check for math mode on the upper level.

```

1300 \if\string'#2%
1301 \let\prim@s\bbl@prim@s

```

```

1302 \let\active@math@prime#1%
1303 \fi
1304 \bbl@usehooks{initiateactive}{\#1}{\#2}{\#3}}

```

The following package options control the behavior of shorthands in math mode.

```

1305 ({*More package options}) ≡
1306 \DeclareOption{math=active}{}
1307 \DeclareOption{math=normal}{\def\bbl@mathnormal{\noexpand\textormath}}
1308 (/More package options)

```

Initiating a shorthand makes active the char. That is not strictly necessary but it is still done for backward compatibility. So we need to restore the original catcode at the end of package *and* the end of the ldf.

```

1309 \ifpackagewith{babel}{KeepShorthandsActive}%
1310 {\let\bbl@restoreactive@gobble}%
1311 {\def\bbl@restoreactive#1{%
1312   \bbl@exp{%
1313     \\AfterBabelLanguage\\CurrentOption
1314     {\catcode`#1=\the\catcode`#1\relax}%
1315     \\AtEndOfPackage
1316     {\catcode`#1=\the\catcode`#1\relax}}}%
1317   \AtEndOfPackage{\let\bbl@restoreactive@gobble}}

```

\bbl@sh@select This command helps the shorthand supporting macros to select how to proceed.

Note that this macro needs to be expandable as do all the shorthand macros in order for them to work in expansion-only environments such as the argument of `\hyphenation`.

This macro expects the name of a group of shorthands in its first argument and a shorthand character in its second argument. It will expand to either `\bbl@firstcs` or `\bbl@scndcs`. Hence two more arguments need to follow it.

```

1318 \def\bbl@sh@select#1#2{%
1319   \expandafter\ifx\csname#1@sh@#2@sel\endcsname\relax
1320     \bbl@afterelse\bbl@scndcs
1321   \else
1322     \bbl@afterfi\csname#1@sh@#2@sel\endcsname
1323   \fi}

```

\active@prefix Used in the expansion of active characters has a function similar to `\OT1-cmd` in that it `\protects` the active character whenever `\protect` is *not* `\@typeset@protect`. The `\@gobble` is needed to remove a token such as `\activechar:` (when the double colon was the active character to be dealt with). There are two definitions, depending of `\ifincsname` is available. If there is, the expansion will be more robust.

```

1324 \begingroup
1325 \bbl@ifunset{ifincsname}
1326 {\gdef\active@prefix#1{%
1327   \ifx\protect\@typeset@protect
1328   \else
1329     \ifx\protect\@unexpandable@protect
1330       \noexpand#1%
1331     \else
1332       \protect#1%
1333     \fi
1334     \expandafter\@gobble
1335   \fi}}
1336 {\gdef\active@prefix#1{%
1337   \ifincsname
1338     \string#1%
1339     \expandafter\@gobble
1340   \else
1341     \ifx\protect\@typeset@protect
1342     \else
1343       \ifx\protect\@unexpandable@protect
1344         \noexpand#1%

```

```

1345         \else
1346         \protect#1%
1347         \fi
1348         \expandafter\expandafter\expandafter\@gobble
1349         \fi
1350     \fi}}
1351 \endgroup

```

if@safe@actives In some circumstances it is necessary to be able to reset the shorthand to its ‘normal’ value (usually the character with catcode ‘other’) on the fly. For this purpose the switch @safe@actives is available. The setting of this switch should be checked in the first level expansion of \active@char⟨char⟩. When this expansion mode is active (with \@safe@activetrue), something like "13"13 becomes "12"12 in an \edef (in other words, shorthands are \string’ed). This contrasts with \protected@edef, where catcodes are always left unchanged. Once converted, they can be used safely even after this expansion mode is deactivated (with \@safe@activefalse).

```

1352 \newif\if@safe@actives
1353 \@safe@activesfalse

```

\bbl@restore@actives When the output routine kicks in while the active characters were made “safe” this must be undone in the headers to prevent unexpected typeset results. For this situation we define a command to make them “unsafe” again.

```

1354 \def\bbl@restore@actives{\if@safe@actives\@safe@activesfalse\fi}

```

\bbl@activate

\bbl@deactivate Both macros take one argument, like \initiate@active@char. The macro is used to change the definition of an active character to expand to \active@char⟨char⟩ in the case of \bbl@activate, or \normal@char⟨char⟩ in the case of \bbl@deactivate.

```

1355 \chardef\bbl@activated\z@
1356 \def\bbl@activate#1{%
1357   \chardef\bbl@activated\@ne
1358   \bbl@withactive{\expandafter\let\expandafter}#1%
1359   \csname bbl@active@\string#1\endcsname}
1360 \def\bbl@deactivate#1{%
1361   \chardef\bbl@activated\tw@
1362   \bbl@withactive{\expandafter\let\expandafter}#1%
1363   \csname bbl@normal@\string#1\endcsname}

```

\bbl@firstcs

\bbl@scndcs These macros are used only as a trick when declaring shorthands.

```

1364 \def\bbl@firstcs#1#2{\csname#1\endcsname}
1365 \def\bbl@scndcs#1#2{\csname#2\endcsname}

```

\declare@shorthand Used to declare a shorthand on a certain level. It takes three arguments:

1. a name for the collection of shorthands, i.e., ‘system’, or ‘dutch’;
2. the character (sequence) that makes up the shorthand, i.e., ~ or "a;
3. the code to be executed when the shorthand is encountered.

The auxiliary macro \babel@texpdf improves the interoperativity with hyperref and takes 4 arguments: (1) The T_EX code in text mode, (2) the string for hyperref, (3) the T_EX code in math mode, and (4), which is currently ignored, but it’s meant for a string in math mode, like a minus sign instead of an hyphen (currently hyperref doesn’t discriminate the mode). This macro may be used in l_df files.

```

1366 \def\babel@texpdf#1#2#3#4{%
1367   \ifx\texorpdfstring\@undefined
1368     \textormath{#1}{#3}%
1369   \else
1370     \texorpdfstring{\textormath{#1}{#3}}{#2}%
1371     % \texorpdfstring{\textormath{#1}{#3}}{\textormath{#2}{#4}}%
1372   \fi}
1373 %
1374 \def\declare@shorthand#1#2{\@decl@short{#1}#2\@nil}

```

```

1375 \def\@decl@short#1#2#3\@nil#4{%
1376   \def\bbl@tempa{#3}%
1377   \ifx\bbl@tempa\@empty
1378     \expandafter\let\csname #1@sh@\string#2@sel\endcsname\bbl@scndcs
1379     \bbl@ifunset{#1@sh@\string#2@}{}%
1380     {\def\bbl@tempa{#4}%
1381      \expandafter\ifx\csname#1@sh@\string#2@\endcsname\bbl@tempa
1382      \else
1383        \bbl@info
1384          {Redefining #1 shorthand \string#2\\%
1385           in language \CurrentOption}%
1386        \fi}%
1387     \@namedef{#1@sh@\string#2@}{#4}%
1388   \else
1389     \expandafter\let\csname #1@sh@\string#2@sel\endcsname\bbl@firstcs
1390     \bbl@ifunset{#1@sh@\string#2@\string#3@}{}%
1391     {\def\bbl@tempa{#4}%
1392      \expandafter\ifx\csname#1@sh@\string#2@\string#3@\endcsname\bbl@tempa
1393      \else
1394        \bbl@info
1395          {Redefining #1 shorthand \string#2\string#3\\%
1396           in language \CurrentOption}%
1397        \fi}%
1398     \@namedef{#1@sh@\string#2@\string#3@}{#4}%
1399   \fi}

```

\textormath Some of the shorthands that will be declared by the language definition files have to be usable in both text and mathmode. To achieve this the helper macro `\textormath` is provided.

```

1400 \def\textormath{%
1401   \ifmmode
1402     \expandafter\@secondoftwo
1403   \else
1404     \expandafter\@firstoftwo
1405   \fi}

```

\user@group

\language@group

\system@group The current concept of ‘shorthands’ supports three levels or groups of shorthands.

For each level the name of the level or group is stored in a macro. The default is to have a user group; use language group ‘english’ and have a system group called ‘system’.

```

1406 \def\user@group{user}
1407 \def\language@group{english}
1408 \def\system@group{system}

```

\useshorthands This is the user level macro. It initializes and activates the character for use as a shorthand character (i.e., it’s active in the preamble). Languages can deactivate shorthands, so a starred version is also provided which activates them always after the language has been switched.

```

1409 \def\useshorthands{%
1410   \@ifstar\bbl@usesh@s{\bbl@usesh@x{}}
1411 \def\bbl@usesh@s#1{%
1412   \bbl@usesh@x
1413   {\AddBabelHook{babel-sh-\string#1}{afterextras}{\bbl@activate{#1}}}%
1414   {#1}}
1415 \def\bbl@usesh@x#1#2{%
1416   \bbl@ifshorthand{#2}%
1417   {\def\user@group{user}%
1418    \initiate@active@char{#2}%
1419    #1%
1420    \bbl@activate{#2}}%
1421   {\bbl@error{shorthand-is-off}{#2}}}

```

\defineshorthand Currently we only support two groups of user level shorthands, named internally user and user@(*language*) (language-dependent user shorthands). By default, only the first one is taken into account, but if the former is also used (in the optional argument of \defineshorthand) a new level is inserted for it (user@generic, done by \bbl@set@user@generic); we make also sure {} and \protect are taken into account in this new top level.

```

1422 \def\user@language@group{user@\language@group}
1423 \def\bbl@set@user@generic#1#2{%
1424   \bbl@ifunset{user@generic@active#1}%
1425   {\bbl@active@def#1\user@language@group{user@active}{user@generic@active}%
1426    \bbl@active@def#1\user@group{user@generic@active}{\language@active}%
1427    \expandafter\edef\csname#2@sh@#1@\endcsname{%
1428      \expandafter\noexpand\csname normal@char#1\endcsname}%
1429      \expandafter\edef\csname#2@sh@#1@\string\protect@\endcsname{%
1430        \expandafter\noexpand\csname user@active#1\endcsname}}%
1431   \@empty}
1432 \newcommand\defineshorthand[3]{user}%
1433 \edef\bbl@tempa{\zap@space#1 \@empty}%
1434 \bbl@for\bbl@tempb\bbl@tempa{%
1435   \if*\expandafter\@car\bbl@tempb\@nil
1436     \edef\bbl@tempb{user@\expandafter\@gobble\bbl@tempb}%
1437     \@expandtwoargs
1438     \bbl@set@user@generic{\expandafter\string\@car#2\@nil}\bbl@tempb
1439   \fi
1440   \declare@shorthand{\bbl@tempb}{#2}{#3}}

```

\languageshorthands A user level command to change the language from which shorthands are used. Unfortunately, babel currently does not keep track of defined groups, and therefore there is no way to catch a possible change in casing to fix it in the same way languages names are fixed.

```

1441 \def\languageshorthands#1{%
1442   \bbl@ifsamestring{none}{#1}{}%
1443   \bbl@once{short-\localename-#1}{%
1444     \bbl@info{'\localename' activates '#1' shorthands.\Reported}}}%
1445   \def\language@group{#1}}

```

\aliasshorthand *Deprecated.* First the new shorthand needs to be initialized. Then, we define the new shorthand in terms of the original one, but note with \aliasshorthands{"}{/} is \active@prefix /\active@char/, so we still need to let the latter to \active@char".

```

1446 \def\aliasshorthand#1#2{%
1447   \bbl@ifshorthand{#2}%
1448   {\expandafter\ifx\csname active@char\string#2\endcsname\relax
1449     \ifx\document\@notprerr
1450       \@notshorthand{#2}%
1451     \else
1452       \initiate@active@char{#2}%
1453       \bbl@ccarg\let{active@char\string#2}{active@char\string#1}%
1454       \bbl@ccarg\let{normal@char\string#2}{normal@char\string#1}%
1455       \bbl@activate{#2}%
1456     \fi
1457   \fi}%
1458   {\bbl@error{shorthand-is-off}{#2}{}}}

```

\@notshorthand

```

1459 \def\@notshorthand#1{\bbl@error{not-a-shorthand}{#1}{}}

```

\shorthandon

\shorthandoff The first level definition of these macros just passes the argument on to \bbl@switch@sh, adding \@nil at the end to denote the end of the list of characters.

```

1460 \newcommand*\shorthandon[1]{\bbl@switch@sh\@ne#1\@nnil}
1461 \DeclareRobustCommand*\shorthandoff{%
1462   \@ifstar{\bbl@shorthandoff\tw@}{\bbl@shorthandoff\z@}}
1463 \def\bbl@shorthandoff#1#2{\bbl@switch@sh#1#2\@nnil}

```

\bbl@switch@sh The macro \bbl@switch@sh takes the list of characters apart one by one and subsequently switches the category code of the shorthand character according to the first argument of \bbl@switch@sh.

But before any of this switching takes place we make sure that the character we are dealing with is known as a shorthand character. If it is, a macro such as \active@char" should exist.

Switching off and on is easy – we just set the category code to ‘other’ (12) and \active. With the starred version, the original catcode and the original definition, saved in @initiate@active@char, are restored.

```

1464 \def\bbl@switch@sh#1#2{%
1465   \ifx#2@nnil\else
1466     \bbl@ifunset{\bbl@active@\string#2}%
1467     {\bbl@error{not-a-shorthand-b}{\string#2}}%
1468     {\ifcase#1%   off, on, off*
1469       \catcode`#2\relax
1470       \or
1471       \catcode`#2\active
1472       \bbl@ifunset{\bbl@shdef@\string#2}%
1473       {}%
1474       {\bbl@withactive{\expandafter\let\expandafter}#2%
1475         \csname bbl@shdef@\string#2\endcsname
1476         \bbl@csarg\let{shdef@\string#2}\relax}%
1477       \ifcase\bbl@activated\or
1478       \bbl@activate{#2}%
1479       \else
1480       \bbl@deactivate{#2}%
1481       \fi
1482       \or
1483       \bbl@ifunset{\bbl@shdef@\string#2}%
1484       {\bbl@withactive{\bbl@csarg\let{shdef@\string#2}}#2}%
1485       {}%
1486       \csname bbl@oricat@\string#2\endcsname
1487       \csname bbl@oridef@\string#2\endcsname
1488       \fi}%
1489   \bbl@afterfi\bbl@switch@sh#1%
1490 \fi}

```

Note the value is that at the expansion time; e.g., in the preamble shorthands are usually deactivated.

```

1491 \def\babelshorthand{\active@prefix\babelshorthand\bbl@putsh}
1492 \def\bbl@putsh#1{%
1493   \bbl@ifunset{\bbl@active@\string#1}%
1494   {\bbl@putsh@i#1@empty@nnil}%
1495   {\csname bbl@active@\string#1\endcsname}}
1496 \def\bbl@putsh@i#1#2@nnil{%
1497   \csname\language@group @sh@\string#1@%
1498   \ifx@empty#2\else\string#2@\fi\endcsname}
1499 %
1500 \ifx\bbl@opt@shorthands@nnil\else
1501   \let\bbl@s@initiate@active@char\initiate@active@char
1502   \def\initiate@active@char#1{%
1503     \bbl@ifshorthand{#1}{\bbl@s@initiate@active@char{#1}}{}}
1504   \let\bbl@s@switch@sh\bbl@switch@sh
1505   \def\bbl@switch@sh#1#2{%
1506     \ifx#2@nnil\else
1507       \bbl@afterfi
1508       \bbl@ifshorthand{#2}{\bbl@s@switch@sh#1{#2}}{\bbl@switch@sh#1}%
1509       \fi}
1510   \let\bbl@s@activate\bbl@activate
1511   \def\bbl@activate#1{%
1512     \bbl@ifshorthand{#1}{\bbl@s@activate{#1}}{}}
1513   \let\bbl@s@deactivate\bbl@deactivate
1514   \def\bbl@deactivate#1{%
1515     \bbl@ifshorthand{#1}{\bbl@s@deactivate{#1}}{}}

```

1516 \fi

You may want to test if a character is a shorthand. Note it does not test whether the shorthand is on or off.

1517 \newcommand\ifbabelshorthand[3]{\bbl@ifunset{\bbl@active@\string#1}{#3}{#2}}

\bbl@prim@s

\bbl@pr@m@s One of the internal macros that are involved in substituting \prime for each right quote in mathmode is \prim@s. This checks if the next character is a right quote. When the right quote is active, the definition of this macro needs to be adapted to look also for an active right quote; the hat could be active, too.

```

1518 \def\bbl@prim@s{%
1519   \prime\futurelet\@let@token\bbl@pr@m@s}
1520 \def\bbl@if@primes#1#2{%
1521   \ifx#1\@let@token
1522     \expandafter\@firstoftwo
1523   \else\ifx#2\@let@token
1524     \bbl@afterelse\expandafter\@firstoftwo
1525   \else
1526     \bbl@afterfi\expandafter\@secondoftwo
1527   \fi\fi}
1528 \begingroup
1529   \catcode`\^=7 \catcode`\*=active \lccode`\*=\^
1530   \catcode`\'=12 \catcode`\`=active \lccode`\`=\'
1531   \lowercase{%
1532     \gdef\bbl@pr@m@s{%
1533       \bbl@if@primes" '%
1534       \pr@@@s
1535       {\bbl@if@primes*\^pr@@@t\egroup}}}
1536 \endgroup

```

Usually the ~ is active and expands to \penalty\@M_. When it is written to the aux file it is written expanded. To prevent that and to be able to use the character ~ as a start character for a shorthand, it is redefined here as a one character shorthand on system level. The system declaration is in most cases redundant (when ~ is still a non-break space), and in some cases is inconvenient (if ~ has been redefined); however, for backward compatibility it is maintained (some existing documents may rely on the babel value).

```

1537 \initiate@active@char{~}
1538 \declare@shorthand{system}{~}{\leavevmode\nobreak\ }
1539 \bbl@activate{~}

```

\OT1dqpos

\T1dqpos The position of the double quote character is different for the OT1 and T1 encodings. It will later be selected using the \f@encoding macro. Therefore we define two macros here to store the position of the character in these encodings.

```

1540 \expandafter\def\csname OT1dqpos\endcsname{127}
1541 \expandafter\def\csname T1dqpos\endcsname{4}

```

When the macro \f@encoding is undefined (as it is in plain T_EX) we define it here to expand to OT1

```

1542 \ifx\f@encoding\undefined
1543   \def\f@encoding{OT1}
1544 \fi

```

4.9 Language attributes

Language attributes provide a means to give the user control over which features of the language definition files he wants to enable.

\languageattribute The macro `\languageattribute` checks whether its arguments are valid and then activates the selected language attribute. First check whether the language is known, and then process each attribute in the list.

To make sure each attribute is selected only once, we store the already selected attributes in `\bbl@known@attrs`. When that control sequence is not yet defined this attribute is certainly not selected before.

When we end up here the attribute is not selected before. So, we add it to the list of selected attributes and execute the associated $\TeX$ code.

```

1545 \bbl@trace{Language attributes}
1546 \newcommand\languageattribute[2]{%
1547   \def\bbl@tempc{#1}%
1548   \bbl@fixname\bbl@tempc
1549   \bbl@iflanguage\bbl@tempc{%
1550     \bbl@vforeach{#2}{%
1551       \ifx\bbl@known@attrs\undefined
1552         \in@false
1553       \else
1554         \bbl@xin@{,\bbl@tempc-##1,}{,\bbl@known@attrs,}%
1555       \fi
1556       \ifin@
1557         \bbl@warning{%
1558           You have more than once selected the attribute '##1'\%
1559           for language #1. Reported}%
1560       \else
1561         \bbl@info{Activated '##1' attribute for\\%
1562           '\bbl@tempc'. Reported}%
1563         \bbl@exp{%
1564           \\ \bbl@add@list\\ \bbl@known@attrs{\bbl@tempc-##1}}%
1565         \edef\bbl@tempa{\bbl@tempc-##1}%
1566         \expandafter\bbl@ifknown@ttrib\expandafter{\bbl@tempa}\bbl@attributes
1567         {\csname\bbl@tempc @attr##1\endcsname}%
1568         {\bbl@error{unknown-attribute}{\bbl@tempc}{##1}}}%
1569       \fi}}
1570 \@onlypreamble\languageattribute

```

\bbl@declare@ttribute This command adds the new language/attribute combination to the list of known attributes.

Then it defines a control sequence to be executed when the attribute is used in a document. The result of this should be that the macro `\extras...` for the current language is extended, otherwise the attribute will not work as its code is removed from memory at `\begin{document}`.

```

1571 \def\bbl@declare@ttribute#1#2#3{%
1572   \bbl@xin@{,#2,}{,\BabelModifiers,}%
1573   \ifin@
1574     \AfterBabelLanguage{#1}{\languageattribute{#1}{#2}}%
1575   \fi
1576   \bbl@add@list\bbl@attributes{#1-#2}%
1577   \expandafter\def\csname#1@attr#2\endcsname{#3}%

```

\bbl@ifattributeset This internal macro has 4 arguments. It can be used to interpret $\TeX$ code based on whether a certain attribute was set. This command should appear inside the argument to `\AtBeginDocument` because the attributes are set in the document preamble, *after* babel is loaded.

The first argument is the language, the second argument the attribute being checked, and the third and fourth arguments are the true and false clauses.

```

1578 \def\bbl@ifattributeset#1#2#3#4{%
1579   \ifx\bbl@known@attrs\undefined
1580     \in@false
1581   \else
1582     \bbl@xin@{,#1-#2,}{,\bbl@known@attrs,}%
1583   \fi
1584   \ifin@
1585     \bbl@afterelse#3%
1586   \else

```



```

1587 \bbl@afterfi#4%
1588 \fi}

```

\bbl@ifknown@ttrib An internal macro to check whether a given language/attribute is known. The macro takes 4 arguments, the language/attribute, the attribute list, the $\TeX$ -code to be executed when the attribute is known and the $\TeX$ -code to be executed otherwise.

We first assume the attribute is unknown. Then we loop over the list of known attributes, trying to find a match.

```

1589 \def\bbl@ifknown@ttrib#1#2{%
1590 \let\bbl@tempa\@secondoftwo
1591 \bbl@loopx\bbl@tempb{#2}{%
1592 \expandafter\in\expandafter{\expandafter,\bbl@tempb,}{, #1,}%
1593 \ifin@
1594 \let\bbl@tempa\@firstoftwo
1595 \else
1596 \fi}%
1597 \bbl@tempa}

```

\bbl@clear@ttribs This macro removes all the attribute code from $\TeX$'s memory at $\begin{document}$ time (if any is present).

```

1598 \def\bbl@clear@ttribs{%
1599 \ifx\bbl@attributes\undefined\else
1600 \bbl@loopx\bbl@tempa{\bbl@attributes}{%
1601 \expandafter\bbl@clear@ttrib\bbl@tempa.}%
1602 \let\bbl@attributes\undefined
1603 \fi}
1604 \def\bbl@clear@ttrib#1-#2.{%
1605 \expandafter\let\csname#1@attr@#2\endcsname\undefined}
1606 \AtBeginDocument{\bbl@clear@ttribs}

```

4.10 Support for saving and redefining macros

To save the meaning of control sequences using $\babel@save$, we use temporary control sequences. To save hash table entries for these control sequences, we don't use the name of the control sequence to be saved to construct the temporary name. Instead we simply use the value of a counter, which is reset to zero each time we begin to save new values. This works well because we release the saved meanings before we begin to save a new set of control sequence meanings (see $\selectlanguage$ and $\originalTeX$). Note undefined macros are not undefined any more when saved – they are $\relax$ 'ed.

\babel@savecnt

\babel@beginsave The initialization of a new save cycle: reset the counter to zero.

```

1607 \bbl@trace{Macros for saving definitions}
1608 \def\babel@beginsave{\babel@savecnt\z@}

```

Before it's forgotten, allocate the counter and initialize all.

```

1609 \newcount\babel@savecnt
1610 \babel@beginsave

```

\babel@save

\babel@savevariable The macro $\babel@save\langle csname \rangle$ saves the current meaning of the control sequence ($csname$) to $\originalTeX$ (which has to be expandable, i.e., you shouldn't let it to $\relax$). To do this, we let the current meaning to a temporary control sequence, the restore commands are appended to $\originalTeX$ and the counter is incremented. The macro $\babel@savevariable\langle variable \rangle$ saves the value of the variable. ($variable$) can be anything allowed after the $\the$ primitive. To avoid messing saved definitions up, they are saved only the very first time.

```

1611 \def\babel@save#1{%
1612 \def\bbl@tempa{, #1,}% Clumsy, for Plain
1613 \expandafter\bbl@add\expandafter\bbl@tempa\expandafter{%
1614 \expandafter{\expandafter,\bbl@savextras,}}%

```

```

1615 \expandafter\in@bbl@tempa
1616 \ifin@else
1617 \bbl@add\bbl@savextras{,#1,}%
1618 \bbl@carg\let{babel@number\babel@savecnt}#1\relax
1619 \toks@{\expandafter{\originalTeX\let#1=}}%
1620 \bbl@exp{%
1621 \def\\originalTeX{\the\toks@<babel@number\babel@savecnt>\relax}}%
1622 \advance\babel@savecnt\@ne
1623 \fi}
1624 \def\babel@savevariable#1{%
1625 \toks@{\expandafter{\originalTeX #1=}}%
1626 \bbl@exp{\def\\originalTeX{\the\toks@{\the#1\relax}}}}

```

\bbl@redefine To redefine a command, we save the old meaning of the macro. Then we redefine it to call the original macro with the ‘sanitized’ argument. The reason why we do it this way is that we don’t want to redefine the $\TeX$ macros completely in case their definitions change (they have changed in the past). A macro named `\macro` will be saved new control sequences named `\org@macro`.

```

1627 \def\bbl@redefine#1{%
1628 \edef\bbl@tempa{\bbl@stripslash#1}%
1629 \expandafter\let\csname org@bbl@tempa\endcsname#1%
1630 \expandafter\def\csname\bbl@tempa\endcsname}
1631 \@onlypreamble\bbl@redefine

```

\bbl@redefine@long This version of `\babel@redefine` can be used to redefine `\long` commands such as `\ifthenelse`.

```

1632 \def\bbl@redefine@long#1{%
1633 \edef\bbl@tempa{\bbl@stripslash#1}%
1634 \expandafter\let\csname org@bbl@tempa\endcsname#1%
1635 \long\expandafter\def\csname\bbl@tempa\endcsname}
1636 \@onlypreamble\bbl@redefine@long

```

\bbl@redefineroobust For commands that are redefined, but which *might* be robust we need a slightly more intelligent macro. A robust command `foo` is defined to expand to `\protect\foo_`. So it is necessary to check whether `\foo_` exists. The result is that the command that is being redefined is always robust afterwards. Therefore all we need to do now is define `\foo_`.

```

1637 \def\bbl@redefineroobust#1{%
1638 \edef\bbl@tempa{\bbl@stripslash#1}%
1639 \bbl@ifunset{\bbl@tempa\space}%
1640 {\expandafter\let\csname org@bbl@tempa\endcsname#1%
1641 \bbl@exp{\def\\#1{\\protect\<\bbl@tempa\space>}}}%
1642 {\bbl@exp{\let\org@bbl@tempa\<\bbl@tempa\space>}}}%
1643 \@namedef{\bbl@tempa\space}}
1644 \@onlypreamble\bbl@redefineroobust

```

4.11 French spacing

\bbl@frenchspacing

\bbl@nonfrenchspacing Some languages need to have `\frenchspacing` in effect. Others don’t want that. The command `\bbl@frenchspacing` switches it on when it isn’t already in effect and `\bbl@nonfrenchspacing` switches it off if necessary.

```

1645 \def\bbl@frenchspacing{%
1646 \ifnum\the\sfcode`\.=\@m
1647 \let\bbl@nonfrenchspacing\relax
1648 \else
1649 \frenchspacing
1650 \let\bbl@nonfrenchspacing\nonfrenchspacing
1651 \fi}
1652 \let\bbl@nonfrenchspacing\nonfrenchspacing

```

A more refined way to switch the catcodes is done with ini files. Here an auxiliary macro is defined, but the main part is in `\babelprovide`. This new method should be ideally the default one.

```

1653 \let\bbl@elt\relax
1654 \edef\bbl@fs@chars{%
1655   \bbl@elt{\string.}\@m{3000}\bbl@elt{\string?}\@m{3000}%
1656   \bbl@elt{\string!}\@m{3000}\bbl@elt{\string:}\@m{2000}%
1657   \bbl@elt{\string;}\@m{1500}\bbl@elt{\string,}\@m{1250}}
1658 \def\bbl@pre@fs{%
1659   \def\bbl@elt##1##2##3{\sfcode`##1=\the\sfcode`##1\relax}%
1660   \edef\bbl@save@sfcodes{\bbl@fs@chars}}%
1661 \def\bbl@post@fs{%
1662   \bbl@save@sfcodes
1663   \edef\bbl@tempa{\bbl@cl{frspc}}%
1664   \edef\bbl@tempa{\expandafter\@car\bbl@tempa\@nil}%
1665   \if u\bbl@tempa      % do nothing
1666   \else\if n\bbl@tempa % non french
1667     \def\bbl@elt##1##2##3{%
1668       \ifnum\sfcode`##1=##2\relax
1669         \babel@savevariable{\sfcode`##1}%
1670         \sfcode`##1=##3\relax
1671       \fi}%
1672     \bbl@fs@chars
1673   \else\if y\bbl@tempa % french
1674     \def\bbl@elt##1##2##3{%
1675       \ifnum\sfcode`##1=##3\relax
1676         \babel@savevariable{\sfcode`##1}%
1677         \sfcode`##1=##2\relax
1678       \fi}%
1679     \bbl@fs@chars
1680   \fi\fi\fi}

```

4.12 Hyphens

\babelhyphenation This macro saves hyphenation exceptions. Two macros are used to store them: `\bbl@hyphenation@` for the global ones and `\bbl@hyphenation@<language>` for language ones. See `\bbl@patterns` above for further details. We make sure there is a space between words when multiple commands are used.

```

1681 \bbl@trace{Hyphens}
1682 \@onlypreamble\babelhyphenation
1683 \AtEndOfPackage{%
1684   \newcommand\babelhyphenation[2][\@empty]{%
1685     \ifx\bbl@hyphenation@\relax
1686       \let\bbl@hyphenation@\@empty
1687     \fi
1688     \ifx\bbl@hyphlist@\@empty\else
1689       \bbl@warning{%
1690         You must not intermingle \string\selectlanguage\space and\\%
1691         \string\babelhyphenation\space or some exceptions will not\\%
1692         be taken into account. Reported}%
1693       \fi
1694       \ifx\@empty#1%
1695         \protected@edef\bbl@hyphenation@{\bbl@hyphenation@\space#2}%
1696       \else
1697         \bbl@vforeach{#1}{%
1698           \def\bbl@tempa{##1}%
1699           \bbl@fixname\bbl@tempa
1700           \bbl@iflanguage\bbl@tempa{%
1701             \bbl@csarg\protected@edef{hyphenation@\bbl@tempa}{%
1702               \bbl@ifunset{bbl@hyphenation@\bbl@tempa}%
1703               {}%
1704               {\csname bbl@hyphenation@\bbl@tempa\endcsname\space}%
1705               #2}}}%
1706         \fi}}

```

\babelhyphenmins Only $\TeX$ (basically because it's defined with a $\TeX$ tool).

```

1707 \ifx\NewDocumentCommand\undefined\else
1708   \NewDocumentCommand\babelhyphenmins{sommo}{%
1709     \IfNoValueTF{#2}%
1710       {\protected@edef\bbl@hyphenmins@{\set@hyphenmins{#3}{#4}}%
1711       \IfValueT{#5}%
1712         \protected@edef\bbl@hyphenatmin@{\hyphenationmin=#5\relax}}%
1713       \IfBooleanT{#1}{%
1714         \lefthyphenmin=#3\relax
1715         \righthyphenmin=#4\relax
1716         \IfValueT{#5}{\hyphenationmin=#5\relax}}}%
1717     {\edef\bbl@tempb{\zap@space#2 \@empty}%
1718     \bbl@for\bbl@tempa\bbl@tempb{%
1719       \@namedef{bbl@hyphenmins@bbl@tempa}{\set@hyphenmins{#3}{#4}}%
1720       \IfValueT{#5}%
1721         \@namedef{bbl@hyphenatmin@bbl@tempa}{\hyphenationmin=#5\relax}}}%
1722     \IfBooleanT{#1}{\bbl@error{hyphenmins-args}{}}}%
1723 \fi

```

\bbl@allowhyphens This macro makes hyphenation possible. Basically its definition is nothing more than `\nobreak\hskip 0pt plus 0pt`. $\TeX$ begins and ends a word for hyphenation at a glue node. The penalty prevents a linebreak at this glue node.

```

1724 \def\bbl@allowhyphens{\ifvmode\else\nobreak\hskip\zap@skip\fi}
1725 \def\bbl@t@one{T1}
1726 \def\allowhyphens{\ifx\cf@encoding\bbl@t@one\else\bbl@allowhyphens\fi}

```

\babelhyphen Macros to insert common hyphens. Note the space before @ in `\babelhyphen`. Instead of protecting it with `\DeclareRobustCommand`, which could insert a `\relax`, we use the same procedure as shorthands, with `\active@prefix`.

```

1727 \newcommand\babelnullhyphen{\char\hyphenchar\font}
1728 \def\babelhyphen{\active@prefix\babelhyphen\bbl@hyphen}
1729 \def\bbl@hyphen{%
1730   \@ifstar{\bbl@hyphen@i @}{\bbl@hyphen@i \@empty}}
1731 \def\bbl@hyphen@i#1#2{%
1732   \lowercase{\bbl@ifunset{\bbl@hy@#1#2\@empty}}%
1733   {\csname bbl@#1usehyphen\endcsname{\discretionary{#2}{}{#2}}}%
1734   {\lowercase{\csname bbl@hy@#1#2\@empty\endcsname}}}

```

The following two commands are used to wrap the “hyphen” and set the behavior of the rest of the word – the version with a single @ is used when further hyphenation is allowed, while that with @@ if no more hyphens are allowed. In both cases, if the hyphen is preceded by a positive space, breaking after the hyphen is disallowed.

There should not be a discretionary after a hyphen at the beginning of a word, so it is prevented if preceded by a skip. Unfortunately, this does handle cases like “(-suffix)”. `\nobreak` is always preceded by `\leavevmode`, in case the shorthand starts a paragraph.

```

1735 \def\bbl@usehyphen#1{%
1736   \leavevmode
1737   \ifdim\lastskip>\z@\mbox{#1}\else\nobreak#1\fi
1738   \nobreak\hskip\zap@skip}
1739 \def\bbl@@usehyphen#1{%
1740   \leavevmode\ifdim\lastskip>\z@\mbox{#1}\else#1\fi}

```

The following macro inserts the hyphen char.

```

1741 \def\bbl@hyphenchar{%
1742   \ifnum\hyphenchar\font=\m@ne
1743     \babelnullhyphen
1744   \else
1745     \char\hyphenchar\font
1746   \fi}

```

Finally, we define the hyphen “types”. Their names will not change, so you may use them in `\ldf`’s. After a space, the `\mbox` in `\bbl@hy@nobreak` is redundant.

```

1747 \def\bbl@hy@soft{\bbl@usehyphen{\discretionary{\bbl@hyphenchar}{}}{}}
1748 \def\bbl@hy@soft{\bbl@usehyphen{\discretionary{\bbl@hyphenchar}{}}{}}
1749 \def\bbl@hy@hard{\bbl@usehyphen\bbl@hyphenchar}
1750 \def\bbl@hy@hard{\bbl@usehyphen\bbl@hyphenchar}
1751 \def\bbl@hy@nobreak{\bbl@usehyphen{\mbox{\bbl@hyphenchar}}}
1752 \def\bbl@hy@nobreak{\mbox{\bbl@hyphenchar}}
1753 \def\bbl@hy@repeat{%
1754   \bbl@usehyphen{%
1755     \discretionary{\bbl@hyphenchar}{\bbl@hyphenchar}{\bbl@hyphenchar}}
1756 \def\bbl@hy@repeat{%
1757   \bbl@usehyphen{%
1758     \discretionary{\bbl@hyphenchar}{\bbl@hyphenchar}{\bbl@hyphenchar}}
1759 \def\bbl@hy@empty{\hskip\z@skip}
1760 \def\bbl@hy@empty{\discretionary{}{}{}}

```

\bbl@disc For some languages the macro `\bbl@disc` is used to ease the insertion of discretionaries for letters that behave ‘abnormally’ at a breakpoint.

```

1761 \def\bbl@disc#1#2{\nobreak\discretionary{#2-}{#1}\bbl@allowhyphens}

```

4.13 Multiencoding strings

The aim following commands is to provide a common interface for strings in several encodings. They also contains several hooks which can be used by `luatex` and `xetex`. The code is organized here with pseudo-guards, so we start with the basic commands.

Tools But first, a tool. It makes global a local variable. This is not the best solution, but it works.

```

1762 \bbl@trace{Multiencoding strings}
1763 \def\bbl@tglobal#1{\global\let#1#1}

```

The following option is currently no-op. It was meant for the deprecated `\SetCase`.

```

1764 ({*More package options}) ≡
1765 \DeclareOption{nocase}{}
1766 ({/More package options})

```

The following package options control the behavior of `\SetString`.

```

1767 ({*More package options}) ≡
1768 \let\bbl@opt@strings@nnil % accept strings=value
1769 \DeclareOption{strings}{\def\bbl@opt@strings{\BabelStringsDefault}}
1770 \DeclareOption{strings=encoded}{\let\bbl@opt@strings\relax}
1771 \def\BabelStringsDefault{generic}
1772 ({/More package options})

```

Main command This is the main command. With the first use it is redefined to omit the basic setup in subsequent blocks. We make sure strings contain actual letters in the range 128-255, not active characters.

```

1773 \@onlypreamble\StartBabelCommands
1774 \def\StartBabelCommands{%
1775   \begingroup
1776   \@tempcnta="7F
1777   \def\bbl@tempa{%
1778     \ifnum\@tempcnta>"FF\else
1779       \catcode\@tempcnta=11
1780       \advance\@tempcnta\@ne
1781       \expandafter\bbl@tempa
1782     \fi}%
1783   \bbl@tempa
1784   <@Macros local to BabelCommands@>
1785   \def\bbl@provstring##1##2{%
1786     \providecommand##1{##2}%
1787     \bbl@tglobal##1}%
1788   \global\let\bbl@scafter\@empty
1789   \let\StartBabelCommands\bbl@startcmds

```

```

1790 \ifx\BabelLanguages\relax
1791   \let\BabelLanguages\CurrentOption
1792 \fi
1793 \begingroup
1794 \let\bbl@screset\@nnil % local flag - disable 1st stopcommands
1795 \StartBabelCommands}
1796 \def\bbl@startcmds{%
1797   \ifx\bbl@screset\@nnil\else
1798     \bbl@usehooks{stopcommands}{}%
1799   \fi
1800 \endgroup
1801 \begingroup
1802 \@ifstar
1803   {\ifx\bbl@opt@strings\@nnil
1804     \let\bbl@opt@strings\BabelStringsDefault
1805     \fi
1806     \bbl@startcmds@i}%
1807   \bbl@startcmds@i}
1808 \def\bbl@startcmds@i#1#2{%
1809   \edef\bbl@L{\zap@space#1 \@empty}%
1810   \edef\bbl@G{\zap@space#2 \@empty}%
1811   \bbl@startcmds@ii}
1812 \let\bbl@startcommands\StartBabelCommands

```

Parse the encoding info to get the label, input, and font parts.

Select the behavior of \SetString. There are two main cases, depending of if there is an optional argument: without it and strings=encoded, strings are defined always; otherwise, they are set only if they are still undefined (i.e., fallback values). With labelled blocks and strings=encoded, define the strings, but with another value, define strings only if the current label or font encoding is the value of strings; otherwise (i.e., no strings or a block whose label is not in strings=) do nothing.

We presume the current block is not loaded, and therefore set (above) a couple of default values to gobble the arguments. Then, these macros are redefined if necessary according to several parameters.

```

1813 \newcommand\bbl@startcmds@ii[1][\@empty]{%
1814   \let\SetString\@gobbletwo
1815   \let\bbl@stringdef\@gobbletwo
1816   \let\AfterBabelCommands\@gobble
1817   \ifx\@empty#1%
1818     \def\bbl@sc@label{generic}%
1819     \def\bbl@encstring##1##2{%
1820       \ProvideTextCommandDefault##1{##2}%
1821       \bbl@tglobal##1%
1822       \expandafter\bbl@tglobal\csname\string?\string##1\endcsname}%
1823     \let\bbl@sctest\in@true
1824   \else
1825     \let\bbl@sc@charset\space % <- zapped below
1826     \let\bbl@sc@fontenc\space % <- " "
1827     \def\bbl@tempa##1=##2\@nil{%
1828       \bbl@csarg\edef{sc@zap@space##1 \@empty}{##2 }}%
1829     \bbl@vforeach{label=#1}{\bbl@tempa##1\@nil}%
1830     \def\bbl@tempa##1 ##2{% space -> comma
1831       ##1%
1832       \ifx\@empty##2\else\ifx,##1,\else,\fi\bbl@afterfi\bbl@tempa##2\fi}%
1833     \edef\bbl@sc@fontenc{\expandafter\bbl@tempa\bbl@sc@fontenc\@empty}%
1834     \edef\bbl@sc@label{\expandafter\zap@space\bbl@sc@label\@empty}%
1835     \edef\bbl@sc@charset{\expandafter\zap@space\bbl@sc@charset\@empty}%
1836     \def\bbl@encstring##1##2{%
1837       \bbl@foreach\bbl@sc@fontenc{%
1838         \bbl@ifunset{T@###1}%
1839         {}%
1840         {\ProvideTextCommand##1{####1}{##2}%
1841         \bbl@tglobal##1%
1842         \expandafter

```

```

1843         \bbl@toglobal\csname####1\string##1\endcsname}}}%
1844     \def\bbl@sctest{%
1845         \bbl@xin@{\,\bbl@opt@strings,}\{\,\bbl@sc@label,\bbl@sc@fontenc,}%
1846     \fi
1847     \ifx\bbl@opt@strings\@nnil           % i.e., no strings key -> defaults
1848     \else\ifx\bbl@opt@strings\relax      % i.e., strings=encoded
1849         \let\AfterBabelCommands\bbl@aftercmds
1850         \let\SetString\bbl@setstring
1851         \let\bbl@stringdef\bbl@encstring
1852     \else                               % i.e., strings=value
1853     \bbl@sctest
1854     \ifin@
1855         \let\AfterBabelCommands\bbl@aftercmds
1856         \let\SetString\bbl@setstring
1857         \let\bbl@stringdef\bbl@provstring
1858     \fi\fi\fi
1859     \bbl@scswitch
1860     \ifx\bbl@G\@empty
1861         \def\SetString##1##2{%
1862             \bbl@error{missing-group}{##1}{}}}%
1863     \fi
1864     \ifx\@empty#1%
1865         \bbl@usehooks{defaultcommands}{}%
1866     \else
1867         \@expandtwoargs
1868         \bbl@usehooks{encodedcommands}{\bbl@sc@charset}{\bbl@sc@fontenc}}}%
1869     \fi}

```

There are two versions of `\bbl@scswitch`. The first version is used when `ldfs` are read, and it makes sure `\(group)\(language)` is reset, but only once (`\bbl@screset` is used to keep track of this). The second version is used in the preamble and packages loaded after `babel` and does nothing.

The macro `\bbl@forlang` loops `\bbl@L` but its body is executed only if the value is in `\BabelLanguages` (inside `babel`) or `\date(language)` is defined (after `babel` has been loaded). There are also two version of `\bbl@forlang`. The first one skips the current iteration if the language is not in `\BabelLanguages` (used in `ldfs`), and the second one skips undefined languages (after `babel` has been loaded).

```

1870 \def\bbl@forlang#1#2{%
1871     \bbl@for#1\bbl@L{%
1872         \bbl@xin@{\,#1,}\{\,\BabelLanguages,}%
1873         \ifin@#2\relax\fi}}
1874 \def\bbl@scswitch{%
1875     \bbl@forlang\bbl@tempa{%
1876         \ifx\bbl@G\@empty\else
1877             \ifx\SetString\@gobbletwo\else
1878                 \edef\bbl@GL{\bbl@G\bbl@tempa}%
1879                 \bbl@xin@{\,\bbl@GL,}\{\,\bbl@screset,}%
1880             \ifin@else
1881                 \global\expandafter\let\csname\bbl@GL\endcsname\@undefined
1882                 \xdef\bbl@screset{\bbl@screset,\bbl@GL}%
1883             \fi
1884         \fi
1885     \fi}}
1886 \AtEndOfPackage{%
1887     \def\bbl@forlang#1#2{\bbl@for#1\bbl@L{\bbl@ifunset{date#1}{}}{#2}}}%
1888     \let\bbl@scswitch\relax}
1889 \@onlypreamble\EndBabelCommands
1890 \def\EndBabelCommands{%
1891     \bbl@usehooks{stopcommands}{}%
1892     \endgroup
1893     \endgroup
1894     \bbl@scafter}
1895 \let\bbl@endcommands\EndBabelCommands

```

Now we define commands to be used inside `\StartBabelCommands`.

Strings The following macro is the actual definition of `\SetString` when it is “active”

First save the “switcher”. Create it if undefined. Strings are defined only if undefined (i.e., like `\providescommand`). With the event `stringprocess` you can preprocess the string by manipulating the value of `\BabelString`. If there are several hooks assigned to this event, preprocessing is done in the same order as defined. Finally, the string is set.

```
1896 \def\bbl@setstring#1#2{% e.g., \prefacename{<string>}
1897   \bbl@forlang\bbl@tempa{%
1898     \edef\bbl@LC{\bbl@tempa\bbl@stripslash#1}%
1899     \bbl@ifunset{\bbl@LC}% e.g., \germanchaptername
1900       {\bbl@exp{%
1901         \global\bbbl@add\<\bbl@G\bbl@tempa>{\bbbl@scset\#1\<\bbl@LC>}}}%
1902       }%
1903   \def\BabelString{#2}%
1904   \bbl@usehooks{stringprocess}{}%
1905   \expandafter\bbl@stringdef
1906     \csname\bbl@LC\expandafter\endcsname\expandafter{\BabelString}}}
```

A little auxiliary command sets the string. Formerly used with casing. Very likely no longer necessary, although it’s used in `\setlocalecaption`.

```
1907 \def\bbl@scset#1#2{\def#1{#2}}
```

Define `\SetStringLoop`, which is actually set inside `\StartBabelCommands`. The current definition is somewhat complicated because we need a count, but `\count@` is not under our control (remember `\SetString` may call hooks). Instead of defining a dedicated count, we just “pre-expand” its value.

```
1908 (< *Macros local to BabelCommands >) ≡
1909 \def\SetStringLoop##1##2{%
1910   \def\bbl@templ####1{\expandafter\noexpand\csname##1\endcsname}%
1911   \count@\z@
1912   \bbl@loop\bbl@tempa{##2}{% empty items and spaces are ok
1913     \advance\count@\@ne
1914     \toks@\expandafter{\bbl@tempa}%
1915     \bbl@exp{%
1916       \\SetString\bbl@templ{\romannumeral\count@}{\the\toks@}%
1917       \count@=\the\count@relax}}}%
1918 (< /Macros local to BabelCommands >)
```

Delaying code Now the definition of `\AfterBabelCommands` when it is activated.

```
1919 \def\bbl@aftercmds#1{%
1920   \toks@\expandafter{\bbl@scafter#1}%
1921   \xdef\bbl@scafter{\the\toks@}}
```

Case mapping The command `\SetCase` is deprecated. Currently it consists in a definition with a hack just for backward compatibility in the macro mapping.

```
1922 (< *Macros local to BabelCommands >) ≡
1923   \newcommand\SetCase[3][]{%
1924     \def\bbl@tempa####1####2{%
1925       \ifx####1\@empty\else
1926         \bbl@carg\bbl@add{extras\CurrentOption}{%
1927           \bbl@carg\babel@save{c__text_uppercase\_string####1_tl}%
1928           \bbl@carg\def{c__text_uppercase\_string####1_tl}{####2}%
1929           \bbl@carg\babel@save{c__text_lowercase\_string####2_tl}%
1930           \bbl@carg\def{c__text_lowercase\_string####2_tl}{####1}}%
1931         \expandafter\bbl@tempa
1932       \fi}%
1933     \bbl@tempa##1\@empty\@empty
1934     \bbl@carg\bbl@tglobal{extras\CurrentOption}}%
1935 (< /Macros local to BabelCommands >)
```

Macros to deal with case mapping for hyphenation. To decide if the document is monolingual or multilingual, we make a rough guess – just see if there is a comma in the languages list, built in the first pass of the package options.

```
1936 (< *Macros local to BabelCommands >) ≡
```



```

1937 \newcommand\SetHyphenMap[1]{%
1938   \bbl@forlang\bbl@tempa{%
1939     \expandafter\bbl@stringdef
1940     \csname\bbl@tempa @bbl@hyphenmap\endcsname{##1}}}%
1941 (
```

There are 3 helper macros which do most of the work for you.

```

1942 \newcommand\BabelLower[2]{% one to one.
1943   \ifnum\lccode#1=#2\else
1944     \babel@savevariable{\lccode#1}%
1945     \lccode#1=#2\relax
1946   \fi}
1947 \newcommand\BabelLowerMM[4]{% many-to-many
1948   \@tempcnta=#1\relax
1949   \@tempcntb=#4\relax
1950   \def\bbl@tempa{%
1951     \ifnum\@tempcnta>#2\else
1952       \@expandtwoargs\BabelLower{\the\@tempcnta}{\the\@tempcntb}%
1953       \advance\@tempcnta#3\relax
1954       \advance\@tempcntb#3\relax
1955       \expandafter\bbl@tempa
1956     \fi}%
1957   \bbl@tempa}
1958 \newcommand\BabelLowerMO[4]{% many-to-one
1959   \@tempcnta=#1\relax
1960   \def\bbl@tempa{%
1961     \ifnum\@tempcnta>#2\else
1962       \@expandtwoargs\BabelLower{\the\@tempcnta}{#4}%
1963       \advance\@tempcnta#3
1964       \expandafter\bbl@tempa
1965     \fi}%
1966   \bbl@tempa}

```

The following package options control the behavior of hyphenation mapping.

```

1967 (
```

Initial setup to provide a default behavior if hyphenmap is not set.

```

1974 \AtEndOfPackage{%
1975   \ifx\bbl@opt@hyphenmap\undefined
1976     \bbl@xin@{,}{\bbl@language@opts}%
1977     \chardef\bbl@opt@hyphenmap\ifin@4\else\@ne\fi
1978   \fi}

```

4.14 Tailor captions

A general tool for resetting the caption names with a unique interface. With the old way, which mixes the switcher and the string, we convert it to the new one, which separates these two steps.

```

1979 \newcommand\setlocalecaption{%
1980   \@ifstar\bbl@setcaption@s\bbl@setcaption@x}
1981 \def\bbl@setcaption@x#1#2#3{% language caption-name string
1982   \bbl@trim@def\bbl@tempa{#2}%
1983   \bbl@xin@{.template}{\bbl@tempa}%
1984   \ifin@
1985     \bbl@ini@captions@template{#3}{#1}%
1986   \else
1987     \edef\bbl@tempd{%
1988       \expandafter\expandafter\expandafter

```

```

1989 \strip@prefix\expandafter\meaning\csname captions#1\endcsname}%
1990 \bbl@xin@
1991 {\expandafter\string\csname #2name\endcsname}%
1992 {\bbl@tempd}%
1993 \ifin@ % Renew caption
1994 \bbl@xin@{\string\bbl@scset}{\bbl@tempd}%
1995 \ifin@
1996 \bbl@exp{%
1997 \\\bbl@ifsamestring{\bbl@tempa}{\language}%
1998 {\\\bbl@scset\<#2name>\<#1#2name>}%
1999 {}}%
2000 \else % Old way converts to new way
2001 \bbl@ifunset{#1#2name}%
2002 {\bbl@exp{%
2003 \\\bbl@add\<captions#1>{\def\<#2name>{\<#1#2name>}}%
2004 \\\bbl@ifsamestring{\bbl@tempa}{\language}%
2005 {\def\<#2name>{\<#1#2name>}}%
2006 {}}}%
2007 {}%
2008 \fi
2009 \else
2010 \bbl@xin@{\string\bbl@scset}{\bbl@tempd}% New
2011 \ifin@ % New way
2012 \bbl@exp{%
2013 \\\bbl@add\<captions#1>{\\\bbl@scset\<#2name>\<#1#2name>}%
2014 \\\bbl@ifsamestring{\bbl@tempa}{\language}%
2015 {\\\bbl@scset\<#2name>\<#1#2name>}%
2016 {}}%
2017 \else % Old way, but defined in the new way
2018 \bbl@exp{%
2019 \\\bbl@add\<captions#1>{\def\<#2name>{\<#1#2name>}}%
2020 \\\bbl@ifsamestring{\bbl@tempa}{\language}%
2021 {\def\<#2name>{\<#1#2name>}}%
2022 {}}%
2023 \fi%
2024 \fi
2025 \@namedef{#1#2name}{#3}%
2026 \toks\expandafter{\bbl@captionslist}%
2027 \bbl@exp{\in@{\<#2name>}{\the\toks@}}%
2028 \ifin@else
2029 \bbl@exp{\\\bbl@add\\bbl@captionslist{\<#2name>}}%
2030 \bbl@tglobal\bbl@captionslist
2031 \fi
2032 \fi}

```

4.15 Making glyphs available

This section makes a number of glyphs available that either do not exist in the OT1 encoding and have to be ‘faked’, or that are not accessible through T1enc.def.

\set@low@box The following macro is used to lower quotes to the same level as the comma. It prepares its argument in box register 0.

```

2033 \bbl@trace{Macros related to glyphs}
2034 \def\set@low@box#1{\setbox\tw@\hbox{,}\setbox\z@\hbox{#1}%
2035 \dimen\z@\ht\z@ \advance\dimen\z@ -\ht\tw@%
2036 \setbox\z@\hbox{\lower\dimen\z@ \box\z@}\ht\z@\ht\tw@ \dp\z@\dp\tw@}

```

\save@sF@q The macro \save@sF@q is used to save and reset the current space factor.

```

2037 \def\save@sF@q#1{\leavevmode
2038 \begingroup
2039 \edef\@SF{\spacefactor\the\spacefactor}#1\@SF
2040 \endgroup}

```

4.15.1 Quotation marks

\quotedblbase In the T1 encoding the opening double quote at the baseline is available as a separate character, accessible via \quotedblbase. In the OT1 encoding it is not available, therefore we make it available by lowering the normal open quote character to the baseline.

```
2041 \ProvideTextCommand{\quotedblbase}{OT1}{%
2042   \save@sf@q{\set@low@box{\textquotedblright\}%
2043     \box\z@\kern-.04em\bbl@allowhyphens}}
```

Make sure that when an encoding other than OT1 or T1 is used this glyph can still be typeset.

```
2044 \ProvideTextCommandDefault{\quotedblbase}{%
2045   \UseTextSymbol{OT1}{\quotedblbase}}
```

\quotesinglbase We also need the single quote character at the baseline.

```
2046 \ProvideTextCommand{\quotesinglbase}{OT1}{%
2047   \save@sf@q{\set@low@box{\textquoteright\}%
2048     \box\z@\kern-.04em\bbl@allowhyphens}}
```

Make sure that when an encoding other than OT1 or T1 is used this glyph can still be typeset.

```
2049 \ProvideTextCommandDefault{\quotesinglbase}{%
2050   \UseTextSymbol{OT1}{\quotesinglbase}}
```

\guillemetleft

\guillemetright The guillemet characters are not available in OT1 encoding. They are faked. (Wrong names with o preserved for compatibility.)

```
2051 \ProvideTextCommand{\guillemetleft}{OT1}{%
2052   \ifmmode
2053     \ll
2054   \else
2055     \save@sf@q{\nobreak
2056       \raise.2ex\hbox{$\scriptscriptstyle\ll$}\bbl@allowhyphens}%
2057   \fi}
2058 \ProvideTextCommand{\guillemetright}{OT1}{%
2059   \ifmmode
2060     \gg
2061   \else
2062     \save@sf@q{\nobreak
2063       \raise.2ex\hbox{$\scriptscriptstyle\gg$}\bbl@allowhyphens}%
2064   \fi}
2065 \ProvideTextCommand{\guillemotleft}{OT1}{%
2066   \ifmmode
2067     \ll
2068   \else
2069     \save@sf@q{\nobreak
2070       \raise.2ex\hbox{$\scriptscriptstyle\ll$}\bbl@allowhyphens}%
2071   \fi}
2072 \ProvideTextCommand{\guillemotright}{OT1}{%
2073   \ifmmode
2074     \gg
2075   \else
2076     \save@sf@q{\nobreak
2077       \raise.2ex\hbox{$\scriptscriptstyle\gg$}\bbl@allowhyphens}%
2078   \fi}
```

Make sure that when an encoding other than OT1 or T1 is used these glyphs can still be typeset.

```
2079 \ProvideTextCommandDefault{\guillemetleft}{%
2080   \UseTextSymbol{OT1}{\guillemetleft}}
2081 \ProvideTextCommandDefault{\guillemetright}{%
2082   \UseTextSymbol{OT1}{\guillemetright}}
2083 \ProvideTextCommandDefault{\guillemotleft}{%
2084   \UseTextSymbol{OT1}{\guillemotleft}}
2085 \ProvideTextCommandDefault{\guillemotright}{%
2086   \UseTextSymbol{OT1}{\guillemotright}}
```

\guilsinglleft

\guilsinglright The single guillemets are not available in OT1 encoding. They are faked.

```
2087 \ProvideTextCommand{\guilsinglleft}{OT1}{%
2088   \ifmmode
2089     <%
2090   \else
2091     \save@sf@q{\nobreak
2092       \raise.2ex\hbox{$\scriptscriptstyle<$}\bbl@allowhyphens}%
2093   \fi}
2094 \ProvideTextCommand{\guilsinglright}{OT1}{%
2095   \ifmmode
2096     >%
2097   \else
2098     \save@sf@q{\nobreak
2099       \raise.2ex\hbox{$\scriptscriptstyle>$}\bbl@allowhyphens}%
2100   \fi}
```

Make sure that when an encoding other than OT1 or T1 is used these glyphs can still be typeset.

```
2101 \ProvideTextCommandDefault{\guilsinglleft}{%
2102   \UseTextSymbol{OT1}{\guilsinglleft}}
2103 \ProvideTextCommandDefault{\guilsinglright}{%
2104   \UseTextSymbol{OT1}{\guilsinglright}}
```

4.15.2 Letters

\ij

\IJ The dutch language uses the letter ‘ij’. It is available in T1 encoded fonts, but not in the OT1 encoded fonts. Therefore we fake it for the OT1 encoding.

```
2105 \DeclareTextCommand{\ij}{OT1}{%
2106   i\kern-0.02em\bbl@allowhyphens j}
2107 \DeclareTextCommand{\IJ}{OT1}{%
2108   I\kern-0.02em\bbl@allowhyphens J}
2109 \DeclareTextCommand{\ij}{T1}{\char188}
2110 \DeclareTextCommand{\IJ}{T1}{\char156}
```

Make sure that when an encoding other than OT1 or T1 is used these glyphs can still be typeset.

```
2111 \ProvideTextCommandDefault{\ij}{%
2112   \UseTextSymbol{OT1}{\ij}}
2113 \ProvideTextCommandDefault{\IJ}{%
2114   \UseTextSymbol{OT1}{\IJ}}
```

\dj

\DJ The croatian language needs the letters \dj and \DJ; they are available in the T1 encoding, but not in the OT1 encoding by default.

Some code to construct these glyphs for the OT1 encoding was made available to me by Stipčević Mario, (stipcevic@olimp.irb.hr).

```
2115 \def\crrtic@{\hrule height0.1ex width0.3em}
2116 \def\crrtic@{\hrule height0.1ex width0.33em}
2117 \def\ddj@{%
2118   \setbox0\hbox{d}\dimen@=\ht0
2119   \advance\dimen@lex
2120   \dimen@.45\dimen@
2121   \dimen@ii\expandafter\rem@pt\the\fontdimen\@ne\font\dimen@
2122   \advance\dimen@ii.5ex
2123   \leavevmode\rlap{\raise\dimen@\hbox{\kern\dimen@ii\vbox{\crrtic@}}}}
2124 \def\DDJ@{%
2125   \setbox0\hbox{D}\dimen@=.55\ht0
2126   \dimen@ii\expandafter\rem@pt\the\fontdimen\@ne\font\dimen@
2127   \advance\dimen@ii.15ex % correction for the dash position
2128   \advance\dimen@ii-.15\fontdimen7\font % correction for cmtt font
2129   \dimen\thr@@\expandafter\rem@pt\the\fontdimen7\font\dimen@
2130   \leavevmode\rlap{\raise\dimen@\hbox{\kern\dimen@ii\vbox{\crrtic@}}}}
```

```

2131 %
2132 \DeclareTextCommand{\dj}{OT1}{\ddj@ d}
2133 \DeclareTextCommand{\DJ}{OT1}{\DDJ@ D}

```

Make sure that when an encoding other than OT1 or T1 is used these glyphs can still be typeset.

```

2134 \ProvideTextCommandDefault{\dj}{%
2135   \UseTextSymbol{OT1}{\dj}}
2136 \ProvideTextCommandDefault{\DJ}{%
2137   \UseTextSymbol{OT1}{\DJ}}

```

ISS For the T1 encoding \SS is defined and selects a specific glyph from the font, but for other encodings it is not available. Therefore we make it available here.

```

2138 \DeclareTextCommand{\SS}{OT1}{\SS}
2139 \ProvideTextCommandDefault{\SS}{\UseTextSymbol{OT1}{\SS}}

```

4.15.3 Shorthands for quotation marks

Shorthands are provided for a number of different quotation marks, which make them usable both outside and inside mathmode. They are defined with \ProvideTextCommandDefault, but this is very likely not required because their definitions are based on encoding-dependent macros.

\glq

\grq The ‘german’ single quotes.

```

2140 \ProvideTextCommandDefault{\glq}{%
2141   \textormath{\quotesinglbase}{\mbox{\quotesinglbase}}}

```

The definition of \grq depends on the fontencoding. With T1 encoding no extra kerning is needed.

```

2142 \ProvideTextCommand{\grq}{T1}{%
2143   \textormath{\kern\z@\textquoteleft}{\mbox{\textquoteleft}}}
2144 \ProvideTextCommand{\grq}{TU}{%
2145   \textormath{\textquoteleft}{\mbox{\textquoteleft}}}
2146 \ProvideTextCommand{\grq}{OT1}{%
2147   \save@sf@q{\kern-.0125em
2148     \textormath{\textquoteleft}{\mbox{\textquoteleft}}}%
2149     \kern.07em\relax}}
2150 \ProvideTextCommandDefault{\grq}{\UseTextSymbol{OT1}\grq}

```

\glqq

\grqq The ‘german’ double quotes.

```

2151 \ProvideTextCommandDefault{\glqq}{%
2152   \textormath{\quotedblbase}{\mbox{\quotedblbase}}}

```

The definition of \grqq depends on the fontencoding. With T1 encoding no extra kerning is needed.

```

2153 \ProvideTextCommand{\grqq}{T1}{%
2154   \textormath{\textquotedblleft}{\mbox{\textquotedblleft}}}
2155 \ProvideTextCommand{\grqq}{TU}{%
2156   \textormath{\textquotedblleft}{\mbox{\textquotedblleft}}}
2157 \ProvideTextCommand{\grqq}{OT1}{%
2158   \save@sf@q{\kern-.07em
2159     \textormath{\textquotedblleft}{\mbox{\textquotedblleft}}}%
2160     \kern.07em\relax}}
2161 \ProvideTextCommandDefault{\grqq}{\UseTextSymbol{OT1}\grqq}

```

\flq

\frq The ‘french’ single guillemets.

```

2162 \ProvideTextCommandDefault{\flq}{%
2163   \textormath{\guilsinglleft}{\mbox{\guilsinglleft}}}
2164 \ProvideTextCommandDefault{\frq}{%
2165   \textormath{\guilsinglright}{\mbox{\guilsinglright}}}

```

\flqq

\frqq The ‘french’ double guillemets.

```
2166 \ProvideTextCommandDefault{\flqq}{%
2167   \textormath{\guillemetleft}{\mbox{\guillemetleft}}}
2168 \ProvideTextCommandDefault{\frqq}{%
2169   \textormath{\guillemetright}{\mbox{\guillemetright}}}
```

4.15.4 Umlauts and tremas

The command `\` needs to have a different effect for different languages. For German for instance, the ‘umlaut’ should be positioned lower than the default position for placing it over the letters a, o, u, A, O and U. When placed over an e, i, E or I it can retain its normal position. For Dutch the same glyph is always placed in the lower position.

\umlauthigh

\umlautlow To be able to provide both positions of `\` we provide two commands to switch the positioning, the default will be `\umlauthigh` (the normal positioning).

```
2170 \def\umlauthigh{%
2171   \def\bbl@umlauta##1{\leavevmode\bgroup%
2172     \accent\csname\fontencoding dqpos\endcsname
2173     ##1\bbl@allowhyphens\egroup}%
2174   \let\bbl@umlaute\bbl@umlauta}
2175 \def\umlautlow{%
2176   \def\bbl@umlauta{\protect\lower@umlaut}}
2177 \def\umlautelow{%
2178   \def\bbl@umlaute{\protect\lower@umlaut}}
2179 \umlauthigh
```

\lower@umlaut Used to position the `\` closer to the letter. We want the umlaut character lowered, nearer to the letter. To do this we need an extra *(dimen)* register.

```
2180 \expandafter\ifx\csname U@D\endcsname\relax
2181   \csname newdimen\endcsname U@D
2182 \fi
```

The following code fools TeX’s `make_accent` procedure about the current x-height of the font to force another placement of the umlaut character. First we have to save the current x-height of the font, because we’ll change this font dimension and this is always done globally.

Then we compute the new x-height in such a way that the umlaut character is lowered to the base character. The value of `.45ex` depends on the METAFONT parameters with which the fonts were built. (Just try out, which value will look best.) If the new x-height is too low, it is not changed. Finally we call the `\accent` primitive, reset the old x-height and insert the base character in the argument.

```
2183 \def\lower@umlaut#1{%
2184   \leavevmode\bgroup
2185   \U@D lex%
2186   {\setbox\z@\hbox{%
2187     \char\csname\fontencoding dqpos\endcsname}%
2188     \dimen@ -.45ex\advance\dimen@\ht\z@
2189     \ifdim lex<\dimen@ \fontdimen5\font\dimen@ \fi}%
2190   \accent\csname\fontencoding dqpos\endcsname
2191   \fontdimen5\font\U@D #1%
2192   \egroup}
```

For all vowels we declare `\` to be a composite command which uses `\bbl@umlauta` or `\bbl@umlaute` to position the umlaut character. We need to be sure that these definitions override the ones that are provided when the package `fontenc` with option `OT1` is used. Therefore these declarations are postponed until the beginning of the document. Note these definitions only apply to some languages, but `babel` sets them for *all* languages – you may want to redefine `\bbl@umlauta` and/or `\bbl@umlaute` for a language in the corresponding `ldf` (using the `babel` switching mechanism, of course).

```
2193 \AtBeginDocument{%
2194   \DeclareTextCompositeCommand{\}{OT1}{a}{\bbl@umlauta{a}}%
2195   \DeclareTextCompositeCommand{\}{OT1}{e}{\bbl@umlaute{e}}%
2196   \DeclareTextCompositeCommand{\}{OT1}{i}{\bbl@umlaute{i}}%
```

```

2197 \DeclareTextCompositeCommand{\}{OT1}{i}{\bbl@umlaut{i}}%
2198 \DeclareTextCompositeCommand{\}{OT1}{o}{\bbl@umlaut{o}}%
2199 \DeclareTextCompositeCommand{\}{OT1}{u}{\bbl@umlaut{u}}%
2200 \DeclareTextCompositeCommand{\}{OT1}{A}{\bbl@umlaut{A}}%
2201 \DeclareTextCompositeCommand{\}{OT1}{E}{\bbl@umlaut{E}}%
2202 \DeclareTextCompositeCommand{\}{OT1}{I}{\bbl@umlaut{I}}%
2203 \DeclareTextCompositeCommand{\}{OT1}{O}{\bbl@umlaut{O}}%
2204 \DeclareTextCompositeCommand{\}{OT1}{U}{\bbl@umlaut{U}}%

```

Finally, make sure the default hyphenrules are defined (even if empty). For internal use, another empty `\language` is defined. Currently used in Amharic.

```

2205 \ifx\l@english\@undefined
2206 \chardef\l@english\z@
2207 \fi
2208 % The following is used to cancel rules in ini files (see Amharic).
2209 \ifx\l@unhyphenated\@undefined
2210 \newlanguage\l@unhyphenated
2211 \fi

```

4.16 Layout

Layout is mainly intended to set bidi documents, but there is at least a tool useful in general.

```

2212 \bbl@trace{Bidi layout}
2213 \providecommand\IfBabelLayout[3]{#3}%

```

4.17 Load engine specific macros

Some macros are not defined in all engines, so, after loading the files define them if necessary to raise an error.

```

2214 \bbl@trace{Input engine specific macros}
2215 \ifcase\bbl@engine
2216 \input txtbabel.def
2217 \or
2218 \input luababel.def
2219 \or
2220 \input xebabel.def
2221 \fi
2222 \providecommand\babelfont{\bbl@error{only-lua-xe}}{}{}
2223 \providecommand\babelprehyphenation{\bbl@error{only-lua}}{}{}
2224 \ifx\babelposthyphenation\@undefined
2225 \let\babelposthyphenation\babelprehyphenation
2226 \let\babelpatterns\babelprehyphenation
2227 \let\babelcharproperty\babelprehyphenation
2228 \fi
2229 (package|core)

```

4.18 Creating and modifying languages

Continue with $\TeX$ only.

`\babelprovide` is a general purpose tool for creating and modifying languages. It creates the language infrastructure, and loads, if requested, an ini file. It may be used in conjunction to previously loaded ldf files.

```

2230 (*package)
2231 \bbl@trace{Creating languages and reading ini files}
2232 \let\bbl@extend@ini\@gobble
2233 \newcommand\babelprovide[2][]{%
2234 \let\bbl@savelangname\languagename
2235 \edef\bbl@savelocaleid{\the\localeid}%
2236 \global\let\bbl@afterload\@empty
2237 % Set name and locale id
2238 \edef\languagename{#2}%
2239 \bbl@id@assign

```

```

2240 % Initialize keys
2241 \bbl@vforeach{captions,date,import,main,script,language,%
2242     hyphenrules,linebreaking,justification,mapfont,maparabic,%
2243     mapdigits,intraspaces,intrapenalty,onchar,transforms,alph,%
2244     Alph,labels,labels*,mapdot,calendar,date,casing,interchar,%
2245     @import}%
2246     {\bbl@csarg\let{KVP@##1}\@nnil}%
2247 \global\let\bbl@release@transforms\@empty
2248 \global\let\bbl@release@casing\@empty
2249 \let\bbl@calendars\@empty
2250 \global\let\bbl@inidata\@empty
2251 \global\let\bbl@extend@ini\@gobble
2252 \global\let\bbl@included@inis\@empty
2253 \gdef\bbl@key@list{;}%
2254 \bbl@ifunset{bbl@passto@#2}%
2255     {\def\bbl@tempa{#1}}%
2256     {\bbl@exp{\def\\bbl@tempa{[\bbl@passto@#2],\unexpanded{#1}}}%
2257 \expandafter\bbl@forkv\expandafter{\bbl@tempa}{%
2258     \in@{/}{##1}% With /, (re)sets a value in the ini
2259     \ifin@
2260         \bbl@renewinikey##1\@{##2}%
2261     \else
2262         \bbl@csarg\ifx{KVP@##1}\@nnil\else
2263             \bbl@error{unknown-provide-key}{##1}{}}%
2264     \fi
2265     \bbl@csarg\def{KVP@##1}{##2}%
2266     \fi}%
2267 \chardef\bbl@howloaded=0: none; 1: ldf without ini; 2: ini
2268 \bbl@ifunset{date#2}\z@{\bbl@ifunset{bbl@llevel@#2}\@ne\tw@}%
2269 % == init ==
2270 \ifx\bbl@screset\undefined
2271     \bbl@ldfinit
2272 \fi
2273 % ==
2274 % If there is no import (last wins), use @import (internal, there
2275 % must be just one). To consider any order (because
2276 % \PassOptionsToLocale).
2277 \ifx\bbl@KVP@import\@nnil
2278     \let\bbl@KVP@import\bbl@KVP@@import
2279 \fi
2280 % == date (as option) ==
2281 % \ifx\bbl@KVP@date\@nnil\else
2282 % \fi
2283 % ==
2284 \let\bbl@lbkflag\relax % \@empty = do setup linebreak, only in 3 cases:
2285 \ifcase\bbl@howloaded
2286     \let\bbl@lbkflag\@empty % new
2287 \else
2288     \ifx\bbl@KVP@hyphenrules\@nnil\else
2289         \let\bbl@lbkflag\@empty
2290     \fi
2291     \ifx\bbl@KVP@import\@nnil\else
2292         \let\bbl@lbkflag\@empty
2293     \fi
2294 \fi
2295 % == import, captions ==
2296 \ifx\bbl@KVP@import\@nnil\else
2297     \bbl@exp{\\bbl@ifblank{\bbl@KVP@import}}%
2298     {\ifx\bbl@initoload\relax
2299         \begingroup
2300             \def\BabelBeforeIni##1##2{\gdef\bbl@KVP@import{##1}\endinput}%
2301             \bbl@input@texini{##2}%
2302         \endgroup

```



```

2303     \else
2304         \xdef\bbk@KVP@import{\bbk@initoload}%
2305     \fi}%
2306     {}%
2307     \let\bbk@KVP@date\@empty
2308 \fi
2309 \let\bbk@KVP@captions@\bbk@KVP@captions
2310 \ifx\bbk@KVP@captions\@nnil
2311     \let\bbk@KVP@captions\bbk@KVP@import
2312 \fi
2313 % ==
2314 \ifx\bbk@KVP@transforms\@nnil\else
2315     \bbk@replace\bbk@KVP@transforms{ }{,}%
2316 \fi
2317 % ==
2318 \ifx\bbk@KVP@mapdot\@nnil\else
2319     \def\bbk@tempa{\@empty}%
2320     \ifx\bbk@KVP@mapdot\bbk@tempa\else
2321         \bbk@exp{\gdef<\bbk@map@. @\language>{\bbk@KVP@mapdot}}}%
2322     \fi
2323 \fi
2324 % Load ini
2325 % -----
2326 \ifcase\bbk@howloaded
2327     \bbk@provide@new{#2}%
2328 \else
2329     \bbk@ifblank{#1}%
2330     {}% With \bbk@load@basic below
2331     {\bbk@provide@renew{#2}}}%
2332 \fi
2333 % Post tasks
2334 % -----
2335 % == subsequent calls after the first provide for a locale ==
2336 \ifx\bbk@inidata\@empty\else
2337     \bbk@extend@ini{#2}%
2338 \fi
2339 % == ensure captions ==
2340 \ifx\bbk@KVP@captions\@nnil\else
2341     \bbk@ifunset{\bbk@extracaps@#2}%
2342     {\bbk@exp{\babelensure[exclude=\\today]{#2}}}%
2343     {\bbk@exp{\babelensure[exclude=\\today,
2344         include=\bbk@extracaps@#2]}{#2}}}%
2345     \let\bbk@tempc\language
2346     \bbk@replace\bbk@tempc{-}{@}%
2347     \bbk@ifunset{\bbk@ensure@\bbk@tempc}%
2348     {\bbk@exp{%
2349         \\DeclareRobustCommand<\bbk@ensure@\bbk@tempc>[1]{%
2350             \\foreignlanguage{\language}%
2351             {###1}}}%
2352     {}%
2353     \bbk@exp{%
2354         \\bbk@tglobal<\bbk@ensure@\bbk@tempc>%
2355         \\bbk@tglobal<\bbk@ensure@\bbk@tempc\space>%
2356 \fi

```

At this point all parameters are defined if 'import'. Now we execute some code depending on them. But what about if nothing was imported? We just set the basic parameters, but still loading the whole ini file.

```

2357 \bbk@load@basic{#2}%
2358 % == script, language ==
2359 % Override the values from ini or defines them
2360 \ifx\bbk@KVP@script\@nnil\else
2361     \bbk@csarg\edef{sname@#2}{\bbk@KVP@script}%
2362 \fi

```

```

2363 \ifx\bbbl@KVP@language\@nnil\else
2364   \bbbl@csarg\edef{lname@#2}{\bbbl@KVP@language}%
2365 \fi
2366 \ifcase\bbbl@engine\or
2367   \bbbl@ifunset{\bbbl@chrng@\languagename}{}%
2368   {\directlua{
2369     Babel.set_chranges_b('\bbbl@cl{sbcpr}', '\bbbl@cl{chrng}') }}%
2370 \fi
2371 % == Line breaking: intraspace, intrapenalty ==
2372 % For CJK, East Asian, Southeast Asian, if interspace in ini
2373 \ifx\bbbl@KVP@intraspace\@nnil\else % We can override the ini or set
2374   \bbbl@csarg\edef{intsp@#2}{\bbbl@KVP@intraspace}%
2375 \fi
2376 \bbbl@provide@intraspace
2377 % == Line breaking: justification ==
2378 \ifx\bbbl@KVP@justification\@nnil\else
2379   \let\bbbl@KVP@linebreaking\bbbl@KVP@justification
2380 \fi
2381 \ifx\bbbl@KVP@linebreaking\@nnil\else
2382   \bbbl@xin@{,\bbbl@KVP@linebreaking,}%
2383   {,elongated,kashida,cjk,padding,unhyphenated,}%
2384   \ifin@
2385     \bbbl@csarg\xdef
2386       {lnbrk@\languagename}{\expandafter\@car\bbbl@KVP@linebreaking\@nil}%
2387   \fi
2388 \fi
2389 \bbbl@xin@{/e}{/\bbbl@cl{lnbrk}}%
2390 \ifin@else\bbbl@xin@{/k}{/\bbbl@cl{lnbrk}}\fi
2391 \ifin@\bbbl@arabicjust\fi
2392 \bbbl@xin@{/p}{/\bbbl@cl{lnbrk}}%
2393 \ifin@\AtBeginDocument{\@nameuse{\bbbl@tibetanjust}}\fi
2394 % == Line breaking: hyphenate.other.(locale|script) ==
2395 \ifx\bbbl@lbfkflag\@empty
2396   \bbbl@ifunset{\bbbl@hyotl@\languagename}{}%
2397   {\bbbl@csarg\bbbl@replace{hyotl@\languagename}{ }{,}%
2398     \bbbl@startcommands*\languagename}{}%
2399     \bbbl@csarg\bbbl@foreach{hyotl@\languagename}{%
2400       \ifcase\bbbl@engine
2401         \ifnum##1<257
2402           \SetHyphenMap{\BabelLower{##1}{##1}}%
2403         \fi
2404       \else
2405         \SetHyphenMap{\BabelLower{##1}{##1}}%
2406       \fi}%
2407     \bbbl@endcommands}%
2408   \bbbl@ifunset{\bbbl@hyots@\languagename}{}%
2409   {\bbbl@csarg\bbbl@replace{hyots@\languagename}{ }{,}%
2410     \bbbl@csarg\bbbl@foreach{hyots@\languagename}{%
2411       \ifcase\bbbl@engine
2412         \ifnum##1<257
2413           \global\lccode##1=##1\relax
2414         \fi
2415       \else
2416         \global\lccode##1=##1\relax
2417       \fi}}%
2418 \fi
2419 % == Counters: maparabic ==
2420 % Native digits, if provided in ini (TeX level, xe and lua)
2421 \ifcase\bbbl@engine\else
2422   \bbbl@ifunset{\bbbl@dgnat@\languagename}{}%
2423   {\expandafter\ifx\csname \bbbl@dgnat@\languagename\endcsname\@empty\else
2424     \expandafter\expandafter\expandafter
2425     \bbbl@setdigits\csname \bbbl@dgnat@\languagename\endcsname

```

```

2426 \ifx\bbbl@KVP@maparabic\@nnil\else
2427 \ifx\bbbl@latinarabic\@undefined
2428 \expandafter\let\expandafter\@arabic
2429 \csname bbl@counter@\language\endcsname
2430 \else % i.e., if layout=counters, which redefines \@arabic
2431 \expandafter\let\expandafter\bbbl@latinarabic
2432 \csname bbl@counter@\language\endcsname
2433 \fi
2434 \fi
2435 \fi}%
2436 \fi
2437 % == Counters: mapdigits ==
2438 % > luababel.def
2439 % == Counters: alph, Alph ==
2440 \ifx\bbbl@KVP@alph\@nnil\else
2441 \bbbl@exp{%
2442 \\\bbbl@add\<bbl@preextras@\language\>{%
2443 \\\babel@save\\\@alph
2444 \let\\\@alph\<bbl@cctr@\bbbl@KVP@alph @\language\>}}%
2445 \fi
2446 \ifx\bbbl@KVP@Alph\@nnil\else
2447 \bbbl@exp{%
2448 \\\bbbl@add\<bbl@preextras@\language\>{%
2449 \\\babel@save\\\@Alph
2450 \let\\\@Alph\<bbl@cctr@\bbbl@KVP@Alph @\language\>}}%
2451 \fi
2452 % == Counters: mapdot ==
2453 \ifx\bbbl@KVP@mapdot\@nnil\else
2454 \bbbl@foreach\bbbl@list@the{%
2455 \bbbl@ifunset{the##1}{}%
2456 {{\bbbl@ncarg\let\bbbl@tempd{the##1}%
2457 \bbbl@carg\bbbl@sreplace{the##1}{.}{\bbbl@map@lbl{.}}%
2458 \expandafter\ifx\csname the##1\endcsname\bbbl@tempd\else
2459 \bbbl@exp{\gdef\<the##1>{{\the##1}}}%
2460 \fi}}%
2461 \edef\bbbl@tempb{enumi,enumii,enumiii,enumiv}%
2462 \bbbl@foreach\bbbl@tempb{%
2463 \bbbl@ifunset{label##1}{}%
2464 {{\bbbl@ncarg\let\bbbl@tempd{label##1}%
2465 \bbbl@carg\bbbl@sreplace{label##1}{.}{\bbbl@map@lbl{.}}%
2466 \expandafter\ifx\csname label##1\endcsname\bbbl@tempd\else
2467 \bbbl@exp{\gdef\<label##1>{{\label##1}}}%
2468 \fi}}%
2469 \fi
2470 % == Casing ==
2471 \bbbl@release@casing
2472 \ifx\bbbl@KVP@casing\@nnil\else
2473 \bbbl@csarg\xdef{casing@\language}%
2474 {\@nameuse{bbl@casing@\language}\bbbl@maybextx\bbbl@KVP@casing}%
2475 \fi
2476 % == Calendars ==
2477 \ifx\bbbl@KVP@calendar\@nnil
2478 \edef\bbbl@KVP@calendar{\bbbl@cl{calpr}}%
2479 \fi
2480 \def\bbbl@tempe##1 ##2\@{@% Get first calendar
2481 \def\bbbl@tempa{##1}}%
2482 \bbbl@exp{\bbbl@tempe\bbbl@KVP@calendar\space\@}%
2483 \def\bbbl@tempe##1.##2.##3.##4\@{@%
2484 \def\bbbl@tempf{##3}% First
2485 \let\bbbl@tempc\@empty \let\bbbl@tempb\@empty
2486 \bbbl@cal@getconvert\bbbl@tempc{##1}%
2487 \bbbl@cal@getconvert\bbbl@tempb{##2}}%
2488 \expandafter\bbbl@tempe\bbbl@tempa...\@

```

```

2489 \bbl@csarg\edef{calpr@\language\name}{%
2490   \ifx\bbl@tempc\@empty\else
2491     calendar=\bbl@tempc
2492   \fi
2493   \ifx\bbl@tempb\@empty\else
2494     ,variant=\bbl@tempb
2495   \fi
2496   \ifx\bbl@tempf\@empty\else
2497     ,convert=\bbl@tempf
2498   \fi}%
2499 % == engine specific extensions ==
2500 % Defined in XXXbabel.def
2501 \bbl@provide@extra{#2}%
2502 % == require.babel in ini ==
2503 % To load or reload the babel-*.tex, if require.babel in ini
2504 \ifx\bbl@beforestart\relax\else % But not in doc aux or body
2505   \bbl@ifunset{bbl@rqtex@\language\name}{}%
2506     {\expandafter\ifx\csname bbl@rqtex@\language\name\endcsname\@empty\else
2507       \let\BabelBeforeIni\@gobbletwo
2508       \chardef\atcatcode=\catcode`\@
2509       \catcode`\@=11\relax
2510       \def\CurrentOption{#2}%
2511       \bbl@input@texini{\bbl@cs{rqtex@\language\name}}%
2512       \catcode`\@=\atcatcode
2513       \let\atcatcode\relax
2514       \global\bbl@csarg\let{rqtex@\language\name}\relax
2515     \fi}%
2516 \bbl@foreach\bbl@calendars{%
2517   \bbl@ifunset{bbl@ca##1}{%
2518     \chardef\atcatcode=\catcode`\@
2519     \catcode`\@=11\relax
2520     \InputIfFileExists{babel-ca-##1.tex}{}{}%
2521     \catcode`\@=\atcatcode
2522     \let\atcatcode\relax}%
2523   {}}%
2524 \fi
2525 % == frenchspacing ==
2526 \ifcase\bbl@howloaded\in@true\else\in@false\fi
2527 \ifin@else\bbl@xin@{typography/frenchspacing}{\bbl@key@list}\fi
2528 \ifin@
2529   \bbl@extras@wrap{\bbl@pre@fs}%
2530   {\bbl@pre@fs}%
2531   {\bbl@post@fs}%
2532 \fi
2533 % == transforms ==
2534 % > luababel.def
2535 \def\CurrentOption{#2}%
2536 \@nameuse{bbl@icsave@#2}%
2537 % == main ==
2538 \ifx\bbl@KVP@main\@nnil % Restore only if not 'main'
2539   \let\language\bbl@savelangname
2540   \chardef\localeid\bbl@savelocaleid\relax
2541 \fi
2542 % == ==
2543 \ifx\ldf@finish\@onlypreamble\else
2544   \bbl@afterload
2545 \fi
2546 % == hyphenrules (apply if current) ==
2547 \ifx\bbl@KVP@hyphenrules\@nnil\else
2548   \ifnum\bbl@savelocaleid=\localeid
2549     \language\@nameuse{l@\language\name}%
2550   \fi
2551 \fi}

```

Depending on whether or not the language exists (based on `\date<language>`), we define two macros. Remember `\bbl@startcommands` opens a group.

```

2552 \def\bbl@provide@new#1{%
2553   \namedef{date#1}{}% marks lang exists - required by \StartBabelCommands
2554   \namedef{extras#1}{}%
2555   \namedef{noextras#1}{}%
2556   \bbl@startcommands*{#1}{captions}%
2557   \ifx\bbl@KVP@captions\@nnil % and also if import, implicit
2558     \def\bbl@tempb##1{% elt for \bbl@captionslist
2559       \ifx##1\@nnil\else
2560         \bbl@exp{%
2561           \\SetString\\##1{%
2562             \\bbl@nocaption{\bbl@stripslash##1}{#1\bbl@stripslash##1}}}%
2563           \expandafter\bbl@tempb
2564         \fi}%
2565     \expandafter\bbl@tempb\bbl@captionslist\@nnil
2566   \else
2567     \ifx\bbl@initoload\relax
2568       \bbl@read@ini{\bbl@KVP@captions}2% % Here letters cat = 11
2569     \else
2570       \bbl@read@ini{\bbl@initoload}2% % Same
2571     \fi
2572   \fi
2573   \StartBabelCommands*{#1}{date}%
2574   \ifx\bbl@KVP@date\@nnil
2575     \bbl@exp{%
2576       \\SetString\\today{\\bbl@nocaption{today}{#1today}}}%
2577   \else
2578     \bbl@savetoday
2579     \bbl@savetoday
2580   \fi
2581   \bbl@endcommands
2582   \bbl@load@basic{#1}%
2583   % == hyphenmins == (only if new)
2584   \bbl@exp{%
2585     \gdef\<#1hyphenmins>{%
2586       {\bbl@ifunset{\bbl@lfthm#1}{2}{\bbl@cs{lfthm#1}}}%
2587       {\bbl@ifunset{\bbl@rgthm#1}{3}{\bbl@cs{rgthm#1}}}%
2588     % == hyphenrules (also in renew) ==
2589     \bbl@provide@hyphens{#1}%
2590     % == main ==
2591     \ifx\bbl@KVP@main\@nnil\else
2592       \expandafter\main@language\expandafter{#1}%
2593     \fi
2594   %
2595   \def\bbl@provide@renew#1{%
2596     \ifx\bbl@KVP@captions\@nnil\else
2597       \StartBabelCommands*{#1}{captions}%
2598       \bbl@read@ini{\bbl@KVP@captions}2% % Here all letters cat = 11
2599       \EndBabelCommands
2600     \fi
2601     \ifx\bbl@KVP@date\@nnil\else
2602       \StartBabelCommands*{#1}{date}%
2603       \bbl@savetoday
2604       \bbl@savetoday
2605     \EndBabelCommands
2606     \fi
2607     % == hyphenrules (also in new) ==
2608     \ifx\bbl@lbkflag\@empty
2609       \bbl@provide@hyphens{#1}%
2610     \fi
2611     % == main ==
2612     \ifx\bbl@KVP@main\@nnil\else

```

```

2613 \expandafter\main@language\expandafter{#1}%
2614 \fi}

```

Load the basic parameters (ids, typography, counters, and a few more), while captions and dates are left out. But it may happen some data has been loaded before automatically, so we first discard the saved values.

```

2615 \def\bbl@load@basic#1{%
2616 \ifcase\bbl@howloaded\or\or
2617 \ifcase\csname bbl@llevel@\language\endcsname
2618 \bbl@csarg\let{lname@\language}\relax
2619 \fi
2620 \fi
2621 \bbl@ifunset\bbl@lname@#1{%
2622 {\def\BabelBeforeIni##1##2{%
2623 \begingroup
2624 \let\bbl@ini@captions@aux\@gobbletwo
2625 \def\bbl@inidate ####1.####2.####3.####4\relax ####5####6}%
2626 \bbl@read@ini{##1}l%
2627 \ifx\bbl@initoload\relax\endinput\fi
2628 \endgroup}%
2629 \begingroup % boxed, to avoid extra spaces:
2630 \ifx\bbl@initoload\relax
2631 \bbl@input@texini{#1}%
2632 \else
2633 \setbox\z@\hbox{\BabelBeforeIni{\bbl@initoload}{}}%
2634 \fi
2635 \endgroup}%
2636 {}}}

```

The following ini reader ignores everything but the identification section. It is called when a font is defined (i.e., when the language is first selected) to know which script/language must be enabled. This means we must make sure a few characters are not active. The ini is not read directly, but with a proxy tex file named as the language (which means any code in it must be skipped, too).

```

2637 \def\bbl@load@info#1{%
2638 \def\BabelBeforeIni##1##2{%
2639 \begingroup
2640 \bbl@read@ini{##1}0%
2641 \endinput % babel- .tex may contain only preamble's
2642 \endgroup}% boxed, to avoid extra spaces:
2643 {\bbl@input@texini{#1}}}}

```

The hyphenrules option is handled with an auxiliary macro. This macro is called in three cases: when a language is first declared with \babelprovide, with hyphenrules and with import.

```

2644 \def\bbl@provide@hyphens#1{%
2645 \@tempcnta@m@ne % a flag
2646 \ifx\bbl@KVP@hyphenrules\@nnil\else
2647 \bbl@replace\bbl@KVP@hyphenrules{ }{,}%
2648 \bbl@foreach\bbl@KVP@hyphenrules{%
2649 \ifnum\@tempcnta=\m@ne % if not yet found
2650 \bbl@ifsamestring{##1}{+}%
2651 {\bbl@carg\addlanguage{l@##1}}%
2652 }%
2653 \bbl@ifunset{l@##1}% After a possible +
2654 {}%
2655 {\@tempcnta\@nameuse{l@##1}}%
2656 \fi}%
2657 \ifnum\@tempcnta=\m@ne
2658 \bbl@warning{%
2659 Requested 'hyphenrules' for '\language' not found:\\%
2660 \bbl@KVP@hyphenrules.\\%
2661 Using the default value. Reported}%
2662 \fi
2663 \fi
2664 \ifnum\@tempcnta=\m@ne % if no opt or no language in opt found

```

```

2665 \ifx\bbbl@KVP@captions@@\@nnil
2666 \bbbl@ifunset{bbbl@hyphr@#1}{}% use value in ini, if exists
2667 {\bbbl@exp{\bbbl@ifblank{\bbbl@cs{hyphr@#1}}}%
2668 {}%
2669 {\bbbl@ifunset{l@bbbl@cl{hyphr}}}%
2670 {}% if hyphenrules found:
2671 {\@tempcnta\@nameuse{l@bbbl@cl{hyphr}}}%
2672 \fi
2673 \fi
2674 \bbbl@ifunset{l@#1}%
2675 {\ifnum\@tempcnta=\m@ne
2676 \bbbl@carg\adddialect{l@#1}\language
2677 \else
2678 \bbbl@carg\adddialect{l@#1}\@tempcnta
2679 \fi}%
2680 {\ifnum\@tempcnta=\m@ne\else
2681 \global\bbbl@carg\chardef{l@#1}\@tempcnta
2682 \fi}}

```

The reader of babel-...tex files. We reset temporarily some catcodes (and make sure no space is accidentally inserted).

```

2683 \def\bbbl@input@texini#1{%
2684 \bbbl@bsphack
2685 \bbbl@exp{%
2686 \catcode`\\=14 \catcode`\\=0
2687 \catcode`\\=1 \catcode`\\=2
2688 \lowercase{\InputIfFileExists{babel-#1.tex}}}%
2689 \catcode`\\=the\catcode`\relax
2690 \catcode`\\=the\catcode`\relax
2691 \catcode`\\=the\catcode`\relax
2692 \catcode`\\=the\catcode`\relax}%
2693 \bbbl@esphack}

```

The following macros read and store ini files (but don't process them). For each line, there are 3 possible actions: ignore if starts with ;, switch section if starts with [, and store otherwise. There are used in the first step of \bbbl@read@ini.

```

2694 \def\bbbl@iniline#1\bbbl@iniline{%
2695 \ifnextchar[\bbbl@inisect{\ifnextchar;\bbbl@iniskip\bbbl@inistore}#1\@@}% ]
2696 \def\bbbl@inisect[#1]#2\@@{\def\bbbl@section{#1}}
2697 \def\bbbl@iniskip#1\@@{% if starts with ;
2698 \def\bbbl@inistore#1=#2\@@{% full (default)
2699 \bbbl@trim@def\bbbl@tempa{#1}%
2700 \bbbl@trim\toks@{#2}%
2701 \bbbl@ifsamestring{\bbbl@tempa}{\include}%
2702 {\bbbl@read@subini{\the\toks@}}%
2703 {\bbbl@xin@;\bbbl@section/\bbbl@tempa;}{\bbbl@key@list}%
2704 \ifin@
2705 \bbbl@xin@{,identification/include.}%
2706 {,\bbbl@section/\bbbl@tempa}%
2707 \ifin@\xdef\bbbl@included@inis{\the\toks@}\fi
2708 \bbbl@exp{%
2709 \\g@addto@macro\\bbbl@inidata{%
2710 \\bbbl@elt{\bbbl@section}{\bbbl@tempa}{\the\toks@}}}%
2711 \fi}}
2712 \def\bbbl@inistore@min#1=#2\@@{% minimal (maybe set in \bbbl@read@ini)
2713 \bbbl@trim@def\bbbl@tempa{#1}%
2714 \bbbl@trim\toks@{#2}%
2715 \bbbl@xin@{.identification.}{.\bbbl@section.}%
2716 \ifin@
2717 \bbbl@exp{\\g@addto@macro\\bbbl@inidata{%
2718 \\bbbl@elt{identification}{\bbbl@tempa}{\the\toks@}}}%
2719 \fi}

```

4.19 Main loop in ‘provide’

Now, the ‘main loop’, `\bbl@read@ini`, which ****must be executed inside a group****. At this point, `\bbl@inidata` may contain data declared in `\babelprovide`, with ‘slashed’ keys. There are 3 steps: first read the ini file and store it; then traverse the stored values, and process some groups if required (date, captions, labels, counters); finally, ‘export’ some values by defining global macros (identification, typography, characters, numbers). The second argument is 0 when called to read the minimal data for fonts; with `\babelprovide` it’s either 1 (without import) or 2 (which import). The value `−1` is used with `\DocumentMetadata`.

`\bbl@loop@ini` is the reader, line by line (1: stream), and calls `\bbl@iniline` to save the key/value pairs. If `\bbl@inistore` finds the `@include` directive, the input stream is switched temporarily and `\bbl@read@subini` is called.

When the language is being set based on the document metadata (#2 in `\bbl@read@ini` is `−1`), there is an interlude to get the name, after the data have been collected, and before it’s processed.

```
2720 \def\bbl@loop@ini#1{%
2721   \loop
2722     \if T\ifeof#1 F\fi T\relax % Trick, because inside \loop
2723     \endlinechar\m@ne
2724     \read#1 to \bbl@line
2725     \endlinechar\^^M
2726     \ifx\bbl@line\empty\else
2727       \expandafter\bbl@iniline\bbl@line\bbl@iniline
2728     \fi
2729   \repeat}
2730 %
2731 \def\bbl@read@subini#1{%
2732   \ifx\bbl@readsubstream\@undefined
2733     \csname newread\endcsname\bbl@readsubstream
2734   \fi
2735   \openin\bbl@readsubstream=babel-#1.ini
2736   \ifeof\bbl@readsubstream
2737     \bbl@error{no-ini-file}{#1}{}{}%
2738   \else
2739     {\bbl@loop@ini\bbl@readsubstream}%
2740   \fi
2741   \closein\bbl@readsubstream}
2742 %
2743 \ifx\bbl@readstream\@undefined
2744   \csname newread\endcsname\bbl@readstream
2745 \fi
2746 \def\bbl@read@ini#1#2{%
2747   \global\let\bbl@extend@ini\@gobble
2748   \openin\bbl@readstream=babel-#1.ini
2749   \ifeof\bbl@readstream
2750     \bbl@error{no-ini-file}{#1}{}{}%
2751   \else
2752     % == Store ini data in \bbl@inidata ==
2753     \catcode\ =10 \catcode\ =12
2754     \catcode\ [=12 \catcode\ ]=12 \catcode\ ==12 \catcode\ &=12
2755     \catcode\ ;=12 \catcode\ |=12 \catcode\ %=14 \catcode\ -=12
2756     \ifnum#2=\m@ne % Just for the info
2757       \edef\language{tag \bbl@metalang}%
2758     \fi
2759     \ifnum#2=\m@ne
2760       \bbl@info{Metadata: '\bbl@metalang' requested, '#1' provided.}%
2761       Fetching the locale name from babel-#1.ini\@gobble}%
2762     \else
2763       \bbl@info{Importing
2764         \ifcase#2font and identification \or basic \fi
2765         data for '\language'%
2766         from babel-#1.ini. Reported}%
2767     \fi
2768     \ifnum#2<\@ne
```



```

2769 \global\let\bbl@inidata\@empty
2770 \let\bbl@inistore\bbl@inistore@min % Remember it's local
2771 \fi
2772 \def\bbl@section{identification}%
2773 \bbl@exp{%
2774   \\bbl@inistore tag.ini=#1\\ \@
2775   \\bbl@inistore load.level=\ifnum#2<\@ne 0\else #2\fi\\ \@}%
2776 \bbl@loop@ini\bbl@readstream
2777 % == Process stored data ==
2778 \ifnum#2=\@ne
2779   \def\bbl@tempa##1 ##2\@{##1}% Get first name
2780   \def\bbl@elt##1##2##3{%
2781     \bbl@ifsamestring{identification/name.babel}{##1/##2}%
2782     {\edef\language\language\bbl@tempa##3 \@}%
2783     \let\localname\language
2784     \bbl@id@assign
2785     \def\bbl@elt###1###2###3{}}%
2786   {}}%
2787   \bbl@inidata
2788 \fi
2789 \bbl@csarg\xdef\l@ini@language{##1}%
2790 \bbl@read@ini@aux
2791 % == 'Export' data ==
2792 \bbl@ini@exports{##2}%
2793 \global\bbl@csarg\let{inidata@language}\bbl@inidata
2794 \global\let\bbl@inidata\@empty
2795 \bbl@xin@{,\language,},{,\bbl@ini@loaded,}%
2796 \ifin@
2797   \bbl@exp{\\bbl@add@list\\bbl@ini@loaded{language}}%
2798   \bbl@tglobal\bbl@ini@loaded
2799 \fi
2800 \@ifpackagewith{babel}{licr=unextended}{}%
2801   {\ifcase\bbl@engine\else % Find a better place
2802     \bbl@xin@{,\bbl@cl{sbc},},{,Cyr,}%
2803     \ifin@
2804       \bbl@once{licr-cyr}{\g@addto@macro\bbl@afterload{\input{cyr2uni.def}}}%
2805     \fi
2806   \fi}%
2807 \fi
2808 \closein\bbl@readstream}
2809 \def\bbl@read@ini@aux{%
2810   \let\bbl@savestrings\@empty
2811   \let\bbl@savetoday\@empty
2812   \let\bbl@savestate\@empty
2813   \def\bbl@elt##1##2##3{%
2814     \def\bbl@section{##1}%
2815     \in@{=date.}{=##1}% Find a better place
2816     \ifin@
2817       \bbl@ifunset{bbl@inikv@##1}%
2818       {\bbl@ini@calendar{##1}}%
2819     {}}%
2820   \fi
2821   \bbl@ifunset{bbl@inikv@##1}{}%
2822   {\csname bbl@inikv@##1\endcsname{##2}{##3}}%
2823   \bbl@inidata}

```

A variant to be used when the ini file has been already loaded, because it's not the first \babelprovide for this language.

```

2824 \def\bbl@extend@ini@aux#1{%
2825   \bbl@startcommands*{##1}{captions}%
2826   % Activate captions/... and modify exports
2827   \bbl@csarg\def{inikv@captions.licr}##1##2{%
2828     \setlocalecaption{##1}{##1}{##2}}%
2829   \def\bbl@inikv@captions##1##2{%

```

```

2830 \bbl@ini@captions@aux{##1}{##2}}%
2831 \def\bbl@stringdef##1##2{\gdef##1{##2}}%
2832 \def\bbl@exportkey##1##2##3{%
2833 \bbl@ifunset\bbl@kv@##2}{}%
2834 {\expandafter\ifx\csname bbl@kv@##2\endcsname\@empty\else
2835 \bbl@exp{\global\let<bbl@##1\language>\<bbl@kv@##2>}}%
2836 \fi}%
2837 % As with \bbl@read@ini, but with some changes
2838 \bbl@read@ini@aux
2839 \bbl@ini@exports\tw@
2840 % Update inidata@lang by pretending the ini is read.
2841 \def\bbl@elt##1##2##3{%
2842 \def\bbl@section{##1}%
2843 \bbl@iniline##2=##3\bbl@iniline}%
2844 \csname bbl@inidata@#1\endcsname
2845 \global\bbl@csarg\let{inidata@#1}\bbl@inidata
2846 \StartBabelCommands*{#1}{date}% And from the import stuff
2847 \def\bbl@stringdef##1##2{\gdef##1{##2}}%
2848 \bbl@savetoday
2849 \bbl@savestate
2850 \bbl@endcommands}

```

A somewhat hackish tool to handle calendar sections.

```

2851 \def\bbl@ini@calendar#1{%
2852 \lowercase{\def\bbl@tempa{=#1=}}%
2853 \bbl@replace\bbl@tempa{=date.gregorian}{}}%
2854 \bbl@replace\bbl@tempa{=date.}{}}%
2855 \in@{.licr=}{#1=}%
2856 \ifin@
2857 \ifcase\bbl@engine
2858 \bbl@replace\bbl@tempa{.licr=}{}}%
2859 \else
2860 \let\bbl@tempa\relax
2861 \fi
2862 \fi
2863 \ifx\bbl@tempa\relax\else
2864 \bbl@replace\bbl@tempa{=}{}}%
2865 \ifx\bbl@tempa\@empty\else
2866 \xdef\bbl@calendars{\bbl@calendars,\bbl@tempa}%
2867 \fi
2868 \bbl@exp{%
2869 \def<bbl@inikv@#1>####1####2{%
2870 \\\bbl@inidate####1...\relax{####2}{\bbl@tempa}}}%
2871 \fi}

```

A key with a slash in \babelprovide replaces the value in the ini file (which is ignored altogether). The mechanism is simple (but suboptimal): add the data to the ini one (at this point the ini file has not yet been read), and define a dummy macro. When the ini file is read, just skip the corresponding key and reset the macro (in \bbl@inistore above).

```

2872 \def\bbl@renewinikey#1/#2\@#3{%
2873 \global\let\bbl@extend@ini\bbl@extend@ini@aux
2874 \edef\bbl@tempa{\zap@space #1 \@empty}% section
2875 \edef\bbl@tempb{\zap@space #2 \@empty}% key
2876 \bbl@trim\toks@{#3}% value
2877 \bbl@exp{%
2878 \edef\\bbl@key@list{\bbl@key@list \bbl@tempa/\bbl@tempb;}%
2879 \\\g@addto@macro\\bbl@inidata{%
2880 \\\bbl@elt{\bbl@tempa}{\bbl@tempb}{\the\toks@}}}%

```

The previous assignments are local, so we need to export them. If the value is empty, we can provide a default value.

```

2881 \def\bbl@exportkey#1#2#3{%
2882 \bbl@ifunset\bbl@kv@##2}{%
2883 {\bbl@csarg\gdef{#1\language}{#3}}%

```

```

2884 {\expandafter\ifx\csname bbl@kv@#2\endcsname\@empty
2885 \bbl@csarg\gdef{#1@{\language}\language}\{#3}%
2886 \else
2887 \bbl@exp{\global\let<\bbl@#1@{\language}\language>\<\bbl@kv@#2>}%
2888 \fi}}

```

Key-value pairs are treated differently depending on the section in the ini file. The following macros are the readers for identification and typography. Note `\bbl@ini@exports` is called always (via `\bbl@inisec`), while `\bbl@after@ini` must be called explicitly after `\bbl@read@ini` if necessary.

Although BCP 47 doesn't treat 'x-' as an extension, the CLDR and many other sources do (as a *private use extension*). For consistency with other single-letter subtags or 'singletons', here is considered an extension, too.

The identification section is used internally by babel in the following places [to be completed]: BCP 47 script tag in the Unicode ranges, which is in turn used by `onchar`; the language system is set with the names, and then `fontspec` maps them to the `opentype` tags, but if the latter package doesn't define them, then babel does it; encodings are used in `pdftex` to select a font encoding valid (and preloaded) for a language loaded on the fly.

```

2889 \def\bbl@iniwarning#1{%
2890 \bbl@ifunset{\bbl@kv@identification.warning#1}{}%
2891 {\bbl@warning{%
2892 From babel-\bbl@cs{lini@{\language}\language}.ini:\\%
2893 \bbl@cs{@kv@identification.warning#1}\\%
2894 Reported}}}%
2895 %
2896 \let\bbl@release@transforms\@empty
2897 \let\bbl@release@casing\@empty

```

Relevant keys are 'exported', i.e., global macros with short names are created with values taken from the corresponding keys. The number of exported keys depends on the loading level (#1): -1 and 0 only info (the identification section), 1 also basic (like linebreaking or character ranges), 2 also (re)new (with date and captions).

```

2898 \def\bbl@ini@exports#1{%
2899 % Identification always exported
2900 \bbl@iniwarning{}%
2901 \ifcase\bbl@engine
2902 \bbl@iniwarning{.pdf\latex}%
2903 \or
2904 \bbl@iniwarning{.lua\latex}%
2905 \or
2906 \bbl@iniwarning{.xel\latex}%
2907 \fi%
2908 \bbl@exportkey{lllevel}{identification.load.level}{}%
2909 \bbl@exportkey{elname}{identification.name.english}{}%
2910 \bbl@exp{\bbl@exportkey{lname}{identification.name.opentype}%
2911 {\csname bbl@elname@{\language}\language\endcsname}}%
2912 \bbl@exportkey{tbc}{identification.tag.bcp47}{}%
2913 \bbl@exportkey{casing}{identification.tag.bcp47}{}%
2914 \bbl@exportkey{lbcp}{identification.language.tag.bcp47}{}%
2915 \bbl@exportkey{lotf}{identification.tag.opentype}{dflt}%
2916 \bbl@exportkey{esname}{identification.script.name}{}%
2917 \bbl@exp{\bbl@exportkey{sname}{identification.script.name.opentype}%
2918 {\csname bbl@esname@{\language}\language\endcsname}}%
2919 \bbl@exportkey{sbcp}{identification.script.tag.bcp47}{}%
2920 \bbl@exportkey{sotf}{identification.script.tag.opentype}{DFLT}%
2921 \bbl@exportkey{rbcp}{identification.region.tag.bcp47}{}%
2922 \bbl@exportkey{vbcp}{identification.variant.tag.bcp47}{}%
2923 \bbl@exportkey{extt}{identification.extension.t.tag.bcp47}{}%
2924 \bbl@exportkey{extu}{identification.extension.u.tag.bcp47}{}%
2925 \bbl@exportkey{extx}{identification.extension.x.tag.bcp47}{}%
2926 % Also maps bcp47 -> language
2927 \bbl@csarg\xdef{bcp@map@{\bbl@cl{tbc}}{\language}}%
2928 \ifcase\bbl@engine\or
2929 \directlua{%

```

```

2930      Babel.locale_props[\the\bbl@cs{id@@\languagename}].script
2931      = '\bbl@cl{sbc}'}%
2932 \fi
2933 % Conditional
2934 \ifnum#1>\z@      % -1 or 0 = only info, 1 = basic, 2 = (re)new
2935   \bbl@exportkey{calpr}{date.calendar.preferred}{}%
2936   \bbl@exportkey{lnbrk}{typography.linebreaking}{h}%
2937   \bbl@exportkey{hyphr}{typography.hyphenrules}{}%
2938   \bbl@exportkey{lfthm}{typography.lefthyphenmin}{2}%
2939   \bbl@exportkey{rgthm}{typography.righthyphenmin}{3}%
2940   \bbl@exportkey{prehc}{typography.prehyphenchar}{}%
2941   \bbl@exportkey{hyotl}{typography.hyphenate.other.locale}{}%
2942   \bbl@exportkey{hyots}{typography.hyphenate.other.script}{}%
2943   \bbl@exportkey{intsp}{typography.intraspaces}{}%
2944   \bbl@exportkey{frspc}{typography.frenchspacing}{u}%
2945   \bbl@exportkey{chrng}{characters.ranges}{}%
2946   \bbl@exportkey{quote}{characters.delimiters.quotes}{}%
2947   \bbl@exportkey{dgnat}{numbers.digits.native}{}%
2948   \ifnum#1=\tw@      % only (re)new
2949     \bbl@exportkey{rqtex}{identification.require.babel}{}%
2950     \bbl@tglobal\bbl@savetoday
2951     \bbl@tglobal\bbl@savestate
2952     \bbl@savestrings
2953 \fi
2954 \fi}

```

4.20 Processing keys in ini

A shared handler for key=val lines to be stored in \bbl@kv@<section>.<key>.

```

2955 \def\bbl@inikv#1#2{%      key=value
2956   \toks@{#2}%             This hides #'s from ini values
2957   \bbl@csarg\edef{@kv@\bbl@section.#1}{\the\toks@}}

```

By default, the following sections are just read. Actions are taken later.

```

2958 \let\bbl@inikv@identification\bbl@inikv
2959 \let\bbl@inikv@date\bbl@inikv
2960 \let\bbl@inikv@typography\bbl@inikv
2961 \let\bbl@inikv@numbers\bbl@inikv

```

The characters section also stores the values, but casing is treated in a different fashion. Much like transforms, a set of commands calling the parser are stored in \bbl@release@casing, which is executed in \babelprovide.

```

2962 \def\bbl@maybextx{-\bbl@csarg\ifx{extx@\languagename}\@empty x-\fi}
2963 \def\bbl@inikv@characters#1#2{%
2964   \bbl@ifsamestring{#1}{casing}% e.g., casing = uV
2965   {\bbl@exp{%
2966     \\g@addto@macro\\bbl@release@casing{%
2967       \\bbl@casemapping}{\languagename}{\unexpanded{#2}}}%
2968   {\in@{#1}{#2}% e.g., casing.uV = uV
2969   \ifin@
2970     \lowercase{\def\bbl@tempb{#1}}%
2971     \bbl@replace\bbl@tempb{casing.}{}%
2972     \bbl@exp{\\g@addto@macro\\bbl@release@casing{%
2973       \\bbl@casemapping
2974       {\\bbl@maybextx\bbl@tempb}{\languagename}{\unexpanded{#2}}}%
2975   \else
2976     \bbl@inikv{#1}{#2}%
2977   \fi}}

```

Additive numerals require an additional definition. When .1 is found, two macros are defined – the basic one, without .1 called by \localnumeral, and another one preserving the trailing .1 for the ‘units’. The asterisk is a ‘terminator’.

```

2978 \def\bbl@inikv@counters#1#2{%

```

```

2979 \bbl@ifsamestring{#1}{digits}%
2980   {\bbl@error{digits-is-reserved}}{}{}%
2981   {}%
2982 \def\bbl@tempc{#1}%
2983 \bbl@trim@def{\bbl@tempb*}{#2}%
2984 \in@{.1$}{#1$}%
2985 \ifin@
2986   \bbl@replace\bbl@tempc{.1}{}%
2987   \bbl@csarg\protected@xdef{cnt@#1\bbl@tempc @\language}%
2988   \noexpand\bbl@alphanumeric{\bbl@tempc}%
2989 \fi
2990 \in@{.F.}{#1}%
2991 \ifin@else\in@{.S.}{#1}\fi
2992 \ifin@
2993   \bbl@csarg\protected@xdef{cnt@#1\bbl@tempc @\language}%
2994   \else
2995     \toks@{}% Required by \bbl@buildifcase, which returns \bbl@tempa
2996     \expandafter\bbl@buildifcase\bbl@tempb* \ \ % Space after \
2997     \bbl@csarg\global\expandafter\let{\cnt@#1\bbl@tempc @\language}%
2998     \fi
2999     \in@{.D$}{#1$}%
3000     \ifin@else\in@{.C$}{#1$}\fi
3001     \ifin@
3002       \bbl@replace\bbl@tempc{.D}{}%
3003       \bbl@replace\bbl@tempc{.C}{}%
3004       \bbl@exp{\gdef\<bbl@cnt@#1\bbl@tempc @\language>###1%
3005         \ \ \expandafter\ \ \bbl@zhnumeral\ \ \expandafter{\ \ \number###1}{\bbl@tempc}}%
3006       \fi}

```

Now captions and captions.licr, depending on the engine. And below also for dates. They rely on a few auxiliary macros. It is expected the ini file provides the complete set in Unicode and LICR, in that order.

```

3007 \ifcase\bbl@engine
3008   \bbl@csarg\def{inikv@captions.licr}#1#2{%
3009     \bbl@ini@captions@aux{#1}{#2}}
3010 \else
3011   \def\bbl@inikv@captions#1#2{%
3012     \bbl@ini@captions@aux{#1}{#2}}
3013 \fi

```

The auxiliary macro for captions define \<caption>name.

```

3014 \def\bbl@ini@captions@template#1#2{% string language tempa=capt-name
3015   \bbl@replace\bbl@tempa{.template}{}%
3016   \def\bbl@toreplace{#1}{}%
3017   \bbl@replace\bbl@toreplace{[ ]}{\nobreakspace}}%
3018   \bbl@replace\bbl@toreplace{[ ]}{\csname}%
3019   \bbl@replace\bbl@toreplace{[ ]}{\csname the}%
3020   \bbl@replace\bbl@toreplace{[ ]}{\name\endcsname}}%
3021   \bbl@replace\bbl@toreplace{[ ]}{\endcsname}}%
3022   \bbl@xin@{,\bbl@tempa,}{,chapter,appendix,part,}%
3023   \ifin@
3024     \@nameuse{\bbl@patch\bbl@tempa}%
3025     \global\bbl@csarg\let{\bbl@tempa fmt@#2}\bbl@toreplace
3026   \fi
3027   \bbl@xin@{,\bbl@tempa,}{,figure,table,}%
3028   \ifin@
3029     \global\bbl@csarg\let{\bbl@tempa fmt@#2}\bbl@toreplace
3030     \bbl@exp{\gdef\<fnum\bbl@tempa>{%
3031       \ \ \bbl@ifunset{\bbl@tempa fmt@\ \ \language}%
3032       {\[fnum\bbl@tempa]}%
3033       {\ \ \@nameuse{\bbl@tempa fmt@\ \ \language}}}%
3034     \fi}
3035 %
3036 \def\bbl@ini@captions@aux#1#2{%

```

```

3037 \bbl@trim@def\bbl@tempa{#1}%
3038 \bbl@xin@{.template}\bbl@tempa}%
3039 \ifin@
3040 \bbl@ini@captions@template{#2}\language name
3041 \else
3042 \bbl@ifblank{#2}%
3043 {\bbl@exp{%
3044 \toks@{\bbl@nocaption\bbl@tempa name}\language name\bbl@tempa name}}}%
3045 {\bbl@trim\toks@{#2}}}%
3046 \bbl@exp{%
3047 \bbl@add\bbl@savestrings{%
3048 \SetString<\bbl@tempa name>\the\toks@}}}%
3049 \toks@expandafter\bbl@captionslist}%
3050 \bbl@exp{\in@<\bbl@tempa name>\the\toks@}}}%
3051 \ifin@else
3052 \bbl@exp{%
3053 \bbl@add<\bbl@extracaps\language name>\<\bbl@tempa name>%
3054 \bbl@tglobal<\bbl@extracaps\language name>%
3055 \fi
3056 \fi}

```

Labels. Captions must contain just strings, no format at all, so there is new group in ini files.

```

3057 \def\bbl@list@the{%
3058 part,chapter,section,subsection,subsubsection,paragraph,%
3059 subparagraph,enumi,enumii,enumiii,enumiv,equation,figure,%
3060 table,page,footnote,mpfootnote,mpfn}
3061 %
3062 \def\bbl@map@cnt#1{% #1:roman,etc, // #2:enumi,etc
3063 \bbl@ifunset\bbl@map@#1\language name}%
3064 {\@nameuse{#1}}}%
3065 {\@nameuse\bbl@map@#1\language name}}
3066 %
3067 \def\bbl@map@lbl#1{% #1:a sign, eg, .
3068 \ifincsname#1\else
3069 \bbl@ifunset\bbl@map@#1\language name}%
3070 {#1}%
3071 {\@nameuse\bbl@map@#1\language name}}}%
3072 \fi}
3073 %
3074 \def\bbl@inikv@labels#1#2{%
3075 \in@{.map}{#1}%
3076 \ifin@
3077 \in@{,dot.map},{,#1,}%
3078 \ifin@
3079 \global\@namedef\bbl@map@. @\language name}{#2}%
3080 \fi
3081 \ifx\bbl@KVP@labels\@nnil\else
3082 \bbl@xin@{ map }\bbl@KVP@labels\space}%
3083 \ifin@
3084 \def\bbl@tempc{#1}%
3085 \bbl@replace\bbl@tempc{.map}{}%
3086 \in@{,#2,},{,arabic,roman,Roman,alph,Alph,fnsymbol,}%
3087 \bbl@exp{%
3088 \gdef\<\bbl@map@\bbl@tempc @\language name>%
3089 {\ifin@<#2>\else\\loCALEcounter{#2}\fi}}}%
3090 \bbl@foreach\bbl@list@the{%
3091 \bbl@ifunset\the##1}{%
3092 {\bbl@ncarg\let\bbl@tempd\the##1}%
3093 \bbl@exp{%
3094 \bbl@sreplace\<\the##1>%
3095 {\<\bbl@tempc>{##1}}}%
3096 {\bbl@map@cnt\bbl@tempc}{##1}}}%
3097 \bbl@sreplace\<\the##1>%
3098 {\<\@empty @\bbl@tempc>\<c@##1>}%

```

```

3099         {\bbl@map@cnt{\bbl@tempc}{##1}}%
3100         \bbl@sreplace\<the##1>%
3101         {\csname @bbl@tempc\endcsname\<c@##1>}%
3102         {\bbl@map@cnt{\bbl@tempc}{##1}}}%
3103         \expandafter\ifx\csname the##1\endcsname\bbl@tempd\else
3104         \bbl@exp{\gdef\<the##1>{\the##1}}}%
3105         \fi}%
3106     \fi
3107 \fi
3108 %
3109 \else
3110 % The following code is still under study. You can test it and make
3111 % suggestions. E.g., enumerate.2 = ([enumi]).([enumii]). It's
3112 % language dependent.
3113 \in@{enumerate.}{#1}%
3114 \ifin@
3115     \def\bbl@tempa{#1}%
3116     \bbl@replace\bbl@tempa{enumerate.}{}%
3117     \def\bbl@toreplace{#2}%
3118     \bbl@replace\bbl@toreplace{[ ]}{\nobreakspace{}}%
3119     \bbl@replace\bbl@toreplace{[ ]}{\csname the}%
3120     \bbl@replace\bbl@toreplace{ ]}{\endcsname{}}}%
3121     \toks@\expandafter{\bbl@toreplace}%
3122     \bbl@exp{%
3123         \bbl@add\<extras\language>{%
3124             \babel@save\<labelenum\romannumeral\bbl@tempa>%
3125             \def\<labelenum\romannumeral\bbl@tempa>{\the\toks@}}%
3126         \bbl@tglobal\<extras\language>}%
3127     \fi
3128 \fi}

```

To show correctly some captions in a few languages, we need to patch some internal macros, because the order is hardcoded. For example, in Japanese the chapter number is surrounded by two string, while in Hungarian is placed after. These replacement works in many classes, but not all. Actually, the following lines are somewhat tentative.

```

3129 \def\bbl@chapttype{chapter}
3130 \ifx\@makechapterhead\undefined
3131     \let\bbl@patchchapter\relax
3132 \else\ifx\thechapter\undefined
3133     \let\bbl@patchchapter\relax
3134 \else\ifx\ps@headings\undefined
3135     \let\bbl@patchchapter\relax
3136 \else
3137     \def\bbl@patchchapter{%
3138         \global\let\bbl@patchchapter\relax
3139         \gdef\bbl@chfmt{%
3140             \bbl@ifunset{\bbl@chapttype fmt@\language}%
3141             {\@chapapp\space\thechapter}%
3142             {\@nameuse{\bbl@chapttype fmt@\language}}}%
3143         \bbl@add\appendix{\def\bbl@chapttype{appendix}}% Not harmful, I hope
3144         \bbl@sreplace\ps@headings{\@chapapp\ \thechapter}{\bbl@chfmt}%
3145         \bbl@sreplace\chaptermark{\@chapapp\ \thechapter}{\bbl@chfmt}%
3146         \bbl@sreplace\@makechapterhead{\@chapapp\space\thechapter}{\bbl@chfmt}%
3147         \bbl@tglobal\appendix
3148         \bbl@tglobal\ps@headings
3149         \bbl@tglobal\chaptermark
3150         \bbl@tglobal\@makechapterhead}
3151     \let\bbl@patchappendix\bbl@patchchapter
3152 \fi\fi\fi
3153 \ifx\@part\undefined
3154     \let\bbl@patchpart\relax
3155 \else
3156     \def\bbl@patchpart{%
3157         \global\let\bbl@patchpart\relax

```

```

3158 \gdef\bbl@partformat{%
3159 \bbl@ifunset\bbl@partfmt@\languagename}%
3160 {\partname\nobreakspace\thepart}%
3161 {\@nameuse\bbl@partfmt@\languagename}}}%
3162 \bbl@sreplace\@part{\partname\nobreakspace\thepart}{\bbl@partformat}%
3163 \bbl@tglobal\@part}
3164 \fi

Date. Arguments (year, month, day) are not protected, on purpose. In \today, arguments are
always gregorian, and therefore always converted with other calendars.

3165 \def\bbl@cal@getconvert#1#2{% Used in provide
3166 \in@{:}{#2}%
3167 \ifin@\def\bbl@tempf{#2}\else\def#1{#2}\fi}
3168 \let\bbl@calendar\@empty
3169 \DeclareRobustCommand\localedate[1][\bbl@localedate{#1}]
3170 \def\bbl@localedate#1#2#3#4{%
3171 \begingroup
3172 \edef\bbl@they{#2}%
3173 \edef\bbl@them{#3}%
3174 \edef\bbl@thed{#4}%
3175 \edef\bbl@tempe{%
3176 \bbl@ifunset\bbl@calpr@\languagename}{\bbl@cl{calpr}},%
3177 #1}%
3178 \bbl@exp{\lowercase{\edef\\bbl@tempe{\bbl@tempe}}}%
3179 \let\bbl@ld@calendar\@empty
3180 \let\bbl@ld@variant\@empty
3181 \let\bbl@ld@convert\relax
3182 \let\bbl@ld@@convert\relax
3183 \expandafter\bbl@forkv\expandafter{\bbl@tempe}{\bbl@csarg\edef{ld@##1}{##2}}%
3184 \bbl@replace\bbl@ld@calendar{gregorian}{}%
3185 \ifx\bbl@ld@convert\relax
3186 \let\bbl@ld@convert\bbl@ld@@convert
3187 \fi
3188 \ifx\bbl@ld@calendar\@empty\else
3189 \ifx\bbl@ld@convert\relax\else
3190 \ifx\bbl@ld@convert\@empty
3191 \let\bbl@ld@convert\bbl@ld@calendar
3192 \fi
3193 \babelcalendar[\bbl@they-\bbl@them-\bbl@thed]%
3194 {\bbl@ld@convert}\bbl@they\bbl@them\bbl@thed
3195 \fi
3196 \fi
3197 \@nameuse\bbl@precalendar}% Remove, e.g., +, -civil (-ca-islamic)
3198 \edef\bbl@calendar{% Used in \month..., too
3199 \bbl@ld@calendar
3200 \ifx\bbl@ld@variant\@empty\else
3201 .\bbl@ld@variant
3202 \fi}%
3203 \bbl@cased
3204 {\@nameuse\bbl@date@\languagename @\bbl@calendar}%
3205 \bbl@they\bbl@them\bbl@thed}%
3206 \endgroup}
3207 %
3208 \def\bbl@printdate#1{%
3209 \@ifnextchar[{\bbl@printdate@i{#1}}{\bbl@printdate@i{#1}[]}]
3210 \def\bbl@printdate@i#1[#2]#3#4#5{%
3211 \bbl@usedategroupttrue
3212 \def\bbl@tempc{#1}%
3213 \bbl@replace\bbl@tempc{-}{@}%
3214 \@nameuse\bbl@ensure@\bbl@tempc{\localedate[#2]{#3}{#4}{#5}}
3215 %
3216 e.g.: 1=months, 2=wide, 3=1, 4=dummy, 5=value, 6=calendar
3217 \def\bbl@inidate#1.#2.#3.#4\relax#5#6{%
3218 \bbl@trim\def\bbl@tempa{#1.#2}%

```



```

3219 \bbl@ifsamestring{\bbl@tempa}{months.wide}%      to savedate
3220 {\bbl@trim@def\bbl@tempa{#3}%
3221 \bbl@trim\toks@{#5}%
3222 \@temptokena\expandafter{\bbl@savestate}%
3223 \bbl@exp{% Reverse order - in ini last wins
3224 \def\\bbl@savestate{%
3225 \\SetString\<month\romannumeral\bbl@tempa#6name>{\the\toks@}%
3226 \the\@temptokena}}}%
3227 {\bbl@ifsamestring{\bbl@tempa}{date.long}%      defined now
3228 {\lowercase{\def\bbl@tempb{#6}}}%
3229 \bbl@trim@def\bbl@toreplace{#5}%
3230 \bbl@TG@@date
3231 \global\bbl@csarg\let{date@\language name @\bbl@tempb}\bbl@toreplace
3232 \ifx\bbl@savetoday\@empty
3233 \bbl@exp{%
3234 \\AfterBabelCommands{%
3235 \gdef\<\language name date>{\\protect\<\language name date >}%
3236 \gdef\<\language name date >{\\bbl@printdate{\language name}}}%
3237 \def\\bbl@savetoday{%
3238 \\SetString\\today{%
3239 \<\language name date>[@convert]%
3240 {\the\year}{\the\month}{\the\day}}}%
3241 \fi}%
3242 {}}}

```

Dates will require some macros for the basic formatting. They may be redefined by language, so “semi-public” names (camel case) are used. Oddly enough, the CLDR places particles like “de” inconsistently in either in the date or in the month name. Note after \bbl@replace \toks@ contains the resulting string, which is used by \bbl@replace@finish@iii (this implicit behavior doesn’t seem a good idea, but it’s efficient).

```

3243 \let\bbl@calendar\@empty
3244 \newcommand\babelcalendar[2][\the\year-\the\month-\the\day]{%
3245 \bbl@xin@{\$calendrica:}{\$#2}%
3246 \ifin@
3247 \directlua{Babel.calendrica = Babel.calendrica or require("calendrica")}%
3248 \edef\bbl@tempa{#2}%
3249 \bbl@replace\bbl@tempa{\$calendrica:}{}%
3250 \bbl@replace\bbl@tempa{-}{_}%
3251 \bbl@afterelse
3252 \bbl@exp{\\bbl@calendrica#1\\@@{\bbl@tempa}}%
3253 \else
3254 \bbl@afterfi
3255 \@nameuse{\bbl@ca@#2}#1\@@
3256 \fi}
3257 \def\bbl@calendrica#1-#2-#3\@@#4#5#6#7{%
3258 \directlua{
3259 local d = Babel.calendrica.new_date(
3260 {year=\number#1, month=\number#2, day=\number#3},
3261 Babel.calendrica.cal_gregorian)
3262 local r = Babel.calendrica.as_date(d, Babel.calendrica.cal_#4)
3263 token.set_macro('\bbl@stripslash#5', r.year)
3264 token.set_macro('\bbl@stripslash#6', r.month)
3265 token.set_macro('\bbl@stripslash#7', r.day) }}
3266 \newcommand\babelDateSpace{\nobreakspace}
3267 \newcommand\babelDateDot{.\@}
3268 \newcommand\babelDated[1][{\number#1}]
3269 \newcommand\babelDatedd[1][{\ifnum#1<10 0\fi\number#1}]
3270 \newcommand\babelDateM[1][{\number#1}]
3271 \newcommand\babelDateMM[1][{\ifnum#1<10 0\fi\number#1}]
3272 \newcommand\babelDateMMMM[1][{%
3273 \csname month\romannumeral#1\bbl@calendar name\endcsname}}%
3274 \newcommand\babelDatey[1][{\number#1}]%
3275 \newcommand\babelDateyy[1][{%
3276 \ifnum#1<10 0\number#1 %

```

```

3277 \else\ifnum#1<100 \number#1 %
3278 \else\ifnum#1<1000 \expandafter\@gobble\number#1 %
3279 \else\ifnum#1<10000 \expandafter\@gobbletwo\number#1 %
3280 \else
3281 \bbl@error{limit-two-digits}{\number#1}%
3282 \fi\fi\fi\fi}}
3283 \newcommand\BabelDateyyy[1]{\number#1}}
3284 \newcommand\BabelDateU[1]{\number#1}}%
3285 \def\bbl@replace@finish@iii#1{%
3286 \bbl@exp{\def\#1###1###2###3{\the\toks@}}%
3287 \def\bbl@TG@@date{%
3288 \bbl@replace\bbl@toreplace{[ ]}{\BabelDateSpace{}}}%
3289 \bbl@replace\bbl@toreplace{[.]}{\BabelDateDot{}}}%
3290 \bbl@replace\bbl@toreplace{[y]}{\BabelDatey{###1}}}%
3291 \bbl@replace\bbl@toreplace{[y]}{\bbl@datecctr{###1|}}%
3292 \bbl@replace\bbl@toreplace{[yy]}{\BabelDateyy{###1}}}%
3293 \bbl@replace\bbl@toreplace{[yyy]}{\BabelDateyyy{###1}}}%
3294 \bbl@replace\bbl@toreplace{[M]}{\BabelDateM{###2}}}%
3295 \bbl@replace\bbl@toreplace{[M]}{\bbl@datecctr{###2|}}%
3296 \bbl@replace\bbl@toreplace{[MM]}{\BabelDateMM{###2}}}%
3297 \bbl@replace\bbl@toreplace{[MMM]}{\BabelDateMMM{###2}}}%
3298 \bbl@replace\bbl@toreplace{[d]}{\BabelDated{###3}}}%
3299 \bbl@replace\bbl@toreplace{[d]}{\bbl@datecctr{###3|}}%
3300 \bbl@replace\bbl@toreplace{[dd]}{\BabelDatedd{###3}}}%
3301 \bbl@replace\bbl@toreplace{[U]}{\BabelDateU{###1}}}%
3302 \bbl@replace\bbl@toreplace{[U]}{\bbl@datecctr{###1|}}%
3303 \bbl@replace@finish@iii\bbl@toreplace}
3304 \def\bbl@datecctr{\expandafter\bbl@xdatecctr\expandafter}
3305 \def\bbl@xdatecctr[#1|#2]{\localenumeral{#2}{#1}}

```

4.21 French spacing (again)

For the following declarations, see issue #240. `\nonfrenchspacing` is set by document too early, so it's a hack.

```

3306 \AddToHook{begindocument/before}{%
3307 \let\bbl@normalsf\normalsfcodes
3308 \let\normalsfcodes\relax}
3309 \AtBeginDocument{%
3310 \ifx\bbl@normalsf\@empty
3311 \ifnum\scode`\.=\@m
3312 \let\normalsfcodes\frenchspacing
3313 \else
3314 \let\normalsfcodes\nonfrenchspacing
3315 \fi
3316 \else
3317 \let\normalsfcodes\bbl@normalsf
3318 \fi}

```

Transforms.

Process the transforms read from ini files, converts them to a form close to the user interface (with `\babelprehyphenation` and `\babelposthyphenation`), wrapped with `\bbl@transforms@aux` ...`\relax`, and stores them in `\bbl@release@transforms`. However, since building a list enclosed in braces isn't trivial, the replacements are added after a comma, and then `\bbl@transforms@aux` adds the braces.

```

3319 \bbl@csarg\let{inikv@transforms.prehyphenation}\bbl@inikv
3320 \bbl@csarg\let{inikv@transforms.posthyphenation}\bbl@inikv
3321 \def\bbl@transforms@aux#1#2#3#4,#5\relax{%
3322 #1[#2]{#3}{#4}{#5}}
3323 \begingroup
3324 \catcode`\%=12
3325 \catcode`\&=14
3326 \gdef\bbl@transforms#1#2#3{&%
3327 \directlua{

```

```

3328     local str = [==[#2]==]
3329     str = str:gsub('%.%d+%.%d+$', '')
3330     token.set_macro('babeltempa', str)
3331 }&%
3332 \def\babeltempc{}&%
3333 \bbl@xin@{\babeltempa,}{\bbl@KVP@transforms}&%
3334 \ifin@else
3335     \bbl@xin@{: \babeltempa,}{\bbl@KVP@transforms}&%
3336 \fi
3337 \ifin@
3338     \bbl@foreach\bbl@KVP@transforms{&%
3339         \bbl@xin@{: \babeltempa,}{##1}&%
3340         \ifin@ &% font:font:transform syntax
3341             \directlua{
3342                 local t = {}
3343                 for m in string.gmatch('##1'..' ':'(.)') do
3344                     table.insert(t, m)
3345                 end
3346                 table.remove(t)
3347                 token.set_macro('babeltempc', ', fonts=' .. table.concat(t, ' '))
3348             }&%
3349         \fi}&%
3350     \in@{.0$}{#2$}&%
3351     \ifin@
3352         \directlua{&% (\attribute) syntax
3353             local str = string.match([[ \bbl@KVP@transforms]],
3354                 '%([^(%[-])^(%))-\babeltempa')
3355             if str == nil then
3356                 token.set_macro('babeltempb', '')
3357             else
3358                 token.set_macro('babeltempb', ', attribute=' .. str)
3359             end
3360         }&%
3361     \toks@{#3}&%
3362     \bbl@exp{&%
3363         \\g@addto@macro\\bbl@release@transforms{&%
3364             \relax &% Closes previous \bbl@transforms@aux
3365             \\bbl@transforms@aux
3366             \#1{label=\babeltempa\babeltempb\babeltempc}&%
3367             {\languagename}{\the\toks@}}&%
3368     \else
3369         \g@addto@macro\bbl@release@transforms{, {#3}}&%
3370     \fi
3371 \fi}
3372 \endgroup

```

4.22 Handle language system

The language system (i.e., Language and Script) to be used when defining a font or setting the direction are set with the following macros. It also deals with unhyphenated line breaking in xetex (e.g., Thai and traditional Sanskrit), which is done with a hack at the font level because this engine doesn't support it.

```

3373 \def\bbl@provide@lsys#1{%
3374     \bbl@ifunset{bbl@lname@#1}%
3375     {\bbl@load@info{#1}}%
3376     {}%
3377     \bbl@csarg\let{lsys@#1}\@empty
3378     \bbl@ifunset{bbl@sname@#1}{\bbl@csarg\gdef{sname@#1}{Default}}{}%
3379     \bbl@ifunset{bbl@sotf@#1}{\bbl@csarg\gdef{sotf@#1}{DFLT}}{}%
3380     \bbl@csarg\bbl@add@list{lsys@#1}{Script=\bbl@cs{sname@#1}}%
3381     \bbl@ifunset{bbl@lname@#1}{}%
3382     {\bbl@csarg\bbl@add@list{lsys@#1}{Language=\bbl@cs{lname@#1}}}%
3383     \ifcase\bbl@engine\or\or

```

```

3384 \bbl@ifunset{\bbl@prehc@#1}{}%
3385 {\bbl@exp{\bbl@ifblank{\bbl@cs{prehc@#1}}}%
3386 }%
3387 {\ifx\bbl@xenohyph\undefined
3388 \global\let\bbl@xenohyph\bbl@xenohyph@d
3389 \ifx\AtBeginDocument\@notprerr
3390 \expandafter\@secondoftwo % to execute right now
3391 \fi
3392 \AtBeginDocument{%
3393 \bbl@patchfont{\bbl@xenohyph}%
3394 {\expandafter\select@language\expandafter{\language\name}}}%
3395 \fi}%
3396 \fi
3397 \bbl@csarg\bbl@toglobal{\sys@#1}}

```

4.23 Numerals

A tool to define the macros for native digits from the list provided in the `ini` file. Somewhat convoluted because there are 10 digits, but only 9 arguments in $\text{T}_{\text{E}}\text{X}$. Non-digits characters are kept. The first macro is the generic “localized” command.

```

3398 \def\bbl@setdigits#1#2#3#4#5{%
3399   \bbl@exp{%
3400     \def<\language name digits>####1%      i.e., \langdigits
3401       <bbl@digi ts@\language name>###1\\\nil}%
3402     \let<bbl@c ntr @digi ts@\language name>\<\language name digi ts>%
3403     \def<\language name counter>####1%      i.e., \langcounter
3404       \\expand after<bbl@count er@\language name>%
3405       \\csname c#####1\end csname}%
3406     \def<bbl@count er@\language name>####1% i.e., \bbl@count er@lang
3407       \\expand after<bbl@digi ts@\language name>%
3408       \\number####1\\\nil}}}%
3409 \def\bbl@tempa##1##2##3##4##5{%
3410   \bbl@exp{%    Wow, quite a lot of hashes! :-(
3411     \def<bbl@digi ts@\language name>#####1%
3412       \\ifx#####1\\\nil          % i.e., \bbl@digi ts@lang
3413       \\else
3414         \\ifx0#####1#1%
3415         \\else\\ifx1#####1#2%
3416         \\else\\ifx2#####1#3%
3417         \\else\\ifx3#####1#4%
3418         \\else\\ifx4#####1#5%
3419         \\else\\ifx5#####1##1%
3420         \\else\\ifx6#####1##2%
3421         \\else\\ifx7#####1##3%
3422         \\else\\ifx8#####1##4%
3423         \\else\\ifx9#####1##5%
3424         \\else#####1%
3425         \\fi\\fi\\fi\\fi\\fi\\fi\\fi\\fi\\fi\\fi\\fi\\fi\\fi\\fi\\fi\\fi
3426         \\expand after<bbl@digi ts@\language name>%
3427         \\fi}}} }%
3428   \bbl@tempa}

```

Alphabetic counters must be converted from a space separated list to an \ifcase structure.

```

3429 \def\bbl@buildifcase#1 {% Returns \bbl@tempa, requires \toks@={%
3430   \ifx\\#1%                % \\ before, in case #1 is multiletter
3431     \bbl@exp{%
3432       \def\\bbl@tempa####1{%
3433         \<ifcase>####1\space\the\toks@\<else>\\@ctrerr\<fi>}}%
3434   \else
3435     \toks@\expandafter{\the\toks@\or #1}%
3436     \expandafter\bbl@buildifcase
3437 \fi}

```

The code for additive counters is somewhat tricky and it's based on the fact the arguments just before @@ collects digits which have been left 'unused' in previous arguments, the first of them being the number of digits in the number to be converted. This explains the reverse set 76543210. Digits above 10000 are not handled yet. When the key contains the subkey .F., the number after is treated as a special case, for a fixed form (see babel-he.ini, for example).

```

3438 \newcommand\localenumerals[2]{%
3439   \bbl@ifunset{bbl@cntr@#1@\language}%
3440   {#2}%
3441   {\bbl@cs{cntr@#1@\language}{#2}}}
3442 \def\bbl@localecntr#1#2{\localenumerals{#2}{#1}}
3443 \newcommand\localecounter[2]{%
3444   \expandafter\bbl@localecntr
3445   \expandafter\number\csname c@#2\endcsname}{#1}}
3446 \def\bbl@alphnumerals#1#2{%
3447   \expandafter\bbl@alphnumerals@i\number#2 76543210\@@{#1}}
3448 \def\bbl@alphnumerals@i#1#2#3#4#5#6#7#8\@@#9{%
3449   \ifcase\@car#8\@nil\or % Currently <10000, but prepared for bigger
3450     \bbl@alphnumerals@ii{#9}000000#1\or
3451     \bbl@alphnumerals@ii{#9}000000#1#2\or
3452     \bbl@alphnumerals@ii{#9}000000#1#2#3\or
3453     \bbl@alphnumerals@ii{#9}000000#1#2#3#4\else
3454     \bbl@error{counter-too-large}{>9999}{}}%
3455   \fi}
3456 \def\bbl@alphnumerals@ii#1#2#3#4#5#6#7#8{%
3457   \bbl@ifunset{bbl@cntr@#1.F.\number#5#6#7#8@\language}%
3458   {\bbl@cs{cntr@#1.4@\language}#5%
3459    \bbl@cs{cntr@#1.3@\language}#6%
3460    \bbl@cs{cntr@#1.2@\language}#7%
3461    \bbl@cs{cntr@#1.1@\language}#8%
3462    \ifnum#6#7#8>\z@
3463      \bbl@ifunset{bbl@cntr@#1.S.321@\language}{}%
3464      {\bbl@cs{cntr@#1.S.321@\language}}%
3465    \fi}%
3466   {\bbl@cs{cntr@#1.F.\number#5#6#7#8@\language}}}
```

Some Chinese counters follow special rules.

```

3467 \def\bbl@zhnumerals#1#2{%
3468   \ifnum#1<10 \bbl@zhnum000#1{#2}%
3469   \else\ifnum#1<100 \bbl@zhnum00#1{#2}%
3470   \else\ifnum#1<1000 \bbl@zhnum0#1{#2}%
3471   \else\ifnum#1<10000 \bbl@zhnum#1{#2}%
3472   \else\bbl@error{counter-too-large}{#1}{}}%
3473   \fi\fi\fi\fi}
3474 \def\bbl@zhnum#1#2#3#4#5{%
3475   \ifnum\numexpr#1#2#3#4\relax=\z@
3476     \localenumerals{#5.C}\@ne
3477   \else
3478     \ifnum#1>\z@\localenumerals{#5.D}{#1}\localenumerals{#5.C}5\fi % Thousands
3479     \ifnum#2>\z@\localenumerals{#5.D}{#2}\localenumerals{#5.C}4\fi % Hundreds
3480     \else\ifnum#1>\z@\ifnum#3#4>\z@\localenumerals{#5.C}\@ne\fi
3481     \fi\fi
3482     \ifnum#3>\z@ % Tens
3483       \ifnum#1#2=\z@\ifnum#3=\@ne\else\localenumerals{#5.D}{#3}\fi % No 千 for 10-19:
3484       \else\localenumerals{#5.D}{#3}%
3485       \fi
3486       \localenumerals{#5.C}\thr@@
3487     \else
3488       \ifnum#2>\z@\ifnum#4>\z@\localenumerals{#5.C}\@ne\fi\fi
3489     \fi
3490     \ifnum#4>\z@\localenumerals{#5.D}{#4}\fi % Units
3491   \fi}
```

4.24 Casing

```
3492 \newcommand\BabelUppercaseMapping[3]{%
3493   \DeclareUppercaseMapping[\@nameuse{bbl@casing@#1}]{#2}{#3}}
3494 \newcommand\BabelTitlecaseMapping[3]{%
3495   \DeclareTitlecaseMapping[\@nameuse{bbl@casing@#1}]{#2}{#3}}
3496 \newcommand\BabelLowercaseMapping[3]{%
3497   \DeclareLowercaseMapping[\@nameuse{bbl@casing@#1}]{#2}{#3}}
```

The parser for casing and casing. (*variant*).

```
3498 \ifcase\bbl@engine % Converts utf8 to its code (expandable)
3499   \def\bbl@uftocode#1{\the\numexpr\decode@UTFviii#1\relax}
3500 \else
3501   \def\bbl@uftocode#1{\expandafter`\string#1}
3502 \fi
3503 \def\bbl@casemapping#1#2#3{% 1:variant
3504   \def\bbl@tempa##1 ##2{% Loop
3505     \bbl@casemapping@i{##1}%
3506     \ifx\@empty##2\else\bbl@afterfi\bbl@tempa##2\fi}%
3507   \edef\bbl@templ{\@nameuse{bbl@casing@#2}#1}% Language code
3508   \def\bbl@tempe{0}% Mode (upper/lower...)
3509   \def\bbl@tempc{#3}% Casing list
3510   \expandafter\bbl@tempa\bbl@tempc\@empty}
3511 \def\bbl@casemapping@i#1{%
3512   \def\bbl@tempb{#1}%
3513   \ifcase\bbl@engine % Handle utf8 in pdftex, by surrounding chars with {}
3514     \@nameuse{regex_replace_all:nnN}%
3515     {[{\x{c0}-\x{ff}}][{\x{80}-\x{bf}}]*}{\{\0}\}\bbl@tempb
3516   \else
3517     \@nameuse{regex_replace_all:nnN}{.}{\{\0}\}\bbl@tempb
3518   \fi
3519   \expandafter\bbl@casemapping@ii\bbl@tempb\@
3520 \def\bbl@casemapping@ii#1#2#3\@@{%
3521   \in@{#1#3}{<>}% i.e., if <u>, <l>, <t>
3522   \ifin@
3523     \edef\bbl@tempe{%
3524       \if#2u1 \else\if#2l2 \else\if#2t3 \fi\fi\fi}%
3525   \else
3526     \ifcase\bbl@tempe\relax
3527       \DeclareUppercaseMapping[\bbl@templ]{\bbl@uftocode{#1}}{#2}%
3528       \DeclareLowercaseMapping[\bbl@templ]{\bbl@uftocode{#2}}{#1}%
3529     \or
3530       \DeclareUppercaseMapping[\bbl@templ]{\bbl@uftocode{#1}}{#2}%
3531     \or
3532       \DeclareLowercaseMapping[\bbl@templ]{\bbl@uftocode{#1}}{#2}%
3533     \or
3534       \DeclareTitlecaseMapping[\bbl@templ]{\bbl@uftocode{#1}}{#2}%
3535   \fi
3536 \fi}
```

4.25 Getting info

The information in the identification section can be useful, so the following macro just exposes it with a user command.

```
3537 \def\bbl@localeinfo#1#2{%
3538   \bbl@ifunset{bbl@info@#2}{#1}%
3539   {\bbl@ifunset{bbl@csname bbl@info@#2\endcsname @\language}{#1}%
3540    {\bbl@cs{csname bbl@info@#2\endcsname @\language}}}%
3541 \newcommand\localeinfo[1]{%
3542   \ifx*#1\@empty
3543     \bbl@afterelse\bbl@localeinfo{}%
3544   \else
3545     \bbl@localeinfo
3546     {\bbl@error{no-ini-info}}{\}\}\}%}
```

```

3547      {#1}%
3548    \fi}
3549 % \@namedef{bbl@info@name.locale}{lcname}
3550 \@namedef{bbl@info@tag.ini}{lini}
3551 \@namedef{bbl@info@name.english}{elname}
3552 \@namedef{bbl@info@name.opentype}{lname}
3553 \@namedef{bbl@info@tag.bcp47}{tbc}
3554 \@namedef{bbl@info@language.tag.bcp47}{lbc}
3555 \@namedef{bbl@info@tag.opentype}{lotf}
3556 \@namedef{bbl@info@script.name}{esname}
3557 \@namedef{bbl@info@script.name.opentype}{sname}
3558 \@namedef{bbl@info@script.tag.bcp47}{sbc}
3559 \@namedef{bbl@info@script.tag.opentype}{sotf}
3560 \@namedef{bbl@info@region.tag.bcp47}{rbcp}
3561 \@namedef{bbl@info@variant.tag.bcp47}{vbc}
3562 \@namedef{bbl@info@extension.t.tag.bcp47}{extt}
3563 \@namedef{bbl@info@extension.u.tag.bcp47}{extu}
3564 \@namedef{bbl@info@extension.x.tag.bcp47}{extx}

```

With version 3.75 \BabelEnsureInfo is executed always, but there is an option to disable it. Since the info in ini files are always loaded, it has been made no-op in version 25.8.

```

3565 ((*More package options)) ≡
3566 \DeclareOption{ensureinfo=off}{}
3567 ((/More package options))
3568 \let\BabelEnsureInfo\relax

```

More general, but non-expandable, is \getLocaleproperty.

```

3569 \newcommand\getLocaleproperty{%
3570   \ifstar\bbl@getproperty@s\bbl@getproperty@x}
3571 \def\bbl@getproperty@s#1#2#3{%
3572   \let#1\relax
3573   \def\bbl@elt##1##2##3{%
3574     \bbl@ifsamestring{##1/##2}{#3}%
3575     {\providecommand#1{##3}%
3576     \def\bbl@elt###1####2####3{}}}%
3577   {}}%
3578   \bbl@cs{inidata@#2}}%
3579 \def\bbl@getproperty@x#1#2#3{%
3580   \bbl@getproperty@s{#1}{#2}{#3}%
3581   \ifx#1\relax
3582     \bbl@error{unknown-locale-key}{#1}{#2}{#3}%
3583   \fi}

```

To inspect every possible loaded ini, we define \LocaleForEach, where \bbl@ini@loaded is a comma-separated list of locales, built by \bbl@read@ini.

```

3584 \let\bbl@ini@loaded\@empty
3585 \newcommand\LocaleForEach{\bbl@foreach\bbl@ini@loaded}
3586 \def\ShowLocaleProperties#1{%
3587   \typeout{#1}%
3588   \typeout{*** Properties for language '#1' ***}%
3589   \def\bbl@elt##1##2##3{\typeout{##1/##2 = \unexpanded{##3}}}%
3590   \@nameuse{bbl@inidata@#1}%
3591   \typeout{*****}}

```

4.26 BCP 47 related commands

This macro is called by language selectors when the language isn't recognized. So, it's the core for (1) mapping from a BCP 47 tag to the actual language, if bcp47.toname is enabled (i.e., if bbl@bcp47toname is true), and (2) lazy loading. With autoload.bcp47 enabled *and* lazy loading, we must first build a name for the language, with the help of autoload.bcp47.prefix. Then we use \provideprovide passing the options set with autoload.bcp47.options (by default import). Finally, and if the locale has not been loaded before, we use \provideprovide with the language name as passed to the selector.

```

3592 \newif\ifbbl@bcpallowed
3593 \bbl@bcpallowedfalse
3594 \def\bbl@autoload@options{@import}
3595 \def\bbl@provide@locale{%
3596   \ifx\babelprovide\undefined
3597     \bbl@error{base-on-the-fly}{}}}%
3598 \fi
3599 \let\bbl@auxname\language
3600 \ifbbl@bcptoname
3601   \bbl@ifunset{bbl@bcp@map@\language}{}% Move uplevel??
3602   {\edef\language{\@nameuse{bbl@bcp@map@\language}}}%
3603   \let\locale\language}%
3604 \fi
3605 \ifbbl@bcpallowed
3606   \expandafter\ifx\csname date\language\endcsname\relax
3607     \expandafter
3608     \bbl@bcplookup{\language}%-\@empty-\@empty-\@empty\@@
3609     \ifx\bbl@bcp\relax\else % Returned by \bbl@bcplookup
3610       \edef\language{\bbl@bcp@prefix\bbl@bcp}%
3611       \let\locale\language
3612       \expandafter\ifx\csname date\language\endcsname\relax
3613         \let\bbl@initload\bbl@bcp
3614         \bbl@exp{\babelprovide[\bbl@autoload@bcptoptions]{\language}}%
3615         \let\bbl@initload\relax
3616       \fi
3617       \bbl@csarg\xdef{bcp@map@\bbl@bcp}{\locale}%
3618     \fi
3619   \fi
3620 \fi
3621 \expandafter\ifx\csname date\language\endcsname\relax
3622   \IfFileExists{babel-\language.tex}%
3623   {\bbl@exp{\babelprovide[\bbl@autoload@options]{\language}}}%
3624   {}%
3625 \fi}

```

$\LaTeX$ needs to know the BCP 47 codes for some features. For that, it expects `\BCPdata` to be defined. While language, region, script, and variant are recognized, extension.`<s>` for singletons may change.

Still somewhat hackish. Note `\str_if_eq:nnTF` is fully expandable (`\bbl@ifsamestring` isn't). The argument is the prefix to `tag.bcp47`.

```

3626 \providecommand\BCPdata{}
3627 \ifx\renewcommand\undefined\else
3628   \renewcommand\BCPdata[1]{\bbl@bcpdata@i#1\@empty\@empty\@empty}
3629   \def\bbl@bcpdata@i#1#2#3#4#5#6\@empty{%
3630     \@nameuse{str_if_eq:nnTF}{#1#2#3#4#5}{main.}%
3631     {\bbl@bcpdata@ii{#6}\bbl@main@language}%
3632     {\bbl@bcpdata@ii{#1#2#3#4#5#6}\language}}%
3633   \def\bbl@bcpdata@ii#1#2{%
3634     \bbl@ifunset{bbl@info@#1.tag.bcp47}%
3635     {\bbl@error{unknown-ini-field}{#1}{}}}%
3636     {\bbl@ifunset{bbl@\csname bbl@info@#1.tag.bcp47\endcsname @#2}{}%
3637     {\bbl@cs{\csname bbl@info@#1.tag.bcp47\endcsname @#2}}}%
3638 \fi
3639 \@namedef{bbl@info@casing.tag.bcp47}{casing}
3640 \@namedef{bbl@info@tag.tag.bcp47}{tbc} % For \BCPdata

```

5 Adjusting the Babel behavior

A generic high level interface is provided to adjust some global and general settings.

```

3641 \newcommand\babeladjust[1]{%
3642   \bbl@forkv{#1}{%
3643     \bbl@ifunset{bbl@ADJ@##1@##2}%

```



```

3644      {\bbl@cs{ADJ@##1}{##2}}%
3645      {\bbl@cs{ADJ@##1@##2}}}%
3646 %
3647 \def\bbl@adjust@lua#1#2{%
3648   \ifvmode
3649     \ifnum\currentgrouplevel=\z@
3650       \directlua{ Babel.#2 }%
3651       \expandafter\expandafter\expandafter\@gobble
3652     \fi
3653   \fi
3654   {\bbl@error{adjust-only-vertical}{#1}{}}}% Gobbled if everything went ok.
3655 \@namedef{bbl@ADJ@bidi.mirroring@on}{%
3656   \bbl@adjust@lua{bidi}{mirroring_enabled=true}}
3657 \@namedef{bbl@ADJ@bidi.mirroring@off}{%
3658   \bbl@adjust@lua{bidi}{mirroring_enabled=false}}
3659 \@namedef{bbl@ADJ@bidi.text@on}{%
3660   \bbl@adjust@lua{bidi}{bidi_enabled=true}}
3661 \@namedef{bbl@ADJ@bidi.text@off}{%
3662   \bbl@adjust@lua{bidi}{bidi_enabled=false}}
3663 \@namedef{bbl@ADJ@bidi.math@on}{%
3664   \let\bbl@noamsmath\empty}
3665 \@namedef{bbl@ADJ@bidi.math@off}{%
3666   \let\bbl@noamsmath\relax}
3667 %
3668 \@namedef{bbl@ADJ@bidi.mapdigits@on}{%
3669   \bbl@adjust@lua{bidi}{digits_mapped=true}}
3670 \@namedef{bbl@ADJ@bidi.mapdigits@off}{%
3671   \bbl@adjust@lua{bidi}{digits_mapped=false}}
3672 %
3673 \@namedef{bbl@ADJ@linebreak.sea@on}{%
3674   \bbl@adjust@lua{linebreak}{sea_enabled=true}}
3675 \@namedef{bbl@ADJ@linebreak.sea@off}{%
3676   \bbl@adjust@lua{linebreak}{sea_enabled=false}}
3677 \@namedef{bbl@ADJ@linebreak.cjk@on}{%
3678   \bbl@adjust@lua{linebreak}{cjk_enabled=true}}
3679 \@namedef{bbl@ADJ@linebreak.cjk@off}{%
3680   \bbl@adjust@lua{linebreak}{cjk_enabled=false}}
3681 \@namedef{bbl@ADJ@justify.arabic@on}{%
3682   \bbl@adjust@lua{linebreak}{arabic.justify_enabled=true}}
3683 \@namedef{bbl@ADJ@justify.arabic@off}{%
3684   \bbl@adjust@lua{linebreak}{arabic.justify_enabled=false}}
3685 %
3686 \def\bbl@adjust@layout#1{%
3687   \ifvmode
3688     #1%
3689     \expandafter\@gobble
3690   \fi
3691   {\bbl@error{layout-only-vertical}{}}}% Gobbled if everything went ok.
3692 \@namedef{bbl@ADJ@layout.tabular@on}{%
3693   \ifnum\bbl@tabular@mode=\tw@
3694     \bbl@adjust@layout{\let\@tabular\bbl@NL@tabular}%
3695   \else
3696     \chardef\bbl@tabular@mode\@ne
3697   \fi}
3698 \@namedef{bbl@ADJ@layout.tabular@off}{%
3699   \ifnum\bbl@tabular@mode=\tw@
3700     \bbl@adjust@layout{\let\@tabular\bbl@OL@tabular}%
3701   \else
3702     \chardef\bbl@tabular@mode\z@
3703   \fi}
3704 \@namedef{bbl@ADJ@layout.lists@on}{%
3705   \bbl@adjust@layout{\let\list\bbl@NL@list}}
3706 \@namedef{bbl@ADJ@layout.lists@off}{%

```

```

3707 \bbl@adjust@layout{\let\list\bbl@OL@list}}
3708 %
3709 \@namedef{bbl@ADJ@autoload.bcp47@on}{%
3710 \bbl@bcppallowedtrue}
3711 \@namedef{bbl@ADJ@autoload.bcp47@off}{%
3712 \bbl@bcppallowedfalse}
3713 \@namedef{bbl@ADJ@autoload.bcp47.prefix}#1{%
3714 \def\bbl@bcp@prefix{#1}}
3715 \def\bbl@bcp@prefix{bcp47-}
3716 \@namedef{bbl@ADJ@autoload.options}#1{%
3717 \def\bbl@autoload@options{#1}}
3718 \def\bbl@autoload@bcptoptions{import}
3719 \@namedef{bbl@ADJ@autoload.bcp47.options}#1{%
3720 \def\bbl@autoload@bcptoptions{#1}}
3721 \newif\ifbbl@bcptoname
3722 %
3723 \@namedef{bbl@ADJ@bcp47.toname@on}{%
3724 \bbl@bcptonametrue}
3725 \@namedef{bbl@ADJ@bcp47.toname@off}{%
3726 \bbl@bcptonamefalse}
3727 %
3728 \@namedef{bbl@ADJ@prehyphenation.disable@nohyphenation}{%
3729 \directlua{ Babel.ignore_pre_char = function(node)
3730 return (node.lang == \the\csname l@nohyphenation\endcsname)
3731 end }}
3732 \@namedef{bbl@ADJ@prehyphenation.disable@off}{%
3733 \directlua{ Babel.ignore_pre_char = function(node)
3734 return false
3735 end }}
3736 %
3737 \@namedef{bbl@ADJ@interchar.disable@nohyphenation}{%
3738 \def\bbl@ignoreinterchar{%
3739 \ifnum\language=\l@nohyphenation
3740 \expandafter\@gobble
3741 \else
3742 \expandafter\@firstofone
3743 \fi}}
3744 \@namedef{bbl@ADJ@interchar.disable@off}{%
3745 \let\bbl@ignoreinterchar\@firstofone}
3746 %
3747 \@namedef{bbl@ADJ@select.write@shift}{%
3748 \let\bbl@restorelastskip\relax
3749 \def\bbl@savelastskip{%
3750 \let\bbl@restorelastskip\relax
3751 \ifvmode
3752 \ifdim\lastskip=\z@
3753 \let\bbl@restorelastskip\nobreak
3754 \else
3755 \bbl@exp{%
3756 \def\\bbl@restorelastskip{%
3757 \skip@=\the\lastskip
3758 \\nobreak \vskip-\skip@ \vskip\skip@}}%
3759 \fi
3760 \fi}}
3761 \@namedef{bbl@ADJ@select.write@keep}{%
3762 \let\bbl@restorelastskip\relax
3763 \let\bbl@savelastskip\relax}
3764 \@namedef{bbl@ADJ@select.write@omit}{%
3765 \AddBabelHook{babel-select}{beforestart}{%
3766 \expandafter\babel@aux\expandafter{\bbl@main@language}}}%
3767 \let\bbl@restorelastskip\relax
3768 \def\bbl@savelastskip##1\bbl@restorelastskip{}
3769 \@namedef{bbl@ADJ@select.encoding@off}{%

```

```
3770 \let\bbl@encoding@select@off\@empty}
```

5.1 Cross referencing macros

The $\LaTeX$ book states:

The key argument is any sequence of letters, digits, and punctuation symbols; upper- and lowercase letters are regarded as different.

When the above quote should still be true when a document is typeset in a language that has active characters, special care has to be taken of the category codes of these characters when they appear in an argument of the cross referencing macros.

When a cross referencing command processes its argument, all tokens in this argument should be character tokens with category ‘letter’ or ‘other’.

The following package options control which macros are to be redefined.

```
3771 <(*More package options)> ≡
3772 \DeclareOption{safe=none}{\let\bbl@opt@safe\@empty}
3773 \DeclareOption{safe=bib}{\def\bbl@opt@safe{B}}
3774 \DeclareOption{safe=ref}{\def\bbl@opt@safe{R}}
3775 \DeclareOption{safe=refbib}{\def\bbl@opt@safe{BR}}
3776 \DeclareOption{safe=bibref}{\def\bbl@opt@safe{BR}}
3777 <(/More package options)>
```

\@newl@bel First we open a new group to keep the changed setting of `\protect` local and then we set the `@safe@actives` switch to true to make sure that any shorthand that appears in any of the arguments immediately expands to its non-active self.

```
3778 \bbl@trace{Cross referencing macros}
3779 \ifx\bbl@opt@safe\@empty\else % i.e., if 'ref' and/or 'bib'
3780   \def\@newl@bel#1#2#3{%
3781     {\@safe@activestrue
3782       \bbl@ifunset{#1@#2}%
3783       \relax
3784       {\gdef\@multiplelabels{%
3785         \@latex@warning@no@line{There were multiply-defined labels}}%
3786         \@latex@warning@no@line{Label `#2' multiply defined}}%
3787       \global\@namedef{#1@#2}{#3}}}
```

\@testdef An internal $\LaTeX$ macro used to test if the labels that have been written on the aux file have changed. It is called by the `\enddocument` macro.

```
3788 \CheckCommand*\@testdef[3]{%
3789   \def\reserved@a{#3}%
3790   \expandafter\ifx\csname#1@#2\endcsname\reserved@a
3791   \else
3792     \@tempswatrue
3793   \fi}
```

Now that we made sure that `\@testdef` still has the same definition we can rewrite it. First we make the shorthands ‘safe’. Then we use `\bbl@tempa` as an ‘alias’ for the macro that contains the label which is being checked. Then we define `\bbl@tempb` just as `\@newl@bel` does it. When the label is defined we replace the definition of `\bbl@tempa` by its meaning. If the label didn’t change, `\bbl@tempa` and `\bbl@tempb` should be identical macros.

```
3794 \def\@testdef#1#2#3{%
3795   \@safe@activestrue
3796   \expandafter\let\expandafter\bbl@tempa\csname #1@#2\endcsname
3797   \def\bbl@tempb{#3}%
3798   \@safe@activestrue
3799   \ifx\bbl@tempa\relax
3800   \else
3801     \edef\bbl@tempa{\expandafter\strip@prefix\meaning\bbl@tempa}%
3802   \fi
3803   \edef\bbl@tempb{\expandafter\strip@prefix\meaning\bbl@tempb}%
3804   \ifx\bbl@tempa\bbl@tempb
3805   \else
```

```

3806      \@tempwattrue
3807      \fi}
3808 \fi

```

\ref

\pageref The same holds for the macro `\ref` that references a label and `\pageref` to reference a page. We make them robust as well (if they weren't already) to prevent problems if they should become expanded at the wrong moment.

```

3809 \bbl@xin@{R}\bbl@opt@safe
3810 \ifin@
3811   \edef\bbl@tempc{\expandafter\string\csname ref code\endcsname}%
3812   \bbl@xin@{\expandafter\strip@prefix\meaning\bbl@tempc}%
3813   {\expandafter\strip@prefix\meaning\ref}%
3814   \ifin@
3815     \bbl@redefine\@kernel@ref#1{%
3816       \@safe@activestruerorg@@kernel@ref{#1}\@safe@activestruerfalse}
3817     \bbl@redefine\@kernel@pageref#1{%
3818       \@safe@activestruerorg@@kernel@pageref{#1}\@safe@activestruerfalse}
3819     \bbl@redefine\@kernel@sref#1{%
3820       \@safe@activestruerorg@@kernel@sref{#1}\@safe@activestruerfalse}
3821     \bbl@redefine\@kernel@spageref#1{%
3822       \@safe@activestruerorg@@kernel@spageref{#1}\@safe@activestruerfalse}
3823   \else
3824     \bbl@redefineroobust\ref#1{%
3825       \@safe@activestruerorg@ref{#1}\@safe@activestruerfalse}
3826     \bbl@redefineroobust\pageref#1{%
3827       \@safe@activestruerorg@pageref{#1}\@safe@activestruerfalse}
3828   \fi
3829 \else
3830   \let\org@ref\ref
3831   \let\org@pageref\pageref
3832 \fi

```

\@citex The macro used to cite from a bibliography, `\cite`, uses an internal macro, `\@citex`. It is this internal macro that picks up the argument(s), so we redefine this internal macro and leave `\cite` alone. The first argument is used for typesetting, so the shorthands need only be deactivated in the second argument.

```

3833 \bbl@xin@{B}\bbl@opt@safe
3834 \ifin@
3835   \bbl@redefine\@citex[#1]#2{%
3836     \@safe@activestrueredef\bbl@tempa{#2}\@safe@activestruerfalse
3837     \org@@citex[#1]{\bbl@tempa}}

```

Unfortunately, the packages `natbib` and `cite` need a different definition of `\@citex`... To begin with, `natbib` has a definition for `\@citex` with *three* arguments... We only know that a package is loaded when `\begin{document}` is executed, so we need to postpone the different redefinition.

Notice that we use `\def` here instead of `\bbl@redefine` because `\org@@citex` is already defined and we don't want to overwrite that definition (it would result in parameter stack overflow because of a circular definition).

(Recent versions of `natbib` change dynamically `\@citex`, so PR4087 doesn't seem fixable in a simple way. Just load `natbib` before.)

```

3838 \AtBeginDocument{%
3839   \@ifpackageloaded{natbib}{%
3840     \def\@citex[#1][#2]#3{%
3841       \@safe@activestrueredef\bbl@tempa{#3}\@safe@activestruerfalse
3842       \org@@citex[#1][#2]{\bbl@tempa}}%
3843   }{}}

```

The package `cite` has a definition of `\@citex` where the shorthands need to be turned off in both arguments.

```

3844 \AtBeginDocument{%
3845   \@ifpackageloaded{cite}{%

```

```

3846 \def\@citex[#1]#2{%
3847   \@safe@activetrue\org@@citex[#1]{#2}\@safe@activesfalse}%
3848 }{}}

```

\nocite The macro `\nocite` which is used to instruct BiB_T_EX to extract uncited references from the database.

```

3849 \bbl@redefine\nocite#1{%
3850   \@safe@activetrue\org@nocite{#1}\@safe@activesfalse}

```

\bibcite The macro that is used in the aux file to define citation labels. When packages such as `natbib` or `cite` are not loaded its second argument is used to typeset the citation label. In that case, this second argument can contain active characters but is used in an environment where `\@safe@activetrue` is in effect. This switch needs to be reset inside the `\hbox` which contains the citation label. In order to determine during aux file processing which definition of `\bibcite` is needed we define `\bbl@cite@choice` in such a way that it redefines itself with the proper definition. We call `\bbl@cite@choice` to select the proper definition for `\bbl@cite@choice`. This new definition is then activated.

```

3851 \bbl@redefine\bibcite{%
3852   \bbl@cite@choice
3853   \bibcite}

```

\bbl@bibcite The macro `\bbl@bibcite` holds the definition of `\bbl@bibcite` needed when neither `natbib` nor `cite` is loaded.

```

3854 \def\bbl@bibcite#1#2{%
3855   \org@bibcite{#1}\@safe@activesfalse#2}}

```

\bbl@cite@choice The macro `\bbl@cite@choice` determines which definition of `\bbl@cite@choice` is needed. First we give `\bbl@cite@choice` its default definition.

```

3856 \def\bbl@cite@choice{%
3857   \global\let\bbl@bibcite\bbl@bibcite
3858   \@ifpackageloaded{natbib}{\global\let\bbl@bibcite\org@bibcite}{}%
3859   \@ifpackageloaded{cite}{\global\let\bbl@bibcite\org@bibcite}{}%
3860   \global\let\bbl@cite@choice\relax}

```

When a document is run for the first time, no aux file is available, and `\bbl@cite@choice` will not yet be properly defined. In this case, this has to happen before the document starts.

```

3861 \AtBeginDocument{\bbl@cite@choice}

```

\@bibitem One of the two internal L^AT_EX macros called by `\bibitem` that write the citation label on the aux file.

```

3862 \bbl@redefine\@bibitem#1{%
3863   \@safe@activetrue\org@@bibitem{#1}\@safe@activesfalse}
3864 \else
3865   \let\org@nocite\nocite
3866   \let\org@@citex\@citex
3867   \let\org@bibcite\bibcite
3868   \let\org@@bibitem\@bibitem
3869 \fi

```

5.2 Layout

```

3870 \newcommand\BabelPatchSection[1]{%
3871   \@ifundefined{#1}{}{%
3872     \bbl@exp{\let<bbl@ss@#1>\<#1>}%
3873     \@namedef{#1}{%
3874       \@ifstar{\bbl@presec@s{#1}}{%
3875         {\@dblarg{\bbl@presec@x{#1}}}}}%
3876 \def\bbl@presec@x#1[#2]#3{%
3877   \bbl@exp{%
3878     \select@language@x{\bbl@main@language}%
3879     \bbl@cs{sspre@#1}%

```

```

3880    \\bbl@cs{ss@#1}%
3881    [\\foreignlanguage{\\language}{\\unexpanded{#2}}}%
3882    {\\foreignlanguage{\\language}{\\unexpanded{#3}}}%
3883    \\select@language{x{\\language}}
3884 \def\bbl@presec@s#1#2{%
3885   \bbl@exp{%
3886     \\select@language{x{\\bbl@main@language}}%
3887     \\bbl@cs{sspre@#1}%
3888     \\bbl@cs{ss@#1}%
3889     {\\foreignlanguage{\\language}{\\unexpanded{#2}}}%
3890     \\select@language{x{\\language}}
3891   }%
3892 \IfBabelLayout{sectioning}%
3893   {\BabelPatchSection{part}%
3894   \BabelPatchSection{chapter}%
3895   \BabelPatchSection{section}%
3896   \BabelPatchSection{subsection}%
3897   \BabelPatchSection{subsubsection}%
3898   \BabelPatchSection{paragraph}%
3899   \BabelPatchSection{subparagraph}%
3900   \def\babel@toc#1{%
3901     \select@language{x{\\bbl@main@language}}
3902 \IfBabelLayout{captions}%
3903   {\BabelPatchSection{caption}}

```

\BabelFootnote Footnotes.

```

3904 \bbl@trace{Footnotes}
3905 \def\bbl@footnote#1#2#3{%
3906   \@ifnextchar[%
3907     {\bbl@footnote@o{#1}{#2}{#3}}%
3908     {\bbl@footnote@x{#1}{#2}{#3}}
3909 \long\def\bbl@footnote@x#1#2#3#4{%
3910   \bgroup
3911   \select@language{x{\\bbl@main@language}}%
3912   \bbl@fn@footnote{#2#1{\ignorespaces#4}#3}%
3913   \egroup}
3914 \long\def\bbl@footnote@o#1#2#3[#4]#5{%
3915   \bgroup
3916   \select@language{x{\\bbl@main@language}}%
3917   \bbl@fn@footnote[#4]{#2#1{\ignorespaces#5}#3}%
3918   \egroup}
3919 \def\bbl@footnotetext#1#2#3{%
3920   \@ifnextchar[%
3921     {\bbl@footnotetext@o{#1}{#2}{#3}}%
3922     {\bbl@footnotetext@x{#1}{#2}{#3}}
3923 \long\def\bbl@footnotetext@x#1#2#3#4{%
3924   \bgroup
3925   \select@language{x{\\bbl@main@language}}%
3926   \bbl@fn@footnotetext{#2#1{\ignorespaces#4}#3}%
3927   \egroup}
3928 \long\def\bbl@footnotetext@o#1#2#3[#4]#5{%
3929   \bgroup
3930   \select@language{x{\\bbl@main@language}}%
3931   \bbl@fn@footnotetext[#4]{#2#1{\ignorespaces#5}#3}%
3932   \egroup}
3933 \def\BabelFootnote#1#2#3#4{%
3934   \ifx\bbl@fn@footnote\undefined
3935     \let\bbl@fn@footnote\footnote
3936   \fi
3937   \ifx\bbl@fn@footnotetext\undefined
3938     \let\bbl@fn@footnotetext\footnotetext
3939   \fi
3940   \bbl@ifblank{#2}%

```

```

3941 {\def#1{\bbl@footnote{\@firstofone}{#3}{#4}}
3942 \namedef{\bbl@stripslash#1text}%
3943 {\bbl@footnotetext{\@firstofone}{#3}{#4}}}%
3944 {\def#1{\bbl@exp{\bbl@footnote{\foreignlanguage{#2}}}{#3}{#4}}}%
3945 \namedef{\bbl@stripslash#1text}%
3946 {\bbl@exp{\bbl@footnotetext{\foreignlanguage{#2}}}{#3}{#4}}}}
3947 \IfBabelLayout{footnotes}%
3948 {\let\bbl@OL@footnote\footnote
3949 \BabelFootnote\footnote\language\language{}{}}%
3950 \BabelFootnote\localfootnote\language\language{}{}}%
3951 \BabelFootnote\mainfootnote{}{}}{}{}}
3952 {}

```

5.3 Marks

\markright Because the output routine is asynchronous, we must pass the current language attribute to the head lines. To achieve this we need to adapt the definition of `\markright` and `\markboth` somewhat. However, headlines and footlines can contain text outside marks; for that we must take some actions in the output routine if the 'headfoot' options is used.

We need to make some redefinitions to the output routine to avoid an endless loop and to correctly handle the page number in bidi documents.

```

3953 \bbl@trace{Marks}
3954 \IfBabelLayout{sectioning}
3955 {\ifx\bbl@opt@headfoot\@nnil
3956 \g@addto@macro\@resetactivechars{%
3957 \set@typeset@protect
3958 \expandafter\select@language@x\expandafter{\bbl@main@language}%
3959 \let\protect\noexpand
3960 \ifcase\bbl@bidimode\else % Only with bidi. See also above
3961 \edef\thepage{%
3962 \noexpand\babelsublr{\unexpanded\expandafter{\thepage}}}%
3963 \fi}%
3964 \fi}
3965 {\ifbbl@single\else
3966 \bbl@ifunset{markright }{\bbl@redefine\bbl@redefineroobust
3967 \markright#1{%
3968 \bbl@ifblank{#1}%
3969 {\org@markright{}}}%
3970 {\toks@{#1}%
3971 \bbl@exp{%
3972 \org@markright{\protect\foreignlanguage{\language}%
3973 \protect\bbl@restore@actives\the\toks@}}}%

```

\markboth

\mkboth The definition of `\markboth` is equivalent to that of `\markright`, except that we need two token registers. The documentclasses report and book define and set the headings for the page. While doing so they also store a copy of `\markboth` in `\mkboth`. Therefore we need to check whether `\mkboth` has already been set. If so we need to do that again with the new definition of `\markboth`. (As of Oct 2019, $\TeX$ stores the definition in an intermediate macro, so it's not necessary anymore, but it's preserved for older versions.)

```

3974 \ifx\mkboth\markboth
3975 \def\bbl@tempc{\let\mkboth\markboth}%
3976 \else
3977 \def\bbl@tempc{}%
3978 \fi
3979 \bbl@ifunset{markboth }{\bbl@redefine\bbl@redefineroobust
3980 \markboth#1#2{%
3981 \protected@edef\bbl@tempb##1{%
3982 \protect\foreignlanguage
3983 {\language\language}{\protect\bbl@restore@actives##1}}}%
3984 \bbl@ifblank{#1}%
3985 {\toks@{}}%

```

```

3986      {\toks@expandafter{\bbl@tempb{#1}}}%
3987      \bbl@ifblank{#2}%
3988      {\@temptokena{}}%
3989      {\@temptokena\expandafter{\bbl@tempb{#2}}}%
3990      \bbl@exp{\@org@markboth{\the\toks@}{\the\@temptokena}}}%
3991      \bbl@tempc
3992      \fi} % end ifbbl@single, end \IfBabelLayout

```

5.4 Other packages

5.4.1 ifthen

\ifthenelse Sometimes a document writer wants to create a special effect depending on the page a certain fragment of text appears on. This can be achieved by the following piece of code:

```

% \ifthenelse{isodd{\pageref{some-label}}}
%      {code for odd pages}
%      {code for even pages}
%

```

In order for this to work the argument of `\isodd` needs to be fully expandable. With the above redefinition of `\pageref` it is not in the case of this example. To overcome that, we add some code to the definition of `\ifthenelse` to make things work.

We want to revert the definition of `\pageref` and `\ref` to their original definition for the first argument of `\ifthenelse`, so we first need to store their current meanings.

Then we can set the `\@safe@actives` switch and call the original `\ifthenelse`. In order to be able to use shorthands in the second and third arguments of `\ifthenelse` the resetting of the switch *and* the definition of `\pageref` happens inside those arguments.

```

3993 \bbl@trace{Preventing clashes with other packages}
3994 \ifx\org@ref\undefined\else
3995   \bbl@xin@{R}\bbl@opt@safe
3996   \ifin@
3997     \AtBeginDocument{%
3998       \@ifpackageloaded{ifthen}{%
3999         \bbl@redefine@long\ifthenelse#1#2#3{%
4000           \let\bbl@temp@pref\pageref
4001           \let\pageref\org@pageref
4002           \let\bbl@temp@ref\ref
4003           \let\ref\org@ref
4004           \@safe@activestrue
4005           \org@ifthenelse{#1}%
4006             {\let\pageref\bbl@temp@pref
4007              \let\ref\bbl@temp@ref
4008              \@safe@activesfalse
4009              #2}%
4010             {\let\pageref\bbl@temp@pref
4011              \let\ref\bbl@temp@ref
4012              \@safe@activesfalse
4013              #3}%
4014           }%
4015         }{}%
4016       }
4017 \fi

```

5.4.2 varioref

\@@vpageref

\vrefpagemum

\Ref When the package `varioref` is in use we need to modify its internal command `\@@vpageref` in order to prevent problems when an active character ends up in the argument of `\vref`. The same needs to happen for `\vrefpagemum`.

```

4018 \AtBeginDocument{%
4019   \@ifpackageloaded{varioref}{%

```



```

4020 \bbl@redefine\@vpageref#1[#2]#3{%
4021 \@safe@activetrue
4022 \org@@vpageref{#1}[#2]#3}%
4023 \@safe@activesfalse}%
4024 \bbl@redefine\vrefpagenum#1#2{%
4025 \@safe@activetrue
4026 \org@vrefpagenum{#1}#2}%
4027 \@safe@activesfalse}%

```

The package `varioref` defines `\Ref` to be a robust command which uppercases the first character of the reference text. In order to be able to do that it needs to access the expandable form of `\ref`. So we employ a little trick here. We redefine the (internal) command `\Ref_` to call `\org@ref` instead of `\ref`. The disadvantage of this solution is that whenever the definition of `\Ref` changes, this definition needs to be updated as well.

```

4028 \expandafter\def\csname Ref \endcsname#1{%
4029 \protected@edef\@tempa{\org@ref{#1}}\expandafter\MakeUppercase\@tempa}
4030 }{}%
4031 }
4032 \fi

```

5.4.3 `hhline`

`\hhline` Delaying the activation of the shorthand characters has introduced a problem with the `hhline` package. The reason is that it uses the ‘`:`’ character which is made active by the french support in `babel`. Therefore we need to *reload* the package when the ‘`:`’ is an active character. Note that this happens *after* the category code of the `@`-sign has been changed to other, so we need to temporarily change it to letter again.

```

4033 \AtEndOfPackage{%
4034 \AtBeginDocument{%
4035 \@ifpackageloaded{hhline}%
4036 {\expandafter\ifx\csname normal@char\string\endcsname\relax
4037 \else
4038 \makeatletter
4039 \def\@currname{hhline}\input{hhline.sty}\makeatother
4040 \fi}%
4041 {}}}

```

`\substitutefontfamily` *Deprecated*. It creates an `fd` file on the fly. The first argument is an encoding mnemonic, the second and third arguments are font family names. Use the tools provided by `TeX` (`\DeclareFontFamilySubstitution`).

```

4042 \def\substitutefontfamily#1#2#3{%
4043 \lowercase{\immediate\openout15=#1#2.fd\relax}%
4044 \immediate\write15{%
4045 \string\ProvidesFile{#1#2.fd}%
4046 [\the\year/\two@digits{\the\month}/\two@digits{\the\day}
4047 \space generated font description file]^J
4048 \string\DeclareFontFamily{#1}{#2}{}^J
4049 \string\DeclareFontShape{#1}{#2}{m}{n}{<->ssub * #3/m/n}{}^J
4050 \string\DeclareFontShape{#1}{#2}{m}{it}{<->ssub * #3/m/it}{}^J
4051 \string\DeclareFontShape{#1}{#2}{m}{sl}{<->ssub * #3/m/sl}{}^J
4052 \string\DeclareFontShape{#1}{#2}{m}{sc}{<->ssub * #3/m/sc}{}^J
4053 \string\DeclareFontShape{#1}{#2}{b}{n}{<->ssub * #3/bx/n}{}^J
4054 \string\DeclareFontShape{#1}{#2}{b}{it}{<->ssub * #3/bx/it}{}^J
4055 \string\DeclareFontShape{#1}{#2}{b}{sl}{<->ssub * #3/bx/sl}{}^J
4056 \string\DeclareFontShape{#1}{#2}{b}{sc}{<->ssub * #3/bx/sc}{}^J
4057 }%
4058 \closeout15
4059 }
4060 \@onlypreamble\substitutefontfamily

```

5.5 Encoding and fonts

Because documents may use non-ASCII font encodings, we make sure that the logos of $\TeX$ and $\LaTeX$ always come out in the right encoding. There is a list of non-ASCII encodings. Requested encodings are currently stored in `\@fontenc@load@list`. If a non-ASCII has been loaded, we define versions of `\TeX` and `\LaTeX` for them using `\ensureascii`. The default ASCII encoding is set, too (in reverse order): the “main” encoding (when the document begins), the last loaded, or OT1.

`\ensureascii`

```
4061 \bbl@trace{Encoding and fonts}
4062 \newcommand\BabelNonASCII{LGR,LGI,X2,OT2,OT3,OT6,LHE,LWN,LMA,LMC,LMS,LMU}
4063 \newcommand\BabelNonText{TS1,T3,TS3}
4064 \let\org@TeX\TeX
4065 \let\org@LaTeX\LaTeX
4066 \let\ensureascii\@firstofone
4067 \let\asciientcoding\@empty
4068 \AtBeginDocument{%
4069   \def\@elt#1{,#1,}%
4070   \edef\bbl@tempa{\expandafter\@gobbletwo\@fontenc@load@list}%
4071   \let\@elt\relax
4072   \let\bbl@tempb\@empty
4073   \def\bbl@tempc{OT1}%
4074   \bbl@foreach\BabelNonASCII{% LGR loaded in a non-standard way
4075     \bbl@ifunset{T@#1}{\def\bbl@tempb{#1}}}%
4076   \bbl@foreach\bbl@tempa{%
4077     \bbl@xin@{,#1,}{,\BabelNonASCII,}%
4078     \ifin@
4079       \def\bbl@tempb{#1}% Store last non-ascii
4080     \else\bbl@xin@{,#1,}{,\BabelNonText,}% Pass
4081       \ifin@else
4082         \def\bbl@tempc{#1}% Store last ascii
4083       \fi
4084     \fi}%
4085   \ifx\bbl@tempb\@empty\else
4086     \bbl@xin@{,\cf@encoding,}{,\BabelNonASCII,\BabelNonText,}%
4087     \ifin@else
4088       \edef\bbl@tempc{\cf@encoding}% The default if ascii wins
4089     \fi
4090     \let\asciientcoding\bbl@tempc
4091     \renewcommand\ensureascii[1]{%
4092       {\fontencoding{\asciientcoding}\selectfont#1}}%
4093     \DeclareTextCommandDefault{\TeX}{\ensureascii{\org@TeX}}%
4094     \DeclareTextCommandDefault{\LaTeX}{\ensureascii{\org@LaTeX}}%
4095   \fi}
```

Now comes the old deprecated stuff (with a little change in 3.9l, for fontspec). The first thing we need to do is to determine, at `\begin{document}`, which latin fontencoding to use.

\latinencoding When text is being typeset in an encoding other than ‘latin’ (OT1 or T1), it would be nice to still have Roman numerals come out in the Latin encoding. So we first assume that the current encoding at the end of processing the package is the Latin encoding.

```
4096 \AtEndOfPackage{\edef\latinencoding{\cf@encoding}}
```

But this might be overruled with a later loading of the package `fontenc`. Therefore we check at the execution of `\begin{document}` whether it was loaded with the T1 option. The normal way to do this (using `\@ifpackageloaded`) is disabled for this package. Now we have to revert to parsing the internal macro `\@filelist` which contains all the filenames loaded.

```
4097 \AtBeginDocument{%
4098   \@ifpackageloaded{fontspec}%
4099   {\xdef\latinencoding{%
4100     \ifx\UTFencname\undefined
4101       EU\ifcase\bbl@engine\or2\or1\fi
4102     \else
```

```

4103      \UTFencname
4104      \fi}}%
4105      {\gdef\latinencoding{OT1}%
4106      \ifx\cf@encoding\bbl@t@one
4107      \xdef\latinencoding{\bbl@t@one}%
4108      \else
4109      \def\@elt#1{,#1,}%
4110      \edef\bbl@tempa{\expandafter\@gobbletwo\@fontenc@load@list}%
4111      \let\@elt\relax
4112      \bbl@xin@{,T1,}\bbl@tempa
4113      \ifin@
4114      \xdef\latinencoding{\bbl@t@one}%
4115      \fi
4116      \fi}}

```

\latintext Then we can define the command `\latintext` which is a declarative switch to a latin font-encoding. Usage of this macro is deprecated.

```

4117 \DeclareRobustCommand{\latintext}{%
4118   \fontencoding{\latinencoding}\selectfont
4119   \def\encodingdefault{\latinencoding}}

```

\textlatin This command takes an argument which is then typeset using the requested font encoding. In order to avoid many encoding switches it operates in a local scope.

```

4120 \ifx\@undefined\DeclareTextFontCommand
4121   \DeclareRobustCommand{\textlatin}[1]{\leavevmode{\latintext #1}}
4122 \else
4123   \DeclareTextFontCommand{\textlatin}{\latintext}
4124 \fi

```

For several functions, we need to execute some code with `\selectfont`. With $\TeX$ 2021-06-01, there is a hook for this purpose.

```

4125 \def\bbl@patchfont#1{\AddToHook{selectfont}{#1}}

```

5.6 Basic bidi support

This code is currently placed here for practical reasons. It will be moved to the correct place soon, I hope.

It is loosely based on `rlbabel.def`, but most of it has been developed from scratch. This babel module (by Johannes Braams and Boris Lavva) has served the purpose of typesetting R documents for two decades, and despite its flaws I think it is still a good starting point (some parts have been copied here almost verbatim), partly thanks to its simplicity. I’ve also looked at ARABI (by Youssef Jabri), which is compatible with babel.

There are two ways of modifying macros to make them “bidi”, namely, by patching the internal low-level macros (which is what I have done with lists, columns, counters, tocs, much like `rlbabel` did), and by introducing a “middle layer” just below the user interface (sectioning, footnotes).

- `pdftex` provides a minimal support for bidi text, and it must be done by hand. Vertical typesetting is not possible.
- `xetex` is somewhat better, thanks to its font engine (even if not always reliable) and a few additional tools. However, very little is done at the paragraph level. Another challenging problem is text direction does not honour $\TeX$ grouping.
- `luatex` can provide the most complete solution, as we can manipulate almost freely the node list, the generated lines, and so on, but bidi text does not work out of the box and some development is necessary. It also provides tools to properly set left-to-right and right-to-left page layouts. As `Lua $\TeX$ -ja` shows, vertical typesetting is possible, too.

```

4126 \bbl@trace{Loading basic (internal) bidi support}
4127 \ifodd\bbl@engine
4128 \else % Any xe+lua bidi
4129   \ifnum\bbl@bidimode>100 \ifnum\bbl@bidimode<200
4130     \bbl@error{bidi-only-lua}{}}{}%
4131     \let\bbl@beforeforeign\leavevmode

```

```

4132 \AtEndOfPackage{%
4133 \EnableBabelHook{babel-bidi}%
4134 \bbl@xebidipar}
4135 \fi\fi
4136 \def\bbl@loadxebidi#1{%
4137 \ifx\RTLfootnotetext\@undefined
4138 \AtEndOfPackage{%
4139 \EnableBabelHook{babel-bidi}%
4140 \ifx\fontspec\@undefined
4141 \usepackage{fontspec}% bidi needs fontspec
4142 \fi
4143 \usepackage#1{bidi}%
4144 \let\bbl@digitsdotdash\DigitsDotDashInterCharToks
4145 \def\DigitsDotDashInterCharToks{% See the 'bidi' package
4146 \ifnum\@nameuse{bbl@wdir\@languagename}=\tw@ % 'AL' bidi
4147 \bbl@digitsdotdash % So ignore in 'R' bidi
4148 \fi}}%
4149 \fi}
4150 \ifnum\bbl@bidimode>200 % Any xe bidi=
4151 \ifcase\expandafter\@gobbletwo\the\bbl@bidimode\or
4152 \bbl@tentative{bidi=bidi}
4153 \bbl@loadxebidi{}
4154 \or
4155 \bbl@loadxebidi{[rldocument]}
4156 \or
4157 \bbl@loadxebidi{}
4158 \fi
4159 \fi
4160 \fi
4161 \ifnum\bbl@bidimode=\@ne % bidi=default
4162 \let\bbl@beforeforeign\leavevmode
4163 \ifodd\bbl@engine % lua
4164 \newattribute\bbl@attr@dir
4165 \directlua{ Babel.attr_dir = luatexbase.registernumber'bbl@attr@dir' }
4166 \bbl@exp{\output{\bodydir\pagedir\the\output}}
4167 \fi
4168 \AtEndOfPackage{%
4169 \EnableBabelHook{babel-bidi}% pdf/lua/xe
4170 \ifodd\bbl@engine\else % pdf/xe
4171 \bbl@xebidipar
4172 \fi}
4173 \fi

```

Now come the macros used to set the direction when a language is switched. Testing are based on script names, because it's the user interface (including language and script in \babelprovide. First the (mostly) common macros.

```

4174 \bbl@trace{Macros to switch the text direction}
4175 \def\bbl@alscripts{%
4176 ,Arabic,Syriac,Thaana,Hanifi Rohingya,Hanifi,Sogdian,}
4177 \def\bbl@rscripts{%
4178 Adlam,Avestan,Chorasmian,Cypriot,Elymaic,Garay,%
4179 Hatran,Hebrew,Imperial Aramaic,Inscriptional Pahlavi,%
4180 Inscriptional Parthian,Kharoshthi,Lydian,Mandaic,Manichaeen,%
4181 Mende Kikakui,Meroitic Cursive,Meroitic Hieroglyphs,Nabataean,%
4182 Nko,Old Hungarian,Old North Arabian,Old Sogdian,%
4183 Old South Arabian,Old Turkic,Old Uyghur,Palmyrene,Phoenician,%
4184 Psalter Pahlavi,Samaritan,Yezidi,Mandaean,%
4185 Meroitic,N'Ko,Orkhon,Todhri}
4186 %
4187 \def\bbl@provide@dirs#1{%
4188 \bbl@xin@{\csname bbl@sname@#1\endcsname}{\bbl@alscripts\bbl@rscripts}%
4189 \ifin@
4190 \global\bbl@csarg\chardef{wdir@#1}\@ne
4191 \bbl@xin@{\csname bbl@sname@#1\endcsname}{\bbl@alscripts}%

```

```

4192 \ifin@
4193 \global\bbl@csarg\chardef{wdir@#1}\tw@
4194 \fi
4195 \else
4196 \global\bbl@csarg\chardef{wdir@#1}\z@
4197 \fi
4198 \ifodd\bbl@engine
4199 \bbl@csarg\ifcase{wdir@#1}%
4200 \directlua{ Babel.locale_props[\the\localeid].texdir = 'l' }%
4201 \or
4202 \directlua{ Babel.locale_props[\the\localeid].texdir = 'r' }%
4203 \or
4204 \directlua{ Babel.locale_props[\the\localeid].texdir = 'al' }%
4205 \fi
4206 \fi}
4207 %
4208 \def\bbl@switchdir{%
4209 \bbl@ifunset{bbl@lsys@\language name}{\bbl@provide@lsys{\language name}}{}%
4210 \bbl@ifunset{bbl@wdir@\language name}{\bbl@provide@dirs{\language name}}{}%
4211 \bbl@exp{\\bbl@setdirs\bbl@cl{wdir}}}%
4212 \def\bbl@setdirs#1{%
4213 \ifcase\bbl@select@type
4214 \bbl@bodydir{#1}%
4215 \bbl@pardir{#1}% <- Must precede \bbl@texdir
4216 \fi
4217 \bbl@texdir{#1}}
4218 \ifnum\bbl@bidimode>\z@
4219 \AddBabelHook{babel-bidi}{afterextras}{\bbl@switchdir}
4220 \DisableBabelHook{babel-bidi}
4221 \fi

Now the engine-dependent macros.
4222 \ifodd\bbl@engine % luatex=1
4223 \else % pdftex=0, xetex=2
4224 \newcount\bbl@dirlevel
4225 \chardef\bbl@thetexdir\z@
4226 \chardef\bbl@thepardir\z@
4227 \def\bbl@texdir#1{%
4228 \ifcase#1\relax
4229 \chardef\bbl@thetexdir\z@
4230 \@nameuse{setlatin}%
4231 \bbl@texdir@i\beginL\endL
4232 \else
4233 \chardef\bbl@thetexdir@ne
4234 \@nameuse{setnonlatin}%
4235 \bbl@texdir@i\beginR\endR
4236 \fi}
4237 \def\bbl@texdir@i#1#2{%
4238 \ifhmode
4239 \ifnum\currentgrouplevel>\z@
4240 \ifnum\currentgrouplevel=\bbl@dirlevel
4241 \bbl@error{multiple-bidi}{}{}%
4242 \bgroup\aftergroup#2\aftergroup\egroup
4243 \else
4244 \ifcase\currentgrouptype\or % 0 bottom
4245 \aftergroup#2% 1 simple {}
4246 \or
4247 \bgroup\aftergroup#2\aftergroup\egroup % 2 hbox
4248 \or
4249 \bgroup\aftergroup#2\aftergroup\egroup % 3 adj hbox
4250 \or\or\or % vbox vtop align
4251 \or
4252 \bgroup\aftergroup#2\aftergroup\egroup % 7 noalign
4253 \or\or\or\or\or\or % output math disc insert vcent mathchoice

```

```

4254         \or
4255         \aftergroup#2% 14 \begingroup
4256         \else
4257         \bgroup\aftergroup#2\aftergroup\egroup % 15 adj
4258         \fi
4259         \fi
4260         \bbl@dirlevel\currentgrouplevel
4261         \fi
4262         #1%
4263         \fi}
4264 \def\bbl@pardir#1{\chardef\bbl@thepardir#1\relax}
4265 \let\bbl@bodydir\@gobble
4266 \def\bbl@dirparastext{\chardef\bbl@thepardir\bbl@thetextdir}

```

The following command is executed only if there is a right-to-left script (once). It activates the `\everypar` hack for xetex, to properly handle the par direction. Note text and par dirs are decoupled to some extent (although not completely).

```

4267 \def\bbl@xebidipar{%
4268   \let\bbl@xebidipar\relax
4269   \TeXeTstate\@ne
4270   \def\bbl@xeeverypar{%
4271     \ifcase\bbl@thepardir
4272       \ifcase\bbl@thetextdir\else\beginR\fi
4273     \else
4274       {\setbox\z@\lastbox\beginR\box\z@}%
4275     \fi}%
4276   \AddToHook{para/begin}{\bbl@xeeverypar}}
4277 \ifnum\bbl@bidimode>200 % Any xe bidi=
4278   \let\bbl@textdir@i\@gobbletwo
4279   \let\bbl@xebidipar\@empty
4280   \AddBabelHook{bidi}{foreign}{%
4281     \ifcase\bbl@thetextdir
4282       \BabelWrapText{\LR{##1}}%
4283     \else
4284       \BabelWrapText{\RL{##1}}%
4285     \fi}
4286   \def\bbl@pardir#1{\ifcase#1\relax\setLR\else\setRL\fi}
4287 \fi
4288 \fi

```

A tool for weak L (mainly digits). We also disable warnings with hyperref.

```

4289 \DeclareRobustCommand\babelsublr[1]{\leavevmode\bbl@textdir\z@#1}}
4290 \AtBeginDocument{%
4291   \ifx\pdfstringdefDisableCommands\undefined\else
4292     \ifx\pdfstringdefDisableCommands\relax\else
4293       \pdfstringdefDisableCommands{\let\babelsublr\@firstofone}%
4294     \fi
4295   \fi}

```

5.7 Local Language Configuration

\loadlocalcfg At some sites it may be necessary to add site-specific actions to a language definition file. This can be done by creating a file with the same name as the language definition file, but with the extension `.cfg`. For instance the file `norsk.cfg` will be loaded when the language definition file `norsk.ldf` is loaded.

For plain-based formats we don't want to override the definition of `\loadlocalcfg` from `plain.def`.

```

4296 \bbl@trace{Local Language Configuration}
4297 \ifx\loadlocalcfg\undefined
4298   \ifpackagewith{babel}{noconfigs}%
4299   {\let\loadlocalcfg\@gobble}%
4300   {\def\loadlocalcfg#1{%
4301     \InputIfFileExists{#1.cfg}%

```

```

4302      {\typeout{*****^J%
4303              * Local config file #1.cfg used^^J%
4304              *}}}%
4305      \@empty}}
4306 \fi

```

5.8 Language options

Languages are loaded when processing the corresponding option *except* if a main language has been set. In such a case, it is not loaded until all options has been processed. The following macro inputs the ldf file and does some additional checks (\input works, too, but possible errors are not caught).

```

4307 \bbl@trace{Language options}
4308 \def\BabelDefinitionFile#1#2#3{
4309 \let\bbl@afterlang\relax
4310 \let\BabelModifiers\relax
4311 \let\bbl@loaded\@empty
4312 \def\bbl@load@language#1{%
4313   \InputIfFileExists{#1.ldf}%
4314   {\edef\bbl@loaded{\CurrentOption
4315     \ifx\bbl@loaded\@empty\else,\bbl@loaded\fi}%
4316     \expandafter\let\expandafter\bbl@afterlang
4317     \csname\CurrentOption.ldf-h@k\endcsname
4318     \expandafter\let\expandafter\BabelModifiers
4319     \csname bbl@mod@\CurrentOption\endcsname
4320     \bbl@exp{\AtBeginDocument{%
4321       \bbl@usehooks@lang{\CurrentOption}{\begindocument}{\CurrentOption}}}%
4322     {\bbl@error{unknown-package-option}}{}}}%

```

Another way to extend the list of ‘known’ options for babel was to create the file `bblopts.cfg` in which one can add option declarations. However, this mechanism is deprecated – if you want an alternative name for a language, just create a new ldf file loading the actual one. You can also set the name of the file with the package option `config=<name>`, which will load `<name>.cfg` instead.

If the language as been set as metadata, read the info from the corresponding ini file and extract the babel name. Then added it as a package option at the end, so that it becomes the main language. The behavior of a metatag with a global language option is not well defined, so if there is not a main option we set here explicitly.

Tagging PDF Span elements requires horizontal mode. With DocumentMetada we also force it with `\foreignlanguage` (this is also done in bidi texts).

```

4323 \ifx\GetDocumentProperties\undefined\else
4324 \let\bbl@beforeforeign\leavevmode
4325 \edef\bbl@tempa{\GetDocumentProperties{document/other-languages}}%
4326 \let\bbl@collect@other\@empty
4327 \bbl@foreach\bbl@tempa{%
4328   \edef\bbl@metalang{#1}%
4329   \ifx\bbl@metalang\@empty\else
4330     \begingroup
4331     \expandafter
4332     \bbl@bcpllookup{\bbl@metalang}%-\@empty-\@empty-\@empty@@
4333     \ifx\bbl@bcp\relax
4334       \ifx\bbl@opt@main\@nnil
4335         \bbl@error{no-locale-for-meta}{\bbl@metalang}{}%
4336       \fi
4337     \else
4338       \bbl@read@ini{\bbl@bcp}\m@ne
4339       \xdef\bbl@collect@other{\bbl@collect@other,\language\metaname}%
4340       \GenericInfo{(babel) \spaces\@spaces\@spaces
4341         Passing '\language\metaname' to babel\@gobble}%
4342     \fi
4343   \endgroup
4344 \fi}
4345 \ifx\bbl@collect@other\@empty\else
4346 \xdef\bbl@language@opts{\bbl@collect@other,\bbl@language@opts}%
4347 \fi

```

```

4348 \edef\bbl@metalang{\GetDocumentProperties{document/language}}%
4349 \ifx\bbl@metalang\empty\else
4350   \begingroup
4351     \expandafter
4352     \bbl@bcplookup{\bbl@metalang}%-\@empty-\@empty-\@empty\@
4353     \ifx\bbl@bcp\relax
4354       \ifx\bbl@opt@main\@nnil
4355         \bbl@error{no-locale-for-meta}{\bbl@metalang}{}}%
4356       \fi
4357     \else
4358       \bbl@read@ini{\bbl@bcp}\m@ne
4359       \xdef\bbl@language@opts{\bbl@language@opts,\language}%
4360       \ifx\bbl@opt@main\@nnil
4361         \global\let\bbl@opt@main\language
4362       \fi
4363       \GenericInfo{{(babel)} \@spaces\@spaces\@spaces
4364         Passing '\language' to babel\@gobble}%
4365     \fi
4366   \endgroup
4367 \fi
4368 \fi
4369 \ifx\bbl@opt@config\@nnil
4370   \@ifpackagewith{babel}{noconfigs}}%
4371   {\InputIfFileExists{bblopts.cfg}%
4372     {\bbl@info{Configuration files are deprecated, as\\%
4373       they can break document portability.\\%
4374       Reported}%
4375     \typeout{*****^J%
4376       * Local config file bblopts.cfg used^^J%
4377       *}%
4378     }{}%
4379   \else
4380     \InputIfFileExists{\bbl@opt@config.cfg}%
4381     {\bbl@info{Configuration files are deprecated, as\\%
4382       they can break document portability.\\%
4383       Reported}%
4384     \typeout{*****^J%
4385       * Local config file \bbl@opt@config.cfg used^^J%
4386       *}%
4387     {\bbl@error{config-not-found}}{}%
4388   \fi

```

Recognizing global options in packages not having a closed set of them is not trivial, as for them to be processed they must be defined explicitly. So, package options not yet taken into account and stored in `\bbl@language@opts` are assumed to be languages. If not declared above, the names of the option and the file are the same. We first pre-process the class and package options to determine the available locales, and which version (ldf or ini) will be loaded. This is done by first loading the corresponding `babel-⟨name⟩.tex` file.

The second argument of `\BabelBeforeIni` may contain a `\BabelDefinitionFile` which defines `\bbl@tempa` and `\bbl@tempb` and saves the third argument for the moment of the actual loading. If there is no `\BabelDefinitionFile` the last element is usually empty, and the ini file is loaded. The values are used to build a list in the form ‘main-or-not’ / ‘ldf-or-ldfini-flag’ // ‘option-name’ // ‘bcp-tag’ / ‘ldf-name-or-none’. The ‘main-or-not’ element is 0 by default and set to 10 later if necessary (by prepending 1). The ‘bcp-tag’ is stored here so that the corresponding ini file can be loaded directly (with `\import`).

```

4389 \def\BabelBeforeIni#1#2{%
4390   \def\bbl@tempa{\@m}% <- Default if no \BDefFile
4391   \let\bbl@tempb\empty
4392   #2%
4393   \edef\bbl@tload{%
4394     \ifx\bbl@tload\empty\else\bbl@tload,\fi
4395     \bbl@tload@last}%
4396   \edef\bbl@tload@last{0/\bbl@tempa//\CurrentOption//\#1/\bbl@tempb}}

```



```

4397 \def\BabelDefinitionFile#1#2#3{%
4398   \def\bbl@tempa{#1}\def\bbl@tempb{#2}%
4399   \namedef\bbl@preldf@{\CurrentOption}{#3}%
4400   \endinput}%

```

For efficiency, first preprocess the class options to remove those with =, which are becoming increasingly frequent (no language should contain this character). Here we use the more robust macro to traverse a clist from the $\text{\TeX}$ layer.

```

4401 \def\bbl@tempf{,}
4402 \@nameuse{clist_map_inline:Nn}\@raw@classoptionslist{%
4403   \in@{=}{#1}%
4404   \ifin@ \else
4405     \edef\bbl@tempf{\bbl@tempf\zap@space#1 \@empty,}%
4406   \fi}

```

Store the class/package options in a list. If there is an explicit main, it's placed as the last option. Then loop it to read the tex files, which can have a `\BabelDefinitionFile`. If there is no tex file, we attempt loading the ldf for the option name; if it fails, an error is raised. Note the option name is surrounded by `//...//`. Class and package options are separated with `@`, because errors and info are dealt with in different ways. Consecutive identical languages count as one.

```

4407 \let\bbl@toload\@empty
4408 \let\bbl@toload@last\@empty
4409 \let\bbl@unkopt\@gobble %% <- Ugly
4410 \edef\bbl@tempc{%
4411   \bbl@tempf,@,\bbl@language@opts
4412   \ifx\bbl@opt@main\@nnil\else,\bbl@opt@main\fi}
4413 \let\BabelLocalesTentative\bbl@tempc
4414 %
4415 \bbl@foreach\bbl@tempc{%
4416   \in@{@@}{#1}% <- Ugly
4417   \ifin@
4418     \def\bbl@unkopt##1{%
4419       \DeclareOption{##1}{\bbl@error{unknown-package-option}{}}{}}%
4420   \else
4421     \def\CurrentOption{#1}%
4422     \bbl@xin@{//#1//}{\bbl@toload@last}% Collapse consecutive
4423     \ifin@ \else
4424       \lowercase{\InputIfFileExists{babel-#1.tex}}{ }{%
4425         \IfFileExists{#1.ldf}%
4426         {\edef\bbl@toload{%
4427           \ifx\bbl@toload\@empty\else\bbl@toload,\fi
4428           \bbl@toload@last}%
4429           \edef\bbl@toload@last{0/0//\CurrentOption//und/#1}}%
4430         {\bbl@unkopt{#1}}}%
4431     \fi
4432   \fi}

```

We have to determine (1) if no language has been loaded (in which case we fallback to 'nil', with a special tag), and (2) the main language. With an explicit 'main' language, remove repeated elements. The number 1 flags it as the main language (relevant in *ini* locales), because with 0 becomes 10.

```

4433 \ifx\bbl@opt@main\@nnil
4434   \ifx\bbl@toload@last\@empty
4435     \def\bbl@toload@last{0/0//nil//und-x-nil-nil}
4436     \bbl@info{%
4437       You haven't specified a language as a class or package\\%
4438       option. I'll load 'nil'. Reported}
4439   \fi
4440 \else
4441   \let\bbl@tempc\@empty
4442   \bbl@foreach\bbl@toload{%
4443     \bbl@xin@{//#1//\bbl@opt@main//}{#1}%
4444     \ifin@ \else
4445       \bbl@add@list\bbl@tempc{#1}%
4446     \fi}

```

```

4447 \let\bbl@toload\bbl@tempc
4448 \fi
4449 \edef\bbl@toload{\bbl@toload,1\bbl@toload@last}

```

Finally, load the ‘ini’ file or the pair ‘ini’/‘ldf’ file. Babel resorts to its own mechanism, not the default one based on \ProcessOptions (which is still present to make some internal clean-up). First, handle provide= and friends (with a recursive call if they are present), and then provide=* and friend. \count@ is used as flag: 0 if ‘ini’, 1 if ‘ldf’.

```

4450 \def\AfterBabelLanguage#1{%
4451 \bbl@ifsamestring\CurrentOption{#1}{\global\bbl@add\bbl@afterlang}{}}
4452 \NewHook{babel/presets}
4453 \UseHook{babel/presets}
4454 %
4455 \let\bbl@tempb\@empty
4456 \def\bbl@tempc#1/#2//#3//#4/#5\@{%
4457 \count@\z@
4458 \ifnum#2=\@m % if no \BabelDefinitionFile
4459 \ifnum#1=\z@ % not main. -- % if provide+=!, provide*=!
4460 \ifnum\bbl@ldfflag>\@ne\bbl@tempc 0/0//#3//#4/#3\@
4461 \else\bbl@tempd{#1}{#2}{#3}{#4}{#5}%
4462 \fi
4463 \else % 10 = main -- % if provide+=!, provide*=!
4464 \ifodd\bbl@ldfflag\bbl@tempc 10/0//#3//#4/#3\@
4465 \else\bbl@tempd{#1}{#2}{#3}{#4}{#5}%
4466 \fi
4467 \fi
4468 \else
4469 \ifnum#1=\z@ % not main
4470 \ifnum\bbl@iniflag>\@ne\else % if ø, provide
4471 \ifcase#2\count@\@ne\else\ifcase\bbl@engine\count@\@ne\fi\fi
4472 \fi
4473 \else % 10 = main
4474 \ifodd\bbl@iniflag\else % if provide+, provide*
4475 \ifcase#2\count@\@ne\else\ifcase\bbl@engine\count@\@ne\fi\fi
4476 \fi
4477 \fi
4478 \bbl@tempd{#1}{#2}{#3}{#4}{#5}%
4479 \fi}

```

Based on the value of \count@, do the actual loading. If ‘ldf’, we load the basic info from the ‘ini’ file before.

```

4480 \def\bbl@tempd#1#2#3#4#5{%
4481 \DeclareOption{#3}{}%
4482 \ifcase\count@
4483 \bbl@exp{\bbl@add\bbl@tempb{%
4484 \global\let\bbl@afterload\@empty
4485 \@@nameuse\bbl@preini@#3}%
4486 \bbl@ldfinit
4487 \def\CurrentOption{#3}%
4488 \bbl@babelprovide[import=#4,\ifnum#1=\z@\else\bbl@opt@provide,main\fi]{#3}%
4489 \bbl@afterldf
4490 \bbl@afterload}}%
4491 \else
4492 \bbl@add\bbl@tempb{%
4493 \global\let\bbl@afterload\@empty
4494 \def\CurrentOption{#3}%
4495 \let\localename\CurrentOption
4496 \let\languagename\localename
4497 \def\BabelIniTag{#4}%
4498 \@@nameuse\bbl@preldf@#3}%
4499 \begingroup
4500 \bbl@id@assign
4501 \bbl@read@ini{\BabelIniTag}0%
4502 \endgroup

```

```

4503      \bbl@load@language{#5}%
4504      \bbl@afterload}%
4505  \fi}
4506 %
4507 \bbl@foreach\bbl@toload{\bbl@tempc#1\@@}
4508 \bbl@tempb
4509 \DeclareOption*{}
4510 \ProcessOptions
4511 %
4512 \bbl@exp{%
4513   \\AtBeginDocument{\\bbl@usehooks@lang{/}{begindocument}}{}}}%
4514 \def\AfterBabelLanguage{\bbl@error{late-after-babel}}{}{}{}
4515 \AddToHook{begindocument/end}{%
4516   \bbl@info{Main language set to '\mainlocalename'\@gobble}}
4517 (/package)

```

6 The kernel of Babel

The kernel of the babel system is currently stored in `babel.def`. The file `babel.def` contains most of the code. The file `hyphen.cfg` is a file that can be loaded into the format, which is necessary when you want to be able to switch hyphenation patterns.

Because plain $\TeX$ users might want to use some of the features of the babel system too, care has to be taken that plain $\TeX$ can process the files. For this reason the current format will have to be checked in a number of places. Some of the code below is common to plain $\TeX$ and $\LaTeX$, some of it is for the $\LaTeX$ case only.

Plain formats based on `etex` (`etex`, `xetex`, `luatex`) don't load `hyphen.cfg` but `etex.src`, which follows a different naming convention, so we need to define the babel names. It presumes `language.def` exists and it is the same file used when formats were created.

A proxy file for `switch.def`

```

4518 (*kernel)
4519 \let\bbl@onlyswitch\@empty
4520 \input babel.def
4521 \let\bbl@onlyswitch\@undefined
4522 (/kernel)

```

7 Error messages

They are loaded when `\bbl@error` is first called. To save space, the main code just identifies them with a tag, and messages are stored in a separate file. Since it can be loaded anywhere, you make sure some catcodes have the right value, although those for `\`, ```, `^`, `M`, `%` and `=` are reset before loading the file.

```

4523 (*errors)
4524 \catcode`\{=1 \catcode`\}=2 \catcode`\#=6
4525 \catcode`\:=12 \catcode`\,=12 \catcode`\.=12 \catcode`\-=12
4526 \catcode`\'=12 \catcode`\(=12 \catcode`\)=12
4527 \catcode`\@=11 \catcode`\^=7
4528 %
4529 \ifx\MessageBreak\@undefined
4530   \gdef\bbl@error@i#1#2{%
4531     \begingroup
4532       \newlinechar=^^J
4533       \def\{^^J(babel) }%
4534       \errhelp{#2}\errmessage{\{#1}%
4535     \endgroup}
4536 \else
4537   \gdef\bbl@error@i#1#2{%
4538     \begingroup
4539     \def\{\MessageBreak}%
4540     \PackageError{babel}{#1}{#2}%
4541     \endgroup}
4542 \fi

```

```

4543 \def\bbl@errmessage#1#2#3{%
4544   \expandafter\gdef\csname bbl@err@#1\endcsname##1##2##3{%
4545     \bbl@error@i{#2}{#3}}}%
4546 % Implicit #2#3#4:
4547 \gdef\bbl@error#1{\csname bbl@err@#1\endcsname}
4548 %
4549 \bbl@errmessage{not-yet-available}
4550   {Not yet available}%
4551   {Find an armchair, sit down and wait}
4552 \bbl@errmessage{bad-package-option}%
4553   {Bad option '#1=#2'. Either you have misspelled the\\%
4554     key or there is a previous setting of '#1'. Valid\\%
4555     keys are, among others, 'shorthands', 'main', 'bidi',\\%
4556     'strings', 'config', 'headfoot', 'safe', 'math'.}%
4557   {See the manual for further details.}
4558 \bbl@errmessage{base-on-the-fly}
4559   {For a language to be defined on the fly 'base'\\%
4560     is not enough, and the whole package must be\\%
4561     loaded. Either delete the 'base' option or\\%
4562     request the languages explicitly}%
4563   {See the manual for further details.}
4564 \bbl@errmessage{undefined-language}
4565   {You haven't defined the language '#1' yet.\\%
4566     Perhaps you misspelled it or your installation\\%
4567     is not complete}%
4568   {Your command will be ignored, type <return> to proceed}
4569 \bbl@errmessage{invalid-ini-name}
4570   {'#1' not valid with the 'ini' mechanism.\\%
4571     I think you want '#2' instead. You may continue,\\%
4572     but you should fix the name. See the babel manual\\%
4573     for the available locales with 'provide'}%
4574   {See the manual for further details.}
4575 \bbl@errmessage{shorthand-is-off}
4576   {I can't declare a shorthand turned off (\string#2)}
4577   {Sorry, but you can't use shorthands which have been\\%
4578     turned off in the package options}
4579 \bbl@errmessage{not-a-shorthand}
4580   {The character '\string #1' should be made a shorthand character;\\%
4581     add the command \string\usesshorthands\string{#1\string} to
4582     the preamble.\\%
4583     I will ignore your instruction}%
4584   {You may proceed, but expect unexpected results}
4585 \bbl@errmessage{not-a-shorthand-b}
4586   {I can't switch '\string#2' on or off--not a shorthand\\%
4587     This character is not a shorthand. Maybe you made\\%
4588     a typing mistake?}%
4589   {I will ignore your instruction.}
4590 \bbl@errmessage{unknown-attribute}
4591   {The attribute #2 is unknown for language #1.}%
4592   {Your command will be ignored, type <return> to proceed}
4593 \bbl@errmessage{missing-group}
4594   {Missing group for string \string#1}%
4595   {You must assign strings to some category, typically\\%
4596     captions or extras, but you set none}
4597 \bbl@errmessage{only-lua-xe}
4598   {This macro is available only in LuaLaTeX and XeLaTeX.}%
4599   {Consider switching to these engines.}
4600 \bbl@errmessage{only-lua}
4601   {This macro is available only in LuaLaTeX}%
4602   {Consider switching to that engine.}
4603 \bbl@errmessage{unknown-provide-key}
4604   {Unknown key '#1' in \string\babelprovide}%
4605   {See the manual for valid keys}%

```

```

4606 \bbl@errmessage{unknown-mapfont}
4607 {Option '\bbl@KVP@mapfont' unknown for\\%
4608   mapfont. Use 'direction'}%
4609 {See the manual for details.}
4610 \bbl@errmessage{no-ini-file}
4611 {There is no ini file for the requested language\\%
4612   (#1: \language). Perhaps you misspelled it or your\\%
4613   installation is not complete}%
4614 {Fix the name or reinstall babel.}
4615 \bbl@errmessage{digits-is-reserved}
4616 {The counter name 'digits' is reserved for mapping\\%
4617   decimal digits}%
4618 {Use another name.}
4619 \bbl@errmessage{limit-two-digits}
4620 {Currently two-digit years are restricted to the\\%
4621   range 0-9999}%
4622 {There is little you can do. Sorry.}
4623 \bbl@errmessage{counter-too-large}
4624 {Counter too large (#1)}%
4625 {Currently this is the limit.}
4626 \bbl@errmessage{no-ini-info}
4627 {I've found no info for the current locale.\\%
4628   The corresponding ini file has not been loaded\\%
4629   Perhaps it doesn't exist}%
4630 {See the manual for details.}
4631 \bbl@errmessage{unknown-ini-field}
4632 {Unknown field '#1' in \string\BCPdata.\\%
4633   Perhaps you misspelled it}%
4634 {See the manual for details.}
4635 \bbl@errmessage{unknown-locale-key}
4636 {Unknown key for locale '#2':\\%
4637   #3\\%
4638   \string#1 will be set to \string\relax}%
4639 {Perhaps you misspelled it.}%
4640 \bbl@errmessage{adjust-only-vertical}
4641 {Currently, #1 related features can be adjusted only\\%
4642   in the main vertical list}%
4643 {Maybe things change in the future, but this is what it is.}
4644 \bbl@errmessage{layout-only-vertical}
4645 {Currently, layout related features can be adjusted only\\%
4646   in vertical mode}%
4647 {Maybe things change in the future, but this is what it is.}
4648 \bbl@errmessage{bidi-only-lua}
4649 {The bidi method 'basic' is available only in\\%
4650   luatex. I'll continue with 'bidi=default', so\\%
4651   expect wrong results.\\%
4652   Suggested actions:\\%
4653   * If possible, switch to luatex, as xetex is not\\%
4654     recommend anymore.\\
4655   * If you can't, try 'bidi=bidi' with xetex.\\%
4656   * With pdftex, only 'bidi=default' is available.}%
4657 {See the manual for further details.}
4658 \bbl@errmessage{multiple-bidi}
4659 {Multiple bidi settings inside a group\\%
4660   I'll insert a new group, but expect wrong results.\\%
4661   Suggested action:\\%
4662   * Add a new group where appropriate.}
4663 {See the manual for further details.}
4664 \bbl@errmessage{unknown-package-option}
4665 {Unknown option '\CurrentOption'.\\%
4666   Suggested actions:\\%
4667   * Make sure you haven't misspelled it\\%
4668   * Check in the babel manual that it's supported\\%

```

```

4669 * If supported and it's a language, you may\\%
4670 \space\space need in some distributions a separate\\%
4671 \space\space installation\\%
4672 * If installed, check there isn't an old\\%
4673 \space\space version of the required files in your system\\%
4674 * If it's an unsupported language, create it with\\%
4675 \string\babelprovide (see the manual)}
4676 {Valid options are, among others: shorthands=, KeepShorthandsActive,\\%
4677 activeacute, activegrave, noconfigs, safe=, main=, math=\\%
4678 headfoot=, strings=, config=, hyphenmap=, or a language name.}
4679 \bbl@errmessage{config-not-found}
4680 {Local config file '\bbl@opt@config.cfg' not found.\\%
4681 Suggested actions:\\%
4682 * Make sure you haven't misspelled it in config=\\%
4683 * Check it exists and it's in the correct path}%
4684 {Perhaps you misspelled it.}
4685 \bbl@errmessage{late-after-babel}
4686 {Too late for \string\AfterBabelLanguage}%
4687 {Languages have been loaded, so I can do nothing}
4688 \bbl@errmessage{double-hyphens-class}
4689 {Double hyphens aren't allowed in \string\babelcharclass\\%
4690 because it's potentially ambiguous}%
4691 {See the manual for further info}
4692 \bbl@errmessage{unknown-interchar}
4693 {'#1' for '\language' cannot be enabled.\\%
4694 Maybe there is a typo}%
4695 {See the manual for further details.}
4696 \bbl@errmessage{unknown-interchar-b}
4697 {'#1' for '\language' cannot be disabled.\\%
4698 Maybe there is a typo}%
4699 {See the manual for further details.}
4700 \bbl@errmessage{charproperty-only-vertical}
4701 {\string\babelcharproperty\space can be used only in\\%
4702 vertical mode (preamble or between paragraphs)}%
4703 {See the manual for further info}
4704 \bbl@errmessage{unknown-char-property}
4705 {No property named '#2'. Allowed values are\\%
4706 direction (bc), mirror (bmg), and linebreak (lb)}%
4707 {See the manual for further info}
4708 \bbl@errmessage{bad-transform-option}
4709 {Bad option '#1' in a transform.\\%
4710 I'll ignore it but expect more errors}%
4711 {See the manual for further info.}
4712 \bbl@errmessage{font-conflict-transforms}
4713 {Transforms cannot be re-assigned to different\\%
4714 fonts. The conflict is in '\bbl@kv@label'.\\%
4715 Apply the same fonts or use a different label}%
4716 {See the manual for further details.}
4717 \bbl@errmessage{transform-not-available}
4718 {'#1' for '\language' cannot be enabled.\\%
4719 Maybe there is a typo or it's a font-dependent transform}%
4720 {See the manual for further details.}
4721 \bbl@errmessage{transform-not-available-b}
4722 {'#1' for '\language' cannot be disabled.\\%
4723 Maybe there is a typo or it's a font-dependent transform}%
4724 {See the manual for further details.}
4725 \bbl@errmessage{year-out-range}
4726 {Year out of range.\\%
4727 The allowed range is #1}%
4728 {See the manual for further details.}
4729 \bbl@errmessage{only-pdfTeX-lang}
4730 {The '#1' ldf style doesn't work with #2,\\%
4731 but you can use the ini locale instead.\\%

```

```

4732 Try adding 'provide=*' to the option list. You may\\%
4733 also want to set 'bidi=' to some value}%
4734 {See the manual for further details.}
4735 \bbl@errmessage{hyphenmins-args}
4736 {\string\babelhyphenmins\ accepts either the optional\\%
4737 argument or the star, but not both at the same time}%
4738 {See the manual for further details.}
4739 \bbl@errmessage{no-locale-for-meta}
4740 {There isn't currently a locale for the 'lang' requested\\%
4741 in the PDF metadata ('#1'). To fix it, you can\\%
4742 set explicitly a similar language (using the same\\%
4743 script) with the key main= when loading babel. If you\\%
4744 continue, I'll fallback to the 'nil' language, with\\%
4745 tag 'und' and script 'Latn', but expect a bad font\\%
4746 rendering with other scripts. You may also need set\\%
4747 explicitly captions and date, too}%
4748 {See the manual for further details.}
4749 (/errors)
4750 (*patterns)

```

8 Loading hyphenation patterns

The following code is meant to be read by `iniTEX` because it should instruct `TEX` to read hyphenation patterns. To this end the `docstrip` option `patterns` is used to include this code in the file `hyphen.cfg`. Code is written with lower level macros.

```

4751 <@Make sure ProvidesFile is defined@>
4752 \ProvidesFile{hyphen.cfg}[<@date@> v<@version@> Babel hyphens]
4753 \xdef\bbl@format{\jobname}
4754 \def\bbl@version{<@version@>}
4755 \def\bbl@date{<@date@>}
4756 \ifx\AtBeginDocument\@undefined
4757 \def\@empty{}
4758 \fi
4759 <@Define core switching macros@>

```

\process@line Each line in the file `language.dat` is processed by `\process@line` after it is read. The first thing this macro does is to check whether the line starts with `=`. When the first token of a line is an `=`, the macro `\process@synonym` is called; otherwise the macro `\process@language` will continue.

```

4760 \def\process@line#1#2 #3 #4 {%
4761 \ifx=#1%
4762 \process@synonym{#2}%
4763 \else
4764 \process@language{#1#2}{#3}{#4}%
4765 \fi
4766 \ignorespaces}

```

\process@synonym This macro takes care of the lines which start with an `=`. It needs an empty token register to begin with. `\bbl@languages` is also set to empty.

```

4767 \toks@{}
4768 \def\bbl@languages{}

```

When no languages have been loaded yet, the name following the `=` will be a synonym for hyphenation register 0. So, it is stored in a token register and executed when the first pattern file has been processed. (The `\relax` just helps to the `\if` below catching synonyms without a language.)

Otherwise the name will be a synonym for the language loaded last.

We also need to copy the `hyphenmin` parameters for the synonym.

```

4769 \def\process@synonym#1{%
4770 \ifnum\last@language=\m@ne
4771 \toks@\expandafter{\the\toks@\relax\process@synonym{#1}}%
4772 \else
4773 \expandafter\chardef\csname l@#1\endcsname\last@language

```

```

4774 \wlog{\string\l@#1=\string\language\the\last@language}%
4775 \expandafter\let\csname #1hyphenmins\expandafter\endcsname
4776 \csname\language\hyphenmins\endcsname
4777 \let\bbl@elt\relax
4778 \edef\bbl@languages{\bbl@languages\bbl@elt{#1}{\the\last@language}{}}}%
4779 \fi}

```

\process@language The macro `\process@language` is used to process a non-empty line from the ‘configuration file’. It has three arguments, each delimited by white space. The first argument is the ‘name’ of a language; the second is the name of the file that contains the patterns. The optional third argument is the name of a file containing hyphenation exceptions.

The first thing to do is call `\addlanguage` to allocate a pattern register and to make that register ‘active’. Then the pattern file is read.

For some hyphenation patterns it is needed to load them with a specific font encoding selected. This can be specified in the file `language.dat` by adding for instance ‘:T1’ to the name of the language. The macro `\bbl@get@enc` extracts the font encoding from the language name and stores it in `\bbl@hyph@enc`. The latter can be used in hyphenation files if you need to set a behavior depending on the given encoding (it is set to empty if no encoding is given).

Pattern files may contain assignments to `\lefthyphenmin` and `\righthyphenmin`. `TEX` does not keep track of these assignments. Therefore we try to detect such assignments and store them in the `\(language)hyphenmins` macro. When no assignments were made we provide a default setting.

Some pattern files contain changes to the `\lccode` or `\uccode` arrays. Such changes should remain local to the language; therefore we process the pattern file in a group; the `\patterns` command acts globally so its effect will be remembered.

Then we globally store the settings of `\lefthyphenmin` and `\righthyphenmin` and close the group.

When the hyphenation patterns have been processed we need to see if a file with hyphenation exceptions needs to be read. This is the case when the third argument is not empty and when it does not contain a space token. (Note however there is no need to save hyphenation exceptions into the format.)

`\bbl@languages` saves a snapshot of the loaded languages in the form `\bbl@elt{<language-name>}{<number>}{<patterns-file>}{<exceptions-file>}`. Note the last 2 arguments are empty in ‘dialects’ defined in `language.dat` with `=`. Note also the language name can have encoding info.

Finally, if the counter `\language` is equal to zero we execute the synonyms stored.

```

4780 \def\process@language#1#2#3{%
4781 \expandafter\addlanguage\csname l@#1\endcsname
4782 \expandafter\language\csname l@#1\endcsname
4783 \edef\language{#1}%
4784 \bbl@hook@everylanguage{#1}%
4785 % > luatex
4786 \bbl@get@enc#1::\@@@
4787 \begingroup
4788 \lefthyphenmin\m@ne
4789 \bbl@hook@loadpatterns{#2}%
4790 % > luatex
4791 \ifnum\lefthyphenmin=\m@ne
4792 \else
4793 \expandafter\xdef\csname #1hyphenmins\endcsname{%
4794 \the\lefthyphenmin\the\righthyphenmin}%
4795 \fi
4796 \endgroup
4797 \def\bbl@tempa{#3}%
4798 \ifx\bbl@tempa\@empty\else
4799 \bbl@hook@loadexceptions{#3}%
4800 % > luatex
4801 \fi
4802 \let\bbl@elt\relax
4803 \edef\bbl@languages{%
4804 \bbl@languages\bbl@elt{#1}{\the\language}{#2}{\bbl@tempa}}%
4805 \ifnum\the\language=\z@
4806 \expandafter\ifx\csname #1hyphenmins\endcsname\relax
4807 \set@hyphenmins\tw@\thr@@\relax
4808 \else

```



```

4809     \expandafter\expandafter\expandafter\set@hyphenmins
4810     \csname #1hyphenmins\endcsname
4811     \fi
4812     \the\toks@
4813     \toks@{}%
4814     \fi}

```

\bbl@get@enc

\bbl@hyph@enc The macro \bbl@get@enc extracts the font encoding from the language name and stores it in \bbl@hyph@enc. It uses delimited arguments to achieve this.

```

4815 \def\bbl@get@enc#1:#2:#3\@@@{\def\bbl@hyph@enc{#2}}

```

Now, hooks are defined. For efficiency reasons, they are dealt here in a special way. Besides luatex, format-specific configuration files are taken into account. loadkernel currently loads nothing, but define some basic macros instead.

```

4816 \def\bbl@hook@everylanguage#1{}
4817 \def\bbl@hook@loadpatterns#1{\input #1\relax}
4818 \let\bbl@hook@loadexceptions\bbl@hook@loadpatterns
4819 \def\bbl@hook@loadkernel#1{%
4820   \def\addlanguage{\csname newlanguage\endcsname}%
4821   \def\adddialect##1##2{%
4822     \global\chardef##1##2\relax
4823     \wlog{\string##1 = a dialect from \string\language##2}}%
4824   \def\iflanguage##1{%
4825     \expandafter\ifx\csname l@##1\endcsname\relax
4826       \@nolanerr{##1}%
4827     \else
4828       \ifnum\csname l@##1\endcsname=\language
4829         \expandafter\expandafter\expandafter\@firstoftwo
4830       \else
4831         \expandafter\expandafter\expandafter\@secondoftwo
4832       \fi
4833     \fi}%
4834   \def\providehyphenmins##1##2{%
4835     \expandafter\ifx\csname ##1hyphenmins\endcsname\relax
4836       \@namedef{##1hyphenmins}{##2}%
4837     \fi}%
4838   \def\set@hyphenmins##1##2{%
4839     \lefthyphenmin##1\relax
4840     \righthyphenmin##2\relax}%
4841   \def\selectlanguage{%
4842     \errhelp{Selecting a language requires a package supporting it}%
4843     \errmessage{No multilingual package has been loaded}}%
4844   \let\foreignlanguage\selectlanguage
4845   \let\otherlanguage\selectlanguage
4846   \expandafter\let\csname otherlanguage*\endcsname\selectlanguage
4847   \def\bbl@usehooks##1##2{%
4848     \def\setlocale{%
4849       \errhelp{Find an armchair, sit down and wait}%
4850       \errmessage{(babel) Not yet available}}%
4851     \let\uselocale\setlocale
4852     \let\locale\setlocale
4853     \let\selectlocale\setlocale
4854     \let\localename\setlocale
4855     \let\textlocale\setlocale
4856     \let\textlanguage\setlocale
4857     \let\languagetext\setlocale}
4858 \begingroup
4859 \def\AddBabelHook#1#2{%
4860   \expandafter\ifx\csname bbl@hook@#2\endcsname\relax
4861     \def\next{\toks1}%
4862   \else
4863     \def\next{\expandafter\gdef\csname bbl@hook@#2\endcsname###1}%

```

```

4864 \fi
4865 \next}
4866 \ifx\directlua\@undefined
4867 \ifx\XeTeXinputencoding\@undefined\else
4868 \input xebabel.def
4869 \fi
4870 \else
4871 \input luababel.def
4872 \fi
4873 \openin1 = babel-\bbl@format.cfg
4874 \ifeof1
4875 \else
4876 \input babel-\bbl@format.cfg\relax
4877 \fi
4878 \closein1
4879 \endgroup
4880 \bbl@hook@loadkernel{switch.def}

```

\readconfigfile The configuration file can now be opened for reading.

```

4881 \openin1 = language.dat

```

See if the file exists, if not, use the default hyphenation file `hyphen.tex`. The user will be informed about this.

```

4882 \def\language{english}%
4883 \ifeof1
4884 \message{I couldn't find the file language.dat,\space
4885          I will try the file hyphen.tex}
4886 \input hyphen.tex\relax
4887 \chardef\l@english\z@
4888 \else

```

Pattern registers are allocated using count register `\last@language`. Its initial value is 0. The definition of the macro `\newlanguage` is such that it first increments the count register and then defines the language. In order to have the first patterns loaded in pattern register number 0 we initialize `\last@language` with the value `-1`.

```

4889 \last@language\m@ne

```

We now read lines from the file until the end is found. While reading from the input, it is useful to switch off recognition of the end-of-line character. This saves us stripping off spaces from the contents of the control sequence.

```

4890 \loop
4891 \endlinechar\m@ne
4892 \read1 to \bbl@line
4893 \endlinechar\^^M

```

If the file has reached its end, exit from the loop here. If not, empty lines are skipped. Add 3 space characters to the end of `\bbl@line`. This is needed to be able to recognize the arguments of `\process@line` later on. The default language should be the very first one.

```

4894 \if T\ifeof1F\fi T\relax
4895 \ifx\bbl@line\@empty\else
4896 \edef\bbl@line{\bbl@line\space\space\space}%
4897 \expandafter\process@line\bbl@line\relax
4898 \fi
4899 \repeat

```

Check for the end of the file. We must reverse the test for `\ifeof` without `\else`. Then reactivate the default patterns, and close the configuration file.

```

4900 \begingroup
4901 \def\bbl@elt#1#2#3#4{%
4902 \global\language=#2\relax
4903 \gdef\language{#1}%
4904 \def\bbl@elt##1##2##3##4{}}%
4905 \bbl@languages

```

```

4906 \endgroup
4907 \fi
4908 \closein1

```

We add a message about the fact that babel is loaded in the format and with which language patterns to the \everyjob register.

```

4909 \if/\the\toks@\else
4910 \errhelp{language.dat loads no language, only synonyms}
4911 \errmessage{Orphan language synonym}
4912 \fi

```

Also remove some macros from memory and raise an error if \toks@ is not empty. Finally load switch.def, but the latter is not required and the line inputting it may be commented out.

```

4913 \let\bbl@line\@undefined
4914 \let\process@line\@undefined
4915 \let\process@synonym\@undefined
4916 \let\process@language\@undefined
4917 \let\bbl@get@enc\@undefined
4918 \let\bbl@hyph@enc\@undefined
4919 \let\bbl@tempa\@undefined
4920 \let\bbl@hook@loadkernel\@undefined
4921 \let\bbl@hook@everylanguage\@undefined
4922 \let\bbl@hook@loadpatterns\@undefined
4923 \let\bbl@hook@loadexceptions\@undefined
4924 (/patterns)

```

Here the code for init_{TeX} ends.

9 luatex + xetex: common stuff

Add the bidi handler just before luaotfload, which is loaded by default by LaTeX. Just in case, consider the possibility it has not been loaded. First, a couple of definitions related to bidi (although default also applies to pdftex).

```

4925 ((*More package options)) ≡
4926 \chardef\bbl@bidimode\z@
4927 \DeclareOption{bidi=default}{\chardef\bbl@bidimode=\@ne}
4928 \DeclareOption{bidi=basic}{\chardef\bbl@bidimode=101 }
4929 \DeclareOption{bidi=basic-r}{\chardef\bbl@bidimode=102 }
4930 \DeclareOption{bidi=bidi}{\chardef\bbl@bidimode=201 }
4931 \DeclareOption{bidi=bidi-r}{\chardef\bbl@bidimode=202 }
4932 \DeclareOption{bidi=bidi-l}{\chardef\bbl@bidimode=203 }
4933 (/More package options)

```

\babelfont With explicit languages, we could define the font at once, but we don't. Just wait and see if the language is actually activated. bbl@font replaces hardcoded font names inside \. . family by the corresponding macro \. . default.

```

4934 ((*Font selection)) ≡
4935 \bbl@trace{Font handling with fontspec}
4936 \AddBabelHook{babel-fontspec}{afterextras}{\bbl@switchfont}
4937 \AddBabelHook{babel-fontspec}{beforestart}{\bbl@ckckstdfonts}
4938 \DisableBabelHook{babel-fontspec}
4939 \@onlypreamble\babelfont
4940 \ifx\NewDocumentCommand\@undefined\else % Not plain
4941 \NewDocumentCommand\babelfont{0}{m0}{m0}{}%
4942 \bbl@bblfont@o[#1]{#2}[#3,#5]{#4}}
4943 \fi
4944 \newcommand\bbl@bblfont@o[2][]{% 1=langs/scripts 2=fam
4945 \ifx\fontspec\@undefined
4946 \usepackage{fontspec}%
4947 \fi
4948 \EnableBabelHook{babel-fontspec}%
4949 \edef\bbl@tempa{#1}%

```

```

4950 \def\bbl@tempb{#2}% Used by \bbl@bblfont
4951 \bbl@bblfont}
4952 \newcommand\bbl@bblfont[2][{}]{% 1=features 2=fontname, @font=rm|sf|tt
4953 \bbl@ifunset{\bbl@tempb family}%
4954 {\bbl@providefam{\bbl@tempb}}%
4955 {}%
4956 % For the default font, just in case:
4957 \bbl@ifunset{\bbl@lsys{\language}\bbl@provide@lsys{\language}}{}%
4958 \expandafter\bbl@ifblank\expandafter{\bbl@tempa}%
4959 {\bbl@csarg\edef{\bbl@tempb dflt@}{<{#1}{#2}}% save bbl@rmdflt@
4960 \bbl@exp{%
4961 \let<\bbl@tempb dflt@\language>\<\bbl@tempb dflt@>%
4962 \\\bbl@font@set<\bbl@tempb dflt@\language>%
4963 \<\bbl@tempb default>\<\bbl@tempb family>}}%
4964 {\bbl@foreach\bbl@tempa{% i.e., bbl@rmdflt@lang / *scrt
4965 \bbl@csarg\def{\bbl@tempb dflt@##1}{<{#1}{#2}}}}%

```

If the family in the previous command does not exist, it must be defined. Here is how:

```

4966 \def\bbl@providefam#1{%
4967 \bbl@exp{%
4968 \\\newcommand\<#1default>{}% Just define it
4969 \\\bbl@add@list\\bbl@font@fams{#1}%
4970 \\\NewHook{#1family}%
4971 \\\DeclareRobustCommand\<#1family>{%
4972 \\\not@math@alphabet\<#1family>\relax
4973 % \\\prepare@family@series@update{#1}\<#1default>% TODO. Fails
4974 \\\fontfamily\<#1default>%
4975 \\\UseHook{#1family}%
4976 \\\selectfont}%
4977 \\\DeclareTextFontCommand{\<text#1>}{\<#1family>}}}

```

The following macro is activated when the hook babel-fontspec is enabled. But before, we define a macro for a warning, which sets a flag to avoid duplicate them.

```

4978 \def\bbl@nostdfont#1{%
4979 \bbl@once{nostdfam-\f@family}%
4980 {\bbl@infowarn{The current font is not a babel standard family:\%
4981 #1%
4982 \fontname\font\\%
4983 There is nothing intrinsically wrong, and you can\\%,
4984 ignore this message altogether if you do not need\\%
4985 this font. If they are used in the document, be aware\\%
4986 'babel' will not set Script and Language for it, so\\%
4987 you may consider defining a new family with \string\babelfont.\\%
4988 See the manual for further details about \string\babelfont.
4989 Reported}}%
4990 {}}%
4991 \gdef\bbl@switchfont{%
4992 \bbl@ifunset{\bbl@lsys{\language}\bbl@provide@lsys{\language}}{}%
4993 \bbl@exp{% e.g., Arabic -> arabic
4994 \lowercase{\edef\\bbl@tempa{\bbl@cl{sname}}}}%
4995 \bbl@foreach\bbl@font@fams{%
4996 \bbl@ifunset{\bbl@##1dflt@\language}% (1) language?
4997 {\bbl@ifunset{\bbl@##1dflt@*\bbl@tempa}% (2) from script?
4998 {\bbl@ifunset{\bbl@##1dflt@}% 2=F - (3) from generic?
4999 {}% 123=F - nothing!
5000 {\bbl@exp{% 3=T - from generic
5001 \global\let<\bbl@##1dflt@\language>%
5002 \<\bbl@##1dflt@>}}}%
5003 {\bbl@exp{% 2=T - from script
5004 \global\let<\bbl@##1dflt@\language>%
5005 \<\bbl@##1dflt@*\bbl@tempa>}}}%
5006 {}}% 1=T - language, already defined
5007 \def\bbl@tempa{\bbl@nostdfont}}%
5008 \bbl@foreach\bbl@font@fams{% don't gather with prev for

```

```

5009 \bbl@ifunset{bbl@##ldflt@\language}%
5010 {\bbl@cs{famrst@##1}%
5011 \global\bbl@csarg\let{famrst@##1}\relax}%
5012 {\bbl@exp{% order is relevant.
5013 \\\bbl@add\\\originalTeX%
5014 \\\bbl@font@rst{bbl@cl{##ldflt}}}%
5015 \<##ldefault>\<##lfamily>{##1}}%
5016 \\\bbl@font@set\<bbl@##ldflt@\language>% the main part!
5017 \<##ldefault>\<##lfamily>}}}%
5018 \bbl@ifrestoring{ }\bbl@tempa}%

```

The following is executed at the beginning of the aux file or the document to warn about fonts not defined with `\babelfont`.

```

5019 \ifx\f@family\undefined\else % if latex
5020 \ifcase\bbl@engine % if pdftex
5021 \let\bbl@ckeckstdfonts\relax
5022 \else
5023 \def\bbl@ckeckstdfonts{%
5024 \begingroup
5025 \global\let\bbl@ckeckstdfonts\relax
5026 \let\bbl@tempa\empty
5027 \bbl@foreach\bbl@font@fams{%
5028 \bbl@ifunset{bbl@##ldflt@}%
5029 {\@nameuse{##lfamily}%
5030 \bbl@csarg\gdef{WFF@f@family}}}% Flag
5031 \bbl@exp{\\\bbl@add\\\bbl@tempa{* \<##lfamily>= \f@family\\}%
5032 \space\space\fontname\font\\}%
5033 \bbl@csarg\xdef{##ldflt@}{f@family}%
5034 \expandafter\xdef\csname ##ldefault\endcsname{f@family}}%
5035 {}}%
5036 \ifx\bbl@tempa\empty\else
5037 \bbl@infowarn{The following font families will use the default\\%
5038 settings for all or some languages:\\%
5039 \bbl@tempa
5040 There is nothing intrinsically wrong with it, but\\%
5041 'babel' will no set Script and Language, which could\\%
5042 be relevant in some languages. If your document uses\\%
5043 these families, consider redefining them with \string\babelfont.\\%
5044 Reported}%
5045 \fi
5046 \endgroup}
5047 \fi
5048 \fi

```

Now the macros defining the font with fontspec.

When there are repeated keys in fontspec, the last value wins. So, we just place the ini settings at the beginning, and user settings will take precedence. We must deactivate temporarily `\bbl@mapselect` because `\selectfont` is called internally when a font is defined.

For historical reasons, $\text{\LaTeX}$ can select two different series (bx and b), for what is conceptually a single one. This can lead to problems when a single family requires several fonts, depending on the language, mainly because ‘substitutions’ with some combinations are not done consistently – sometimes bx/sc is the correct font, but sometimes points to b/n, even if b/sc exists. So, some substitutions are redefined (in a somewhat hackish way, by inspecting if the variant declaration contains `>ssub*`).

```

5049 \def\bbl@font@set#1#2#3{% e.g., \bbl@rmdflt@lang \rmdefault \rmfamily
5050 \bbl@xin@{<>}{#1}%
5051 \ifin@
5052 \bbl@exp{\\\bbl@fontspec@set\\#1\expandafter\@gobbletwo#1\\#3}%
5053 \fi
5054 \bbl@exp{%
5055 \def\\#2{#1}% e.g., \rmdefault{\bbl@rmdflt@lang}
5056 \\\bbl@ifsamestring{#2}{f@family}%
5057 {\#3%
5058 \\\bbl@ifsamestring{f@series}{\bfdefault}{\bfseries}}}%

```

```

5059 \let\\bbl@tempa\relax}%
5060 {}}}}

```

Loaded locally, which does its job, but very must be global. The problem is how. This actually defines a font predeclared with `\babelfont`, making sure Script and Language names are defined. If they are not, the corresponding data in the ini file is used. The font is actually set temporarily to get the family name (`\f@family`). There is also a hack because by default some replacements related to the bold series are sometimes assigned to the wrong font (see issue #92).

```

5061 \def\bbl@fontspec@set#1#2#3#4{% eg \bbl@rmdflt@lang fnt-opt fnt-nme \xxfamily
5062 \let\bbl@tempe\bbl@mapselect
5063 \edef\bbl@tempb{\bbl@stripslash#4}% Catcodes hack (better pass it).
5064 \bbl@exp{\bbl@replace\bbl@tempb{\bbl@stripslash\family/}}}%
5065 \let\bbl@mapselect\relax
5066 \let\bbl@temp@fam#4% e.g., '\rmfamily', to be restored below
5067 \let#4\empty % Make sure \renewfontfamily is valid
5068 \bbl@set@renderer
5069 \bbl@exp{%
5070 \let\\bbl@temp@pfam<\bbl@stripslash#4\space>% e.g., '\rmfamily '
5071 \<keys_if_exist:nnF>{fontspec-opentype}{Script/\bbl@cl{sname}}}%
5072 {\bbl@cl{sname}}{\bbl@cl{sotf}}}%
5073 \<keys_if_exist:nnF>{fontspec-opentype}{Language/\bbl@cl{lname}}}%
5074 {\bbl@cl{lname}}{\bbl@cl{lotf}}}%
5075 \\renewfontfamily\\#4%
5076 [\bbl@cl{lsys},% xetex removes unknown features :-(
5077 \ifcase\bbl@engine\or RawFeature={family=\bbl@tempb},\fi
5078 #2}}{#3}% i.e., \bbl@exp{.}{#3}
5079 \bbl@unset@renderer
5080 \begingroup
5081 #4%
5082 \xdef#1{\f@family}% e.g., \bbl@rmdflt@lang{FreeSerif(0)}
5083 \endgroup
5084 \bbl@xin@{\detokenize{>ssub*}}}%
5085 {\expandafter\meaning\csname TU/#1/bx/sc\endcsname}%
5086 \ifin@
5087 \global\bbl@ccarg\let{TU/#1/bx/sc}{TU/#1/b/sc}%
5088 \fi
5089 \bbl@xin@{\detokenize{>ssub*}}}%
5090 {\expandafter\meaning\csname TU/#1/bx/scit\endcsname}%
5091 \ifin@
5092 \global\bbl@ccarg\let{TU/#1/bx/scit}{TU/#1/b/scit}%
5093 \fi
5094 \let#4\bbl@temp@fam
5095 \bbl@exp{\let<\bbl@stripslash#4\space>\bbl@temp@pfam
5096 \let\bbl@mapselect\bbl@tempe}%

```

`font@rst` and `famrst` are only used when there is no global settings, to save and restore de previous families. Not really necessary, but done for optimization.

```

5097 \def\bbl@font@rst#1#2#3#4{%
5098 \bbl@csarg\def{famrst@#4}{\bbl@font@set{#1}#2#3}}

```

The default font families. They are eurocentric, but the list can be expanded easily with `\babelfont`.

```

5099 \def\bbl@font@fams{rm,sf,tt}
5100 (
```

10 Hooks for XeTeX and LuaTeX

10.1 XeTeX

Unfortunately, the current encoding cannot be retrieved and therefore it is reset always to `utf8`, which seems a sensible default.

Now, the code.

```

5101 (*xetex)

```

```

5102 \def\BabelStringsDefault{unicode}
5103 \let\xebbl@stop\relax
5104 \AddBabelHook{xetex}{encodedcommands}{%
5105   \def\bbl@tempa{#1}%
5106   \ifx\bbl@tempa\@empty
5107     \XeTeXinputencoding"bytes"%
5108   \else
5109     \XeTeXinputencoding"#1"%
5110   \fi
5111   \def\xebbl@stop{\XeTeXinputencoding"utf8"}}
5112 \AddBabelHook{xetex}{stopcommands}{%
5113   \xebbl@stop
5114   \let\xebbl@stop\relax}
5115 \def\bbl@input@classes{% Used in CJK intraspaces
5116   \input{load-unicode-xetex-classes.tex}%
5117   \let\bbl@input@classes\relax}
5118 \def\bbl@intraspace#1 #2 #3\@{%
5119   \bbl@csarg\gdef{xeisp@\language}%
5120   {\XeTeXlinebreakskip #1em plus #2em minus #3em\relax}}
5121 \def\bbl@intrapenalty#1\@{%
5122   \bbl@csarg\gdef{xeipn@\language}%
5123   {\XeTeXlinebreakpenalty #1\relax}}
5124 \def\bbl@provide@intraspace{%
5125   \bbl@xin@{/s}{/\bbl@cl{lnbrk}}}%
5126   \ifin@else\bbl@xin@{/c}{/\bbl@cl{lnbrk}}\fi
5127   \ifin@
5128     \bbl@ifunset{bbl@intsp@\language}{}%
5129     {\expandafter\ifx\csname bbl@intsp@\language\endcsname\@empty\else
5130       \ifx\bbl@KVP@intraspace\@nnil
5131         \bbl@exp{%
5132           \\bbl@intraspace\bbl@cl{intsp}\@@}%
5133         \fi
5134         \ifx\bbl@KVP@intrapenalty\@nnil
5135           \bbl@intrapenalty0\@@
5136         \fi
5137       \fi
5138       \ifx\bbl@KVP@intraspace\@nnil\else % We may override the ini
5139         \expandafter\bbl@intraspace\bbl@KVP@intraspace\@@
5140       \fi
5141       \ifx\bbl@KVP@intrapenalty\@nnil\else
5142         \expandafter\bbl@intrapenalty\bbl@KVP@intrapenalty\@@
5143       \fi
5144       \bbl@exp{%
5145         \\bbl@add\<extras\language>{%
5146           \XeTeXlinebreaklocale "\bbl@cl{tbcpr}"%
5147           \<bbl@xeisp@\language>%
5148           \<bbl@xeipn@\language>}%
5149         \\bbl@tglobal\<extras\language>%
5150         \\bbl@add\<noextras\language>{%
5151           \XeTeXlinebreaklocale ""}%
5152         \\bbl@tglobal\<noextras\language>}%
5153       \ifx\bbl@ispacesize\@undefined
5154         \gdef\bbl@ispacesize{\bbl@cl{xeisp}}}%
5155       \ifx\AtBeginDocument\@notprerr
5156         \expandafter\@secondoftwo % to execute right now
5157       \fi
5158       \AtBeginDocument{\bbl@patchfont{\bbl@ispacesize}}%
5159     \fi}%
5160   \fi}
5161 \ifx\DisableBabelHook\@undefined\endinput\fi
5162 \let\bbl@set@renderer\relax
5163 \let\bbl@unset@renderer\relax
5164 <@Font selection@>

```

```
5165 \def\bbl@provide@extra#1{}
```

Hack for unhyphenated line breaking. See `\bbl@provide@lsys` in the common code.

```
5166 \def\bbl@xenoxyph@d{%
5167   \bbl@ifset{bbl@prehc@}\languagename}%
5168   {\ifnum\hyphenchar\font=\defaultthyphenchar
5169     \iffontchar\font\bbl@cl{prehc}\relax
5170     \hyphenchar\font\bbl@cl{prehc}\relax
5171   \else\iffontchar\font"200B
5172     \hyphenchar\font"200B
5173   \else
5174     \bbl@warning
5175     {Neither 0 nor ZERO WIDTH SPACE are available\\%
5176     in the current font, and therefore the hyphen\\%
5177     will be printed. Try changing the fontspec's\\%
5178     'HyphenChar' to another value, but be aware\\%
5179     this setting is not safe (see the manual).\\%
5180     Reported}%
5181     \hyphenchar\font\defaultthyphenchar
5182   \fi\fi
5183   \fi}%
5184   {\hyphenchar\font\defaultthyphenchar}}
```

10.2 Support for interchar

`xetex` reserves some values for CJK (although they are not set in `XELATEX`), so we make sure they are skipped. Define some user names for the global classes, too.

```
5185 \ifnum\Xe@alloc@intercharclass<\thr@@
5186   \Xe@alloc@intercharclass\thr@@
5187 \fi
5188 \chardef\bbl@xeclass@default@=\z@
5189 \chardef\bbl@xeclass@cjkideogram@=\@ne
5190 \chardef\bbl@xeclass@cjkleftpunctuation@=\tw@
5191 \chardef\bbl@xeclass@cjkrightpunctuation@=\thr@@
5192 \chardef\bbl@xeclass@boundary@=4095
5193 \chardef\bbl@xeclass@ignore@=4096
```

The machinery is activated with a hook (enabled only if actually used). Here `\bbl@tempc` is pre-set with `\bbl@usingxeclass`, defined below. The standard mechanism based on `\originalTeX` to save, set and restore values is used. `\count@` stores the previous char to be set, except at the beginning (0) and after `\bbl@upto`, which is the previous char negated, as a flag to mark a range.

```
5194 \AddBabelHook{babel-interchar}{beforeextras}{%
5195   \@nameuse{bbl@xechars@\languagename}}
5196 \DisableBabelHook{babel-interchar}
5197 \protected\def\bbl@charclass#1{%
5198   \ifnum\count@<\z@
5199     \count@-\count@
5200   \loop
5201     \bbl@exp{%
5202       \\babel@savevariable{\XeTeXcharclass`\Uchar\count@}}%
5203       \XeTeXcharclass\count@ \bbl@tempc
5204     \ifnum\count@<`#1\relax
5205       \advance\count@\@ne
5206     \repeat
5207   \else
5208     \babel@savevariable{\XeTeXcharclass`#1}%
5209     \XeTeXcharclass`#1 \bbl@tempc
5210   \fi
5211   \count@`#1\relax}
```

Now the two user macros. Char classes are declared implicitly, and then the macro to be executed at the `babel-interchar` hook is created. The list of chars to be handled by the hook defined above has internally the form `\bbl@usingxeclass\bbl@xeclass@punct@english\bbl@charclass{.}` `\bbl@charclass{,}` (etc.), where `\bbl@usingxeclass` stores the class to be applied to the

subsequent characters. The `\ifcat` part deals with the alternative way to enter characters as macros (e.g., `\}`). As a special case, hyphens are stored as `\bbl@upto`, to deal with ranges.

```

5212 \newcommand\bbl@ifinterchar[1]{%
5213   \let\bbl@tempa\@gobble           % Assume to ignore
5214   \edef\bbl@tempb{\zap@space#1 \@empty}%
5215   \ifx\bbl@KVP@interchar\@nnil\else
5216     \bbl@replace\bbl@KVP@interchar{ }{,}%
5217     \bbl@foreach\bbl@tempb{%
5218       \bbl@xin@{,##1,}{, \bbl@KVP@interchar,}%
5219     }
5220     \let\bbl@tempa\@firstofone
5221   \fi}%
5222 \fi
5223 \bbl@tempa}
5224 \newcommand\IfBabelIntercharT[2]{%
5225   \bbl@carg\bbl@add{\bbl@icsave@CurrentOption}{\bbl@ifinterchar{#1}{#2}}}%
5226 \newcommand\babelcharclass[3]{%
5227   \EnableBabelHook{babel-interchar}%
5228   \bbl@csarg\newXeTeXintercharclass{xeclass@#2@#1}%
5229   \def\bbl@tempb##1{%
5230     \ifx##1\@empty\else
5231       \ifx##1-%
5232         \bbl@upto
5233       \else
5234         \bbl@charclass{%
5235           \ifcat\noexpand##1\relax\bbl@stripslash##1\else\string##1\fi}%
5236         \fi
5237         \expandafter\bbl@tempb
5238       \fi}%
5239   \bbl@ifunset{\bbl@xechars@#1}%
5240   {\toks@{%
5241     \babel@savevariable\XeTeXinterchartokenstate
5242     \XeTeXinterchartokenstate\@ne
5243   }}%
5244   {\toks@\expandafter\expandafter\expandafter{%
5245     \csname bbl@xechars@#1\endcsname}}%
5246   \bbl@csarg\edef{xechars@#1}{%
5247     \the\toks@
5248     \bbl@usingxeclass\csname bbl@xeclass@#2@#1\endcsname
5249     \bbl@tempb#3\@empty}}
5250 \protected\def\bbl@usingxeclass#1{\count@\z@ \let\bbl@tempc#1}
5251 \protected\def\bbl@upto{%
5252   \ifnum\count@>\z@
5253     \advance\count@\@ne
5254     \count@-\count@
5255   \else\ifnum\count@=\z@
5256     \bbl@charclass{-}%
5257   \else
5258     \bbl@error{double-hyphens-class}{\count@}{\count@}%
5259   \fi\fi}

```

And finally, the command with the code to be inserted. If the language doesn't define a class, then use the global one, as defined above. For the definition there is an intermediate macro, which can be 'disabled' with `\bbl@ic@<label>@<language>`.

```

5260 \def\bbl@ignoreinterchar{%
5261   \ifnum\language=\@nohyphenation
5262     \expandafter\@gobble
5263   \else
5264     \expandafter\@firstofone
5265   \fi}
5266 \newcommand\babelinterchar[5][{}]{%
5267   \let\bbl@kv@label\@empty
5268   \bbl@forkv{#1}{\bbl@csarg\edef{kv@##1}{##2}}}%

```

```

5269 \@namedef{\zap@space bbl@xeinter@bbl@kv@label @#3@#4@#2 \@empty}%
5270 {\bbl@ignoreinterchar{#5}}%
5271 \bbl@csarg\let{ic@bbl@kv@label @#2}\@firstofone
5272 \bbl@expf{\bbl@for{\bbl@tempa{\zap@space#3 \@empty}}{%
5273 \bbl@expf{\bbl@for{\bbl@tempb{\zap@space#4 \@empty}}{%
5274 \XeTeXinterchartoks
5275 \@nameuse{bbl@xeclasse@bbl@tempa @#2}%
5276 \bbl@ifunset{bbl@xeclasse@bbl@tempa @#2}{#2}} %
5277 \@nameuse{bbl@xeclasse@bbl@tempb @#2}%
5278 \bbl@ifunset{bbl@xeclasse@bbl@tempb @#2}{#2}} %
5279 = \expandafter{%
5280 \csname bbl@ic@bbl@kv@label @#2\expandafter\endcsname
5281 \csname\zap@space bbl@xeinter@bbl@kv@label
5282 @#3@#4@#2 \@empty\endcsname}}}}
5283 \DeclareRobustCommand\enablelocaleinterchar[1]{%
5284 \bbl@ifunset{bbl@ic@#1@language}%
5285 {\bbl@error{unknown-interchar}{#1}{}}}%
5286 {\bbl@csarg\let{ic@#1@language}\@firstofone}}
5287 \DeclareRobustCommand\disablelocaleinterchar[1]{%
5288 \bbl@ifunset{bbl@ic@#1@language}%
5289 {\bbl@error{unknown-interchar-b}{#1}{}}}%
5290 {\bbl@csarg\let{ic@#1@language}\@gobble}}
5291 (/xetex)

```

10.3 Layout

Note elements like headlines and margins can be modified easily with packages like fancyhdr, typearea or titleps, and geometry.

\bbl@startskip and \bbl@endskip are available to package authors. Thanks to the T_EX expansion mechanism the following constructs are valid: \adim\bbl@startskip, \advance\bbl@startskip\adim, \bbl@startskip\adim.

Consider txtbabel as a shorthand for *tex-xet babel*, which is the bidi model in both pdftex and xetex.

```

5292 (*xetex|texxet)
5293 \providecommand\bbl@provide@intraspace{}
5294 \bbl@trace{Redefinitions for bidi layout}

Finish here if there in no layout.

5295 \ifx\bbl@opt@layout\@nnil\else % if layout=.
5296 \ifx\bbl@opt@layout\@undefined\else % Plain
5297 \IfBabelLayout{nopars}
5298 {}
5299 {\edef\bbl@opt@layout{\bbl@opt@layout.pars.}}%
5300 \fi
5301 \def\bbl@startskip{\ifcase\bbl@thepardir\leftskip\else\rightskip\fi}
5302 \def\bbl@endskip{\ifcase\bbl@thepardir\rightskip\else\leftskip\fi}
5303 \ifnum\bbl@bidimode>\z@
5304 \IfBabelLayout{pars}
5305 {\def\hangfrom#1{%
5306 \setbox\@tempboxa\hbox{#1}}%
5307 \hangindent\ifcase\bbl@thepardir\wd\@tempboxa\else-\wd\@tempboxa\fi
5308 \noindent\box\@tempboxa}
5309 \def\raggedright{%
5310 \let\@centercr
5311 \bbl@startskip\z@skip
5312 \@rightskip\@flushglue
5313 \bbl@endskip\@rightskip
5314 \parindent\z@
5315 \parfillskip\bbl@startskip}
5316 \def\raggedleft{%
5317 \let\@centercr
5318 \bbl@startskip\@flushglue
5319 \bbl@endskip\z@skip

```

```

5320     \parindent\z@
5321     \parfillskip\bbl@endskip}}
5322 {}
5323 \fi
5324 \IfBabelLayout{lists}
5325 {\bbl@sreplace\list
5326   {\@totalleftmargin\leftmargin}{\@totalleftmargin\bbl@listleftmargin}%
5327   \def\bbl@listleftmargin{%
5328     \ifcase\bbl@thepardir\leftmargin\else\rightmargin\fi}%
5329   \ifcase\bbl@engine
5330     \def\labelenumii{}\theenumii{}\pdfTeX doesn't reverse ()
5331     \def\p@enumiii{\p@enumii}\theenumii{}\fi
5332   \fi
5333   \bbl@sreplace\@verbatim
5334   {\leftskip\@totalleftmargin}%
5335   {\bbl@startskip\textwidth
5336     \advance\bbl@startskip-\linewidth}%
5337   \bbl@sreplace\@verbatim
5338   {\rightskip\z@skip}%
5339   {\bbl@endskip\z@skip}}%
5340 {}
5341 \IfBabelLayout{contents}
5342 {\bbl@sreplace\@dottedtocline{\leftskip}{\bbl@startskip}%
5343   \bbl@sreplace\@dottedtocline{\rightskip}{\bbl@endskip}}
5344 {}
5345 \IfBabelLayout{columns}
5346 {\bbl@sreplace\@outputdblcol{\hbxt@\textwidth}{\bbl@outputbox}%
5347   \def\bbl@outputbox#1{%
5348     \hbxt@\textwidth{%
5349       \hskip\columnwidth
5350       \hfil
5351       {\normalcolor\vrule \@width\columnseprule}%
5352       \hfil
5353       \hbxt@\columnwidth{\box\@leftcolumn \hss}%
5354       \hskip-\textwidth
5355       \hbxt@\columnwidth{\box\@outputbox \hss}%
5356       \hskip\columnsep
5357       \hskip\columnwidth}}}%
5358 {}

```

Implicitly reverses sectioning labels in `bidibasic`, because the full stop is not in contact with L numbers any more. I think there must be a better way.

```

5359 \IfBabelLayout{counters*}%
5360 {\bbl@add\bbl@opt@layout{.counters.}%
5361   \AddToHook{shipout/before}{%
5362     \let\bbl@tempa\babelsublr
5363     \let\babelsublr\@firstofone
5364     \let\bbl@save@thepage\thepage
5365     \protected@edef\thepage{\thepage}%
5366     \let\babelsublr\bbl@tempa}%
5367   \AddToHook{shipout/after}{%
5368     \let\thepage\bbl@save@thepage}}{}
5369 \IfBabelLayout{counters}%
5370 {\let\bbl@latinarabic=\@arabic
5371   \def\@arabic#1{\babelsublr{\bbl@latinarabic#1}}%
5372   \let\bbl@asciroman=\@roman
5373   \def\@roman#1{\babelsublr{\ensureascii{\bbl@asciroman#1}}}%
5374   \let\bbl@asciiRoman=\@Roman
5375   \def\@Roman#1{\babelsublr{\ensureascii{\bbl@asciiRoman#1}}}}{}
5376 \fi % end if layout
5377 (/xetex|texet)

```

10.4 8-bit TeX

Which start just above, because some code is shared with xetex. Now, 8-bit specific stuff. If just one encoding has been declared, then assume no switching is necessary (1).

```
5378 (*texxet)
5379 \def\bbl@provide@extra#1{%
5380   % == auto-select encoding ==
5381   \ifx\bbl@encoding@select@off\@empty\else
5382     \bbl@ifunset{\bbl@encoding@#1}%
5383     {\def\elt##1{,##1,}%
5384      \edef\bbl@tempe{\expandafter\@gobbletwo\@fontenc@load@list}%
5385      \count@\z@
5386      \bbl@foreach\bbl@tempe{%
5387        \def\bbl@tempd{##1}% Save last declared
5388        \advance\count@\@ne}%
5389      \ifnum\count@>\@ne % (1)
5390        \getlocaleproperty*\bbl@tempa{#1}{identification/encodings}%
5391        \ifx\bbl@tempa\relax \let\bbl@tempa\@empty \fi
5392        \bbl@replace\bbl@tempa{ },}%
5393        \global\bbl@csarg\let{encoding@#1}\@empty
5394        \bbl@xin@{,\bbl@tempd,}{,\bbl@tempa,}%
5395        \ifin@else % if main encoding included in ini, do nothing
5396          \let\bbl@tempb\relax
5397          \bbl@foreach\bbl@tempa{%
5398            \ifx\bbl@tempb\relax
5399              \bbl@xin@{,##1,}{,\bbl@tempe,}%
5400              \ifin@\def\bbl@tempb{##1}\fi
5401            \fi}%
5402          \ifx\bbl@tempb\relax\else
5403            \bbl@exp{%
5404              \global\<bbl@add>\<bbl@preextras@#1>\<bbl@encoding@#1>}%
5405            \gdef\<bbl@encoding@#1>{%
5406              \\babel@save\\f@encoding
5407              \\bbl@add\\originalTeX{\\selectfont}%
5408              \\fontencoding{\bbl@tempb}%
5409              \\selectfont}}%
5410            \fi
5411          \fi
5412        \fi}%
5413      }%
5414    \fi}
5415 (/texxet)
```

10.5 LuaTeX

The loader for luatex is based solely on language.dat, which is read on the fly. The code shouldn't be executed when the format is build, so we check if \AddBabelHook is defined. Then comes a modified version of the loader in hyphen.cfg (without the hyphenmins stuff, which is under the direct control of babel).

The names \l@<language> are defined and take some value from the beginning because all ldf files assume this for the corresponding language to be considered valid, but patterns are not loaded (except the first one). This is done later, when the language is first selected (which usually means when the ldf finishes). If a language has been loaded, \bbl@hyphendata@<num> exists (with the names of the files read).

The default setup preloads the first language into the format. This is intended mainly for 'english', so that it's available without further intervention from the user. To avoid duplicating it, the following rule applies: if the "0th" language and the first language in language.dat have the same name then just ignore the latter. If there are new synonymous, they are added, but note if the language patterns have not been preloaded they won't at run time.

Other preloaded languages could be read twice, if they have been preloaded into the format. This is not optimal, but it shouldn't happen very often – with luatex patterns are best loaded when the document is typeset, and the "0th" language is preloaded just for backwards compatibility.

As of 1.1b, lua(e)tex is taken into account. Formerly, loading of patterns on the fly didn't work in this format, but with the new loader it does. Unfortunately, the format is not based on babel, and data could be duplicated, because languages are reassigned above those in the format (nothing serious, anyway). Note even with this format language.dat is used (under the principle of a single source), instead of language.def.

Of course, there is room for improvements, like tools to read and reassign languages, which would require modifying the language list, and better error handling.

We need catcode tables, but no format (targeted by babel) provide a command to allocate them (although there are packages like ctablestack). FIX - This isn't true anymore. For the moment, a dangerous approach is used - just allocate a high random number and cross the fingers. To complicate things, etex.sty changes the way languages are allocated.

This files is read at three places: (1) when plain.def, babel.sty starts, to read the list of available languages from language.dat (for the base option); (2) at hyphen.cfg, to modify some macros; (3) in the middle of plain.def and babel.sty, by babel.def, with the commands and other definitions for luatex (e.g., \babelpatterns).

```

5416 (*luatex)
5417 \directlua{ Babel = Babel or {} } % DL2
5418 \ifx\AddBabelHook\undefined % When plain.def, babel.sty starts
5419 \bbl@trace{Read language.dat}
5420 \ifx\bbl@readstream\undefined
5421 \csname newread\endcsname\bbl@readstream
5422 \fi
5423 \begingroup
5424 \toks@{}
5425 \count@ \z@ % 0=start, 1=0th, 2=normal
5426 \def\bbl@process@line#1#2 #3 #4 {%
5427 \ifx=#1%
5428 \bbl@process@synonym{#2}%
5429 \else
5430 \bbl@process@language{#1#2}{#3}{#4}%
5431 \fi
5432 \ignorespaces}
5433 \def\bbl@manylang{%
5434 \ifnum\bbl@last>\@ne
5435 \bbl@info{Non-standard hyphenation setup}%
5436 \fi
5437 \let\bbl@manylang\relax}
5438 \def\bbl@process@language#1#2#3{%
5439 \ifcase\count@
5440 \ifundefined{zth#1}{\count@\tw@}{\count@\@ne}%
5441 \or
5442 \count@\tw@
5443 \fi
5444 \ifnum\count@=\tw@
5445 \expandafter\addlanguage\csname l@#1\endcsname
5446 \language\allocationnumber
5447 \chardef\bbl@last\allocationnumber
5448 \bbl@manylang
5449 \let\bbl@elt\relax
5450 \xdef\bbl@languages{%
5451 \bbl@languages\bbl@elt{#1}{\the\language}{#2}{#3}}%
5452 \fi
5453 \the\toks@
5454 \toks@{}}
5455 \def\bbl@process@synonym@aux#1#2{%
5456 \global\expandafter\chardef\csname l@#1\endcsname#2\relax
5457 \let\bbl@elt\relax
5458 \xdef\bbl@languages{%
5459 \bbl@languages\bbl@elt{#1}{#2}{}}}%
5460 \def\bbl@process@synonym#1{%
5461 \ifcase\count@
5462 \toks@\expandafter{\the\toks@\relax\bbl@process@synonym{#1}}%
5463 \or

```

```

5464 \ifundefined{zth#1}{\bbl@process@synonym@aux{#1}{0}}{}%
5465 \else
5466 \bbl@process@synonym@aux{#1}{\the\bbl@last}%
5467 \fi}
5468 \ifx\bbl@languages\undefined % Just a (sensible?) guess
5469 \chardef\l@english\z@
5470 \chardef\l@USenglish\z@
5471 \chardef\bbl@last\z@
5472 \global\@namedef{bbl@hyphendata@0}{{hyphen.tex}}{}
5473 \gdef\bbl@languages{%
5474 \bbl@elt{english}{0}{hyphen.tex}}{}%
5475 \bbl@elt{USenglish}{0}{}}{}
5476 \else
5477 \global\let\bbl@languages@format\bbl@languages
5478 \def\bbl@elt#1#2#3#4{% Remove all except language 0
5479 \ifnum#2>\z@\else
5480 \noexpand\bbl@elt{#1}{#2}{#3}{#4}%
5481 \fi}%
5482 \xdef\bbl@languages{\bbl@languages}%
5483 \fi
5484 \def\bbl@elt#1#2#3#4{\@namedef{zth#1}}{} % Define flags
5485 \bbl@languages
5486 \openin\bbl@readstream=language.dat
5487 \ifeof\bbl@readstream
5488 \bbl@warning{I couldn't find language.dat. No additional\\%
5489 patterns loaded. Reported}%
5490 \else
5491 \loop
5492 \endlinechar\m@ne
5493 \read\bbl@readstream to \bbl@line
5494 \endlinechar\^^M
5495 \if T\ifeof\bbl@readstream F\fi T\relax
5496 \ifx\bbl@line\@empty\else
5497 \edef\bbl@line{\bbl@line\space\space\space}%
5498 \expandafter\bbl@process@line\bbl@line\relax
5499 \fi
5500 \repeat
5501 \fi
5502 \closein\bbl@readstream
5503 \endgroup
5504 \bbl@trace{Macros for reading patterns files}
5505 \def\bbl@get@enc#1:#2:#3\@@{\def\bbl@hyph@enc{#2}}
5506 \ifx\babelcatcodetablenum\undefined
5507 \ifx\newcatcodetable\undefined
5508 \def\babelcatcodetablenum{5211}
5509 \def\bbl@pattcodes{\numexpr\babelcatcodetablenum+1\relax}
5510 \else
5511 \newcatcodetable\babelcatcodetablenum
5512 \newcatcodetable\bbl@pattcodes
5513 \fi
5514 \else
5515 \def\bbl@pattcodes{\numexpr\babelcatcodetablenum+1\relax}
5516 \fi
5517 \def\bbl@luapatterns#1#2{%
5518 \bbl@get@enc#1:.\@@@
5519 \setbox\z@\hbox\bgroup
5520 \begin{group}
5521 \savecatcodetable\babelcatcodetablenum\relax
5522 \initcatcodetable\bbl@pattcodes\relax
5523 \catcodetable\bbl@pattcodes\relax
5524 \catcode\#=6 \catcode\$=3 \catcode\&=4 \catcode\^=7
5525 \catcode\_ =8 \catcode\{=1 \catcode\}=2 \catcode\~=13
5526 \catcode\@=11 \catcode\^^I=10 \catcode\^^J=12

```

```

5527 \catcode\<=12 \catcode\>=12 \catcode\*=12 \catcode\.=12
5528 \catcode\-=12 \catcode\/=12 \catcode\[=12 \catcode\]=12
5529 \catcode\`=12 \catcode\'=12 \catcode\"=12
5530 \input #1\relax
5531 \catcodetable\babelcatcodetablenum\relax
5532 \endgroup
5533 \def\bbl@tempa{#2}%
5534 \ifx\bbl@tempa\empty\else
5535 \input #2\relax
5536 \fi
5537 \egroup}%
5538 \def\bbl@patterns@lua#1{%
5539 \language=\expandafter\ifx\csname l@#1:\f@encoding\endcsname\relax
5540 \csname l@#1\endcsname
5541 \edef\bbl@tempa{#1}%
5542 \else
5543 \csname l@#1:\f@encoding\endcsname
5544 \edef\bbl@tempa{#1:\f@encoding}%
5545 \fi\relax
5546 \namedef{lu@texhyphen@loaded@the\language}{}% Temp
5547 \ifundefined{bbl@hyphendata@the\language}%
5548 {\def\bbl@elt##1##2###4{%
5549 \ifnum##2=\csname l@bbl@tempa\endcsname % #2=spanish, dutch:OT1...
5550 \def\bbl@tempb{##3}%
5551 \ifx\bbl@tempb\empty\else % if not a synonymous
5552 \def\bbl@tempc{{##3}{##4}}%
5553 \fi
5554 \bbl@csarg\xdef{hyphendata@##2}{\bbl@tempc}%
5555 \fi}%
5556 \bbl@languages
5557 \ifundefined{bbl@hyphendata@the\language}%
5558 {\bbl@info{No hyphenation patterns were set for\%
5559 language '\bbl@tempa'. Reported}}%
5560 {\expandafter\expandafter\expandafter\bbl@luapatterns
5561 \csname bbl@hyphendata@the\language\endcsname}}}%
5562 \endinput\fi

```

Here ends \ifx\AddBabelHook\@undefined. A few lines are only read by HYPHEN.CFG.

```

5563 \ifx\DisableBabelHook\@undefined
5564 \AddBabelHook{luatex}{everylanguage}{%
5565 \def\process@language##1##2##3{%
5566 \def\process@line####1####2 ####3 ####4 {}}}
5567 \AddBabelHook{luatex}{loadpatterns}{%
5568 \input #1\relax
5569 \expandafter\gdef\csname bbl@hyphendata@the\language\endcsname
5570 {{#1}}}%
5571 \AddBabelHook{luatex}{loadexceptions}{%
5572 \input #1\relax
5573 \def\bbl@tempb##1##2{{##1}{##1}}%
5574 \expandafter\xdef\csname bbl@hyphendata@the\language\endcsname
5575 {\expandafter\expandafter\expandafter\bbl@tempb
5576 \csname bbl@hyphendata@the\language\endcsname}}
5577 \endinput\fi

```

Here stops reading code for HYPHEN.CFG. The following is read the 2nd time it's loaded. First, global declarations for lua.

```

5578 \begingroup
5579 \catcode\%=12
5580 \catcode\'=12
5581 \catcode\"=12
5582 \catcode\:=12
5583 \directlua{
5584 Babel.locale_props = Babel.locale_props or {}
5585

```

```

5586 function Babel.lua_error(e, a)
5587     tex.print([[noexpand\csname bbl@error\endcsname{]] ..
5588         e .. '}' .. (a or '') .. '}{'}])
5589 end
5590
5591 function Babel.bytes(line)
5592     return line:gsub(".",
5593         function (chr) return unicode.utf8.char(string.byte(chr)) end)
5594 end
5595
5596 function Babel.priority_in_callback(name,description)
5597     for i,v in ipairs(luatexbase.callback_descriptions(name)) do
5598         if v == description then return i end
5599     end
5600     return false
5601 end
5602
5603 function Babel.begin_process_input()
5604     if luatexbase and luatexbase.add_to_callback then
5605         luatexbase.add_to_callback('process_input_buffer',
5606             Babel.bytes,'Babel.bytes')
5607     else
5608         Babel.callback = callback.find('process_input_buffer')
5609         callback.register('process_input_buffer',Babel.bytes)
5610     end
5611 end
5612 function Babel.end_process_input ()
5613     if luatexbase and luatexbase.remove_from_callback then
5614         luatexbase.remove_from_callback('process_input_buffer','Babel.bytes')
5615     else
5616         callback.register('process_input_buffer',Babel.callback)
5617     end
5618 end
5619
5620 function Babel.str_to_nodes(fn, matches, base)
5621     local n, head, last
5622     if fn == nil then return nil end
5623     for s in string.utfvalues(fn(matches)) do
5624         if base.id == 7 then
5625             base = base.replace
5626         end
5627         n = node.copy(base)
5628         n.char = s
5629         if not head then
5630             head = n
5631         else
5632             last.next = n
5633         end
5634         last = n
5635     end
5636     return head
5637 end
5638
5639 Babel.linebreaking = Babel.linebreaking or {}
5640 Babel.linebreaking.before = {}
5641 Babel.linebreaking.after = {}
5642 Babel.locale = {}
5643 function Babel.linebreaking.add_before(func, pos)
5644     tex.print([[noexpand\csname bbl@luahyphenate\endcsname]])
5645     if pos == nil then
5646         table.insert(Babel.linebreaking.before, func)
5647     else
5648         table.insert(Babel.linebreaking.before, pos, func)

```



```

5649     end
5650 end
5651 function Babel.linebreaking.add_after(func)
5652     tex.print([[\\noexpand\\csname bbl@lua hyphenate\\endcsname]])
5653     table.insert(Babel.linebreaking.after, func)
5654 end
5655
5656 function Babel.addpatterns(pp, lg)
5657     local lg = lang.new(lg)
5658     local pats = lang.patterns(lg) or ''
5659     lang.clear_patterns(lg)
5660     for p in pp:gmatch('[^%s]+') do
5661         ss = ''
5662         for i in string.utfcharacters(p:gsub('%d', '')) do
5663             ss = ss .. '%d?' .. i
5664         end
5665         ss = ss:gsub('^%d%?%.', '%%.') .. '%d?'
5666         ss = ss:gsub('%.%d%?$', '%%.')
5667         pats, n = pats:gsub('%s' .. ss .. '%s', ' ' .. p .. ' ')
5668         if n == 0 then
5669             tex.sprint(
5670                 [[\\string\\csname\\space bbl@info\\endcsname{New pattern: }
5671                 .. p .. [{}]])
5672             pats = pats .. ' ' .. p
5673         else
5674             tex.sprint(
5675                 [[\\string\\csname\\space bbl@info\\endcsname{Renew pattern: }
5676                 .. p .. [{}]])
5677         end
5678     end
5679     lang.patterns(lg, pats)
5680 end
5681
5682 Babel.characters = Babel.characters or {}
5683 Babel.ranges = Babel.ranges or {}
5684 function Babel.hlist_has_bidi(head)
5685     local has_bidi = false
5686     local ranges = Babel.ranges
5687     for item in node.traverse(head) do
5688         if item.id == node.id'glyph' then
5689             local itemchar = item.char
5690             local chardata = Babel.characters[itemchar]
5691             local dir = chardata and chardata.d or nil
5692             if not dir then
5693                 for nn, et in ipairs(ranges) do
5694                     if itemchar < et[1] then
5695                         break
5696                     elseif itemchar <= et[2] then
5697                         dir = et[3]
5698                         break
5699                     end
5700                 end
5701             end
5702             if dir and (dir == 'al' or dir == 'r') then
5703                 has_bidi = true
5704             end
5705         end
5706     end
5707     return has_bidi
5708 end
5709 function Babel.set_chranges_b (script, chrng)
5710     if chrng == '' then return end
5711     texio.write('Package babel Info: Replacing ' .. script .. ' script ranges')

```

```

5712     Babel.script_blocks[script] = {}
5713     for s, e in string.gmatch(chrng..' ', '(.-%.(-)%s') do
5714         table.insert(
5715             Babel.script_blocks[script], {tonumber(s,16), tonumber(e,16)})
5716     end
5717 end
5718
5719 function Babel.discard_sublr(str)
5720     if str:find( [[\string\indexentry]] ) and
5721         str:find( [[\string\babelsublr]] ) then
5722         str = str:gsub( [[\string\babelsublr%s*(%b{})]],
5723             function(m) return m:sub(2,-2) end )
5724     end
5725     return str
5726 end
5727 }
5728 \endgroup
5729 \ifx\newattribute\@undefined\else % Test for plain
5730     \newattribute\bbl@attr@locale % DL4
5731     \directlua{ Babel.attr_locale = luatexbase.registernumber'bbl@attr@locale' }
5732     \AddBabelHook{luatex}{beforeextras}{%
5733         \setattribute\bbl@attr@locale\localeid}
5734 \fi
5735 %
5736 \def\BabelStringsDefault{unicode}
5737 \let\luabbl@stop\relax
5738 \AddBabelHook{luatex}{encodedcommands}{%
5739     \def\bbl@tempa{utf8}\def\bbl@tempb{#1}%
5740     \ifx\bbl@tempa\bbl@tempb\else
5741         \directlua{Babel.begin_process_input()}%
5742         \def\luabbl@stop{%
5743             \directlua{Babel.end_process_input()}}%
5744     \fi}%
5745 \AddBabelHook{luatex}{stopcommands}{%
5746     \luabbl@stop
5747     \let\luabbl@stop\relax}
5748 %
5749 \AddBabelHook{luatex}{patterns}{%
5750     \@ifundefined{bbl@hyphendata@the\language}%
5751     {\def\bbl@elt##1##2##3##4{%
5752         \ifnum##2=\csname l@#2\endcsname % #2=spanish, dutch:OT1...
5753         \def\bbl@tempb{##3}%
5754         \ifx\bbl@tempb\empty\else % if not a synonymous
5755             \def\bbl@tempc{{##3}{##4}}%
5756         \fi
5757         \bbl@csarg\xdef{hyphendata@##2}{\bbl@tempc}%
5758         \fi}%
5759     \bbl@languages
5760     \@ifundefined{bbl@hyphendata@the\language}%
5761     {\bbl@info{No hyphenation patterns were set for\%
5762         language '#2'. Reported}}%
5763     {\expandafter\expandafter\expandafter\bbl@luapatterns
5764         \csname bbl@hyphendata@the\language\endcsname}}}%
5765     \@ifundefined{bbl@patterns@}{}%
5766     \begin{group}
5767         \bbl@xin@{, \number\language,}{, \bbl@pttnlist}%
5768         \ifin@ \else
5769             \ifx\bbl@patterns@\empty\else
5770                 \directlua{ Babel.addpatterns(
5771                     [[\bbl@patterns@]], \number\language) }%
5772             \fi
5773             \@ifundefined{bbl@patterns@#1}%
5774             \@empty

```

```

5775         {\directlua{ Babel.addpatterns(
5776             [[\space\csname bbl@patterns@#1\endcsname]],
5777             \number\language) }}%
5778         \xdef\bbl@pttnlist{\bbl@pttnlist\number\language,}%
5779         \fi
5780     \endgroup}%
5781 \bbl@exp{%
5782     \bbl@ifunset{\bbl@prehc@\languagename}{}%
5783     {\bbl@ifblank{\bbl@cs{\prehc@\languagename}}{}%
5784     {\prehyphenchar=\bbl@cl{\prehc}\relax}}}%

```

\babelpatterns This macro adds patterns. Two macros are used to store them: `\bbl@patterns@` for the global ones and `\bbl@patterns@<language>` for language ones. We make sure there is a space between words when multiple commands are used.

```

5785 \@onlypreamble\babelpatterns
5786 \AtEndOfPackage{%
5787     \newcommand\babelpatterns[2][\@empty]{%
5788         \ifx\bbl@patterns@\relax
5789             \let\bbl@patterns@\@empty
5790         \fi
5791         \ifx\bbl@pttnlist@\@empty\else
5792             \bbl@warning{%
5793                 You must not intermingle \string\selectlanguage\space and\%
5794                 \string\babelpatterns\space or some patterns will not\%
5795                 be taken into account. Reported}%
5796             \fi
5797             \ifx\@empty#1%
5798                 \protected@edef\bbl@patterns@{\bbl@patterns@\space#2}%
5799             \else
5800                 \edef\bbl@tempb{\zap@space#1 \@empty}%
5801                 \bbl@for\bbl@tempa\bbl@tempb{%
5802                     \bbl@fixname\bbl@tempa
5803                     \bbl@iflanguage\bbl@tempa{%
5804                         \bbl@csarg\protected@edef{patterns@\bbl@tempa}{%
5805                             \@ifundefined{\bbl@patterns@\bbl@tempa}%
5806                             \@empty
5807                             {\csname bbl@patterns@\bbl@tempa\endcsname\space}%
5808                             #2}}}%
5809             \fi}}

```

10.6 Southeast Asian scripts

First, some general code for line breaking, used by `\babelposthyphenation`.

Replace regular (i.e., implicit) discretionaries by spaceskips, based on the previous glyph (which I think makes sense, because the hyphen and the previous char go always together). Other discretionaries are not touched. See Unicode UAX 14.

```

5810 \def\bbl@intraspace#1 #2 #3\@{%
5811     \directlua{
5812         Babel.intraspaces = Babel.intraspaces or {}
5813         Babel.intraspaces['\csname bbl@sbc@languagename\endcsname'] = %
5814             {b = #1, p = #2, m = #3}
5815         Babel.locale_props[\the\localeid].intraspace = %
5816             {b = #1, p = #2, m = #3}
5817     }}
5818 \def\bbl@intrapenalty#1\@{%
5819     \directlua{
5820         Babel.intrapenalties = Babel.intrapenalties or {}
5821         Babel.intrapenalties['\csname bbl@sbc@languagename\endcsname'] = #1
5822         Babel.locale_props[\the\localeid].intrapenalty = #1
5823     }}
5824 \begingroup
5825 \catcode`\%=12

```

```

5826 \catcode`\&=14
5827 \catcode`\'=12
5828 \catcode`\~ =12
5829 \gdef\bbl@seaintraspace{&
5830 \let\bbl@seaintraspace\relax
5831 \directlua{
5832   Babel.sea_enabled = true
5833   Babel.sea_ranges = Babel.sea_ranges or {}
5834   function Babel.set_chranges (script, chrng)
5835     local c = 0
5836     for s, e in string.gmatch(chrng..' ', '(.-%.%.(.-%s)') do
5837       Babel.sea_ranges[script..c]={tonumber(s,16), tonumber(e,16)}
5838       c = c + 1
5839     end
5840   end
5841   function Babel.sea_disc_to_space (head)
5842     local sea_ranges = Babel.sea_ranges
5843     local last_char = nil
5844     local quad = 655360      &% 10 pt = 655360 = 10 * 65536
5845     for item in node.traverse(head) do
5846       local i = item.id
5847       if i == node.id'glyph' then
5848         last_char = item
5849       elseif i == 7 and item.subtype == 3 and last_char
5850         and last_char.char > 0x0C99 then
5851         quad = font.getfont(last_char.font).size
5852         for lg, rg in pairs(sea_ranges) do
5853           if last_char.char > rg[1] and last_char.char < rg[2] then
5854             lg = lg:sub(1, 4) &% Remove trailing number of, e.g., Cyril
5855             local intraspace = Babel.intraspaces[lg]
5856             local intrapenalty = Babel.intrapenalties[lg]
5857             local n
5858             if intrapenalty ~= 0 then
5859               n = node.new(14, 0)      &% penalty
5860               n.penalty = intrapenalty
5861               node.insert_before(head, item, n)
5862             end
5863             n = node.new(12, 13)      &% (glue, spaceskip)
5864             node.setglue(n, intraspace.b * quad,
5865               intraspace.p * quad,
5866               intraspace.m * quad)
5867             node.insert_before(head, item, n)
5868             node.remove(head, item)
5869           end
5870         end
5871       end
5872     end
5873   end
5874 }&
5875 \bbl@luahyphenate}

```

10.7 CJK line breaking

Minimal line breaking for CJK scripts, mainly intended for simple documents and short texts as a secondary language. Only line breaking, with a little stretching for justification, without any attempt to adjust the spacing. It is based on (but does not strictly follow) the Unicode algorithm.

We first need a little table with the corresponding line breaking properties. A few characters have an additional key for the width (fullwidth vs. halfwidth), not yet used. There is a separate file, defined below.

```

5876 \catcode`\%=14
5877 \gdef\bbl@cjkkintraspace{%
5878 \let\bbl@cjkkintraspace\relax
5879 \directlua{

```

```

5880     require('babel-data-cjk.lua')
5881     Babel.cjk_enabled = true
5882     function Babel.cjk_linebreak(head)
5883         local GLYPH = node.id'glyph'
5884         local last_char = nil
5885         local quad = 655360      % 10 pt = 655360 = 10 * 65536
5886         local last_class = nil
5887         local last_lang = nil
5888         for item in node.traverse(head) do
5889             if item.id == GLYPH then
5890                 local lang = item.lang
5891                 local LOCALE = node.get_attribute(item,
5892                     Babel.attr_locale)
5893                 local props = Babel.locale_props[LOCALE] or {}
5894                 local class = Babel.cjk_class[item.char].c
5895                 if props.cjk_quotes and props.cjk_quotes[item.char] then
5896                     class = props.cjk_quotes[item.char]
5897                 end
5898                 if class == 'cp' then class = 'cl' % ]) as CL
5899                 elseif class == 'id' then class = 'I'
5900                 elseif class == 'cj' then class = 'I' % loose
5901                 end
5902                 local br = 0
5903                 if class and last_class and Babel.cjk_breaks[last_class][class] then
5904                     br = Babel.cjk_breaks[last_class][class]
5905                 end
5906                 if br == 1 and props.linebreak == 'c' and
5907                     lang ~= \the\l@nohyphenation\space and
5908                     last_lang ~= \the\l@nohyphenation then
5909                     local intrapenalty = props.intrapenalty
5910                     if intrapenalty ~= 0 then
5911                         local n = node.new(14, 0)      % penalty
5912                         n.penalty = intrapenalty
5913                         node.insert_before(head, item, n)
5914                     end
5915                     local intraspace = props.intraspace
5916                     local n = node.new(12, 13)      % (glue, spaceskip)
5917                     node.setglue(n, intraspace.b * quad,
5918                         intraspace.p * quad,
5919                         intraspace.m * quad)
5920                     node.insert_before(head, item, n)
5921                 end
5922                 if font.getfont(item.font) then
5923                     quad = font.getfont(item.font).size
5924                 end
5925                 last_class = class
5926                 last_lang = lang
5927                 else % if penalty, glue or anything else
5928                     last_class = nil
5929                 end
5930             end
5931             lang.hyphenate(head)
5932         end
5933     }%
5934     \bbl@luahyphenate}
5935 \gdef\bbl@luahyphenate{%
5936     \let\bbl@luahyphenate\relax
5937     \directlua{
5938         luatexbase.add_to_callback('hyphenate',
5939             function (head, tail)
5940                 if Babel.linebreaking.before then
5941                     for k, func in ipairs(Babel.linebreaking.before) do
5942                         func(head)

```

```

5943     end
5944 end
5945 lang.hyphenate(head)
5946 if Babel.cjk_enabled then
5947     Babel.cjk_linebreak(head)
5948 end
5949 if Babel.linebreaking.after then
5950     for k, func in ipairs(Babel.linebreaking.after) do
5951         func(head)
5952     end
5953 end
5954 if Babel.set_hboxed then
5955     Babel.set_hboxed(head)
5956 end
5957 if Babel.sea_enabled then
5958     Babel.sea_disc_to_space(head)
5959 end
5960 end,
5961 'Babel.hyphenate')
5962 }}
5963 \endgroup
5964 %
5965 \def\bbl@provide@intraspace{%
5966   \bbl@ifunset{\bbl@intsp@{language}}{%
5967     {\expandafter\ifx\csname\bbl@intsp@{language}\endcsname\@empty\else
5968       \bbl@xin@{/c}{\bbl@cl{lnbrk}}}%
5969     \ifin@           % cjk
5970       \bbl@cjk@intraspace
5971       \directlua{
5972         Babel.locale_props = Babel.locale_props or {}
5973         Babel.locale_props[\the\localeid].linebreak = 'c'
5974       }%
5975       \bbl@exp{\bbl@intraspace\bbl@cl{intsp}}\@}%
5976       \ifx\bbl@KVP@intrapenalty\@nnil
5977         \bbl@intrapenalty0\@@
5978       \fi
5979     \else           % sea
5980       \bbl@sea@intraspace
5981       \bbl@exp{\bbl@intraspace\bbl@cl{intsp}}\@}%
5982       \directlua{
5983         Babel.sea_ranges = Babel.sea_ranges or {}
5984         Babel.set_chranges('\bbl@cl{sbcpr}',
5985           '\bbl@cl{chrng}')
5986       }%
5987       \ifx\bbl@KVP@intrapenalty\@nnil
5988         \bbl@intrapenalty0\@@
5989       \fi
5990     \fi
5991   \fi
5992   \ifx\bbl@KVP@intrapenalty\@nnil\else
5993     \expandafter\bbl@intrapenalty\bbl@KVP@intrapenalty\@@
5994   \fi}}

```

10.8 Arabic justification

WIP. \bbl@arabicjust is executed with both elongated and kashida. This must be fine tuned. The attribute kashida is set by transforms with kashida.

```

5995 \ifnum\bbl@bidimode>100 \ifnum\bbl@bidimode<200
5996 \def\bblar@chars{%
5997   0628,0629,062A,062B,062C,062D,062E,062F,0630,0631,0632,0633,%
5998   0634,0635,0636,0637,0638,0639,063A,063B,063C,063D,063E,063F,%
5999   0640,0641,0642,0643,0644,0645,0646,0647,0649}
6000 \def\bblar@elongated{%

```

```

6001 0626,0628,062A,062B,0633,0634,0635,0636,063B,%
6002 063C,063D,063E,063F,0641,0642,0643,0644,0646,%
6003 0649,064A}
6004 \beginngroup
6005 \catcode\_ =11 \catcode\:=11
6006 \gdef\bblar@nofswarn{\gdef\msg_warning:nx##1##2##3{}}
6007 \endgroup
6008 \gdef\bbl@arabicjust{%
6009 \let\bbl@arabicjust\relax
6010 \newattribute\bblar@kashida
6011 \directlua{ Babel.attr_kashida = luatexbase.registernumber'bblar@kashida' }%
6012 \bblar@kashida=\z@
6013 \bbl@patchfont{{\bbl@parsejalt}}%
6014 \directlua{
6015   Babel.arabic.elong_map = Babel.arabic.elong_map or {}
6016   Babel.arabic.elong_map[\the\localeid] = {}
6017   luatexbase.add_to_callback('post_linebreak_filter',
6018     Babel.arabic.justify, 'Babel.arabic.justify')
6019   luatexbase.add_to_callback('hpack_filter',
6020     Babel.arabic.justify_hbox, 'Babel.arabic.justify_hbox')
6021 }%

```

Save both node lists to make replacement.

```

6022 \def\bblar@fetchjalt#1#2#3#4{%
6023 \bbl@exp{\bbl@foreach{#1}}{%
6024 \bbl@ifunset{bblar@JE@##1}%
6025 {\setbox\z@\hbox{\texdir TRT ^^^200d\char"##1#2}}%
6026 {\setbox\z@\hbox{\texdir TRT ^^^200d\char"\@nameuse{bblar@JE@##1}#2}}%
6027 \directlua{%
6028   local last = nil
6029   for item in node.traverse(tex.box[0].head) do
6030     if item.id == node.id'glyph' and item.char > 0x600 and
6031       not (item.char == 0x200D) then
6032       last = item
6033     end
6034   end
6035   Babel.arabic.#3['##1#4'] = last.char
6036 }}

```

Elongated forms. Brute force. No rules at all, yet. The ideal: look at jalt table. And perhaps other tables (falt?, csw?). What about kaf? And diacritic positioning?

```

6037 \gdef\bbl@parsejalt{%
6038 \ifx\addfontfeature\undefined\else
6039 \bbl@xin@{/e}{/\bbl@cl{\lnbrk}}%
6040 \ifin@
6041 \directlua{%
6042   if Babel.arabic.elong_map[\the\localeid][\fontid\font] == nil then
6043     Babel.arabic.elong_map[\the\localeid][\fontid\font] = {}
6044     tex.print([[string\cswname\space bbl@parsejalti\endcswname]])
6045   end
6046 }%
6047 \fi
6048 \fi}
6049 \gdef\bbl@parsejalti{%
6050 \beginngroup
6051 \let\bbl@parsejalt\relax % To avoid infinite loop
6052 \edef\bbl@tempb{\fontid\font}%
6053 \bblar@nofswarn
6054 \@nameuse{tracinglostchars}\z@
6055 \bblar@fetchjalt\bblar@elongated{{from}}{%
6056 \bblar@fetchjalt\bblar@chars{^^^064a}{from}{a}% Alef maksura
6057 \bblar@fetchjalt\bblar@chars{^^^0649}{from}{y}% Yeh
6058 \addfontfeature{RawFeature+=jalt}%
6059 % \@namedef{bblar@JE@0643}{06AA}% todo: catch medial kaf

```

```

6060 \bblar@fetchjalt\bblar@elongated{}{dest}{}%
6061 \bblar@fetchjalt\bblar@chars{^^^064a}{dest}{a}%
6062 \bblar@fetchjalt\bblar@chars{^^^0649}{dest}{y}%
6063 \directlua{%
6064     for k, v in pairs(Babel.arabic.from) do
6065         if Babel.arabic.dest[k] and
6066             not (Babel.arabic.from[k] == Babel.arabic.dest[k]) then
6067             Babel.arabic.elong_map[\the\localeid][\bbl@tempb]
6068             [Babel.arabic.from[k]] = Babel.arabic.dest[k]
6069         end
6070     end
6071 }%
6072 \endgroup}

```

The actual justification (inspired by CHICKENIZE).

```

6073 \begingroup
6074 \catcode`#=11
6075 \catcode`~=11
6076 \directlua{
6077
6078 Babel.arabic = Babel.arabic or {}
6079 Babel.arabic.from = {}
6080 Babel.arabic.dest = {}
6081 Babel.arabic.justify_factor = 0.95
6082 Babel.arabic.justify_enabled = true
6083 Babel.arabic.kashida_limit = -1
6084
6085 function Babel.arabic.justify(head)
6086   if not Babel.arabic.justify_enabled then return head end
6087   for line in node.traverse_id(node.id'hlist', head) do
6088     Babel.arabic.justify_hlist(head, line)
6089   end
6090   % In case the very first item is a line (eg, in \vbox):
6091   while head.prev do head = head.prev end
6092   return head
6093 end
6094
6095 function Babel.arabic.justify_hbox(head, gc, size, pack)
6096   local has_inf = false
6097   if Babel.arabic.justify_enabled and pack == 'exactly' then
6098     for n in node.traverse_id(12, head) do
6099       if n.stretch_order > 0 then has_inf = true end
6100     end
6101     if not has_inf then
6102       Babel.arabic.justify_hlist(head, nil, gc, size, pack)
6103     end
6104   end
6105   return head
6106 end
6107
6108 function Babel.arabic.justify_hlist(head, line, gc, size, pack)
6109   local d, new
6110   local k_list, k_item, pos_inline
6111   local width, width_new, full, k_curr, wt_pos, goal, shift
6112   local subst_done = false
6113   local elong_map = Babel.arabic.elong_map
6114   local cnt
6115   local last_line
6116   local GLYPH = node.id'glyph'
6117   local KASHIDA = Babel.attr_kashida
6118   local LOCALE = Babel.attr_locale
6119
6120   if line == nil then
6121     line = {}

```



```

6122     line.glue_sign = 1
6123     line.glue_order = 0
6124     line.head = head
6125     line.shift = 0
6126     line.width = size
6127 end
6128
6129 % Exclude last line.
6130 if ((line.next ~= nil or line.glue_sign == 1) and line.glue_order == 0) then
6131     elongs = {}      % Stores elongated candidates of each line
6132     k_list = {}      % And all letters with kashida
6133     pos_inline = 0   % Not yet used
6134
6135     for n in node.traverse_id(GLYPH, line.head) do
6136         pos_inline = pos_inline + 1 % To find where it is. Not used.
6137
6138         % Elongated glyphs
6139         if elong_map then
6140             local locale = node.get_attribute(n, LOCALE)
6141             if elong_map[locale] and elong_map[locale][n.font] and
6142                 elong_map[locale][n.font][n.char] then
6143                 table.insert(elongs, {node = n, locale = locale} )
6144                 node.set_attribute(n.prev, KASHIDA, 0)
6145             end
6146         end
6147
6148         % Tatwil. First create a list of nodes marked with kashida. The
6149         % rest of nodes can be ignored. The list of used weights is build
6150         % when transforms with the key kashida= are declared.
6151         if Babel.kashida_wts then
6152             local k_wt = node.get_attribute(n, KASHIDA)
6153             if k_wt > 0 then % todo. parameter for multi inserts
6154                 table.insert(k_list, {node = n, weight = k_wt, pos = pos_inline})
6155             end
6156         end
6157
6158     end % of node.traverse_id
6159
6160     if #elongs == 0 and #k_list == 0 then goto next_line end
6161     full = line.width
6162     shift = line.shift
6163     goal = full * Babel.arabic.justify_factor % A bit crude
6164     width = node.dimensions(line.head) % The 'natural' width
6165
6166     % == Elongated ==
6167     % Original idea taken from 'chickenize'
6168     while (#elongs > 0 and width < goal) do
6169         subst_done = true
6170         local x = #elongs
6171         local curr = elongs[x].node
6172         local oldchar = curr.char
6173         curr.char = elong_map[elongs[x].locale][curr.font][curr.char]
6174         width = node.dimensions(line.head) % Check if the line is too wide
6175         % Substitute back if the line would be too wide and break:
6176         if width > goal then
6177             curr.char = oldchar
6178             break
6179         end
6180         % If continue, pop the just substituted node from the list:
6181         table.remove(elongs, x)
6182     end
6183
6184     % == Tatwil ==

```

```

6185 % Traverse the kashida node list so many times as required, until
6186 % the line is filled. The first pass adds a tatweel after each
6187 % node with kashida in the line, the second pass adds another one,
6188 % and so on. In each pass, add first the kashida with the highest
6189 % weight, then with lower weight and so on.
6190 if #k_list == 0 then goto next_line end
6191
6192 width = node.dimensions(line.head) % The 'natural' width
6193 k_curr = #k_list % Traverse backwards, from the end
6194 wt_pos = 1
6195
6196 while width < goal do
6197     subst_done = true
6198     k_item = k_list[k_curr].node
6199     if k_list[k_curr].weight == Babel.kashida_wts[wt_pos] then
6200         d = node.copy(k_item)
6201         d.char = 0x0640
6202         d.yoffset = 0 % TODO. From the prev char. But 0 seems safe.
6203         d.xoffset = 0
6204         line.head, new = node.insert_after(line.head, k_item, d)
6205         width_new = node.dimensions(line.head)
6206         if width > goal or width == width_new then
6207             node.remove(line.head, new) % Better compute before
6208             break
6209         end
6210         if Babel.fix_diacr then
6211             Babel.fix_diacr(k_item.next)
6212         end
6213         width = width_new
6214     end
6215     if k_curr == 1 then
6216         k_curr = #k_list
6217         wt_pos = (wt_pos >= table.getn(Babel.kashida_wts)) and 1 or wt_pos+1
6218     else
6219         k_curr = k_curr - 1
6220     end
6221 end
6222
6223 % Limit the number of tatweel by removing them. Not very efficient,
6224 % but it does the job in a quite predictable way.
6225 if Babel.arabic.kashida_limit > -1 then
6226     cnt = 0
6227     for n in node.traverse_id(GLYPH, line.head) do
6228         if n.char == 0x0640 then
6229             cnt = cnt + 1
6230             if cnt > Babel.arabic.kashida_limit then
6231                 node.remove(line.head, n)
6232             end
6233         else
6234             cnt = 0
6235         end
6236     end
6237 end
6238
6239 ::next_line::
6240
6241 % Must take into account marks and ins, see luatex manual.
6242 % Have to be executed only if there are changes. Investigate
6243 % what's going on exactly.
6244 if subst_done and not gc then
6245     d = node.hpack(line.head, full, 'exactly')
6246     d.shift = shift
6247     node.insert_before(head, line, d)

```

```

6248     node.remove(head, line)
6249   end
6250 end % if process line
6251 end
6252 }
6253 \endgroup
6254 \fi\fi % ends Arabic just block: \ifnum\bbl@bidimode>100...

```

10.9 Common stuff

First, a couple of auxiliary macros to set the renderer according to the script. This is done by patching temporarily the low-level fontspec macro containing the current features set with `\defaultfontfeatures`. Admittedly this is somewhat dangerous, but that way the latter command still works as expected, because the renderer is set just before other settings. In xetex they are set to `\relax`.

```

6255 \def\bbl@scr@node@list{%
6256   ,Armenian,Coptic,Cyrillic,Georgian,,Glagolitic,Gothic,%
6257   ,Greek,Latin,Old Church Slavonic Cyrillic,}
6258 \ifnum\bbl@bidimode=102 % bidi-r
6259   \bbl@add\bbl@scr@node@list{Arabic,Hebrew,Syriac}
6260 \fi
6261 \def\bbl@set@renderer{%
6262   \bbl@xin@{\bbl@cl{sname}}{\bbl@scr@node@list}%
6263   \ifin@
6264     \let\bbl@unset@renderer\relax
6265   \else
6266     \bbl@exp{%
6267       \def\\bbl@unset@renderer{%
6268         \def<g__fontspec_default_fontopts_clist>{%
6269           [g__fontspec_default_fontopts_clist]}%
6270         \def<g__fontspec_default_fontopts_clist>{%
6271           Renderer=Harfbuzz,[g__fontspec_default_fontopts_clist]}%
6272       \fi}
6273 <@Font selection@>

```

10.10 Automatic fonts and ids switching

After defining the blocks for a number of scripts (must be extended and very likely fine tuned), we define a the function `Babel.locale_map`, which just traverse the node list to carry out the replacements. The table `loc_to_scr` stores the script range for each locale (whose id is the key), copied from this table (so that it can be modified on a locale basis); there is an intermediate table named `chr_to_loc` built on the fly for optimization, which maps a char to the locale. This locale is then used to get the `\language` as stored in `locale_props`, as well as the font (as requested). In the latter table a key starting with `/` maps the font from the global one (the key) to the local one (the value). Maths are skipped and discretionaries are handled in a special way.

There are two situations where the replacement is not carried out: either the `letters` option has been set and the character is not a letter (in the $\TeX$ sense), or the current script is the same as the new one.

```

6274 \directlua{% DL6
6275 Babel.script_blocks = {
6276   ['dflt'] = {},
6277   ['Arab'] = {{0x0600, 0x06FF}, {0x08A0, 0x08FF}, {0x0750, 0x077F},
6278               {0xFE70, 0xFEFF}, {0xFB50, 0xFDFF}, {0x1EE00, 0x1EEFF}},
6279   ['Armn'] = {{0x0530, 0x058F}},
6280   ['Beng'] = {{0x0980, 0x09FF}},
6281   ['Cher'] = {{0x13A0, 0x13FF}, {0xAB70, 0xABBF}},
6282   ['Copt'] = {{0x03E2, 0x03EF}, {0x2C80, 0x2CFF}, {0x102E0, 0x102FF}},
6283   ['Cyr'] = {{0x0400, 0x04FF}, {0x0500, 0x052F}, {0x1C80, 0x1C8F},
6284              {0x2DE0, 0x2DFF}, {0xA640, 0xA69F}},
6285   ['Deva'] = {{0x0900, 0x097F}, {0xA8E0, 0xA8FF}},
6286   ['Ethi'] = {{0x1200, 0x137F}, {0x1380, 0x139F}, {0x2D80, 0x2DDF},
6287              {0xAB00, 0xAB2F}},

```

```

6288 ['Geor'] = {{0x10A0, 0x10FF}, {0x2D00, 0x2D2F}},
6289 % Don't follow strictly Unicode, which places some Coptic letters in
6290 % the 'Greek and Coptic' block
6291 ['Grek'] = {{0x0370, 0x03E1}, {0x03F0, 0x03FF}, {0x1F00, 0x1FFF}},
6292 ['Hans'] = {{0x2E80, 0x2EFF}, {0x3000, 0x303F}, {0x31C0, 0x31EF},
6293             {0x3300, 0x33FF}, {0x3400, 0x4DBF}, {0x4E00, 0x9FFF},
6294             {0xF900, 0xFAFF}, {0xFE30, 0xFE4F}, {0xFF00, 0xFFEF},
6295             {0x2000, 0x2A6DF}, {0x2A700, 0x2B73F},
6296             {0x2B740, 0x2B81F}, {0x2B820, 0x2CEAF},
6297             {0x2CEB0, 0x2EBEF}, {0x2F800, 0x2FA1F}},
6298 ['Hebr'] = {{0x0590, 0x05FF},
6299             {0xFB1F, 0xFB4E}}, % <- Includes some <reserved>
6300 ['Jpan'] = {{0x3000, 0x303F}, {0x3040, 0x309F}, {0x30A0, 0x30FF},
6301             {0x4E00, 0x9FAF}, {0xFF00, 0xFFEF}},
6302 ['Khmr'] = {{0x1780, 0x17FF}, {0x19E0, 0x19FF}},
6303 ['Knda'] = {{0x0C80, 0x0CFF}},
6304 ['Kore'] = {{0x1100, 0x11FF}, {0x3000, 0x303F}, {0x3130, 0x318F},
6305             {0x4E00, 0x9FAF}, {0xA960, 0xA97F}, {0xAC00, 0xD7AF},
6306             {0xD7B0, 0xD7FF}, {0xFF00, 0xFFEF}},
6307 ['Lao'] = {{0x0E80, 0x0EFF}},
6308 ['Latn'] = {{0x0000, 0x007F}, {0x0080, 0x00FF}, {0x0100, 0x017F},
6309             {0x0180, 0x024F}, {0x1E00, 0x1EFF}, {0x2C60, 0x2C7F},
6310             {0xA720, 0xA7FF}, {0xAB30, 0xAB6F}},
6311 ['Mahj'] = {{0x11150, 0x1117F}},
6312 ['Mlym'] = {{0x0D00, 0x0D7F}},
6313 ['Mymr'] = {{0x1000, 0x109F}, {0xAA60, 0xAA7F}, {0xA9E0, 0xA9FF}},
6314 ['Orya'] = {{0x0B00, 0x0B7F}},
6315 ['Sinh'] = {{0x0D80, 0x0DFF}, {0x111E0, 0x111FF}},
6316 ['Syr'] = {{0x0700, 0x074F}, {0x0860, 0x086F}},
6317 ['Taml'] = {{0x0B80, 0x0BFF}},
6318 ['Telu'] = {{0x0C00, 0x0C7F}},
6319 ['Tfng'] = {{0x2D30, 0x2D7F}},
6320 ['Thai'] = {{0x0E00, 0x0E7F}},
6321 ['Tibt'] = {{0x0F00, 0x0FFF}},
6322 ['Vaii'] = {{0xA500, 0xA63F}},
6323 ['Yiii'] = {{0xA000, 0xA48F}, {0xA490, 0xA4CF}}
6324 }
6325
6326 Babel.script_blocks.Cyrs = Babel.script_blocks.Cyrl
6327 Babel.script_blocks.Hant = Babel.script_blocks.Hans
6328 Babel.script_blocks.Kana = Babel.script_blocks.Jpan
6329
6330 function Babel.locale_map(head)
6331   if not Babel.locale_mapped then return head end
6332
6333   local LOCALE = Babel.attr_locale
6334   local GLYPH = node.id('glyph')
6335   local inmath = false
6336   local toloc_save
6337   for item in node.traverse(head) do
6338     local toloc
6339     if not inmath and item.id == GLYPH then
6340       % Optimization: build a table with the chars found
6341       if Babel.chr_to_loc[item.char] then
6342         toloc = Babel.chr_to_loc[item.char]
6343       else
6344         for lc, maps in pairs(Babel.loc_to_scr) do
6345           for _, rg in pairs(maps) do
6346             if item.char >= rg[1] and item.char <= rg[2] then
6347               Babel.chr_to_loc[item.char] = lc
6348               toloc = lc
6349               break
6350             end
6351           end
6352         end
6353       end
6354     end
6355     toloc_save = toloc
6356     item.toloc = toloc_save
6357   end

```

```

6351         end
6352     end
6353     % Treat composite chars in a different fashion, because they
6354     % 'inherit' the previous locale.
6355     if (item.char >= 0x0300 and item.char <= 0x036F) or
6356        (item.char >= 0x1AB0 and item.char <= 0x1AFF) or
6357        (item.char >= 0x1DC0 and item.char <= 0x1DFF) then
6358         Babel.chr_to_loc[item.char] = -2000
6359         toloc = -2000
6360     end
6361     if not toloc then
6362         Babel.chr_to_loc[item.char] = -1000
6363     end
6364     end
6365     if toloc == -2000 then
6366         toloc = toloc_save
6367     elseif toloc == -1000 then
6368         toloc = nil
6369     end
6370     if toloc and Babel.locale_props[toloc] and
6371        Babel.locale_props[toloc].letters and
6372        tex.getcatcode(item.char) \string~= 11 then
6373         toloc = nil
6374     end
6375     if toloc and Babel.locale_props[toloc].script
6376        and Babel.locale_props[node.get_attribute(item, LOCALE)].script
6377        and Babel.locale_props[toloc].script ==
6378        Babel.locale_props[node.get_attribute(item, LOCALE)].script then
6379         toloc = nil
6380     end
6381     if toloc then
6382         if Babel.locale_props[toloc].lg then
6383             item.lang = Babel.locale_props[toloc].lg
6384             node.set_attribute(item, LOCALE, toloc)
6385         end
6386         if Babel.locale_props[toloc]['/'..item.font] then
6387             item.font = Babel.locale_props[toloc]['/'..item.font]
6388         end
6389     end
6390     toloc_save = toloc
6391     elseif not inmath and item.id == 7 then % Apply recursively
6392         item.replace = item.replace and Babel.locale_map(item.replace)
6393         item.pre      = item.pre and Babel.locale_map(item.pre)
6394         item.post      = item.post and Babel.locale_map(item.post)
6395     elseif item.id == node.id'math' then
6396         inmath = (item.subtype == 0)
6397     end
6398 end
6399 return head
6400 end
6401 }

```

The code for \babelcharproperty is straightforward. Just note the modified lua table can be different.

```

6402 \newcommand\babelcharproperty[1]{%
6403   \count@=#1\relax
6404   \ifvmode
6405     \expandafter\bbl@chprop
6406   \else
6407     \bbl@error{charproperty-only-vertical}{}}}%
6408 \fi}
6409 \newcommand\bbl@chprop[3][\the\count@]{%
6410   \@tempcnta=#1\relax
6411   \bbl@ifunset\bbl@chprop@#2}% {unknown-char-property}

```

```

6412     {\bbl@error{unknown-char-property}}{#2}}}%
6413     }%
6414     \loop
6415     \bbl@cs{chprop@#2}{#3}%
6416     \ifnum\count@<\@tempcnta
6417     \advance\count@\@ne
6418     \repeat}
6419 %
6420 \def\bbl@chprop@direction#1{%
6421   \directlua{
6422     Babel.characters[\the\count@] = Babel.characters[\the\count@] or {}
6423     Babel.characters[\the\count@]['d'] = '#1'
6424   }}
6425 \let\bbl@chprop@bc\bbl@chprop@direction
6426 %
6427 \def\bbl@chprop@mirror#1{%
6428   \directlua{
6429     Babel.characters[\the\count@] = Babel.characters[\the\count@] or {}
6430     Babel.characters[\the\count@]['m'] = '\number#1'
6431   }}
6432 \let\bbl@chprop@bmg\bbl@chprop@mirror
6433 %
6434 \def\bbl@chprop@linebreak#1{%
6435   \directlua{
6436     Babel.cjk_characters[\the\count@] = Babel.cjk_characters[\the\count@] or {}
6437     Babel.cjk_characters[\the\count@]['c'] = '#1'
6438   }}
6439 \let\bbl@chprop@lb\bbl@chprop@linebreak
6440 %
6441 \def\bbl@chprop@locale#1{%
6442   \directlua{
6443     Babel.chr_to_loc = Babel.chr_to_loc or {}
6444     Babel.chr_to_loc[\the\count@] =
6445       \bbl@ifblank{#1}{-1000}{\the\bbl@cs{id@#1}}\space
6446   }}

```

Post-handling hyphenation patterns for non-standard rules, like ff to ff-f. There are still some issues with speed (not very slow, but still slow). The Lua code is below.

```

6447 \directlua{% DL7
6448   Babel.nohyphenation = \the\l@nohyphenation
6449 }

```

Now the TeX high level interface, which requires the function defined above for converting strings to functions returning a string. These functions handle the $\{n\}$ syntax. For example, $\text{pre}=\{1\}\{1\}$ - becomes $\text{function}(m) \text{ return } m[1] \dots m[1] \dots '-' \text{ end}$, where m are the matches returned after applying the pattern. With a mapped capture the functions are similar to $\text{function}(m) \text{ return } \text{Babel.capt_map}(m[1], 1) \text{ end}$, where the last argument identifies the mapping to be applied to $m[1]$. The way it is carried out is somewhat tricky, but the effect is not dissimilar to lua load – save the code as string in a TeX macro, and expand this macro at the appropriate place. As $\text{\directlua}$ does not take into account the current catcode of $@$, we just avoid this character in macro names (which explains the internal group, too).

```

6450 \begingroup
6451 \catcode`\-=12
6452 \catcode`\%=12
6453 \catcode`\&=14
6454 \catcode`\|=12
6455 \gdef\babelprehyphenation{%&
6456   \@ifnextchar[{\bbl@settransform{0}}{\bbl@settransform{0}[]}]
6457 \gdef\babelposthyphenation{%&
6458   \@ifnextchar[{\bbl@settransform{1}}{\bbl@settransform{1}[]}]
6459 %
6460 \gdef\bbl@settransform#1[#2]#3#4#5{%&
6461   \ifcase#1
6462     \bbl@activateprehyphen

```

```

6463 \or
6464 \bbl@activateposthyphen
6465 \fi
6466 \beginingroup
6467 \def\bbl@tempa{\bbl@add@list\bbl@tempb}&%
6468 \let\bbl@tempb\@empty
6469 \def\bbl@tempa{#5}&%
6470 \bbl@replace\bbl@tempa{,}{ ,}&% TODO. Ugly trick to preserve {}
6471 \expandafter\bbl@foreach\expandafter{\bbl@tempa}{&%
6472 \bbl@ifsamestring{##1}{remove}&%
6473 {\bbl@add@list\bbl@tempb{nil}}&%
6474 {\directlua{
6475 local rep = {[##1]=}
6476 local three_args = '%s*=%s*([%-d%.%a{}|]|+)%s+([%-d%.%a{}|]|+)%s+([%-d%.%a{}|]|+)'
6477 &% Numeric passes directly: kern, penalty...
6478 rep = rep:gsub('^%s*(remove)%s*$', 'remove = true')
6479 rep = rep:gsub('^%s*(insert)%s*', ', 'insert = true, ')
6480 rep = rep:gsub('^%s*(after)%s*', ', 'after = true, ')
6481 rep = rep:gsub('(string)%s*=%s*([%^s,]*)', Babel.capture_func)
6482 rep = rep:gsub('node%s*=%s*(%a+)%s*(%a*)', Babel.capture_node)
6483 rep = rep:gsub(' (norule)' .. three_args,
6484 'norule = {' .. '%2, %3, %4' .. '})')
6485 if #1 == 0 or #1 == 2 then
6486 rep = rep:gsub(' (space)' .. three_args,
6487 'space = {' .. '%2, %3, %4' .. '})')
6488 rep = rep:gsub(' (spacefactor)' .. three_args,
6489 'spacefactor = {' .. '%2, %3, %4' .. '})')
6490 rep = rep:gsub('(kashida)%s*=%s*([%^s,]*)', Babel.capture_kashida)
6491 &% Transform values
6492 rep, n = rep:gsub(' {[(%a%-%.|.)]|([%a%_%.|.)+}}',
6493 function(v,d)
6494 return string.format (
6495 '\the\csname bbl@id@@#3\endcsname,"%s",%s}',
6496 v,
6497 load( 'return Babel.locale_props'..
6498 '\the\csname bbl@id@@#3\endcsname].' .. d)() )
6499 end )
6500 rep, n = rep:gsub(' {[(%a%-%.|.)+]|([%-d%.|.)+}}',
6501 '\the\csname bbl@id@@#3\endcsname,"%1",%2}')
6502 end
6503 if #1 == 1 then
6504 rep = rep:gsub(' (no)%s*=%s*([%^s,]*)', Babel.capture_func)
6505 rep = rep:gsub(' (pre)%s*=%s*([%^s,]*)', Babel.capture_func)
6506 rep = rep:gsub(' (post)%s*=%s*([%^s,]*)', Babel.capture_func)
6507 end
6508 tex.print([[\string\bbl@tempa{[]] .. rep .. [[]]])
6509 }}&%
6510 \bbl@foreach\bbl@tempb{&%
6511 \bbl@forkv{##1}}{&%
6512 \in@{,###1,}{,nil,step,data,remove,insert,string,no,pre,no,&%
6513 post,penalty,kashida,space,spacefactor,kern,node,after,norule,}&%
6514 \ifin@else
6515 \bbl@error{bad-transform-option}{###1}{,}{&%
6516 \fi}}&%
6517 \let\bbl@kv@attribute\relax
6518 \let\bbl@kv@label\relax
6519 \let\bbl@kv@fonts\@empty
6520 \let\bbl@kv@prepend\relax
6521 \bbl@forkv{#2}{\bbl@csarg\edef{kv{##1}{##2}}&%
6522 \ifx\bbl@kv@fonts\@empty\else\bbl@settransfont\fi
6523 \ifx\bbl@kv@attribute\relax
6524 \ifx\bbl@kv@label\relax\else
6525 \bbl@exp{\bbl@trim@def\bbl@kv@fonts{\bbl@kv@fonts}}&%

```

```

6526 \bbl@replace\bbl@kv@fonts{ }{,}&%
6527 \edef\bbl@kv@attribute{\bbl@ATR@\bbl@kv@label @#3@\bbl@kv@fonts}&%
6528 \count@\z@
6529 \def\bbl@elt##1##2##3{&%
6530 \bbl@ifsamestring{#3,\bbl@kv@label}{##1,##2}&%
6531 {\bbl@ifsamestring{\bbl@kv@fonts}{##3}&%
6532 {\count@\@ne}&%
6533 {\bbl@error{font-conflict-transforms}{}}{}}}&%
6534 {}}&%
6535 \bbl@transfont@list
6536 \ifnum\count@=\z@
6537 \bbl@exp{\global\\bbl@add\\bbl@transfont@list
6538 {\bbl@elt{#3}{\bbl@kv@label}{\bbl@kv@fonts}}}&%
6539 \fi
6540 \bbl@ifunset{\bbl@kv@attribute}&%
6541 {\global\bbl@carg\newattribute{\bbl@kv@attribute}}&%
6542 {}&%
6543 \global\bbl@carg\setattribute{\bbl@kv@attribute}\@ne
6544 \fi
6545 \else
6546 \edef\bbl@kv@attribute{\expandafter\bbl@stripslash\bbl@kv@attribute}&%
6547 \fi
6548 \directlua{
6549 local lbkr = Babel.linebreaking.replacements[#1]
6550 local u = unicode.utf8
6551 local id, attr, label
6552 if #1 == 0 then
6553 id = \the\csname bbl@id@@#3\endcsname\space
6554 else
6555 id = \the\csname l@#3\endcsname\space
6556 end
6557 \ifx\bbl@kv@attribute\relax
6558 attr = -1
6559 \else
6560 attr = luatexbase.registernumber'\bbl@kv@attribute'
6561 \fi
6562 \ifx\bbl@kv@label\relax\else &% Same refs:
6563 label = [==[\bbl@kv@label]==]
6564 \fi
6565 &% Convert pattern:
6566 local patt = string.gsub([==[#4]==], '%s', '')
6567 if #1 == 0 then
6568 patt = string.gsub(patt, '|', ' ')
6569 end
6570 if not u.find(patt, '()', nil, true) then
6571 patt = '()' .. patt .. '()'
6572 end
6573 patt = string.gsub(patt, '%(%)%^', '^()')
6574 patt = string.gsub(patt, '%$(%)', '()$')
6575 patt = u.gsub(patt, '{(.)}',
6576 function (n)
6577 return '%' .. (tonumber(n) and (tonumber(n)+1) or n)
6578 end)
6579 patt = u.gsub(patt, '{(%x%x%x%x+)}',
6580 function (n)
6581 return u.gsub(u.char(tonumber(n, 16)), '(%p)', '%%1')
6582 end)
6583 lbkr[id] = lbkr[id] or {}
6584 table.insert(lbkr[id], \ifx\bbl@kv@prepend\relax\else 1,\fi
6585 { label=label, attr=attr, pattern=patt, replace={\babeltempb} })
6586 }&%
6587 \endgroup}
6588 \endgroup

```



```

6589 %
6590 \let\bbl@transfont@list\@empty
6591 \def\bbl@settransfont{%
6592   \global\let\bbl@settransfont\relax % Execute only once
6593   \gdef\bbl@transfont{%
6594     \def\bbl@elt####1####2####3{%
6595       \bbl@ifblank{####3}%
6596       {\count@tw@}% Do nothing if no fonts
6597       {\count@z@
6598         \bbl@vforeach{####3}{%
6599           \def\bbl@tempd{#####1}%
6600           \edef\bbl@tempe{\bbl@transfam/\f@series/\f@shape}%
6601           \ifx\bbl@tempd\bbl@tempe
6602             \count@ne
6603           \else\ifx\bbl@tempd\bbl@transfam
6604             \count@ne
6605           \fi\fi}%
6606           \ifcase\count@
6607             \bbl@csarg\unsetattribute{ATR####2@####1@####3}%
6608           \or
6609             \bbl@csarg\setattribute{ATR####2@####1@####3}\@ne
6610           \fi}}%
6611     \bbl@transfont@list}%
6612 \AddToHook{selectfont}{\bbl@transfont}% Hooks are global.
6613 \gdef\bbl@transfam{-unknown-}%
6614 \bbl@foreach\bbl@font@fams{%
6615   \AddToHook{##1family}{\def\bbl@transfam{##1}}%
6616   \bbl@ifsamestring{\@nameuse{##1default}}\familydefault
6617   {\xdef\bbl@transfam{##1}}%
6618   {}}}
6619 %
6620 \DeclareRobustCommand\enablelocaletransform[1]{%
6621   \bbl@ifunset{\bbl@ATR#1@\language @}%
6622   {\bbl@error{transform-not-available}{#1}{}}}%
6623   {\bbl@csarg\setattribute{ATR#1@\language @}\@ne}}
6624 \DeclareRobustCommand\disablelocaletransform[1]{%
6625   \bbl@ifunset{\bbl@ATR#1@\language @}%
6626   {\bbl@error{transform-not-available-b}{#1}{}}}%
6627   {\bbl@csarg\unsetattribute{ATR#1@\language @}}}%

```

The following two macros load the Lua code for transforms, but only once. The only difference is in add_after and add_before.

```

6628 \def\bbl@activateposthyphen{%
6629   \let\bbl@activateposthyphen\relax
6630   \ifx\bbl@attr@hboxed\@undefined
6631     \newattribute\bbl@attr@hboxed
6632   \fi
6633   \directlua{
6634     require('babel-transforms.lua')
6635     Babel.linebreaking.add_after(Babel.post_hyphenate_replace)
6636   }}
6637 \def\bbl@activateprehyphen{%
6638   \let\bbl@activateprehyphen\relax
6639   \ifx\bbl@attr@hboxed\@undefined
6640     \newattribute\bbl@attr@hboxed
6641   \fi
6642   \directlua{
6643     require('babel-transforms.lua')
6644     Babel.linebreaking.add_before(Babel.pre_hyphenate_replace)
6645   }}
6646 \newcommand\SetTransformValue[3]{%
6647   \directlua{
6648     Babel.locale_props[\the\csname bbl@id@#1\endcsname].vars["#2"] = #3
6649   }}

```

The code in `babel-transforms.lua` prints at some points the current string being transformed. This macro first make sure this file is loaded. Then, activates temporarily this feature and typeset inside a box the text in the argument.

```
6650 \newcommand\ShowBabelTransforms[1]{%
6651   \bbl@activateprehyphen
6652   \bbl@activateposthyphen
6653   \begingroup
6654     \directlua{ Babel.show_transforms = true }%
6655     \setbox\z@\vbox{#1}%
6656     \directlua{ Babel.show_transforms = false }%
6657   \endgroup}
```

The following experimental (and unfinished) macro applies the prehyphenation transforms for the current locale to a string (characters and spaces) and processes it in a fully expandable way (among other limitations, the string can't contain `]==]`). The way it operates is admittedly rather cumbersome: it converts the string to a node list, processes it, and converts it back to a string. The lua code is in the lua file below.

```
6658 \newcommand\localeprehyphenation[1]{%
6659   \directlua{ Babel.string_prehyphenation([==[#1]==], \the\localeid) }}
```

10.11 Bidi

Make sure the `\pagedir` is always the same as the `\bodydir` when the document starts. At this point the main language has been selected, and therefore its `\bodydir` has been set.

```
6660 \AtBeginDocument{\pagedir\bodydir}
```

As a first step, add a handler for bidi and digits (and potentially other processes) just before `luaotfload` is applied, which is loaded by default by $\TeX$. Just in case, consider the possibility it has not been loaded.

```
6661 \def\bbl@activate@preotf{%
6662   \let\bbl@activate@preotf\relax % only once
6663   \directlua{
6664     function Babel.pre_otfload_v(head)
6665       if Babel.numbers and Babel.digits_mapped then
6666         head = Babel.numbers(head)
6667       end
6668       if Babel.bidi_enabled then
6669         head = Babel.bidi(head, false, dir)
6670       end
6671       return head
6672     end
6673     %
6674     function Babel.pre_otfload_h(head, gc, sz, pt, dir)
6675       if Babel.numbers and Babel.digits_mapped then
6676         head = Babel.numbers(head)
6677       end
6678       if Babel.bidi_enabled then
6679         head = Babel.bidi(head, false, dir)
6680       end
6681       return head
6682     end
6683     %
6684     luatexbase.add_to_callback('pre_linebreak_filter',
6685       Babel.pre_otfload_v,
6686       'Babel.pre_otfload_v',
6687       Babel.priority_in_callback('pre_linebreak_filter',
6688         'luaotfload.node_processor') or nil)
6689     %
6690     luatexbase.add_to_callback('hpack_filter',
6691       Babel.pre_otfload_h,
6692       'Babel.pre_otfload_h',
6693       Babel.priority_in_callback('hpack_filter',
6694         'luaotfload.node_processor') or nil)
```

```
6695  }}
```

The basic setup. The output is modified at a very low level to set the `\bodydir` to the `\pagedir`. Sadly, we have to deal with boxes in math with basic, so the `\bbl@insidemath` hack is activated (set to 1) every math with the package option `bidi=`. The hack for the PUA is no longer necessary with basic (24.8), but it's kept in `basic-r`.

```
6696 \ifnum\bbl@bidimode>\@ne % Any bidi= except default (=1)
6697 \let\bbl@beforeforeign\leavevmode
6698 \AtEndOfPackage{\EnableBabelHook{babel-bidi}}
6699 \RequirePackage{luatexbase}
6700 \bbl@activate@preotf
6701 \directlua{
6702   require('babel-data-bidi.lua')
6703   \ifcase\expandafter\@gobbletwo\the\bbl@bidimode\or
6704     require('babel-bidi-basic.lua')
6705   \or
6706     require('babel-bidi-basic-r.lua')
6707     table.insert(Babel.ranges, {0xE000, 0xF8FF, 'on'})
6708     table.insert(Babel.ranges, {0xF000, 0xFFFFD, 'on'})
6709     table.insert(Babel.ranges, {0x10000, 0x10FFFD, 'on'})
6710   \fi}
6711 \newattribute\bbl@attr@dir
6712 \directlua{ Babel.attr_dir = luatexbase.registernumber'bbl@attr@dir' }
6713 \bbl@exp{\output{\bodydir\pagedir\the\output}}
6714 \fi
6715 %
6716 \chardef\bbl@thetextdir\z@
6717 \chardef\bbl@thepardir\z@
6718 \def\bbl@setluadir#1#2{% 1=\text/pardirection 2= 0:l 1:r 2:a:l
6719   \ifcase#2\relax
6720     \ifcase#1\else#1=\z@\fi
6721   \else
6722     \ifcase#1#1=\@ne\fi
6723   \fi}
```

`\bbl@attr@dir` stores the directions with a mask: `..00PPTT`, with masks `0xC` (PP is the par dir) and `0x3` (TT is the text dir). These macro names are shared by the 3 engines, with different definitions.

```
6724 \def\bbl@thedir{0}
6725 \def\bbl@textdir#1{%
6726   \bbl@setluadir\textdirection{#1}%
6727   \chardef\bbl@thetextdir#1\relax
6728   \edef\bbl@thedir{\the\numexpr\bbl@thepardir*4+#1}%
6729   \setattribute\bbl@attr@dir{\numexpr\bbl@thepardir*4+#1}}
6730 \def\bbl@pardir#1{% Used twice
6731   \bbl@setluadir\pardirection{#1}%
6732   \chardef\bbl@thepardir#1\relax}
6733 \def\bbl@bodydir{\bbl@setluadir\bodydirection}% Used once
6734 \def\bbl@dirparastext{\pardirection=\textdirection\relax}% Used once
```

RTL text inside math needs special attention. It affects not only to actual math stuff, but also to ‘tabular’, which is based on a fake math.

```
6735 \ifnum\bbl@bidimode>\z@ % Any bidi=
6736 \def\bbl@insidemath{0}%
6737 \def\bbl@everymath{\def\bbl@insidemath{1}}
6738 \def\bbl@everydisplay{\def\bbl@insidemath{2}}
6739 \frozen@everymath\expandafter{%
6740   \expandafter\bbl@everymath\the\frozen@everymath}
6741 \frozen@everydisplay\expandafter{%
6742   \expandafter\bbl@everydisplay\the\frozen@everydisplay}
6743 \AtBeginDocument{
6744   \directlua{
6745     function Babel.math_box_dir(head)
6746       if not (token.get_macro('bbl@insidemath') == '0') then
6747         if Babel.hlist_has_bidi(head) then
```

```

6748         local d = node.new(node.id'dir')
6749         d.dir = '+TRT'
6750         for item in node.traverse(head) do
6751             if item.id == 11 or item.id == node.id'glyph' then
6752                 head = node.insert_before(head, item, d)
6753                 break
6754             end
6755         end
6756         local inmath = false
6757         for item in node.traverse(head) do
6758             if item.id == 11 then
6759                 inmath = (item.subtype == 0)
6760             elseif not inmath then
6761                 node.set_attribute(item,
6762                     Babel.attr_dir, token.get_macro('bbl@thedir'))
6763             end
6764         end
6765     end
6766 end
6767 return head
6768 end
6769 luatexbase.add_to_callback("hpack_filter", Babel.math_box_dir,
6770     "Babel.math_box_dir", 0)
6771 if Babel.unset_atdir then
6772     luatexbase.add_to_callback("pre_linebreak_filter", Babel.unset_atdir,
6773         "Babel.unset_atdir")
6774     luatexbase.add_to_callback("hpack_filter", Babel.unset_atdir,
6775         "Babel.unset_atdir")
6776 end
6777 luatexbase.add_to_callback("post_mlist_to_hlist_filter", function(head)
6778     local dir = tex.mathdirection
6779     local n = node.new("dir")
6780     n.direction = dir
6781     head = node.insert_before(head, head, n)
6782     n = node.new("dir", 1)
6783     n.direction = dir
6784     head = node.insert_after(head, node.tail(head), n)
6785     return head
6786 end, "Babel.ensure_math_dir")
6787 } }%
6788 \fi

```

Experimental. Tentative name.

```

6789 \DeclareRobustCommand\localebox[1]{%
6790     {\def\bbl@insidemath{0}%
6791         \mbox{\foreignlanguage{\language}{#1}}}}

```

10.12 Layout

Unlike xetex, luatex requires only minimal changes for right-to-left layouts, particularly in monolingual documents (the engine itself reverses boxes – including column order or headings –, margins, etc.) with `bidirectional=basic`, without having to patch almost any macro where text direction is relevant.

Still, there are three areas deserving special attention, namely, tabular, math, and graphics, text and intrinsically left-to-right elements are intermingled. I've made some progress in graphics, but they're essentially hacks; I've also made some progress in 'tabular', but when I decided to tackle math (both standard math and 'amsmath') the nightmare began. I'm still not sure how 'amsmath' should be modified, but the main problem is that, boxes are "generic" containers that can hold text, math, and graphics (even at the same time; remember that inline math is included in the list of text nodes marked with 'math' (11) nodes too).

`\@hangfrom` is useful in many contexts and it is redefined always with the layout option.

There are, however, a number of issues when the text direction is not the same as the box direction (as set by `\bodydir`), and when `\parbox` and `\hangindent` are involved. Fortunately, latest releases of luatex simplify a lot the solution with `\shapemode`.

With the issue #15 I realized commands are best patched, instead of redefined. With a few lines, a modification could be applied to several classes and packages. Now, tabular seems to work (at least in simple cases) with array, tabularx, hhline, colortbl, longtable, booktabs, etc. However, dcolumn still fails.

```
6792 \bbl@trace{Redefinitions for bidi layout}
6793 %
6794 ({*More package options}) ≡
6795 \chardef\bbl@eqnpos\z@
6796 \DeclareOption{leqno}{\chardef\bbl@eqnpos@ne}
6797 \DeclareOption{fleqn}{\chardef\bbl@eqnpos\tw@}
6798 (/More package options)
```

Fixes in luatex are sometimes done with flags. We first activate all all them.

```
6799 \ifnum\bbl@bidimode>\z@ % Any bidi=
6800 \matheqdirmode@ne
6801 \mathemptydisplaymode@ne
6802 \breakafterdirmode@ne
6803 \fixupboxesmode@ne
6804 \let\bbl@eqnodir\relax
6805 \def\bbl@eqdel{()}
6806 \def\bbl@eqnum{%
6807   {\normalfont\normalcolor
6808     \expandafter\@firstoftwo\bbl@eqdel
6809     \theequation
6810     \expandafter\@secondoftwo\bbl@eqdel}}
6811 \def\bbl@puteqno#1{\eqno\hbox{#1}}
6812 \def\bbl@putleqno#1{\leqno\hbox{#1}}
6813 \def\bbl@eqno@flip#1{% So
6814   \ifdim\predisplaysize=-\maxdimen % For consecutive displays
6815     \eqno
6816     \hb@xt@.01pt{%
6817       \hb@xt@\displaywidth{\hss{#1}\glet\bbl@upset\@currentlabel}}\hss}%
6818   \else
6819     \leqno\hbox{#1}\glet\bbl@upset\@currentlabel}%
6820   \fi
6821   \bbl@exp{\def\\@currentlabel{\[bbl@upset]}}}}
6822 \def\bbl@leqno@flip#1{%
6823   \ifdim\predisplaysize=-\maxdimen % For consecutive displays
6824     \leqno
6825     \hb@xt@.01pt{%
6826       \hss\hb@xt@\displaywidth{#1}\glet\bbl@upset\@currentlabel}\hss}%
6827   \else
6828     \eqno\hbox{#1}\glet\bbl@upset\@currentlabel}%
6829   \fi
6830   \bbl@exp{\def\\@currentlabel{\[bbl@upset]}}}}
6831 %
6832 \AtBeginDocument{%
6833   \ifx\bbl@noamsmath\relax\else
6834     \ifx\maketag@@@\@undefined % Normal equation, eqnarray
6835       \ifnum\bbl@eqnpos=\tw@
6836         \bbl@replace\equation{\hb@xt@\linewidth}%
6837           {\hbox bdir\mathdirection to\linewidth}%
6838         \bbl@carg\bbl@sreplace{[ ]}{\hb@xt@\linewidth}%
6839           {\hbox bdir\mathdirection to\linewidth}%
6840       \fi
6841       \AddToHook{env/equation/begin}{%
6842         \ifnum\bbl@thetextdir>\z@
6843           \let\@eqnnum\bbl@eqnum
6844           \edef\bbl@eqnodir{\noexpand\bbl@textdir{\the\bbl@thetextdir}}%
6845           \chardef\bbl@thetextdir\z@
6846           \bbl@add\normalfont{\bbl@eqnodir}%
6847           \ifcase\bbl@eqnpos
6848             \let\bbl@puteqno\bbl@eqno@flip
```

```

6849         \or
6850         \let\bbl@puteqno\bbl@leqno@flip
6851         \fi
6852     \fi}%
6853 \ifnum\bbl@eqnpos=\tw@%else
6854 \def\endequation{\bbl@puteqno{\@eqnnum}$\@ignoretrue}%
6855 \fi
6856 \AddToHook{env/eqnarray/begin}{%
6857     \ifnum\bbl@thetextdir>\z@
6858     \edef\bbl@eqnodir{\noexpand\bbl@textdir{\the\bbl@thetextdir}}%
6859     \chardef\bbl@thetextdir\z@
6860     \bbl@add\normalfont{\bbl@eqnodir}%
6861     \ifnum\bbl@eqnpos=\@ne
6862     \def\@eqnnum{%
6863         \setbox\z@\hbox{\bbl@eqnum}%
6864         \hbox to0.01pt{\hss\hbox to\displaywidth{\box\z@\hss}}}%
6865     \else
6866     \let\@eqnnum\bbl@eqnum
6867     \fi
6868 \fi}
6869 \else % amstex
6870 \bbl@exp{% Hack to hide maybe undefined conditionals:
6871     \chardef\bbl@eqnpos=0<\iftagsleft>1<\else>\if@fleqn>2<\fi>\relax}%
6872 \ifnum\bbl@eqnpos=\@ne
6873 \let\bbl@ams@lap\hbox
6874 \else
6875 \let\bbl@ams@lap\llap
6876 \fi
6877 \ExplSyntaxOn % Required by \bbl@sreplace with \intertext@
6878 \bbl@sreplace\intertext@{\normalbaselines}%
6879 {\normalbaselines
6880 \ifx\bbl@eqnodir\relax\else\bbl@pardir\@ne\bbl@eqnodir\fi}%
6881 \ExplSyntaxOff
6882 \def\bbl@ams@tagbox#1#2{\bbl@eqnodir#2}% #1=hbox|@lap|flip
6883 \ifx\bbl@ams@lap\hbox % leqno
6884 \def\bbl@ams@flip#1{%
6885     \hbox to 0.01pt{\hss\hbox to\displaywidth{\#1\hss}}}%
6886 \else % eqno
6887 \def\bbl@ams@flip#1{%
6888     \hbox to 0.01pt{\hbox to\displaywidth{\hss\#1\hss}}}%
6889 \fi
6890 \def\bbl@ams@preset#1{%
6891     \ifnum\bbl@thetextdir>\z@
6892     \edef\bbl@eqnodir{\noexpand\bbl@textdir{\the\bbl@thetextdir}}%
6893     \bbl@sreplace\textdef@{\hbox}{\bbl@ams@tagbox\hbox}%
6894     \bbl@sreplace\maketag@@@{\hbox}{\bbl@ams@tagbox#1}%
6895     \fi}%
6896 \def\bbl@ams@equation{%
6897     \ifnum\bbl@thetextdir>\z@
6898     \bbl@ams@preset\hbox
6899     \chardef\bbl@thetextdir\z@
6900     \bbl@add\normalfont{\bbl@eqnodir}%
6901     \ifcase\bbl@eqnpos
6902     \def\veqno##1##2{\bbl@eqno@flip{##1##2}}%
6903     \or
6904     \def\veqno##1##2{\bbl@leqno@flip{##1##2}}%
6905     \fi
6906 \fi}%
6907 \AddToHook{env/equation/begin}{\bbl@ams@equation}%
6908 \AddToHook{env/equation*/begin}{\bbl@ams@equation}%
6909 \AddToHook{env/cases/begin}{\bbl@ams@preset\bbl@ams@lap}%
6910 \AddToHook{env/multline/begin}{\bbl@ams@preset\hbox}%
6911 \AddToHook{env/gather/begin}{\bbl@ams@preset\bbl@ams@lap}%

```

```

6912 \AddToHook{env/gather*/begin}{\bbl@ams@preset\bbl@ams@lap}%
6913 \AddToHook{env/align/begin}{\bbl@ams@preset\bbl@ams@lap}%
6914 \AddToHook{env/align*/begin}{\bbl@ams@preset\bbl@ams@lap}%
6915 \AddToHook{env/alignat/begin}{\bbl@ams@preset\bbl@ams@lap}%
6916 \AddToHook{env/alignat*/begin}{\bbl@ams@preset\bbl@ams@lap}%
6917 \AddToHook{env/eqnalign/begin}{\bbl@ams@preset\bbl@ams@lap}%
6918 % Hackish, for proper alignment. Don't ask me why it works!:
6919 \bbl@exp{% Avoid a 'visible' conditional
6920   \\ \AddToHook{env/align*/end}{\<iftag>\<else>\\tag*{\<fi>}}%
6921   \\ \AddToHook{env/alignat*/end}{\<iftag>\<else>\\tag*{\<fi>}}%
6922 \AddToHook{env/flalign/begin}{\bbl@ams@preset\bbl@ams@lap}%
6923 \AddToHook{env/split/before}{%
6924   \ifnum\bbl@thetextdir>z@
6925     \bbl@ifsamestring\@currentenv{equation}%
6926     {\ifx\bbl@ams@lap\hbox % leqno
6927       \def\bbl@ams@flip#1{%
6928         \hbox to 0.01pt{\hbox to\displaywidth{#1}\hss}\hss}}%
6929       \else
6930       \def\bbl@ams@flip#1{%
6931         \hbox to 0.01pt{\hss\hbox to\displaywidth{\hss{#1}}}%
6932         \fi}%
6933     }%
6934   \fi}%
6935 \fi\fi}
6936 \fi

Declarations specific to lua, called by \babelprovide.
6937 \def\bbl@provide@extra#1{%
6938   % == onchar ==
6939   \ifx\bbl@KVP@onchar\@nnil\else
6940     \bbl@luahyphenate
6941     \bbl@exp{%
6942       \\ \AddToHook{env/document/before}{%
6943         {\let\\ \bbl@ifrestoring\\ \@firstoftwo
6944           \\ \select@language{#1}}}%
6945       \directlua{
6946         if Babel.locale_mapped == nil then
6947           Babel.locale_mapped = true
6948           Babel.linebreaking.add_before(Babel.locale_map, 1)
6949           Babel.loc_to_scr = {}
6950           Babel.chr_to_loc = Babel.chr_to_loc or {}
6951         end
6952         Babel.locale_props[\the\localeid].letters = false
6953       }%
6954       \bbl@xin@{ letters }{ \bbl@KVP@onchar\space}%
6955       \ifin@
6956         \directlua{
6957           Babel.locale_props[\the\localeid].letters = true
6958         }%
6959       \fi
6960       \bbl@xin@{ ids }{ \bbl@KVP@onchar\space}%
6961       \ifin@
6962         \ifx\bbl@starthyphens\@undefined % Needed if no explicit selection
6963           \AddBabelHook{babel-onchar}{beforestart}{\bbl@starthyphens}%
6964         \fi
6965         \bbl@exp{\\ \bbl@add\\ \bbl@starthyphens
6966           {\bbl@patterns@lua{\language\language}}}%
6967         \directlua{
6968           if Babel.script_blocks['\bbl@cl{sbc}'] then
6969             Babel.loc_to_scr[\the\localeid] = Babel.script_blocks['\bbl@cl{sbc}']
6970             Babel.locale_props[\the\localeid].lg = \the\@nameuse{l\language}\space
6971           end
6972         }%
6973       \fi

```

```

6974 \bbl@xin@{ fonts }{ \bbl@KVP@onchar\space}%
6975 \ifin@
6976 \bbl@ifunset{bbl@lsys@\languageName}{\bbl@provide@lsys{\languageName}}{}%
6977 \bbl@ifunset{bbl@wdir@\languageName}{\bbl@provide@dirs{\languageName}}{}%
6978 \directlua{
6979   if Babel.script_blocks['\bbl@cl{sbcP}'] then
6980     Babel.loc_to_scr[\the\localeid] =
6981       Babel.script_blocks['\bbl@cl{sbcP}']
6982   end}%
6983 \ifx\bbl@mapselect\undefined
6984 \AtBeginDocument{%
6985   \bbl@patchfont{\bbl@mapselect}}%
6986   {\selectfont}}%
6987 \def\bbl@mapselect{%
6988   \let\bbl@mapselect\relax
6989   \edef\bbl@prefontid{\fontid\font}}%
6990 \def\bbl@mapdir##1{%
6991   \begingroup
6992     \setbox\z@\hbox{% Force text mode
6993       \def\languageName{##1}%
6994       \let\bbl@ifrestoring\@firstoftwo % To avoid font warning
6995       \bbl@switchfont
6996       \ifnum\fontid\font>\z@ % A hack, for the pgf nullfont hack
6997         \directlua{
6998           Babel.locale_props[\the\csname bbl@id@##1\endcsname]%
6999             [\bbl@prefontid'] = \fontid\font\space}%
7000         \fi}%
7001     \endgroup}%
7002 \fi
7003 \bbl@exp{\bbl@add\bbl@mapselect{\bbl@mapdir{\languageName}}}%
7004 \fi
7005 \fi
7006 % == mapfont ==
7007 % For bidi texts, to switch the font based on direction. Deprecated
7008 \ifx\bbl@KVP@mapfont\@nnil\else
7009   \bbl@ifsamestring{\bbl@KVP@mapfont}{direction}}{}%
7010   {\bbl@error{unknown-mapfont}}{}{}%
7011   \bbl@ifunset{bbl@lsys@\languageName}{\bbl@provide@lsys{\languageName}}{}%
7012   \bbl@ifunset{bbl@wdir@\languageName}{\bbl@provide@dirs{\languageName}}{}%
7013   \ifx\bbl@mapselect\undefined
7014     \AtBeginDocument{%
7015       \bbl@patchfont{\bbl@mapselect}}%
7016       {\selectfont}}%
7017     \def\bbl@mapselect{%
7018       \let\bbl@mapselect\relax
7019       \edef\bbl@prefontid{\fontid\font}}%
7020     \def\bbl@mapdir##1{%
7021       {\def\languageName{##1}%
7022         \let\bbl@ifrestoring\@firstoftwo % avoid font warning
7023         \bbl@switchfont
7024         \directlua{Babel.fontmap
7025           [\the\csname bbl@wdir@##1\endcsname]%
7026           [\bbl@prefontid]=\fontid\font}}}%
7027     \fi
7028     \bbl@exp{\bbl@add\bbl@mapselect{\bbl@mapdir{\languageName}}}%
7029   \fi
7030 % == Line breaking: CJK quotes ==
7031 \ifcase\bbl@engine\or
7032   \bbl@xin@{/c}{\bbl@cl{lbrk}}}%
7033 \ifin@
7034   \bbl@ifunset{bbl@quote@\languageName}}{}%
7035   {\directlua{
7036     Babel.locale_props[\the\localeid].cjk_quotes = {}

```



```

7037         local cs = 'op'
7038         for c in string.utfvalues(%
7039             [[\csname bbl@quote@\languagename\endcsname]]) do
7040             if Babel.cjk_characters[c].c == 'qu' then
7041                 Babel.locale_props[\the\localeid].cjk_quotes[c] = cs
7042             end
7043             cs = ( cs == 'op') and 'cl' or 'op'
7044         end
7045     }}%
7046 \fi
7047 \fi
7048 % == Counters: mapdigits ==
7049 % Native digits
7050 \ifx\bbl@KVP@mapdigits\@nnil\else
7051     \bbl@ifunset{bbl@dgnat@\languagename}{}%
7052     {\bbl@activate@preotf
7053         \directlua{
7054             Babel.digits_mapped = true
7055             Babel.digits = Babel.digits or {}
7056             Babel.digits[\the\localeid] =
7057                 table.pack(string.utfvalue('\bbl@cl{dgnat}'))
7058             if not Babel.numbers then
7059                 function Babel.numbers(head)
7060                     local LOCALE = Babel.attr_locale
7061                     local GLYPH = node.id'glyph'
7062                     local inmath = false
7063                     for item in node.traverse(head) do
7064                         if not inmath and item.id == GLYPH then
7065                             local temp = node.get_attribute(item, LOCALE)
7066                             if Babel.digits[temp] then
7067                                 local chr = item.char
7068                                 if chr > 47 and chr < 58 then
7069                                     item.char = Babel.digits[temp][chr-47]
7070                                 end
7071                             end
7072                         elseif item.id == node.id'math' then
7073                             inmath = (item.subtype == 0)
7074                         end
7075                     end
7076                     return head
7077                 end
7078             end
7079         }}%
7080 \fi
7081 % == transforms ==
7082 \ifx\bbl@KVP@transforms\@nnil\else
7083     \def\bbl@elt##1##2##3{%
7084         \in@{$transforms.}{$##1}%
7085         \ifin@
7086             \def\bbl@tempa{##1}%
7087             \bbl@replace\bbl@tempa{transforms.}{}%
7088             \bbl@carg\bbl@transforms{babel\bbl@tempa}{##2}{##3}%
7089         \fi}%
7090     \bbl@exp{%
7091         \\bbl@ifblank{\bbl@cl{dgnat}}%
7092         {\let\\bbl@tempa\relax}%
7093         {\def\\bbl@tempa{%
7094             \\bbl@elt{transforms.prehyphenation}%
7095             {digits.native.1.0}{([0-9])}%
7096             \\bbl@elt{transforms.prehyphenation}%
7097             {digits.native.1.1}{string={1|string|0123456789|string|\bbl@cl{dgnat}}}}}%
7098     \ifx\bbl@tempa\relax\else
7099         \toks@{\expandafter\expandafter\expandafter}%

```

```

7100      \csname bbl@inidata@\language\endcsname}%
7101      \bbl@csarg\edef{inidata@\language}%
7102      \unexpanded\expandafter{\bbl@tempa}%
7103      \the\toks@}%
7104      \fi
7105      \csname bbl@inidata@\language\endcsname
7106      \bbl@release@transforms\relax % \relax closes the last item.
7107      \fi}

      Start tabular here:
7108 \def\localerestoredirs{%
7109   \ifcase\bbl@thetextdir
7110     \ifnum\textdirection=\z@\else\textdirection=\z@\fi
7111   \else
7112     \ifnum\textdirection=\@ne\else\textdirection=\@ne\fi
7113   \fi
7114   \ifcase\bbl@thepardir
7115     \ifnum\pardirection=\z@\else\pardirection=\z@\bodydirection=\z@\fi
7116   \else
7117     \ifnum\pardirection=\@ne\else\pardirection=\@ne\bodydirection=\@ne\fi
7118   \fi}
7119 %
7120 \IfBabelLayout{tabular}%
7121   {\chardef\bbl@tabular@mode\z@}% All RTL
7122   {\IfBabelLayout{lrtabular}%
7123     {\chardef\bbl@tabular@mode\@ne}%
7124     {\chardef\bbl@tabular@mode\z@}}}% Mixed, with LTR cols
7125 %
7126 \ifnum\bbl@bidimode>\@ne % Any lua bidi= except default=1
7127 % Redefine: vrules mess up dirs (why?).
7128 \AtBeginDocument{\def\@arstrut{\relax\copy\@arstrutbox}}%
7129 \ifcase\bbl@tabular@mode\else % 1 = Mixed
7130   \let\bbl@parabefore\relax
7131   \AddToHook{para/before}{\bbl@parabefore}
7132   \AtBeginDocument{%
7133     \bbl@replace@tabular{\hbox\bgroup}{\hbox bdir\z@\bgroup}%
7134     \ifnum\bbl@tabular@mode=\@ne
7135       \bbl@ifunset{@tabclassz}{\}%
7136       \bbl@exp{% Hide conditionals
7137         \\bbl@sreplace\\@tabclassz
7138         {\<ifcase>\\@chnum}%
7139         {\localerestoredirs\<ifcase>\\@chnum}}}%
7140       \@ifpackageloaded{colortbl}%
7141       {\bbl@sreplace@classz
7142         {\hbox\bgroup\bgroup}{\hbox\bgroup\bgroup\localerestoredirs}}%
7143       {\@ifpackageloaded{array}%
7144         {\bbl@exp{% Hide conditionals
7145           \\bbl@sreplace\\@classz
7146           {\<ifcase>\\@chnum}%
7147           {\bgroup\\localerestoredirs\<ifcase>\\@chnum}%
7148           \\bbl@sreplace\\@classz
7149           {\do@row@strut\<fi>}{\do@row@strut\<fi>\egroup}}}%
7150         {}}}%
7151     \fi}%
7152   \fi

```

Very likely the \output routine must be patched in a quite general way to make sure the \bodydir is set to \pagedir. Note outside \output they can be different (and often are). For the moment, two *ad hoc* changes.

```

7153 \AtBeginDocument{%
7154   \@ifpackageloaded{multicol}%
7155     {\toks@\expandafter{\multi@column@out}%
7156       \edef\multi@column@out{\bodydir\pagedir\the\toks@}}%
7157   }%

```

```

7158 \@ifpackageloaded{paracol}%
7159 {\edef\pcol@output{%
7160 \bodydir\pagedir\unexpanded\expandafter{\pcol@output}}}%
7161 {}}%
7162 \fi

```

Finish here if there in no layout.

```

7163 \ifx\bbl@opt@layout\@nnil\endinput\fi

```

\hangindent does not honour direction changes by default, so we need to redefine \@hangfrom.

```

7164 \chardef\bbl@trace@vboxdir\z@
7165 \ifnum\bbl@bidimode>\z@ % Any bidi=
7166 \IfBabelLayout{nopars}{}
7167 {\edef\bbl@opt@layout{\bbl@opt@layout.pars.}}%
7168 \IfBabelLayout{pars}
7169 {\chardef\bbl@trace@vboxdir\@ne
7170 \def\@hangfrom#1{%
7171 \setbox\@tempboxa\hbox{#1}}%
7172 \hangindent\wd\@tempboxa
7173 \ifnum\bbl@vbox@bodydir=\pardirection\else
7174 \shapemode\@ne
7175 \fi
7176 \noindent\box\@tempboxa}}
7177 {}
7178 \fi

```

We need to patch lists in documents with both LTR and RTL paragraphs. See issue #395 in GitHub. There was a partial solution, but a better one has been devised by Udi Fogiel (in 26.4).

```

7179 \IfBabelLayout{lists}
7180 {\chardef\bbl@trace@vboxdir\@ne
7181 \let\bbl@OL@list\list
7182 \bbl@sreplace\list{\parshape}{\bbl@listparshape}%
7183 \let\bbl@NL@list\list
7184 \def\bbl@listparshape#1#2#3{%
7185 \parshape #1 #2 #3 %
7186 \ifnum\bbl@vbox@bodydir=\pardirection\else
7187 \shapemode\tw@
7188 \fi}}
7189 {}
7190 %
7191 \ifcase\bbl@trace@vboxdir\else
7192 \AtBeginDocument{\chardef\bbl@vbox@bodydir\pagedirection}%
7193 \def\bbl@vbox@lists{\chardef\bbl@vbox@bodydir\bodydirection}%
7194 \let\bbl@bidi@vbox\everyvbox
7195 \@nameuse{newtoks}\everyvbox % \outer in Plain
7196 \everyvbox\expandafter{\the\bbl@bidi@vbox}%
7197 \bbl@bidi@vbox{\bbl@vbox@lists\the\everyvbox}%
7198 \fi
7199 %
7200 \IfBabelLayout{graphics}
7201 {\let\bbl@pictresetdir\relax
7202 \def\bbl@pictsetdir#1{%
7203 \ifcase\bbl@thetextdir
7204 \let\bbl@pictresetdir\relax
7205 \else
7206 \ifcase#1\bodydir TLT % Remember this sets the inner boxes
7207 \or\textdir TLT
7208 \else\bodydir TLT \textdir TLT
7209 \fi
7210 % \(\text|par)dir required in pgf:
7211 \def\bbl@pictresetdir{\bodydir TRT\pardir TRT\textdir TRT\relax}%
7212 \fi}%
7213 \AddToHook{env/picture/begin}{\bbl@pictsetdir\tw@}%
7214 \directlua{

```

```

7215 Babel.get_picture_dir = true
7216 Babel.picture_has_bidi = 0
7217 %
7218 function Babel.picture_dir (head)
7219   if not Babel.get_picture_dir then return head end
7220   if Babel.hlist_has_bidi(head) then
7221     Babel.picture_has_bidi = 1
7222   end
7223   return head
7224 end
7225 luatexbase.add_to_callback("hpack_filter", Babel.picture_dir,
7226   "Babel.picture_dir")
7227 }%
7228 \AddToHook{package/graphics/after}{%
7229   \bbl@exp{%
7230     \\bbl@sreplace\\rotatebox{\hbox{{\string#2}}}
7231     {\hbox bdir\z@{\hbox bdir<ifcase>\bbl@thetextdir\z@\<else>\@ne<fi>{\string#2}}}}}%
7232 \AddToHook{package/graphicsx/after}{%
7233   \bbl@exp{%
7234     \\bbl@sreplace\\Grot@box@std{\hbox{{\string#2}}}
7235     {\hbox bdir\z@{\hbox bdir<ifcase>\bbl@thetextdir\z@\<else>\@ne<fi>{\string#2}}}}}%
7236     \\bbl@sreplace\\Grot@box@kv{\hbox{\string#3}}
7237     {\hbox bdir\z@{\hbox bdir<ifcase>\bbl@thetextdir\z@\<else>\@ne<fi>{\string#3}}}}}%
7238     \\bbl@sreplace\\Grot@box{\hbox{\hbox bdir\z@}}}%
7239 \AtBeginDocument{%
7240   \long\def\put(#1,#2)#3{%
7241     \@killglue
7242     % Try:
7243     \ifx\bbl@pictresetdir\relax
7244       \def\bbl@tempc{0}%
7245     \else
7246       \directlua{
7247         Babel.get_picture_dir = true
7248         Babel.picture_has_bidi = 0
7249       }%
7250       \setbox\z@\hb@xt@{z@}%
7251       \@defaultunitsset\@tempdimc{#1}\unitlength
7252       \kern\@tempdimc
7253       #3\hss}%
7254       \edef\bbl@tempc{\directlua{tex.print(Babel.picture_has_bidi)}}}%
7255     \fi
7256     % Do:
7257     \@defaultunitsset\@tempdimc{#2}\unitlength
7258     \raise\@tempdimc\hb@xt@{z@}%
7259     \@defaultunitsset\@tempdimc{#1}\unitlength
7260     \kern\@tempdimc
7261     {\ifnum\bbl@tempc>z@\bbl@pictresetdir\fi#3}\hss}%
7262     \ignorespaces}%
7263   \MakeRobust\put}%
7264 \AtBeginDocument
7265 {\AddToHook{cmd/diagbox@pict/before}{\let\bbl@pictsetdir\@gobble}%
7266   \ifx\pgfpicture\@undefined\else
7267     \AddToHook{env/pgfpicture/begin}{\bbl@pictsetdir\@ne}%
7268     \bbl@add\pgfinterruptpicture{\bbl@pictresetdir}%
7269     \bbl@add\pgfsys@beginpicture{\bbl@pictsetdir\z@}%
7270   \fi
7271   \ifx\tikzpicture\@undefined\else
7272     \AddToHook{env/tikzpicture/begin}{\bbl@pictsetdir\tw}%
7273     \bbl@add\tikz@atbegin@node{\bbl@pictresetdir}%
7274     \bbl@sreplace\tikz{\begingroup}\begingroup\bbl@pictsetdir\tw}%
7275     \bbl@sreplace\tikzpicture{\begingroup}\begingroup\bbl@pictsetdir\tw}%
7276   \fi
7277 }}

```

```
7278 {}
```

Implicitly reverses sectioning labels in bidi=basic-r, because the full stop is not in contact with L numbers any more. I think there must be a better way. Assumes bidi=basic, but there are some additional readjustments for bidi=default.

```
7279 \IfBabelLayout{counters*}%
7280 {\bbl@add\bbl@opt@layout{.counters.}%
7281 \directlua{
7282   luatexbase.add_to_callback("process_output_buffer",
7283     Babel.discard_sublr , "Babel.discard_sublr") }%
7284 }{}
7285 \IfBabelLayout{counters}%
7286 {\let\bbl@latinarabic=\@arabic
7287 \let\bbl@OL@arabic\@arabic
7288 \def\@arabic#1{\babelsublr{\bbl@latinarabic#1}}%
7289 \@ifpackagewith{babel}{bidi=default}%
7290 {\let\bbl@asciroman=\@roman
7291 \let\bbl@OL@roman\@roman
7292 \def\@roman#1{\babelsublr{\ensureascii{\bbl@asciroman#1}}}%
7293 \let\bbl@asciiRoman=\@Roman
7294 \let\bbl@OL@roman\@Roman
7295 \def\@Roman#1{\babelsublr{\ensureascii{\bbl@asciiRoman#1}}}%
7296 \let\bbl@OL@labelenumii\labelenumii
7297 \def\labelenumii{}\theenumii}%
7298 \let\bbl@OL@p@enumiii\p@enumiii
7299 \def\p@enumiii{\p@enumii}\theenumii{}}{}{}}
```

Some $\TeX$ macros use internally the math mode for text formatting. They have very little in common and are grouped here, as a single option.

```
7300 \IfBabelLayout{extras}%
7301 {\bbl@ncarg\let\bbl@OL@underline{underline }%
7302 \bbl@carg\bbl@sreplace{underline }{\hbox}%
7303 {\def\bbl@insidemath{0}\hbox bdir@ifcase\bbl@thetextdir\z@else\@ne\fi}%
7304 \let\bbl@OL@LaTeXe\LaTeXe
7305 \DeclareRobustCommand{\LaTeXe}{\mbox{\math
7306   \if b\expandafter\@car\@f@series\@nil\boldmath\fi
7307   \babelsublr{\LaTeX\kern.15em2_{\textstyle\varepsilon}}}}}
7308 {}
7309 (/luatex)
```

10.13 Lua: transforms

After declaring the table containing the patterns with their replacements, we define some auxiliary functions: `str_to_nodes` converts the string returned by a function to a node list, taking the node at base as a model (font, language, etc.); `fetch_word` fetches a series of glyphs and discretionaries, which pattern is matched against (if there is a match, it is called again before trying other patterns, and this is very likely the main bottleneck).

`post_hyphenate_replace` is the callback applied after `lang.hyphenate`. This means the automatic hyphenation points are known. As empty captures return a byte position (as explained in the `luatex manual`), we must convert it to a utf8 position. With `first`, the last byte can be the leading byte in a utf8 sequence, so we just remove it and add 1 to the resulting length. With `last` we must take into account the capture position points to the next character. Here `word_head` points to the starting node of the text to be matched.

```
7310 (*transforms)
7311 Babel.linebreaking.replacements = {}
7312 Babel.linebreaking.replacements[0] = {} -- pre
7313 Babel.linebreaking.replacements[1] = {} -- post
7314
7315 function Babel.tovalue(v)
7316   if type(v) == 'table' then
7317     return Babel.locale_props[v[1]].vars[v[2]] or v[3]
7318   else
7319     return v
```

```

7320 end
7321 end
7322
7323 Babel.attr_hboxed = luatexbase.registernumber'bbl@attr@hboxed'
7324
7325 function Babel.set_hboxed(head, gc)
7326   for item in node.traverse(head) do
7327     node.set_attribute(item, Babel.attr_hboxed, 1)
7328   end
7329   return head
7330 end
7331
7332 Babel.fetch_subtext = {}
7333
7334 Babel.ignore_pre_char = function(node)
7335   return (node.lang == Babel.nohyphenation)
7336 end
7337
7338 Babel.show_transforms = false
7339
7340 -- Merging both functions doesn't seem feasible, because there are too
7341 -- many differences.
7342 Babel.fetch_subtext[0] = function(head)
7343   local word_string = ''
7344   local word_nodes = {}
7345   local lang
7346   local item = head
7347   local inmath = false
7348
7349   while item do
7350
7351     if item.id == 11 then
7352       inmath = (item.subtype == 0)
7353     end
7354
7355     if inmath then
7356       -- pass
7357
7358     elseif item.id == 29 then
7359       local locale = node.get_attribute(item, Babel.attr_locale)
7360
7361       if lang == locale or lang == nil then
7362         lang = lang or locale
7363         if Babel.ignore_pre_char(item) then
7364           word_string = word_string .. Babel.us_char
7365         else
7366           if node.has_attribute(item, Babel.attr_hboxed) then
7367             word_string = word_string .. Babel.us_char
7368           else
7369             word_string = word_string .. unicode.utf8.char(item.char)
7370           end
7371         end
7372         word_nodes[#word_nodes+1] = item
7373       else
7374         break
7375       end
7376
7377     elseif item.id == 12 and item.subtype == 13 then
7378       if node.has_attribute(item, Babel.attr_hboxed) then
7379         word_string = word_string .. Babel.us_char
7380       else
7381         word_string = word_string .. ' '
7382       end

```

```

7383     word_nodes[#word_nodes+1] = item
7384
7385     -- Ignore leading unrecognized nodes, too.
7386     elseif word_string ~= '' then
7387         word_string = word_string .. Babel.us_char
7388         word_nodes[#word_nodes+1] = item -- Will be ignored
7389     end
7390
7391     item = item.next
7392 end
7393
7394 -- Here and above we remove some trailing chars but not the
7395 -- corresponding nodes. But they aren't accessed.
7396 if word_string:sub(-1) == ' ' then
7397     word_string = word_string:sub(1,-2)
7398 end
7399 if Babel.show_transforms then texio.write_nl(word_string) end
7400 word_string = unicode.utf8.gsub(word_string, Babel.us_char .. '+$', '')
7401 return word_string, word_nodes, item, lang
7402 end
7403
7404 Babel.fetch_subtext[1] = function(head)
7405     local word_string = ''
7406     local word_nodes = {}
7407     local lang
7408     local item = head
7409     local inmath = false
7410
7411     while item do
7412
7413         if item.id == 11 then
7414             inmath = (item.subtype == 0)
7415         end
7416
7417         if inmath then
7418             -- pass
7419
7420         elseif item.id == 29 then
7421             if item.lang == lang or lang == nil then
7422                 lang = lang or item.lang
7423                 if node.has_attribute(item, Babel.attr_hboxed) then
7424                     word_string = word_string .. Babel.us_char
7425                 elseif (item.char == 124) or (item.char == 61) then -- not =, not |
7426                     word_string = word_string .. Babel.us_char
7427                 else
7428                     word_string = word_string .. unicode.utf8.char(item.char)
7429                 end
7430                 word_nodes[#word_nodes+1] = item
7431             else
7432                 break
7433             end
7434
7435         elseif item.id == 7 and item.subtype == 2 then
7436             if node.has_attribute(item, Babel.attr_hboxed) then
7437                 word_string = word_string .. Babel.us_char
7438             else
7439                 word_string = word_string .. '='
7440             end
7441             word_nodes[#word_nodes+1] = item
7442
7443         elseif item.id == 7 and item.subtype == 3 then
7444             if node.has_attribute(item, Babel.attr_hboxed) then
7445                 word_string = word_string .. Babel.us_char

```

```

7446     else
7447         word_string = word_string .. '|'
7448     end
7449     word_nodes[#word_nodes+1] = item
7450
7451     -- (1) Go to next word if nothing was found, and (2) implicitly
7452     -- remove leading USs.
7453     elseif word_string == '' then
7454         -- pass
7455
7456     -- This is the responsible for splitting by words.
7457     elseif (item.id == 12 and item.subtype == 13) then
7458         break
7459
7460     else
7461         word_string = word_string .. Babel.us_char
7462         word_nodes[#word_nodes+1] = item -- Will be ignored
7463     end
7464
7465     item = item.next
7466 end
7467 if Babel.show_transforms then texio.write_nl(word_string) end
7468 word_string = unicode.utf8.gsub(word_string, Babel.us_char .. '+$', '')
7469 return word_string, word_nodes, item, lang
7470 end
7471
7472 function Babel.pre_hyphenate_replace(head)
7473     Babel.hyphenate_replace(head, 0)
7474 end
7475
7476 function Babel.post_hyphenate_replace(head)
7477     Babel.hyphenate_replace(head, 1)
7478 end
7479
7480 Babel.us_char = string.char(31)
7481
7482 function Babel.hyphenate_replace(head, mode)
7483     local u = unicode.utf8
7484     local lbkr = Babel.linebreaking.replacements[mode]
7485     local tovalue = Babel.tovalue
7486
7487     local word_head = head
7488
7489     if Babel.show_transforms then
7490         texio.write_nl('\n==== Showing ' .. (mode == 0 and 'pre' or 'post') .. 'hyphenation ====')
7491     end
7492
7493     while true do -- for each subtext block
7494
7495         local w, w_nodes, nw, lang = Babel.fetch_subtext[mode](word_head)
7496
7497         if Babel.debug then
7498             print()
7499             print((mode == 0) and '@@@<' or '@@@>', w)
7500         end
7501
7502         if nw == nil and w == '' then break end
7503
7504         if not lang then goto next end
7505         if not lbkr[lang] then goto next end
7506
7507         -- For each saved (pre|post)hyphenation. TODO. Reconsider how
7508         -- loops are nested.

```



```

7509     for k=1, #lbr[lang] do
7510         local p = lbr[lang][k].pattern
7511         local r = lbr[lang][k].replace
7512         local attr = lbr[lang][k].attr or -1
7513
7514         if Babel.debug then
7515             print('*****', p, mode)
7516         end
7517
7518         -- This variable is set in some cases below to the first *byte*
7519         -- after the match, either as found by u.match (faster) or the
7520         -- computed position based on sc if w has changed.
7521         local last_match = 0
7522         local step = 0
7523
7524         -- For every match.
7525         while true do
7526             if Babel.debug then
7527                 print('====')
7528             end
7529             local new -- used when inserting and removing nodes
7530             local dummy_node -- used by after
7531
7532             local matches = { u.match(w, p, last_match) }
7533
7534             if #matches < 2 then break end
7535
7536             -- Get and remove empty captures (with ())'s, which return a
7537             -- number with the position), and keep actual captures
7538             -- (from (...)), if any, in matches.
7539             local first = table.remove(matches, 1)
7540             local last = table.remove(matches, #matches)
7541             -- Non re-fetched substrings may contain \31, which separates
7542             -- subsubstrings.
7543             if string.find(w:sub(first, last-1), Babel.us_char) then break end
7544
7545             local save_last = last -- with A()BC()D, points to D
7546
7547             -- Fix offsets, from bytes to unicode. Explained above.
7548             first = u.len(w:sub(1, first-1)) + 1
7549             last = u.len(w:sub(1, last-1)) -- now last points to C
7550
7551             -- This loop stores in a small table the nodes
7552             -- corresponding to the pattern. Used by 'data' to provide a
7553             -- predictable behavior with 'insert' (w_nodes is modified on
7554             -- the fly), and also access to 'remove'd nodes.
7555             local sc = first-1 -- Used below, too
7556             local data_nodes = {}
7557
7558             local enabled = true
7559             for q = 1, last-first+1 do
7560                 data_nodes[q] = w_nodes[sc+q]
7561                 if enabled
7562                     and attr > -1
7563                     and not node.has_attribute(data_nodes[q], attr)
7564                 then
7565                     enabled = false
7566                 end
7567             end
7568
7569             -- This loop traverses the matched substring and takes the
7570             -- corresponding action stored in the replacement list.
7571             -- sc = the position in substr nodes / string

```

```

7572         -- rc = the replacement table index
7573         local rc = 0
7574
7575         ----- TODO0. dummy_node?
7576         while rc < last-first+1 or dummy_node do -- for each replacement
7577             if Babel.debug then
7578                 print('.....', rc + 1)
7579             end
7580             sc = sc + 1
7581             rc = rc + 1
7582
7583             if Babel.debug then
7584                 Babel.debug_hyph(w, w_nodes, sc, first, last, last_match)
7585                 local ss = ''
7586                 for itt in node.traverse(head) do
7587                     if itt.id == 29 then
7588                         ss = ss .. unicode.utf8.char(itt.char)
7589                     else
7590                         ss = ss .. '{' .. itt.id .. '}'
7591                     end
7592                 end
7593                 print('*****', ss)
7594
7595             end
7596
7597             local crep = r[rc]
7598             local item = w_nodes[sc]
7599             local item_base = item
7600             local placeholder = Babel.us_char
7601             local d
7602
7603             if crep and crep.data then
7604                 item_base = data_nodes[crep.data]
7605             end
7606
7607             if crep then
7608                 step = crep.step or step
7609             end
7610
7611             if crep and crep.after then
7612                 crep.insert = true
7613                 if dummy_node then
7614                     item = dummy_node
7615                 else -- TODO0. if there is a node after?
7616                     d = node.copy(item_base)
7617                     head, item = node.insert_after(head, item, d)
7618                     dummy_node = item
7619                 end
7620             end
7621
7622             if crep and not crep.after and dummy_node then
7623                 node.remove(head, dummy_node)
7624                 dummy_node = nil
7625             end
7626
7627             if not enabled then
7628                 last_match = save_last
7629                 goto next
7630
7631             elseif crep and next(crep) == nil then -- = {}
7632                 if step == 0 then
7633                     last_match = save_last -- Optimization
7634                 else

```

```

7635         last_match = utf8.offset(w, sc+step)
7636     end
7637     goto next
7638
7639 elseif crep == nil or crep.remove then
7640     node.remove(head, item)
7641     table.remove(w_nodes, sc)
7642     w = u.sub(w, 1, sc-1) .. u.sub(w, sc+1)
7643     sc = sc - 1 -- Nothing has been inserted.
7644     last_match = utf8.offset(w, sc+1+step)
7645     goto next
7646
7647 elseif crep and crep.kashida then
7648     node.set_attribute(item,
7649         Babel.attr_kashida,
7650         crep.kashida)
7651     last_match = utf8.offset(w, sc+1+step)
7652     goto next
7653
7654 elseif crep and crep.string then
7655     local str = crep.string(matches)
7656     if str == '' then -- Gather with nil
7657         node.remove(head, item)
7658         table.remove(w_nodes, sc)
7659         w = u.sub(w, 1, sc-1) .. u.sub(w, sc+1)
7660         sc = sc - 1 -- Nothing has been inserted.
7661     else
7662         local loop_first = true
7663         for s in string.utfvalues(str) do
7664             d = node.copy(item_base)
7665             d.char = s
7666             if loop_first then
7667                 loop_first = false
7668                 head, new = node.insert_before(head, item, d)
7669                 if sc == 1 then
7670                     word_head = head
7671                 end
7672                 w_nodes[sc] = d
7673                 w = u.sub(w, 1, sc-1) .. u.char(s) .. u.sub(w, sc+1)
7674             else
7675                 sc = sc + 1
7676                 head, new = node.insert_before(head, item, d)
7677                 table.insert(w_nodes, sc, new)
7678                 w = u.sub(w, 1, sc-1) .. u.char(s) .. u.sub(w, sc)
7679             end
7680             if Babel.debug then
7681                 print('.....', 'str')
7682                 Babel.debug_hyph(w, w_nodes, sc, first, last, last_match)
7683             end
7684         end -- for
7685         node.remove(head, item)
7686     end -- if ''
7687     last_match = utf8.offset(w, sc+1+step)
7688     goto next
7689
7690 elseif mode == 1 and crep and (crep.pre or crep.no or crep.post) then
7691     d = node.new(7, 3) -- (disc, regular)
7692     d.pre = Babel.str_to_nodes(crep.pre, matches, item_base)
7693     d.post = Babel.str_to_nodes(crep.post, matches, item_base)
7694     d.replace = Babel.str_to_nodes(crep.no, matches, item_base)
7695     d.attr = item_base.attr
7696     if crep.pre == nil then -- TeXbook p96
7697         d.penalty = tovalue(crep.penalty) or tex.hyphenpenalty

```

```

7698         else
7699             d.penalty = tovalue(crep.penalty) or tex.exhyphenpenalty
7700         end
7701         placeholder = '|'
7702         head, new = node.insert_before(head, item, d)
7703
7704     elseif mode == 0 and crep and (crep.pre or crep.no or crep.post) then
7705         -- ERROR
7706
7707     elseif crep and crep.penalty then
7708         d = node.new(14, 0) -- (penalty, userpenalty)
7709         d.attr = item_base.attr
7710         d.penalty = tovalue(crep.penalty)
7711         head, new = node.insert_before(head, item, d)
7712
7713     elseif crep and crep.space then
7714         -- 655360 = 10 pt = 10 * 65536 sp
7715         d = node.new(12, 13) -- (glue, spaceskip)
7716         local quad = font.getfont(item_base.font).size or 655360
7717         node.setglue(d, tovalue(crep.space[1]) * quad,
7718                        tovalue(crep.space[2]) * quad,
7719                        tovalue(crep.space[3]) * quad)
7720         if mode == 0 then
7721             placeholder = ' '
7722         end
7723         head, new = node.insert_before(head, item, d)
7724
7725     elseif crep and crep.norule then
7726         -- 655360 = 10 pt = 10 * 65536 sp
7727         d = node.new(2, 3) -- (rule, empty) = \no*rule
7728         local quad = font.getfont(item_base.font).size or 655360
7729         d.width = tovalue(crep.norule[1]) * quad
7730         d.height = tovalue(crep.norule[2]) * quad
7731         d.depth = tovalue(crep.norule[3]) * quad
7732         head, new = node.insert_before(head, item, d)
7733
7734     elseif crep and crep.spacefactor then
7735         d = node.new(12, 13) -- (glue, spaceskip)
7736         local base_font = font.getfont(item_base.font)
7737         node.setglue(d,
7738                     tovalue(crep.spacefactor[1]) * base_font.parameters['space'],
7739                     tovalue(crep.spacefactor[2]) * base_font.parameters['space_stretch'],
7740                     tovalue(crep.spacefactor[3]) * base_font.parameters['space_shrink'])
7741         if mode == 0 then
7742             placeholder = ' '
7743         end
7744         head, new = node.insert_before(head, item, d)
7745
7746     elseif mode == 0 and crep and crep.space then
7747         -- ERROR
7748
7749     elseif crep and crep.kern then
7750         d = node.new(13, 1) -- (kern, user)
7751         local quad = font.getfont(item_base.font).size or 655360
7752         d.attr = item_base.attr
7753         d.kern = tovalue(crep.kern) * quad
7754         head, new = node.insert_before(head, item, d)
7755
7756     elseif crep and crep.node then
7757         d = node.new(crep.node[1], crep.node[2])
7758         d.attr = item_base.attr
7759         head, new = node.insert_before(head, item, d)
7760

```

```

7761         end -- i.e., replacement cases
7762
7763         -- Shared by disc, space(factor), kern, node and penalty.
7764         if sc == 1 then
7765             word_head = head
7766         end
7767         if crep.insert then
7768             w = u.sub(w, 1, sc-1) .. placeholder .. u.sub(w, sc)
7769             table.insert(w_nodes, sc, new)
7770             last = last + 1
7771         else
7772             w_nodes[sc] = d
7773             node.remove(head, item)
7774             w = u.sub(w, 1, sc-1) .. placeholder .. u.sub(w, sc+1)
7775         end
7776
7777         last_match = utf8.offset(w, sc+1+step)
7778
7779         ::next::
7780
7781     end -- for each replacement
7782
7783     if Babel.show_transforms then texio.write_nl('> ' .. w) end
7784     if Babel.debug then
7785         print('.....', '/')
7786         Babel.debug_hyph(w, w_nodes, sc, first, last, last_match)
7787     end
7788
7789     if dummy_node then
7790         node.remove(head, dummy_node)
7791         dummy_node = nil
7792     end
7793
7794     end -- for match
7795
7796 end -- for patterns
7797
7798 ::next::
7799 word_head = nw
7800 end -- for substring
7801
7802 if Babel.show_transforms then texio.write_nl(string.rep('-', 32) .. '\n') end
7803 return head
7804 end
7805
7806 -- This table stores capture maps, numbered consecutively
7807 Babel.capture_maps = {}
7808
7809 function Babel.esc_hex_to_char(h)
7810     if tex.getcatcode(tonumber(h, 16)) ~= 11 and
7811         tex.getcatcode(tonumber(h, 16)) ~= 12 then
7812         return string.format([[\\Uchar"%X ]], tonumber(h,16))
7813     else
7814         return unicode.utf8.char(tonumber(h, 16))
7815     end
7816 end
7817
7818 -- The following functions belong to the next macro
7819 function Babel.capture_func(key, cap)
7820     local ret = "[[" .. cap:gsub('{{([0-9])}}', "[ ]..m[%1]..[[") .. "]"
7821     local cnt
7822     local u = unicode.utf8
7823     ret = u.gsub(ret, '{(%x%x%x%x+)}', '\\x01%1\\x04')

```

```

7824 ret, cnt = ret:gsub('{{([0-9])|([^\]|+)|(.-)}}', Babel.capture_func_map)
7825 ret = u.gsub(ret, '\x01(%x%x%x%x+)\x04', Babel.esc_hex_to_char)
7826 ret = ret:gsub("%[%[%]%.%.%", '')
7827 ret = ret:gsub("%.%.%[%[%]%.%.%", '')
7828 return key .. [[=function(m) return ]] .. ret .. [[ end]]
7829 end
7830
7831 function Babel.capt_map(from, mapno)
7832 return Babel.capture_maps[mapno][from] or from
7833 end
7834
7835 -- Handle the {n|abc|ABC} syntax in captures
7836 function Babel.capture_func_map(capno, from, to)
7837 local u = unicode.utf8
7838 from = u.gsub(from, '\x01(%x%x%x%x+)\x04',
7839 function (n)
7840 return u.char(tonumber(n, 16))
7841 end)
7842 to = u.gsub(to, '\x01(%x%x%x%x+)\x04',
7843 function (n)
7844 return u.char(tonumber(n, 16))
7845 end)
7846 local froms = {}
7847 for s in string.utfcharacters(from) do
7848 table.insert(froms, s)
7849 end
7850 local cnt = 1
7851 table.insert(Babel.capture_maps, {})
7852 local mlen = table.getn(Babel.capture_maps)
7853 for s in string.utfcharacters(to) do
7854 Babel.capture_maps[mlen][froms[cnt]] = s
7855 cnt = cnt + 1
7856 end
7857 return "]"..Babel.capt_map(m[" .. capno .. "], " ..
7858 (mlen) .. " ).." .. "[["
7859 end
7860
7861 -- Create/Extend reversed sorted list of kashida weights:
7862 function Babel.capture_kashida(key, wt)
7863 wt = tonumber(wt)
7864 if Babel.kashida_wts then
7865 for p, q in ipairs(Babel.kashida_wts) do
7866 if wt == q then
7867 break
7868 elseif wt > q then
7869 table.insert(Babel.kashida_wts, p, wt)
7870 break
7871 elseif table.getn(Babel.kashida_wts) == p then
7872 table.insert(Babel.kashida_wts, wt)
7873 end
7874 end
7875 else
7876 Babel.kashida_wts = { wt }
7877 end
7878 return 'kashida = ' .. wt
7879 end
7880
7881 function Babel.capture_node(id, subtype)
7882 local sbt = 0
7883 for k, v in pairs(node.subtypes(id)) do
7884 if v == subtype then sbt = k end
7885 end
7886 return 'node = { ' .. node.id(id) .. ', ' .. sbt .. ' }'

```

```

7887 end
7888
7889 -- Experimental: applies prehyphenation transforms to a string (letters
7890 -- and spaces).
7891 function Babel.string_prehyphenation(str, locale)
7892   local n, head, last, res
7893   head = node.new(8, 0) -- dummy (hack just to start)
7894   last = head
7895   for s in string.utfvalues(str) do
7896     if s == 20 then
7897       n = node.new(12, 0)
7898     else
7899       n = node.new(29, 0)
7900       n.char = s
7901     end
7902     node.set_attribute(n, Babel.attr_locale, locale)
7903     last.next = n
7904     last = n
7905   end
7906   head = Babel.hyphenate_replace(head, 0)
7907   res = ''
7908   for n in node.traverse(head) do
7909     if n.id == 12 then
7910       res = res .. ' '
7911     elseif n.id == 29 then
7912       res = res .. unicode.utf8.char(n.char)
7913     end
7914   end
7915   tex.print(res)
7916 end
7917 (/transforms)

```

10.14 Lua: Auto bidi with basic and basic-r

The file `babel-data-bidi.lua` currently only contains data. It is a large and boring file and it is not shown here (see the generated file), but here is a sample:

```

% [0x25]={d='et'},
% [0x26]={d='on'},
% [0x27]={d='on'},
% [0x28]={d='on', m=0x29},
% [0x29]={d='on', m=0x28},
% [0x2A]={d='on'},
% [0x2B]={d='es'},
% [0x2C]={d='cs'},
%

```

For the meaning of these codes, see the Unicode standard.

Now the `basic-r` bidi mode. One of the aims is to implement a fast and simple bidi algorithm, with a single loop. I managed to do it for R texts, with a second smaller loop for a special case. The code is still somewhat chaotic, but its behavior is essentially correct. I cannot resist copying the following text from Emacs `bidi.c` (which also attempts to implement the bidi algorithm with a single loop):

Arrrrgh!! The UAX#9 algorithm is too deeply entrenched in the assumption of batch-style processing [...]. May the fleas of a thousand camels infest the armpits of those who design supposedly general-purpose algorithms by looking at their own implementations, and fail to consider other possible implementations!

Well, it took me some time to guess what the batch rules in UAX#9 actually mean (in other word, *what* they do and *why*, and not only *how*), but I think (or I hope) I've managed to understand them.

In some sense, there are two bidi modes, one for numbers, and the other for text. Furthermore, setting just the direction in R text is not enough, because there are actually *two* R modes (set explicitly in Unicode with RLM and ALM). In `babel` the `dir` is set by a higher protocol based on the language/script, which in turn sets the correct `dir` (`<l>`, `<r>` or `<al>`).

From UAX#9: “Where available, markup should be used instead of the explicit formatting characters”. So, this simple version just ignores formatting characters. Actually, most of that annex is devoted to how to handle them.

BD14-BD16 are not implemented. Unicode (and the W3C) are making a great effort to deal with some special problematic cases in “streamed” plain text. I don’t think this is the way to go – particular issues should be fixed by a high level interface taking into account the needs of the document. And here is where luatex excels, because everything related to bidi writing is under our control.

```

7918 (*basic-r)
7919 Babel.bidi_enabled = true
7920
7921 require('babel-data-bidi.lua')
7922
7923 local characters = Babel.characters
7924 local ranges = Babel.ranges
7925
7926 local DIR = node.id("dir")
7927
7928 local function dir_mark(head, from, to, outer)
7929   dir = (outer == 'r') and 'TLT' or 'TRT' -- i.e., reverse
7930   local d = node.new(DIR)
7931   d.dir = '+' .. dir
7932   node.insert_before(head, from, d)
7933   d = node.new(DIR)
7934   d.dir = '-' .. dir
7935   node.insert_after(head, to, d)
7936 end
7937
7938 function Babel.bidi(head, ispar)
7939   local first_n, last_n          -- first and last char with nums
7940   local last_es                 -- an auxiliary 'last' used with nums
7941   local first_d, last_d         -- first and last char in L/R block
7942   local dir, dir_real

```

Next also depends on script/lang (<al>/<r>). To be set by babel.tex.pardir is dangerous, could be (re)set but it should be changed only in vmode. There are two strong’s – strong = l/al/r and strong_lr = l/r (there must be a better way):

```

7943   local strong = ('TRT' == tex.pardir) and 'r' or 'l'
7944   local strong_lr = (strong == 'l') and 'l' or 'r'
7945   local outer = strong
7946
7947   local new_dir = false
7948   local first_dir = false
7949   local inmath = false
7950
7951   local last_lr
7952
7953   local type_n = ''
7954
7955   for item in node.traverse(head) do
7956
7957     -- three cases: glyph, dir, otherwise
7958     if item.id == node.id'glyph'
7959       or (item.id == 7 and item.subtype == 2) then
7960
7961       local itemchar
7962       if item.id == 7 and item.subtype == 2 then
7963         itemchar = item.replace.char
7964       else
7965         itemchar = item.char
7966       end
7967       local chardata = characters[itemchar]
7968       dir = chardata and chardata.d or nil
7969       if not dir then

```



```

7970     for nn, et in ipairs(ranges) do
7971         if itemchar < et[1] then
7972             break
7973         elseif itemchar <= et[2] then
7974             dir = et[3]
7975             break
7976         end
7977     end
7978 end
7979 dir = dir or 'l'
7980 if inmath then dir = ('TRT' == tex.mathdir) and 'r' or 'l' end

```

Next is based on the assumption babel sets the language *and* switches the script with its dir. We treat a language block as a separate Unicode sequence. The following piece of code is executed at the first glyph after a 'dir' node. We don't know the current language until then. This is not exactly true, as the math mode may insert explicit dirs in the node list, so, for the moment there is a hack by brute force (just above).

```

7981     if new_dir then
7982         attr_dir = 0
7983         for at in node.traverse(item.attr) do
7984             if at.number == Babel.attr_dir then
7985                 attr_dir = at.value & 0x3
7986             end
7987         end
7988         if attr_dir == 1 then
7989             strong = 'r'
7990         elseif attr_dir == 2 then
7991             strong = 'al'
7992         else
7993             strong = 'l'
7994         end
7995         strong_lr = (strong == 'l') and 'l' or 'r'
7996         outer = strong_lr
7997         new_dir = false
7998     end
7999
8000     if dir == 'nsm' then dir = strong end -- W1

```

Numbers. The dual <al>/<r> system for R is somewhat cumbersome.

```

8001     dir_real = dir -- We need dir_real to set strong below
8002     if dir == 'al' then dir = 'r' end -- W3

```

By W2, there are no <en> <et> <es> if strong == <al>, only <an>. Therefore, there are not <et en> nor <en et>, W5 can be ignored, and W6 applied:

```

8003     if strong == 'al' then
8004         if dir == 'en' then dir = 'an' end -- W2
8005         if dir == 'et' or dir == 'es' then dir = 'on' end -- W6
8006         strong_lr = 'r' -- W3
8007     end

```

Once finished the basic setup for glyphs, consider the two other cases: dir node and the rest.

```

8008     elseif item.id == node.id'dir' and not inmath then
8009         new_dir = true
8010         dir = nil
8011     elseif item.id == node.id'math' then
8012         inmath = (item.subtype == 0)
8013     else
8014         dir = nil -- Not a char
8015     end

```

Numbers in R mode. A sequence of <en>, <et>, <an>, <es> and <cs> is typeset (with some rules) in L mode. We store the starting and ending points, and only when anything different is found (including nil, i.e., a non-char), the textdir is set. This means you cannot insert, say, a whatsit, but this is what I

would expect (with luacolor you may colorize some digits). Anyway, this behavior could be changed with a switch in the future. Note in the first branch only <an> is relevant if <al>.

```

8016   if dir == 'en' or dir == 'an' or dir == 'et' then
8017       if dir ~= 'et' then
8018           type_n = dir
8019       end
8020       first_n = first_n or item
8021       last_n = last_es or item
8022       last_es = nil
8023   elseif dir == 'es' and last_n then -- W3+W6
8024       last_es = item
8025   elseif dir == 'cs' then           -- it's right - do nothing
8026   elseif first_n then -- & if dir = any but en, et, an, es, cs, inc nil
8027       if strong_lr == 'r' and type_n ~= '' then
8028           dir_mark(head, first_n, last_n, 'r')
8029       elseif strong_lr == 'l' and first_d and type_n == 'an' then
8030           dir_mark(head, first_n, last_n, 'r')
8031           dir_mark(head, first_d, last_d, outer)
8032           first_d, last_d = nil, nil
8033       elseif strong_lr == 'l' and type_n ~= '' then
8034           last_d = last_n
8035       end
8036       type_n = ''
8037       first_n, last_n = nil, nil
8038   end

```

R text in L, or L text in R. Order of dir_ mark's are relevant: d goes outside n, and therefore it's emitted after. See dir_mark to understand why (but is the nesting actually necessary or is a flat dir structure enough?). Only L, R (and AL) chars are taken into account – everything else, including spaces, whatsits, etc., are ignored:

```

8039   if dir == 'l' or dir == 'r' then
8040       if dir ~= outer then
8041           first_d = first_d or item
8042           last_d = item
8043       elseif first_d and dir ~= strong_lr then
8044           dir_mark(head, first_d, last_d, outer)
8045           first_d, last_d = nil, nil
8046       end
8047   end

```

Mirroring. Each chunk of text in a certain language is considered a “closed” sequence. If <r on r> and <l on l>, it's clearly <r> and <l>, resptly, but with other combinations depends on outer. From all these, we select only those resolving <on> → <r>. At the beginning (when last_lr is nil) of an R text, they are mirrored directly. Numbers in R mode are processed. It should not be done, but it doesn't hurt.

```

8048   if dir and not last_lr and dir ~= 'l' and outer == 'r' then
8049       item.char = characters[item.char] and
8050           characters[item.char].m or item.char
8051   elseif (dir or new_dir) and last_lr ~= item then
8052       local mir = outer .. strong_lr .. (dir or outer)
8053       if mir == 'rrr' or mir == 'lrr' or mir == 'rrl' or mir == 'rlr' then
8054           for ch in node.traverse(node.next(last_lr)) do
8055               if ch == item then break end
8056               if ch.id == node.id'glyph' and characters[ch.char] then
8057                   ch.char = characters[ch.char].m or ch.char
8058               end
8059           end
8060       end
8061   end

```

Save some values for the next iteration. If the current node is 'dir', open a new sequence. Since dir could be changed, strong is set with its real value (dir_real).

```

8062   if dir == 'l' or dir == 'r' then

```

```

8063     last_lr = item
8064     strong = dir_real          -- Don't search back - best save now
8065     strong_lr = (strong == 'l') and 'l' or 'r'
8066     elseif new_dir then
8067         last_lr = nil
8068     end
8069 end

```

Mirror the last chars if they are no directed. And make sure any open block is closed, too.

```

8070 if last_lr and outer == 'r' then
8071     for ch in node.traverse_id(node.id('glyph', node.next(last_lr)) do
8072         if characters[ch.char] then
8073             ch.char = characters[ch.char].m or ch.char
8074         end
8075     end
8076 end
8077 if first_n then
8078     dir_mark(head, first_n, last_n, outer)
8079 end
8080 if first_d then
8081     dir_mark(head, first_d, last_d, outer)
8082 end

```

In boxes, the dir node could be added before the original head, so the actual head is the previous node.

```

8083 return node.prev(head) or head
8084 end
8085 (/basic-r)

```

And here the Lua code for bidi=basic:

```

8086 (*basic)
8087 -- e.g., Babel.fontmap[1][<prefontid>]=<dirfontid>
8088
8089 Babel.fontmap = Babel.fontmap or {}
8090 Babel.fontmap[0] = {}          -- l
8091 Babel.fontmap[1] = {}          -- r
8092 Babel.fontmap[2] = {}          -- al/an
8093
8094 -- To cancel mirroring. Also OML, OMS, U?
8095 Babel.symbol_fonts = Babel.symbol_fonts or {}
8096 Babel.symbol_fonts[font.id('tenln')] = true
8097 Babel.symbol_fonts[font.id('tenlnw')] = true
8098 Babel.symbol_fonts[font.id('tencirc')] = true
8099 Babel.symbol_fonts[font.id('tencircw')] = true
8100
8101 Babel.bidi_enabled = true
8102 Babel.mirroring_enabled = true
8103
8104 require('babel-data-bidi.lua')
8105
8106 local characters = Babel.characters
8107 local ranges = Babel.ranges
8108
8109 local DIR = node.id('dir')
8110 local GLYPH = node.id('glyph')
8111
8112 local function insert_implicit(head, state, outer)
8113     local new_state = state
8114     if state.sim and state.eim and state.sim ~= state.eim then
8115         dir = ((outer == 'r') and 'TLT' or 'TRT') -- i.e., reverse
8116         local d = node.new(DIR)
8117         d.dir = '+' .. dir
8118         node.insert_before(head, state.sim, d)
8119         local d = node.new(DIR)

```

```

8120     d.dir = '-' .. dir
8121     node.insert_after(head, state.eim, d)
8122 end
8123 new_state.sim, new_state.eim = nil, nil
8124 return head, new_state
8125 end
8126
8127 local function insert_numeric(head, state)
8128     local new
8129     local new_state = state
8130     if state.san and state.ean and state.san ~= state.ean then
8131         local d = node.new(DIR)
8132         d.dir = '+TLT'
8133         _, new = node.insert_before(head, state.san, d)
8134         if state.san == state.sim then state.sim = new end
8135         local d = node.new(DIR)
8136         d.dir = '-TLT'
8137         _, new = node.insert_after(head, state.ean, d)
8138         if state.ean == state.eim then state.eim = new end
8139     end
8140     new_state.san, new_state.ean = nil, nil
8141     return head, new_state
8142 end
8143
8144 local function glyph_not_symbol_font(node)
8145     if node.id == GLYPH then
8146         return not Babel.symbol_fonts[node.font]
8147     else
8148         return false
8149     end
8150 end
8151
8152 -- TODO - \hbox with an explicit dir can lead to wrong results
8153 -- <R \hbox dir TLT{<R>}> and <L \hbox dir TRT{<L>}>. A small attempt
8154 -- was made to improve the situation, but the problem is the 3-dir
8155 -- model in babel/Unicode and the 2-dir model in LuaTeX don't fit
8156 -- well.
8157
8158 function Babel.bidi(head, ispar, hdir)
8159     local d -- d is used mainly for computations in a loop
8160     local prev_d = ''
8161     local new_d = false
8162
8163     local nodes = {}
8164     local outer_first = nil
8165     local inmath = false
8166
8167     local glue_d = nil
8168     local glue_i = nil
8169
8170     local has_en = false
8171     local first_et = nil
8172
8173     local has_hyperlink = false
8174
8175     local ATDIR = Babel.attr_dir
8176     local attr_d, temp
8177     local locale_d
8178
8179     local save_outer
8180     local locale_d = node.get_attribute(head, ATDIR)
8181     if locale_d then
8182         locale_d = locale_d & 0x3

```

```

8183     save_outer = (locale_d == 0 and 'l') or
8184                   (locale_d == 1 and 'r') or
8185                   (locale_d == 2 and 'al')
8186 elseif ispar then      -- Or error? Shouldn't happen
8187     -- when the callback is called, we are just _after_ the box,
8188     -- and the textdir is that of the surrounding text
8189     save_outer = ('TRT' == tex.pardir) and 'r' or 'l'
8190 else                  -- Empty box
8191     save_outer = ('TRT' == hdir) and 'r' or 'l'
8192 end
8193 local outer = save_outer
8194 local last = outer
8195 -- 'al' is only taken into account in the first, current loop
8196 if save_outer == 'al' then save_outer = 'r' end
8197
8198 local fontmap = Babel.fontmap
8199
8200 for item in node.traverse(head) do
8201
8202     -- Mask: DxxxPPTT (Done, Paddir [0-2], Textdir [0-2])
8203     locale_d = node.get_attribute(item, ATDIR)
8204     node.set_attribute(item, ATDIR, 0x80)
8205
8206     -- In what follows, #node is the last (previous) node, because the
8207     -- current one is not added until we start processing the neutrals.
8208     -- three cases: glyph, dir, otherwise
8209     if glyph_not_symbol_font(item)
8210       or (item.id == 7 and item.subtype == 2) then
8211
8212         if inmath then goto nextnode end
8213
8214         if locale_d == 0x80 then goto nextnode end
8215         locale_d = locale_d or ((save_outer=='l') and 0 or 1)
8216
8217         local d_font = nil
8218         local item_r
8219         if item.id == 7 and item.subtype == 2 then
8220             item_r = item.replace      -- automatic discs have just 1 glyph
8221         else
8222             item_r = item
8223         end
8224
8225         local chardata = characters[item_r.char]
8226         d = chardata and chardata.d or nil
8227         if not d or d == 'nsm' then
8228             for nn, et in ipairs(ranges) do
8229                 if item_r.char < et[1] then
8230                     break
8231                 elseif item_r.char <= et[2] then
8232                     if not d then d = et[3]
8233                     elseif d == 'nsm' then d_font = et[3]
8234                     end
8235                     break
8236                 end
8237             end
8238             end
8239             d = d or 'l'
8240
8241             -- A short 'pause' in bidi for mapfont
8242             -- %%% TODO. move if fontmap here
8243             d_font = d_font or d
8244             d_font = (d_font == 'l' and 0) or
8245                     (d_font == 'nsm' and 0) or

```

```

8246         (d_font == 'r' and 1) or
8247         (d_font == 'al' and 2) or
8248         (d_font == 'an' and 2) or nil
8249     if d_font and fontmap and fontmap[d_font][item_r.font] then
8250         item_r.font = fontmap[d_font][item_r.font]
8251     end
8252
8253     if new_d then
8254         table.insert(nodes, {nil, (outer == 'l') and 'l' or 'r', nil})
8255         if inmath then
8256             attr_d = 0
8257         else
8258             attr_d = locale_d & 0x3
8259         end
8260         if attr_d == 1 then
8261             outer_first = 'r'
8262             last = 'r'
8263         elseif attr_d == 2 then
8264             outer_first = 'r'
8265             last = 'al'
8266         else
8267             outer_first = 'l'
8268             last = 'l'
8269         end
8270         outer = last
8271         has_en = false
8272         first_et = nil
8273         new_d = false
8274     end
8275
8276     if glue_d then
8277         if (d == 'l' and 'l' or 'r') ~= glue_d then
8278             table.insert(nodes, {glue_i, 'on', nil})
8279         end
8280         glue_d = nil
8281         glue_i = nil
8282     end
8283
8284     elseif item.id == DIR then
8285         if inmath then goto nextnode end
8286         d = nil
8287         new_d = true
8288
8289     elseif item.id == node.id'glue' and item.subtype == 13 then
8290         if inmath then goto nextnode end
8291         glue_d = d
8292         glue_i = item
8293         d = nil
8294
8295     elseif item.id == node.id'math' then
8296         if item.subtype == 0 then -- mathon
8297             inmath = true
8298             d = 'on'
8299             table.insert(nodes, {item, 'on', outer_first})
8300             outer_first = nil
8301             goto nextnode
8302         else -- mathoff
8303             inmath = false
8304         end
8305
8306     elseif item.id == 8 and item.subtype == 19 then
8307         has_hyperlink = true
8308

```

```

8309 else
8310     d = nil
8311 end
8312
8313 -- AL <= EN/ET/ES      -- W2 + W3 + W6
8314 if last == 'al' and d == 'en' then
8315     d = 'an'           -- W3
8316 elseif last == 'al' and (d == 'et' or d == 'es') then
8317     d = 'on'           -- W6
8318 end
8319
8320 -- EN + CS/ES + EN      -- W4
8321 if d == 'en' and #nodes >= 2 then
8322     if (nodes[#nodes][2] == 'es' or nodes[#nodes][2] == 'cs')
8323         and nodes[#nodes-1][2] == 'en' then
8324         nodes[#nodes][2] = 'en'
8325     end
8326 end
8327
8328 -- AN + CS + AN          -- W4 too, because uax9 mixes both cases
8329 if d == 'an' and #nodes >= 2 then
8330     if (nodes[#nodes][2] == 'cs')
8331         and nodes[#nodes-1][2] == 'an' then
8332         nodes[#nodes][2] = 'an'
8333     end
8334 end
8335
8336 -- ET/EN                  -- W5 + W7->l / W6->on
8337 if d == 'et' then
8338     first_et = first_et or (#nodes + 1)
8339 elseif d == 'en' then
8340     has_en = true
8341     first_et = first_et or (#nodes + 1)
8342 elseif first_et then      -- d may be nil here !
8343     if has_en then
8344         if last == 'l' then
8345             temp = 'l'    -- W7
8346         else
8347             temp = 'en'   -- W5
8348         end
8349     else
8350         temp = 'on'       -- W6
8351     end
8352     for e = first_et, #nodes do
8353         if glyph_not_symbol_font(nodes[e][1]) then nodes[e][2] = temp end
8354     end
8355     first_et = nil
8356     has_en = false
8357 end
8358
8359 -- Force mathdir in math if ON (currently works as expected only
8360 -- with 'l')
8361
8362 if inmath and d == 'on' then
8363     d = ('TRT' == tex.mathdir) and 'r' or 'l'
8364 end
8365
8366 if d then
8367     if d == 'al' then
8368         d = 'r'
8369         last = 'al'
8370     elseif d == 'l' or d == 'r' then
8371         last = d

```

```

8372     end
8373     prev_d = d
8374     table.insert(nodes, {item, d, outer_first})
8375 end
8376
8377 outer_first = nil
8378
8379 ::nextnode::
8380
8381 end -- for each node
8382
8383 -- TODO -- repeated here in case EN/ET is the last node. Find a
8384 -- better way of doing things:
8385 if first_et then      -- dir may be nil here !
8386     if has_en then
8387         if last == 'l' then
8388             temp = 'l'    -- W7
8389         else
8390             temp = 'en'    -- W5
8391         end
8392     else
8393         temp = 'on'        -- W6
8394     end
8395     for e = first_et, #nodes do
8396         if glyph_not_symbol_font(nodes[e][1]) then nodes[e][2] = temp end
8397     end
8398 end
8399
8400 -- dummy node, to close things
8401 table.insert(nodes, {nil, (outer == 'l') and 'l' or 'r', nil})
8402
8403 ----- NEUTRAL -----
8404
8405 outer = save_outer
8406 last = outer
8407
8408 local first_on = nil
8409
8410 for q = 1, #nodes do
8411     local item
8412
8413     local outer_first = nodes[q][3]
8414     outer = outer_first or outer
8415     last = outer_first or last
8416
8417     local d = nodes[q][2]
8418     if d == 'an' or d == 'en' then d = 'r' end
8419     if d == 'cs' or d == 'et' or d == 'es' then d = 'on' end --- W6
8420
8421     if d == 'on' then
8422         first_on = first_on or q
8423     elseif first_on then
8424         if last == d then
8425             temp = d
8426         else
8427             temp = outer
8428         end
8429         for r = first_on, q - 1 do
8430             nodes[r][2] = temp
8431             item = nodes[r][1]    -- MIRRORING
8432             if Babel.mirroring_enabled and glyph_not_symbol_font(item)
8433                 and temp == 'r' and characters[item.char] then
8434                 local font_mode = ''

```



```

8435         if item.font > 0 and font.fonts[item.font].properties then
8436             font_mode = font.fonts[item.font].properties.mode
8437         end
8438         if font_mode ~= 'harf' and font_mode ~= 'plug' then
8439             item.char = characters[item.char].m or item.char
8440         end
8441     end
8442 end
8443     first_on = nil
8444 end
8445
8446     if d == 'r' or d == 'l' then last = d end
8447 end
8448
8449 ----- IMPLICIT, REORDER -----
8450
8451 outer = save_outer
8452 last = outer
8453
8454 local state = {}
8455 state.has_r = false
8456
8457 for q = 1, #nodes do
8458
8459     local item = nodes[q][1]
8460
8461     outer = nodes[q][3] or outer
8462
8463     local d = nodes[q][2]
8464
8465     if d == 'nsm' then d = last end          -- W1
8466     if d == 'en' then d = 'an' end
8467     local isdir = (d == 'r' or d == 'l')
8468
8469     if outer == 'l' and d == 'an' then
8470         state.san = state.san or item
8471         state.ean = item
8472     elseif state.san then
8473         head, state = insert_numeric(head, state)
8474     end
8475
8476     if outer == 'l' then
8477         if d == 'an' or d == 'r' then      -- im -> implicit
8478             if d == 'r' then state.has_r = true end
8479             state.sim = state.sim or item
8480             state.eim = item
8481         elseif d == 'l' and state.sim and state.has_r then
8482             head, state = insert_implicit(head, state, outer)
8483         elseif d == 'l' then
8484             state.sim, state.eim, state.has_r = nil, nil, false
8485         end
8486     else
8487         if d == 'an' or d == 'l' then
8488             if nodes[q][3] then -- nil except after an explicit dir
8489                 state.sim = item -- so we move sim 'inside' the group
8490             else
8491                 state.sim = state.sim or item
8492             end
8493             state.eim = item
8494         elseif d == 'r' and state.sim then
8495             head, state = insert_implicit(head, state, outer)
8496         elseif d == 'r' then
8497             state.sim, state.eim = nil, nil

```

```

8498     end
8499 end
8500
8501 if isdir then
8502     last = d          -- Don't search back - best save now
8503 elseif d == 'on' and state.san then
8504     state.san = state.san or item
8505     state.ean = item
8506 end
8507
8508 end
8509
8510 head = node.prev(head) or head
8511 % \end{macrocode}
8512 %
8513 % Now direction nodes has been distributed with relation to characters
8514 % and spaces, we need to take into account \TeX-specific elements in
8515 % the node list, to move them at an appropriate place. Firstly, with
8516 % hyperlinks. Secondly, we avoid them between penalties and spaces, so
8517 % that the latter are still discardable.
8518 %
8519 % \begin{macrocode}
8520 --- FIXES ---
8521 if has_hyperlink then
8522     local flag, linking = 0, 0
8523     for item in node.traverse(head) do
8524         if item.id == DIR then
8525             if item.dir == '+TRT' or item.dir == '+TLT' then
8526                 flag = flag + 1
8527             elseif item.dir == '-TRT' or item.dir == '-TLT' then
8528                 flag = flag - 1
8529             end
8530             elseif item.id == 8 and item.subtype == 19 then
8531                 linking = flag
8532             elseif item.id == 8 and item.subtype == 20 then
8533                 if linking > 0 then
8534                     if item.prev.id == DIR and
8535                        (item.prev.dir == '-TRT' or item.prev.dir == '-TLT') then
8536                         d = node.new(DIR)
8537                         d.dir = item.prev.dir
8538                         node.remove(head, item.prev)
8539                         node.insert_after(head, item, d)
8540                     end
8541                 end
8542                 linking = 0
8543             end
8544         end
8545     end
8546
8547     for item in node.traverse_id(10, head) do
8548         local p = item
8549         local flag = false
8550         while p.prev and p.prev.id == 14 do
8551             flag = true
8552             p = p.prev
8553         end
8554         if flag then
8555             node.insert_before(head, p, node.copy(item))
8556             node.remove(head, item)
8557         end
8558     end
8559
8560     return head

```

```

8561 end
8562 function Babel.unset_atdir(head)
8563   local ATDIR = Babel.attr_dir
8564   for item in node.traverse(head) do
8565     node.set_attribute(item, ATDIR, 0x80)
8566   end
8567   return head
8568 end
8569 (/basic)

```

11 Data for CJK

It is a boring file and it is not shown here (see the generated file), but here is a sample:

```

% [0x0021]={c='ex'},
% [0x0024]={c='pr'},
% [0x0025]={c='po'},
% [0x0028]={c='op'},
% [0x0029]={c='cp'},
% [0x002B]={c='pr'},
%

```

For the meaning of these codes, see the Unicode standard.

12 The ‘nil’ language

This ‘language’ does nothing, except setting the hyphenation patterns to nohyphenation. For this language currently no special definitions are needed or available.

The macro `\LdfInit` takes care of preventing that this file is loaded more than once, checking the category code of the `@` sign, etc.

```

8570 (*nil)
8571 \ProvidesLanguage{nil}[<@date@> v<@version@> Nil language]
8572 \LdfInit{nil}{datenil}

```

When this file is read as an option, i.e., by the `\usepackage` command, `nil` could be an ‘unknown’ language in which case we have to make it known.

```

8573 \ifx\l@nil\undefined
8574   \newlanguage\l@nil
8575   \@namedef{bbl@hyphendata@the\l@nil}{}{}{}% Remove warning
8576   \let\bbl@elt\relax
8577   \edef\bbl@languages{% Add it to the list of languages
8578     \bbl@languages\bbl@elt{nil}{the\l@nil}{}{}
8579 \fi

```

This macro is used to store the values of the hyphenation parameters `\lefthyphenmin` and `\righthyphenmin`.

```

8580 \providehyphenmins{\CurrentOption}{\m@ne\m@ne}

```

The next step consists of defining commands to switch to (and from) the ‘nil’ language.

`\captionnil`

`\datenil`

```

8581 \let\captionnil\empty
8582 \let\datenil\empty

```

There is no locale file for this pseudo-language, so the corresponding fields are defined here.

```

8583 \def\bbl@inidata@nil{%
8584   \bbl@elt{identification}{tag.ini}{und}%
8585   \bbl@elt{identification}{load.level}{0}%
8586   \bbl@elt{identification}{name.english}{nil}}
8587 \@namedef{bbl@tbc@nil}{und}

```

```

8588 \@namedef{bbl@lbcpl@nil}{und}
8589 \@namedef{bbl@casing@nil}{und}
8590 \@namedef{bbl@lotf@nil}{dflt}
8591 \@namedef{bbl@elname@nil}{nil}
8592 \@namedef{bbl@lname@nil}{nil}
8593 \@namedef{bbl@esname@nil}{Latin}
8594 \@namedef{bbl@sname@nil}{Latin}
8595 \@namedef{bbl@sbcpl@nil}{Latn}
8596 \@namedef{bbl@sotf@nil}{latn}

```

The macro `\ldf@finish` takes care of looking for a configuration file, setting the main language to be switched on at `\begin{document}` and resetting the category code of `@` to its original value.

```

8597 \ldf@finish{nil}
8598 (/nil)

```

13 Calendars

The code for specific calendars are placed in the specific files, loaded when requested by an ini file in the identification section with `require.calendars`.

Start with function to compute the Julian day. It's based on the little library `calendar.js`, by John Walker, in the public domain.

```

8599 ((*Compute Julian day)) ≡
8600 \def\bbl@fpmmod#1#2{(#1-#2*floor((#1)/(#2)))}
8601 \def\bbl@cs@gregleap#1{%
8602   (\bbl@fpmmod{#1}{4} == 0) &&
8603   (!((\bbl@fpmmod{#1}{100} == 0) && (\bbl@fpmmod{#1}{400} != 0)))}
8604 \def\bbl@cs@jd#1#2#3{ % year, month, day
8605   \fpeval{ 1721424.5 + (365 * (#1 - 1)) +
8606     floor((#1 - 1) / 4) + (-floor((#1 - 1) / 100)) +
8607     floor((#1 - 1) / 400) + floor(((367 * #2) - 362) / 12) +
8608     ((#2 <= 2) ? 0 : (\bbl@cs@gregleap{#1} ? -1 : -2)) + #3 } }
8609 (/Compute Julian day)

```

13.1 Islamic

The code for the Civil calendar is based on it, too.

```

8610 (*ca-islamic)
8611 <@Compute Julian day@>
8612 % == islamic (default)
8613 % Not yet implemented
8614 \def\bbl@ca@islamic#1-#2-#3\@#4#5#6{

```

The Civil calendar.

```

8615 \def\bbl@cs@isltojd#1#2#3{ % year, month, day
8616   ((#3 + ceil(29.5 * (#2 - 1)) +
8617     (#1 - 1) * 354 + floor((3 + (11 * #1)) / 30) +
8618     1948439.5) - 1) }
8619 \@namedef{bbl@ca@islamic-civil++}{\bbl@ca@islamicvl@x{+2}}
8620 \@namedef{bbl@ca@islamic-civil+}{\bbl@ca@islamicvl@x{+1}}
8621 \@namedef{bbl@ca@islamic-civil}{\bbl@ca@islamicvl@x{}}
8622 \@namedef{bbl@ca@islamic-civil-}{\bbl@ca@islamicvl@x{-1}}
8623 \@namedef{bbl@ca@islamic-civil--}{\bbl@ca@islamicvl@x{-2}}
8624 \def\bbl@ca@islamicvl@x#1#2-#3-#4\@#5#6#7{%
8625   \edef\bbl@tempa{%
8626     \fpeval{ floor(\bbl@cs@jd{#2}{#3}{#4})+0.5 #1}}%
8627   \edef#5{%
8628     \fpeval{ floor(((30*(\bbl@tempa-1948439.5)) + 10646)/10631) }}%
8629   \edef#6{\fpeval{
8630     min(12, ceil((\bbl@tempa - (29+\bbl@cs@isltojd{#5}{1}{1}))/29.5)+1) }}%
8631   \edef#7{\fpeval{ \bbl@tempa - \bbl@cs@isltojd{#5}{#6}{1} + 1 } }

```

The Umm al-Qura calendar, used mainly in Saudi Arabia, is based on moment-hijri, by Abdullah Alsigar (license MIT).

Since the main aim is to provide a suitable \today, and maybe some close dates, data just covers Hijri ~1435/~1460 (Gregorian ~2014/~2038).

```

8632 \def\bbl@cs@umalqura@data{56660, 56690,56719,56749,56778,56808,%
8633 56837,56867,56897,56926,56956,56985,57015,57044,57074,57103,%
8634 57133,57162,57192,57221,57251,57280,57310,57340,57369,57399,%
8635 57429,57458,57487,57517,57546,57576,57605,57634,57664,57694,%
8636 57723,57753,57783,57813,57842,57871,57901,57930,57959,57989,%
8637 58018,58048,58077,58107,58137,58167,58196,58226,58255,58285,%
8638 58314,58343,58373,58402,58432,58461,58491,58521,58551,58580,%
8639 58610,58639,58669,58698,58727,58757,58786,58816,58845,58875,%
8640 58905,58934,58964,58994,59023,59053,59082,59111,59141,59170,%
8641 59200,59229,59259,59288,59318,59348,59377,59407,59436,59466,%
8642 59495,59525,59554,59584,59613,59643,59672,59702,59731,59761,%
8643 59791,59820,59850,59879,59909,59939,59968,59997,60027,60056,%
8644 60086,60115,60145,60174,60204,60234,60264,60293,60323,60352,%
8645 60381,60411,60440,60469,60499,60528,60558,60588,60618,60648,%
8646 60677,60707,60736,60765,60795,60824,60853,60883,60912,60942,%
8647 60972,61002,61031,61061,61090,61120,61149,61179,61208,61237,%
8648 61267,61296,61326,61356,61385,61415,61445,61474,61504,61533,%
8649 61563,61592,61621,61651,61680,61710,61739,61769,61799,61828,%
8650 61858,61888,61917,61947,61976,62006,62035,62064,62094,62123,%
8651 62153,62182,62212,62242,62271,62301,62331,62360,62390,62419,%
8652 62448,62478,62507,62537,62566,62596,62625,62655,62685,62715,%
8653 62744,62774,62803,62832,62862,62891,62921,62950,62980,63009,%
8654 63039,63069,63099,63128,63157,63187,63216,63246,63275,63305,%
8655 63334,63363,63393,63423,63453,63482,63512,63541,63571,63600,%
8656 63630,63659,63689,63718,63747,63777,63807,63836,63866,63895,%
8657 63925,63955,63984,64014,64043,64073,64102,64131,64161,64190,%
8658 64220,64249,64279,64309,64339,64368,64398,64427,64457,64486,%
8659 64515,64545,64574,64603,64633,64663,64692,64722,64752,64782,%
8660 64811,64841,64870,64899,64929,64958,64987,65017,65047,65076,%
8661 65106,65136,65166,65195,65225,65254,65283,65313,65342,65371,%
8662 65401,65431,65460,65490,65520}
8663 \@namedef\bbl@ca@islamic-umalqura+{\bbl@ca@islamcuqr@x{+1}}
8664 \@namedef\bbl@ca@islamic-umalqura-{\bbl@ca@islamcuqr@x{-1}}
8665 \@namedef\bbl@ca@islamic-umalqura-{\bbl@ca@islamcuqr@x{-1}}
8666 \def\bbl@ca@islamcuqr@x#1#2-#3-#4\@#5#6#7{%
8667 \ifnum#2>2014 \ifnum#2<2038
8668 \bbl@afterfi\expandafter\@gobble
8669 \fi\fi
8670 {\bbl@error{year-out-range}{2014-2038}}}%
8671 \edef\bbl@tempd{\fpeval{ % (Julian) day
8672 \bbl@cs@jd{#2}{#3}{#4} + 0.5 - 2400000 #1}}%
8673 \count@\@ne
8674 \bbl@foreach\bbl@cs@umalqura@data{%
8675 \advance\count@\@ne
8676 \ifnum##1>\bbl@tempd\else
8677 \edef\bbl@tempe{\the\count@}%
8678 \edef\bbl@tempb{##1}%
8679 \fi}%
8680 \edef\bbl@templ{\fpeval{ \bbl@tempe + 16260 + 949 }}% month~lunar
8681 \edef\bbl@tempa{\fpeval{ floor((\bbl@templ - 1 ) / 12) }}% annus
8682 \edef#5{\fpeval{ \bbl@tempa + 1 }}%
8683 \edef#6{\fpeval{ \bbl@templ - (12 * \bbl@tempa) }}%
8684 \edef#7{\fpeval{ \bbl@tempd - \bbl@tempb + 1 }}%
8685 \bbl@add\bbl@precalendar{%
8686 \bbl@replace\bbl@ld@calendar{-civil}}}%
8687 \bbl@replace\bbl@ld@calendar{-umalqura}}}%
8688 \bbl@replace\bbl@ld@calendar{+}}}%
8689 \bbl@replace\bbl@ld@calendar{-}}}%
8690 (/ca-islamic)

```

13.2 Hebrew

This is basically the set of macros written by Michail Rozman in 1991, with corrections and adaption by Rama Porrat, Misha, Dan Haran and Boris Lavva. This must be eventually replaced by computations with l3fp. An explanation of what's going on can be found in `hebcsl.sty`

```
8691 (*ca-hebrew)
8692 \newcount\bbl@cntcommon
8693 \def\bbl@remainder#1#2#3{%
8694   #3=#1\relax
8695   \divide #3 by #2\relax
8696   \multiply #3 by -#2\relax
8697   \advance #3 by #1\relax}%
8698 \newif\ifbbl@divisible
8699 \def\bbl@checkifdivisible#1#2{%
8700   {\countdef\tmp=0
8701     \bbl@remainder{#1}{#2}{\tmp}%
8702     \ifnum \tmp=0
8703       \global\bbl@divisibletrue
8704     \else
8705       \global\bbl@divisiblefalse
8706     \fi}}
8707 \newif\ifbbl@gregleap
8708 \def\bbl@ifgregleap#1{%
8709   \bbl@checkifdivisible{#1}{4}%
8710   \ifbbl@divisible
8711     \bbl@checkifdivisible{#1}{100}%
8712     \ifbbl@divisible
8713       \bbl@checkifdivisible{#1}{400}%
8714       \ifbbl@divisible
8715         \bbl@gregleaptrue
8716       \else
8717         \bbl@gregleapfalse
8718       \fi
8719     \else
8720       \bbl@gregleaptrue
8721     \fi
8722   \else
8723     \bbl@gregleapfalse
8724   \fi
8725   \ifbbl@gregleap}
8726 \def\bbl@gregdayspriormonths#1#2#3{%
8727   {#3=\ifcase #1 0 \or 0 \or 31 \or 59 \or 90 \or 120 \or 151 \or
8728     181 \or 212 \or 243 \or 273 \or 304 \or 334 \fi
8729   \bbl@ifgregleap{#2}%
8730   \ifnum #1 > 2
8731     \advance #3 by 1
8732   \fi
8733   \fi
8734   \global\bbl@cntcommon=#3}%
8735   #3=\bbl@cntcommon}
8736 \def\bbl@gregdaysprioryears#1#2{%
8737   {\countdef\tmpc=4
8738     \countdef\tmpb=2
8739     \tmpb=#1\relax
8740     \advance \tmpb by -1
8741     \tmpc=\tmpb
8742     \multiply \tmpc by 365
8743     #2=\tmpc
8744     \tmpc=\tmpb
8745     \divide \tmpc by 4
8746     \advance #2 by \tmpc
8747     \tmpc=\tmpb
8748     \divide \tmpc by 100
```

```

8749 \advance #2 by -\tmpc
8750 \tmpc=\tmpb
8751 \divide \tmpc by 400
8752 \advance #2 by \tmpc
8753 \global\bbl@cntcommon=#2\relax}%
8754 #2=\bbl@cntcommon}
8755 \def\bbl@absfromgreg#1#2#3#4{%
8756 {\countdef\tmpd=0
8757 #4=#1\relax
8758 \bbl@gregdayspriormonths{#2}{#3}{\tmpd}%
8759 \advance #4 by \tmpd
8760 \bbl@gregdaysprioryears{#3}{\tmpd}%
8761 \advance #4 by \tmpd
8762 \global\bbl@cntcommon=#4\relax}%
8763 #4=\bbl@cntcommon}
8764 \newif\ifbbl@hebrleap
8765 \def\bbl@checkleaphebryear#1{%
8766 {\countdef\tmpa=0
8767 \countdef\tmpb=1
8768 \tmpa=#1\relax
8769 \multiply \tmpa by 7
8770 \advance \tmpa by 1
8771 \bbl@remainder{\tmpa}{19}{\tmpb}%
8772 \ifnum \tmpb < 7
8773 \global\bbl@hebrleaptrue
8774 \else
8775 \global\bbl@hebrleapfalse
8776 \fi}}
8777 \def\bbl@hebreleapsedmonths#1#2{%
8778 {\countdef\tmpa=0
8779 \countdef\tmpb=1
8780 \countdef\tmpc=2
8781 \tmpa=#1\relax
8782 \advance \tmpa by -1
8783 #2=\tmpa
8784 \divide #2 by 19
8785 \multiply #2 by 235
8786 \bbl@remainder{\tmpa}{19}{\tmpb}% \tmpa=years%19-years this cycle
8787 \tmpc=\tmpb
8788 \multiply \tmpb by 12
8789 \advance #2 by \tmpb
8790 \multiply \tmpc by 7
8791 \advance \tmpc by 1
8792 \divide \tmpc by 19
8793 \advance #2 by \tmpc
8794 \global\bbl@cntcommon=#2}%
8795 #2=\bbl@cntcommon}
8796 \def\bbl@hebreleapseddays#1#2{%
8797 {\countdef\tmpa=0
8798 \countdef\tmpb=1
8799 \countdef\tmpc=2
8800 \bbl@hebreleapsedmonths{#1}{#2}%
8801 \tmpa=#2\relax
8802 \multiply \tmpa by 13753
8803 \advance \tmpa by 5604
8804 \bbl@remainder{\tmpa}{25920}{\tmpc}% \tmpc == ConjunctionParts
8805 \divide \tmpa by 25920
8806 \multiply #2 by 29
8807 \advance #2 by 1
8808 \advance #2 by \tmpa
8809 \bbl@remainder{#2}{7}{\tmpa}%
8810 \ifnum \tmpc < 19440
8811 \ifnum \tmpc < 9924

```

```

8812     \else
8813         \ifnum \tmpa=2
8814             \bbl@checkleaphebyear{#1}% of a common year
8815             \ifbbl@hebrleap
8816                 \else
8817                     \advance #2 by 1
8818                 \fi
8819             \fi
8820         \fi
8821     \ifnum \tmpc < 16789
8822     \else
8823         \ifnum \tmpa=1
8824             \advance #1 by -1
8825             \bbl@checkleaphebyear{#1}% at the end of leap year
8826             \ifbbl@hebrleap
8827                 \advance #2 by 1
8828             \fi
8829         \fi
8830     \fi
8831 \else
8832     \advance #2 by 1
8833 \fi
8834 \bbl@remainder{#2}{7}{\tmpa}%
8835 \ifnum \tmpa=0
8836     \advance #2 by 1
8837 \else
8838     \ifnum \tmpa=3
8839         \advance #2 by 1
8840     \else
8841         \ifnum \tmpa=5
8842             \advance #2 by 1
8843         \fi
8844     \fi
8845 \fi
8846 \global\bbl@cntcommon=#2\relax}%
8847 #2=\bbl@cntcommon}
8848 \def\bbl@daysinhebyear#1#2{%
8849 {\countdef\tmpe=12
8850 \bbl@hebreleaseddays{#1}{\tmpe}%
8851 \advance #1 by 1
8852 \bbl@hebreleaseddays{#1}{#2}%
8853 \advance #2 by -\tmpe
8854 \global\bbl@cntcommon=#2}%
8855 #2=\bbl@cntcommon}
8856 \def\bbl@hebrdayspriormonths#1#2#3{%
8857 {\countdef\tmpf= 14
8858 #3=\ifcase #1
8859     0 \or
8860     0 \or
8861     30 \or
8862     59 \or
8863     89 \or
8864     118 \or
8865     148 \or
8866     148 \or
8867     177 \or
8868     207 \or
8869     236 \or
8870     266 \or
8871     295 \or
8872     325 \or
8873     400
8874 \fi

```



```

8875 \bbl@checkleaphebyear{#2}%
8876 \ifbbl@hebrleap
8877     \ifnum #1 > 6
8878         \advance #3 by 30
8879     \fi
8880 \fi
8881 \bbl@daysinhebyear{#2}{\tmpf}%
8882 \ifnum #1 > 3
8883     \ifnum \tmpf=353
8884         \advance #3 by -1
8885     \fi
8886     \ifnum \tmpf=383
8887         \advance #3 by -1
8888     \fi
8889 \fi
8890 \ifnum #1 > 2
8891     \ifnum \tmpf=355
8892         \advance #3 by 1
8893     \fi
8894     \ifnum \tmpf=385
8895         \advance #3 by 1
8896     \fi
8897 \fi
8898 \global\bbl@cntcommon=#3\relax}%
8899 #3=\bbl@cntcommon}
8900 \def\bbl@absfromhebr#1#2#3#4{%
8901 {#4=#1\relax
8902 \bbl@hebrdayspriormonths{#2}{#3}{#1}%
8903 \advance #4 by #1\relax
8904 \bbl@hebrelapseddays{#3}{#1}%
8905 \advance #4 by #1\relax
8906 \advance #4 by -1373429
8907 \global\bbl@cntcommon=#4\relax}%
8908 #4=\bbl@cntcommon}
8909 \def\bbl@hebrfromgreg#1#2#3#4#5#6{%
8910 {\countdef\tmpx= 17
8911 \countdef\tmpy= 18
8912 \countdef\tmpz= 19
8913 #6=#3\relax
8914 \global\advance #6 by 3761
8915 \bbl@absfromgreg{#1}{#2}{#3}{#4}%
8916 \tmpz=1 \tmpy=1
8917 \bbl@absfromhebr{\tmpz}{\tmpy}{#6}{\tmpx}%
8918 \ifnum \tmpx > #4\relax
8919     \global\advance #6 by -1
8920     \bbl@absfromhebr{\tmpz}{\tmpy}{#6}{\tmpx}%
8921 \fi
8922 \advance #4 by -\tmpx
8923 \advance #4 by 1
8924 #5=#4\relax
8925 \divide #5 by 30
8926 \loop
8927     \bbl@hebrdayspriormonths{#5}{#6}{\tmpx}%
8928     \ifnum \tmpx < #4\relax
8929         \advance #5 by 1
8930         \tmpy=\tmpx
8931 \repeat
8932 \global\advance #5 by -1
8933 \global\advance #4 by -\tmpy}}
8934 \newcount\bbl@hebrday \newcount\bbl@hebrmonth \newcount\bbl@hebyear
8935 \newcount\bbl@gregday \newcount\bbl@gregmonth \newcount\bbl@gregyear
8936 \def\bbl@ca@hebrew#1-#2-#3\@#4#5#6{%
8937 \bbl@gregday=#3\relax \bbl@gregmonth=#2\relax \bbl@gregyear=#1\relax

```

```

8938 \bbl@hebrfromgreg
8939 {\bbl@gregday}{\bbl@gregmonth}{\bbl@gregyear}%
8940 {\bbl@hebrday}{\bbl@hebrmonth}{\bbl@hebyear}%
8941 \edef#4{\the\bbl@hebyear}%
8942 \edef#5{\the\bbl@hebrmonth}%
8943 \edef#6{\the\bbl@hebrday}%
8944 (/ca-hebrew)

```

13.3 Persian

There is an algorithm written in TeX by Jabri, Abolhassani, Pournader and Esfahbod, created for the first versions of the FarsiTeX system (no longer available), but the original license is GPL, so its use with LPL is problematic. The code here follows loosely that by John Walker, which is free and accurate, but sadly very complex, so the relevant data for the years 2013-2050 have been pre-calculated and stored. Actually, all we need is the first day (either March 20 or March 21).

```

8945 (*ca-persian)
8946 <@Compute Julian day@>
8947 \def\bbl@cs@firstjal@xx{2012,2016,2020,2024,2028,2029,% March 20
8948 2032,2033,2036,2037,2040,2041,2044,2045,2048,2049}
8949 \def\bbl@ca@persian#1-#2-#3\@@#4#5#6{%
8950 \edef\bbl@tempa{#1}% 20XX-03-\bbl@tempe = 1 farvardin:
8951 \ifnum\bbl@tempa>2012 \ifnum\bbl@tempa<2051
8952 \bbl@afterfi\expandafter\@gobble
8953 \fi\fi
8954 {\bbl@error{year-out-range}{2013-2050}{}}}%
8955 \bbl@xin@{\bbl@tempa}{\bbl@cs@firstjal@xx}%
8956 \ifin\def\bbl@tempe{20}\else\def\bbl@tempe{21}\fi
8957 \edef\bbl@tempc{\fpeval{\bbl@cs@jd{\bbl@tempa}{#2}{#3}+.5}}% current
8958 \edef\bbl@tempb{\fpeval{\bbl@cs@jd{\bbl@tempa}{03}{\bbl@tempe}+.5}}% begin
8959 \ifnum\bbl@tempc<\bbl@tempb
8960 \edef\bbl@tempa{\fpeval{\bbl@tempa-1}}% go back 1 year and redo
8961 \bbl@xin@{\bbl@tempa}{\bbl@cs@firstjal@xx}%
8962 \ifin\def\bbl@tempe{20}\else\def\bbl@tempe{21}\fi
8963 \edef\bbl@tempb{\fpeval{\bbl@cs@jd{\bbl@tempa}{03}{\bbl@tempe}+.5}}%
8964 \fi
8965 \edef#4{\fpeval{\bbl@tempa-621}}% set Jalali year
8966 \edef#6{\fpeval{\bbl@tempc-\bbl@tempb+1}}% days from 1 farvardin
8967 \edef#5{\fpeval{% set Jalali month
8968 (#6 <= 186) ? ceil(#6 / 31) : ceil((#6 - 6) / 30)}}
8969 \edef#6{\fpeval{% set Jalali day
8970 (#6 - ((#5 <= 7) ? ((#5 - 1) * 31) : (((#5 - 1) * 30) + 6))}}}%
8971 (/ca-persian)

```

13.4 Coptic and Ethiopic

Adapted from `jquery.calendars.package-1.1.4`, written by Keith Wood, 2010. Dual license: GPL and MIT. The only difference is the epoch.

```

8972 (*ca-coptic)
8973 <@Compute Julian day@>
8974 \def\bbl@ca@coptic#1-#2-#3\@@#4#5#6{%
8975 \edef\bbl@tempd{\fpeval{floor(\bbl@cs@jd{#1}{#2}{#3}) + 0.5}}%
8976 \edef\bbl@tempc{\fpeval{\bbl@tempd - 1825029.5}}%
8977 \edef#4{\fpeval{%
8978 floor((\bbl@tempc - floor((\bbl@tempc+366) / 1461)) / 365) + 1}}%
8979 \edef\bbl@tempc{\fpeval{%
8980 \bbl@tempd - (#4-1) * 365 - floor(#4/4) - 1825029.5}}%
8981 \edef#5{\fpeval{floor(\bbl@tempc / 30) + 1}}%
8982 \edef#6{\fpeval{\bbl@tempc - (#5 - 1) * 30 + 1}}%
8983 (/ca-coptic)
8984 (*ca-ethiopic)
8985 <@Compute Julian day@>
8986 \def\bbl@ca@ethiopic#1-#2-#3\@@#4#5#6{%

```

```

8987 \edef\bbl@tempd{\fpeval{floor(\bbl@cs@jd{#1}{#2}{#3}) + 0.5}}%
8988 \edef\bbl@tempc{\fpeval{\bbl@tempd - 1724220.5}}%
8989 \edef#4{\fpeval{%
8990   floor((\bbl@tempc - floor((\bbl@tempc+366) / 1461)) / 365) + 1}}%
8991 \edef\bbl@tempc{\fpeval{%
8992   \bbl@tempd - (#4-1) * 365 - floor(#4/4) - 1724220.5}}%
8993 \edef#5{\fpeval{floor(\bbl@tempc / 30) + 1}}%
8994 \edef#6{\fpeval{\bbl@tempc - (#5 - 1) * 30 + 1}}
8995 (/ca-ethiopic)

```

13.5 Julian

Based on [ReinDersh].

```

8996 (*ca-julian)
8997 <@Compute Julian day@>
8998 \def\bbl@ca@julian#1-#2-#3\@@#4#5#6{%
8999   \edef\bbl@tempj{\fpeval{floor(\bbl@cs@jd{#1}{#2}{#3}) + .5}}%
9000   \edef\bbl@tempa{\fpeval{\bbl@tempj + 32082.5}}%
9001   \edef\bbl@tempb{\fpeval{floor((4 * \bbl@tempa + 3) / 1461)}}%
9002   \edef\bbl@tempc{\fpeval{\bbl@tempa - floor(1461*\bbl@tempb/4)}}%
9003   \edef\bbl@tempd{\fpeval{floor((5 * \bbl@tempc + 2) / 153)}}%
9004   \edef#6{\fpeval{\bbl@tempc - floor((153*\bbl@tempd+2) / 5) + 1}}%
9005   \edef#5{\fpeval{\bbl@tempd + 3 - 12 * floor(\bbl@tempd / 10)}}%
9006   \edef#4{\fpeval{\bbl@tempb - 4800 + floor(\bbl@tempd / 10)}}
9007 (/ca-julian)

```

The Indian National Calendar, Saka era.

```

9008 (*ca-indian)
9009 <@Compute Julian day@>
9010 \def\bbl@sakachaitra#1{%
9011   \bbl@cs@jd{#1}{3}{%
9012     \fpeval{ ((\bbl@fpmmod{#1}{4})==0 && \bbl@fpmmod{#1}{100})!=0)
9013     || \bbl@fpmmod{#1}{400})==0) ? 21 : 22 }}
9014 \def\bbl@ca@indian#1-#2-#3\@@#4#5#6{%
9015   \edef\bbl@tempa{\fpeval{\bbl@cs@jd{#1}{#2}{#3}}}%
9016   \edef\bbl@tempb{\fpeval{\bbl@sakachaitra{#1}}}%
9017   \edef\bbl@tempc{\fpeval{\bbl@tempa < \bbl@tempb ? #1-1 : #1}}%
9018   \edef\bbl@tempd{\fpeval{\bbl@tempa-\bbl@sakachaitra{\bbl@tempc}}}%
9019   \edef\bbl@tempe{\fpeval{ ((\bbl@fpmmod{\bbl@tempc}{4})==0 &&
9020     \bbl@fpmmod{\bbl@tempc}{100})!=0) || \bbl@fpmmod{\bbl@tempc}{400})==0)?31:30}}%
9021   \edef#4{\fpeval{\bbl@tempc-78}}%
9022   \edef#5{\fpeval{\bbl@tempd<\bbl@tempe ? 1 :
9023     (\bbl@tempd-\bbl@tempe < 155 ?
9024       2+trunc((\bbl@tempd-\bbl@tempe)/31,0) :
9025       7+trunc((\bbl@tempd-\bbl@tempe-155)/30,0))}}%
9026   \edef#6{\fpeval{\bbl@tempd<\bbl@tempe ? \bbl@tempd+1 :
9027     (\bbl@tempd-\bbl@tempe < 155 ?
9028       \bbl@fpmmod{\bbl@tempd-\bbl@tempe}{31}+1 :
9029       \bbl@fpmmod{\bbl@tempd-\bbl@tempe-155}{30}+1)}}
9030 (/ca-indian)
9031 %

```

13.6 Buddhist

That's very simple.

```

9032 (*ca-buddhist)
9033 \def\bbl@ca@buddhist#1-#2-#3\@@#4#5#6{%
9034   \edef#4{\number\numexpr#1+543\relax}%
9035   \edef#5{#2}%
9036   \edef#6{#3}}
9037 (/ca-buddhist)
9038 %

```

```

9039 % \subsection{Chinese}
9040 %
9041 % Brute force, with the Julian day of first day of each month. The
9042 % table has been computed with the help of \textsf{python-lunardate} by
9043 % Ricky Yeung, GPLv2 (but the code itself has not been used). The range
9044 % is 2015-2044.
9045 %
9046 % \begin{macrocode}
9047 (*ca-chinese)
9048 \ExplSyntaxOn
9049 <@Compute Julian day@>
9050 \def\bbl@ca@chinese#1-#2-#3\@#4#5#6{%
9051 \edef\bbl@tempd{\fpeval{%
9052 \bbl@cs@jd{#1}{#2}{#3} - 2457072.5 }}%
9053 \count@\z@
9054 \@tempcnta=2015
9055 \bbl@foreach\bbl@cs@chinese@data{%
9056 \ifnum##1>\bbl@tempd\else
9057 \advance\count@\@ne
9058 \ifnum\count@>12
9059 \count@\@ne
9060 \advance\@tempcnta\@ne\fi
9061 \bbl@xin@{,##1,}{,\bbl@cs@chinese@leap,}%
9062 \ifin@
9063 \advance\count@\m@ne
9064 \edef\bbl@tempe{\the\numexpr\count@+12\relax}%
9065 \else
9066 \edef\bbl@tempe{\the\count@}%
9067 \fi
9068 \edef\bbl@tempb{##1}%
9069 \fi}%
9070 \edef#4{\the\@tempcnta}%
9071 \edef#5{\bbl@tempe}%
9072 \edef#6{\the\numexpr\bbl@tempd-\bbl@tempb+1\relax}}
9073 \def\bbl@cs@chinese@leap{%
9074 885,1920,2953,3809,4873,5906,6881,7825,8889,9893,10778}
9075 \def\bbl@cs@chinese@data{0,29,59,88,117,147,176,206,236,266,295,325,
9076 354,384,413,443,472,501,531,560,590,620,649,679,709,738,%
9077 768,797,827,856,885,915,944,974,1003,1033,1063,1093,1122,%
9078 1152,1181,1211,1240,1269,1299,1328,1358,1387,1417,1447,1477,%
9079 1506,1536,1565,1595,1624,1653,1683,1712,1741,1771,1801,1830,%
9080 1860,1890,1920,1949,1979,2008,2037,2067,2096,2126,2155,2185,%
9081 2214,2244,2274,2303,2333,2362,2392,2421,2451,2480,2510,2539,%
9082 2569,2598,2628,2657,2687,2717,2746,2776,2805,2835,2864,2894,%
9083 2923,2953,2982,3011,3041,3071,3100,3130,3160,3189,3219,3248,%
9084 3278,3307,3337,3366,3395,3425,3454,3484,3514,3543,3573,3603,%
9085 3632,3662,3691,3721,3750,3779,3809,3838,3868,3897,3927,3957,%
9086 3987,4016,4046,4075,4105,4134,4163,4193,4222,4251,4281,4311,%
9087 4341,4370,4400,4430,4459,4489,4518,4547,4577,4606,4635,4665,%
9088 4695,4724,4754,4784,4814,4843,4873,4902,4931,4961,4990,5019,%
9089 5049,5079,5108,5138,5168,5197,5227,5256,5286,5315,5345,5374,%
9090 5403,5433,5463,5492,5522,5551,5581,5611,5640,5670,5699,5729,%
9091 5758,5788,5817,5846,5876,5906,5935,5965,5994,6024,6054,6083,%
9092 6113,6142,6172,6201,6231,6260,6289,6319,6348,6378,6408,6437,%
9093 6467,6497,6526,6556,6585,6615,6644,6673,6703,6732,6762,6791,%
9094 6821,6851,6881,6910,6940,6969,6999,7028,7057,7087,7116,7146,%
9095 7175,7205,7235,7264,7294,7324,7353,7383,7412,7441,7471,7500,%
9096 7529,7559,7589,7618,7648,7678,7708,7737,7767,7796,7825,7855,%
9097 7884,7913,7943,7972,8002,8032,8062,8092,8121,8151,8180,8209,%
9098 8239,8268,8297,8327,8356,8386,8416,8446,8475,8505,8534,8564,%
9099 8593,8623,8652,8681,8711,8740,8770,8800,8829,8859,8889,8918,%
9100 8948,8977,9007,9036,9066,9095,9124,9154,9183,9213,9243,9272,%
9101 9302,9331,9361,9391,9420,9450,9479,9508,9538,9567,9597,9626,%

```

```

9102 9656,9686,9715,9745,9775,9804,9834,9863,9893,9922,9951,9981,%
9103 10010,10040,10069,10099,10129,10158,10188,10218,10247,10277,%
9104 10306,10335,10365,10394,10423,10453,10483,10512,10542,10572,%
9105 10602,10631,10661,10690,10719,10749,10778,10807,10837,10866,%
9106 10896,10926,10956,10986,11015,11045,11074,11103}
9107 \ExplSyntaxOff
9108 (/ca-chinese)

```

14 Support for Plain T_EX (plain.def)

14.1 Not renaming hyphen.tex

As Don Knuth has declared that the filename `hyphen.tex` may only be used to designate *his* version of the american English hyphenation patterns, a new solution has to be found in order to be able to load hyphenation patterns for other languages in a plain-based T_EX-format. When asked he responded:

That file name is “sacred”, and if anybody changes it they will cause severe upward/downward compatibility headaches.

People can have a file `locallyhyphen.tex` or whatever they like, but they mustn’t diddle with `hyphen.tex` (or `plain.tex` except to preload additional fonts).

The files `bplain.tex` and `blplain.tex` can be used as replacement wrappers around `plain.tex` and `lplain.tex` to achieve the desired effect, based on the `babel` package. If you load each of them with `iniTEX`, you will get a file called either `bplain.fmt` or `blplain.fmt`, which you can use as replacements for `plain.fmt` and `lplain.fmt`.

As these files are going to be read as the first thing `iniTEX` sees, we need to set some category codes just to be able to change the definition of `\input`.

```

9109 (*bplain|blplain)
9110 \catcode\{=1 % left brace is begin-group character
9111 \catcode\}=2 % right brace is end-group character
9112 \catcode\#=6 % hash mark is macro parameter character

```

If a file called `hyphen.cfg` can be found, we make sure that *it* will be read instead of the file `hyphen.tex`. We do this by first saving the original meaning of `\input` (and I use a one letter control sequence for that so as not to waste multi-letter control sequence on this in the format).

```

9113 \openin 0 hyphen.cfg
9114 \ifeof0
9115 \else
9116   \let\input

```

Then `\input` is defined to forget about its argument and load `hyphen.cfg` instead. Once that’s done the original meaning of `\input` can be restored and the definition of `\a` can be forgotten.

```

9117 \def\input #1 {%
9118   \let\input\input
9119   \a hyphen.cfg
9120   \let\input\input
9121 }
9122 \fi
9123 (/bplain|blplain)

```

Now that we have made sure that `hyphen.cfg` will be loaded at the right moment it is time to load `plain.tex`.

```

9124 (bplain)\a plain.tex
9125 (blplain)\a lplain.tex

```

Finally we change the contents of `\fmtname` to indicate that this is *not* the plain format, but a format based on plain with the `babel` package preloaded.

```

9126 (bplain)\def\fmtname{babel-plain}
9127 (blplain)\def\fmtname{babel-lplain}

```

When you are using a different format, based on `plain.tex` you can make a copy of `blplain.tex`, rename it and replace `plain.tex` with the name of your format file.

14.2 Emulating some $\text{\LaTeX}$ features

The file `babel.def` expects some definitions made in the $\text{\LaTeX} 2_{\epsilon}$ style file. So, in Plain we must provide at least some predefined values as well some tools to set them (even if not all options are available). There are no package options, and therefore an alternative mechanism is provided. For the moment, only `\babeloptionstrings` and `\babeloptionmath` are provided, which can be defined before loading `babel`. `\BabelModifiers` can be set too (but not sure it works).

```
9128 <<*Emulate LaTeX>>  ≡
9129 \def\@empty{}
9130 \def\loadlocalcfg#1{%
9131   \openin0#1.cfg
9132   \ifeof0
9133     \closein0
9134   \else
9135     \closein0
9136     {\immediate\write16{*****}%
9137      \immediate\write16{* Local config file #1.cfg used}%
9138      \immediate\write16{*}%
9139     }
9140   \input #1.cfg\relax
9141   \fi
9142   \@endofldf}
```

14.3 General tools

A number of $\text{\LaTeX}$ macro's that are needed later on.

```
9143 \long\def\@firstofone#1{#1}
9144 \long\def\@firstoftwo#1#2{#1}
9145 \long\def\@secondoftwo#1#2{#2}
9146 \def\@nnil{\@nil}
9147 \def\@gobbletwo#1#2{}
9148 \def\@ifstar#1{\@ifnextchar *{\@firstoftwo{#1}}}
9149 \def\@star@or@long#1{%
9150   \@ifstar
9151   {\let\l@ngrel@x\relax#1}%
9152   {\let\l@ngrel@x\long#1}}
9153 \let\l@ngrel@x\relax
9154 \def\@car#1#2\@nil{#1}
9155 \def\@cdr#1#2\@nil{#2}
9156 \let\@typeset@protect\relax
9157 \let\protected@edef\edef
9158 \long\def\@gobble#1{}
9159 \edef\@backslashchar{\expandafter\@gobble\string\}
9160 \def\strip@prefix#1>{}
9161 \def\g@addto@macro#1#2{{%
9162   \toks@\expandafter{#1#2}%
9163   \xdef#1{\the\toks@}}}
9164 \def\@namedef#1{\expandafter\def\csname #1\endcsname}
9165 \def\@nameuse#1{\csname #1\endcsname}
9166 \def\@ifundefined#1{%
9167   \expandafter\ifx\csname#1\endcsname\relax
9168     \expandafter\@firstoftwo
9169   \else
9170     \expandafter\@secondoftwo
9171   \fi}
9172 \def\@expandtwoargs#1#2#3{%
9173   \edef\reserved@a{\noexpand#1{#2}{#3}}\reserved@a}
9174 \def\zap@space#1 #2{%
9175   #1%
9176   \ifx#2\@empty\else\expandafter\zap@space\fi
9177   #2}
9178 \let\bbl@trace\@gobble
9179 \def\bbl@error#1{% Implicit #2#3#4}
```

```

9180 \begingroup
9181 \catcode`\=0 \catcode`\==12 \catcode`\`=12
9182 \catcode`\^M=5 \catcode`\%=14
9183 \input errbabel.def
9184 \endgroup
9185 \bbl@error{#1}}
9186 \def\bbl@warning#1{%
9187 \begingroup
9188 \newlinechar=`^^J
9189 \def\{`^^J(babel) }%
9190 \message{\{#1}%
9191 \endgroup}
9192 \let\bbl@infowarn\bbl@warning
9193 \def\bbl@info#1{%
9194 \begingroup
9195 \newlinechar=`^^J
9196 \def\{`^^J}%
9197 \wlog{#1}%
9198 \endgroup}

```

$\text{\LaTeX} 2_{\epsilon}$ has the command `\@onlypreamble` which adds commands to a list of commands that are no longer needed after `\begin{document}`.

```

9199 \ifx\@preamblecmds\undefined
9200 \def\@preamblecmds{}
9201 \fi
9202 \def\@onlypreamble#1{%
9203 \expandafter\gdef\expandafter\@preamblecmds\expandafter{%
9204 \@preamblecmds\do#1}}
9205 \@onlypreamble\@onlypreamble

```

Mimic $\text{\LaTeX}$'s `\AtBeginDocument`; for this to work the user needs to add `\begindocument` to his file.

```

9206 \def\begindocument{%
9207 \@begindocumenthook
9208 \global\let\@begindocumenthook\undefined
9209 \def\do##1{\global\let##1\undefined}%
9210 \@preamblecmds
9211 \global\let\do\noexpand}
9212 \ifx\@begindocumenthook\undefined
9213 \def\@begindocumenthook{}
9214 \fi
9215 \@onlypreamble\@begindocumenthook
9216 \def\AtBeginDocument{\g@addto@macro\@begindocumenthook}

```

We also have to mimic $\text{\LaTeX}$'s `\AtEndOfPackage`. Our replacement macro is much simpler; it stores its argument in `\@endoflfd`.

```

9217 \def\AtEndOfPackage#1{\g@addto@macro\@endoflfd{#1}}
9218 \@onlypreamble\AtEndOfPackage
9219 \def\@endoflfd{}
9220 \@onlypreamble\@endoflfd
9221 \let\bbl@afterlang\@empty
9222 \chardef\bbl@opt@hyphenmap\z@

```

$\text{\LaTeX}$ needs to be able to switch off writing to its auxiliary files; plain doesn't have them by default. There is a trick to hide some conditional commands from the outer `\ifx`. The same trick is applied below.

```

9223 \catcode`\&=\z@
9224 \ifx&\if@files\@undefined
9225 \expandafter\let\csname if@files\expandafter\endcsname
9226 \csname iffalse\endcsname
9227 \fi
9228 \catcode`\&=4

```

Mimic $\text{\LaTeX}$'s commands to define control sequences.

```

9229 \def\newcommand{\@star@or@long\newcommand}

```

```

9230 \def\new@command#1{%
9231   \@testopt{\@newcommand#1}0}
9232 \def\@newcommand#1[#2]{%
9233   \ifnextchar [{\@xargdef#1[#2]}%
9234               {\@argdef#1[#2]}}
9235 \long\def\@argdef#1[#2]#3{%
9236   \@yargdef#1\@ne{#2}{#3}}
9237 \long\def\@xargdef#1[#2][#3]#4{%
9238   \expandafter\def\expandafter#1\expandafter{%
9239     \expandafter\@protected@testopt\expandafter #1%
9240     \csname\string#1\expandafter\endcsname{#3}}}%
9241   \expandafter\@yargdef \csname\string#1\endcsname
9242   \tw@{#2}{#4}}
9243 \long\def\@yargdef#1#2#3{%
9244   \@tempcnta#3\relax
9245   \advance \@tempcnta \@ne
9246   \let\@hash@\relax
9247   \edef\reserved@a{\ifx#2\tw@ [\@hash@1]\fi}%
9248   \@tempcntb #2%
9249   \@whilenum \@tempcntb < \@tempcnta
9250   \do{%
9251     \edef\reserved@a{\reserved@a\@hash@\the\@tempcntb}%
9252     \advance\@tempcntb \@ne}%
9253   \let\@hash@##%
9254   \l@ngrelx\expandafter\def\expandafter#1\reserved@a}
9255 \def\providecommand{\@star@or@long\provide@command}
9256 \def\provide@command#1{%
9257   \begingroup
9258     \escapechar\m@ne\xdef\@gtempa{\string#1}%
9259   \endgroup
9260   \expandafter\ifundefined\@gtempa
9261     {\def\reserved@a{\new@command#1}}%
9262     {\let\reserved@a\relax
9263     \def\reserved@a{\new@command\reserved@a}}%
9264   \reserved@a}%

9265 \def\DeclareRobustCommand{\@star@or@long\declare@robustcommand}
9266 \def\declare@robustcommand#1{%
9267   \edef\reserved@a{\string#1}%
9268   \def\reserved@b{#1}%
9269   \edef\reserved@b{\expandafter\strip@prefix\meaning\reserved@b}%
9270   \edef#1{%
9271     \ifx\reserved@a\reserved@b
9272       \noexpand\x@protect
9273       \noexpand#1%
9274     \fi
9275     \noexpand\protect
9276     \expandafter\noexpand\csname
9277       \expandafter\@gobble\string#1 \endcsname
9278   }%
9279   \expandafter\new@command\csname
9280     \expandafter\@gobble\string#1 \endcsname
9281 }
9282 \def\x@protect#1{%
9283   \ifx\protect\@typeset@protect\else
9284     \@x@protect#1%
9285   \fi
9286 }
9287 \catcode`\&=\z@ % Trick to hide conditionals
9288 \def\@x@protect#1&fi#2#3{&fi\protect#1}

```

The following little macro `\in@` is taken from `latex.ltx`; it checks whether its first argument is part of its second argument. It uses the boolean `\in@`; allocating a new boolean inside conditionally executed code is not possible, hence the construct with the temporary definition of `\bbl@tempa`.


```

9289 \def\bbbl@tempa{\csname newif\endcsname&ifin@}
9290 \catcode`\&=4
9291 \ifx\in@\@undefined
9292 \def\in@#1#2{%
9293 \def\in@@##1#1##2##3\in@@{%
9294 \ifx\in@@##2\in@false\else\in@true\fi}%
9295 \in@@##2#1\in@\in@@}
9296 \else
9297 \let\bbbl@tempa\@empty
9298 \fi
9299 \bbbl@tempa

```

$\LaTeX$ has a macro to check whether a certain package was loaded with specific options. The command has two extra arguments which are code to be executed in either the true or false case. This is used to detect whether the document needs one of the accents to be activated (activegrave and activeacute). For plain $\TeX$ we assume that the user wants them to be active by default. Therefore the only thing we do is execute the third argument (the code for the true case).

```

9300 \def\@ifpackagewith#1#2#3#4{#3}

```

The $\LaTeX$ macro `\@ifl@aded` checks whether a file was loaded. This functionality is not needed for plain $\TeX$ but we need the macro to be defined as a no-op.

```

9301 \def\@ifl@aded#1#2#3#4{}

```

For the following code we need to make sure that the commands `\newcommand` and `\providecommand` exist with some sensible definition. They are not fully equivalent to their $\LaTeX 2_{\epsilon}$ versions; just enough to make things work in plain $\TeX$ environments.

```

9302 \ifx\@tempcnta\@undefined
9303 \csname newcount\endcsname\@tempcnta\relax
9304 \fi
9305 \ifx\@tempcntb\@undefined
9306 \csname newcount\endcsname\@tempcntb\relax
9307 \fi

```

To prevent wasting two counters in $\LaTeX$ (because counters with the same name are allocated later by it) we reset the counter that holds the next free counter (`\count10`).

```

9308 \ifx\bye\@undefined
9309 \advance\count10 by -2\relax
9310 \fi
9311 \ifx\@ifnextchar\@undefined
9312 \def\@ifnextchar#1#2#3{%
9313 \let\reserved@d=#1%
9314 \def\reserved@a{#2}\def\reserved@b{#3}%
9315 \futurelet\@let@token\@ifnch}
9316 \def\@ifnch{%
9317 \ifx\@let@token\@sptoken
9318 \let\reserved@c\@xifnch
9319 \else
9320 \ifx\@let@token\reserved@d
9321 \let\reserved@c\reserved@a
9322 \else
9323 \let\reserved@c\reserved@b
9324 \fi
9325 \fi
9326 \reserved@c}
9327 \def\:{\let\@sptoken= }\: % this makes \@sptoken a space token
9328 \def\{\@xifnch} \expandafter\def\:{\futurelet\@let@token\@ifnch}
9329 \fi
9330 \def\@testopt#1#2{%
9331 \@ifnextchar[#{1}{#1{#2}}]
9332 \def\@protected@testopt#1{%
9333 \ifx\protect\@typeset@protect
9334 \expandafter\@testopt
9335 \else
9336 \@x@protect#1%

```

```

9337 \fi}
9338 \long\def\@whilenum#1\do #2{\ifnum #1\relax #2\relax\@iwhilenum{#1\relax
9339 #2\relax}\fi}
9340 \long\def\@iwhilenum#1{\ifnum #1\expandafter\@iwhilenum
9341 \else\expandafter\@gobble\fi{#1}}

```

14.4 Encoding related macros

Code from `ltoutenc.dtx`, adapted for use in the plain $\TeX$ environment.

```

9342 \def\DeclareTextCommand{%
9343 \@dec@text@cmd\providecommand
9344 }
9345 \def\ProvideTextCommand{%
9346 \@dec@text@cmd\providecommand
9347 }
9348 \def\DeclareTextSymbol#1#2#3{%
9349 \@dec@text@cmd\chardef#1{#2}#3\relax
9350 }
9351 \def\@dec@text@cmd#1#2#3{%
9352 \expandafter\def\expandafter#2%
9353 \expandafter{%
9354 \csname#3-cmd\expandafter\endcsname
9355 \expandafter#2%
9356 \csname#3\string#2\endcsname
9357 }%
9358 % \let\@ifdefinable\@rc@ifdefinable
9359 \expandafter#1\csname#3\string#2\endcsname
9360 }
9361 \def\@current@cmd#1{%
9362 \ifx\protect\@typeset@protect\else
9363 \noexpand#1\expandafter\@gobble
9364 \fi
9365 }
9366 \def\@changed@cmd#1#2{%
9367 \ifx\protect\@typeset@protect
9368 \expandafter\ifx\csname\cf@encoding\string#1\endcsname\relax
9369 \expandafter\ifx\csname ?\string#1\endcsname\relax
9370 \expandafter\def\csname ?\string#1\endcsname{%
9371 \@changed@x@err{#1}%
9372 }%
9373 \fi
9374 \global\expandafter\let
9375 \csname\cf@encoding\string#1\expandafter\endcsname
9376 \csname ?\string#1\endcsname
9377 \fi
9378 \csname\cf@encoding\string#1%
9379 \expandafter\endcsname
9380 \else
9381 \noexpand#1%
9382 \fi
9383 }
9384 \def\@changed@x@err#1{%
9385 \errhelp{Your command will be ignored, type <return> to proceed}%
9386 \errmessage{Command \protect#1 undefined in encoding \cf@encoding}}
9387 \def\DeclareTextCommandDefault#1{%
9388 \DeclareTextCommand#1?%
9389 }
9390 \def\ProvideTextCommandDefault#1{%
9391 \ProvideTextCommand#1?%
9392 }
9393 \expandafter\let\csname OT1-cmd\endcsname\@current@cmd
9394 \expandafter\let\csname?-cmd\endcsname\@changed@cmd
9395 \def\DeclareTextAccent#1#2#3{%

```

```

9396 \DeclareTextCommand#1{#2}[1]{\accent#3 #1}
9397 }
9398 \def\DeclareTextCompositeCommand#1#2#3#4{%
9399 \expandafter\let\expandafter\reserved@a\csname#2\string#1\endcsname
9400 \edef\reserved@b{\string##1}%
9401 \edef\reserved@c{%
9402 \expandafter\@strip@args\meaning\reserved@a:-\@strip@args}%
9403 \ifx\reserved@b\reserved@c
9404 \expandafter\expandafter\expandafter\ifx
9405 \expandafter\@car\reserved@a\relax\relax\@nil
9406 \@text@composite
9407 \else
9408 \edef\reserved@b##1{%
9409 \def\expandafter\noexpand
9410 \csname#2\string#1\endcsname###1{%
9411 \noexpand\@text@composite
9412 \expandafter\noexpand\csname#2\string#1\endcsname
9413 ###1\noexpand\@empty\noexpand\@text@composite
9414 {##1}%
9415 }%
9416 }%
9417 \expandafter\reserved@b\expandafter{\reserved@a{##1}}%
9418 \fi
9419 \expandafter\def\csname\expandafter\string\csname
9420 #2\endcsname\string#1-\string#3\endcsname{#4}
9421 \else
9422 \errhelp{Your command will be ignored, type <return> to proceed}%
9423 \errmessage{\string\DeclareTextCompositeCommand\space used on
9424 inappropriate command \protect#1}
9425 \fi
9426 }
9427 \def\@text@composite#1#2#3\@text@composite{%
9428 \expandafter\@text@composite@x
9429 \csname\string#1-\string#2\endcsname
9430 }
9431 \def\@text@composite@x#1#2{%
9432 \ifx#1\relax
9433 #2%
9434 \else
9435 #1%
9436 \fi
9437 }
9438 %
9439 \def\@strip@args#1:#2-#3\@strip@args{#2}
9440 \def\DeclareTextComposite#1#2#3#4{%
9441 \def\reserved@a{\DeclareTextCompositeCommand#1{#2}{#3}}%
9442 \bgroup
9443 \lccode`\@=#4%
9444 \lowercase{%
9445 \egroup
9446 \reserved@a @%
9447 }%
9448 }
9449 %
9450 \def\UseTextSymbol#1#2{#2}
9451 \def\UseTextAccent#1#2#3{}
9452 \def\@use@text@encoding#1{}
9453 \def\DeclareTextSymbolDefault#1#2{%
9454 \DeclareTextCommandDefault#1{\UseTextSymbol{#2}#1}%
9455 }
9456 \def\DeclareTextAccentDefault#1#2{%
9457 \DeclareTextCommandDefault#1{\UseTextAccent{#2}#1}%
9458 }

```

```
9459 \def\cf@encoding{OT1}
```

Currently we only use the $\text{\LaTeX 2}\epsilon$ method for accents for those that are known to be made active in *some* language definition file.

```
9460 \DeclareTextAccent{"}{OT1}{127}
9461 \DeclareTextAccent{'}{OT1}{19}
9462 \DeclareTextAccent{^}{OT1}{94}
9463 \DeclareTextAccent`}{OT1}{18}
9464 \DeclareTextAccent{~}{OT1}{126}
```

The following control sequences are used in `babel.def` but are not defined for PLAIN $\text{\TeX}$.

```
9465 \DeclareTextSymbol{\textquotedblleft}{OT1}{92}
9466 \DeclareTextSymbol{\textquotedblright}{OT1}{`\"}
9467 \DeclareTextSymbol{\textquoteleft}{OT1}{``}
9468 \DeclareTextSymbol{\textquoteright}{OT1}{``}
9469 \DeclareTextSymbol{\i}{OT1}{16}
9470 \DeclareTextSymbol{\ss}{OT1}{25}
```

For a couple of languages we need the $\text{\LaTeX}$ -control sequence `\scriptsize` to be available. Because plain $\text{\TeX}$ doesn't have such a sophisticated font mechanism as $\text{\LaTeX}$ has, we just `\let` it to `\sevenrm`.

```
9471 \ifx\scriptsize\undefined
9472   \let\scriptsize\sevenrm
9473 \fi
```

And a few more “dummy” definitions.

```
9474 \def\language{english}%
9475 \let\bbl@opt@shorthands\@nnil
9476 \def\bbl@ifshorthand#1#2#3{#2}%
9477 \let\bbl@language@opts\@empty
9478 \let\bbl@provide@locale\relax
9479 \ifx\babeloptionstrings\undefined
9480   \let\bbl@opt@strings\@nnil
9481 \else
9482   \let\bbl@opt@strings\babeloptionstrings
9483 \fi
9484 \def\BabelStringsDefault{generic}
9485 \def\bbl@tempa{normal}
9486 \ifx\babeloptionmath\bbl@tempa
9487   \def\bbl@mathnormal{\noexpand\textormath}
9488 \fi
9489 \def\AfterBabelLanguage#1#2{}
9490 \ifx\BabelModifiers\undefined\let\BabelModifiers\relax\fi
9491 \let\bbl@afterlang\relax
9492 \def\bbl@opt@safe{BR}
9493 \ifx\@uclclist\undefined\let\@uclclist\@empty\fi
9494 \ifx\bbl@trace\undefined\def\bbl@trace#1{}\fi
9495 \expandafter\newif\csname ifbbl@single\endcsname
9496 \chardef\bbl@bidimode\z@
9497 <(/Emulate LaTeX)>
```

A proxy file:

```
9498 (<*plain>)
9499 \input babel.def
9500 (</plain>)
```

15 Acknowledgements

In the initial stages of the development of `babel`, Bernd Raichle provided many helpful suggestions and Michel Goossens supplied contributions for many languages. Ideas from Nico Poppelier, Piet van Oostrum and many others have been used. Paul Wackers and Werenfried Spit helped find and repair bugs.

More recently, there are significant contributions by Salim Bou, Ulrike Fischer, Loren Davis and Udi Fogiel.

Barbara Beeton has helped in improving the manual.

There are also many contributors for specific languages, which are mentioned in the respective files. Without them, babel just wouldn't exist.

References

- [1] Huda Smitshuijzen Abifares, *Arabic Typography*, Saqi, 2001.
- [2] Johannes Braams, Victor Eijkhout and Nico Poppelier, *The development of national $\text{\LaTeX}$ styles*, *TUGboat* 10 (1989) #3, pp. 401–406.
- [3] Yannis Haralambous, *Fonts & Encodings*, O'Reilly, 2007.
- [4] Donald E. Knuth, *The $\text{\TeX}$ book*, Addison-Wesley, 1986.
- [5] Jukka K. Korpela, *Unicode Explained*, O'Reilly, 2006.
- [6] Leslie Lamport, *$\text{\LaTeX}$, A document preparation System*, Addison-Wesley, 1986.
- [7] Leslie Lamport, in: $\text{\TeX}$ hax Digest, Volume 89, #13, 17 February 1989.
- [8] Ken Lunde, *CJKV Information Processing*, O'Reilly, 2nd ed., 2009.
- [9] Edward M. Reingold and Nachum Dershowitz, *Calendrical Calculations: The Ultimate Edition*, Cambridge University Press, 2018.
- [10] Hubert Partl, *German $\text{\TeX}$* , *TUGboat* 9 (1988) #1, pp. 70–72.
- [11] Joachim Schrod, *International $\text{\LaTeX}$ is ready to use*, *TUGboat* 11 (1990) #1, pp. 87–90.
- [12] Apostolos Syropoulos, Antonis Tsolomitis and Nick Sofroniu, *Digital typography using $\text{\LaTeX}$* , Springer, 2002, pp. 301–373.
- [13] K.F. Treebus. *Tekstwijzer; een gids voor het grafisch verwerken van tekst*, SDU Uitgeverij ('s-Gravenhage, 1988).